

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

LIMA: *MIDDLEWARE* DE ASISTENCIA POR VOZ PARA GNU/LINUX CON ENFOQUE EN INTELIGENCIA ARTIFICIAL CENTRADA EN EL HUMANO

TESIS

PARA OBTENER EL TÍTULO DE INGENIERO EN COMPUTACIÓN

PRESENTA

JOSE RAMON ARAGON TOLEDO

DIRECTOR DE TESIS

M.C. RICARDO RUIZ RODRÍGUEZ

HUAJUAPAN DE LEÓN, OAX., SEPTIEMBRE DE 2026

Dedicatoria

*A mis padres, por su amor incondicional,
su paciencia y su apoyo en cada paso de este camino.*

A mi hermana, por siempre creer en mí.

A mis amigos, por hacer más ligero el trayecto.

Agradecimientos

Agradezco a Dios por permitirme llegar hasta este momento y por darme la fortaleza necesaria para culminar esta etapa de mi formación.

A mis padres, por su sacrificio, su confianza y su apoyo incondicional a lo largo de toda mi vida y, en especial, durante mis estudios universitarios. Todo lo que soy se los debo a ustedes.

A las Escuelas Pías, donde cursé mi educación primaria, secundaria y bachillerato, por sentar las bases de mi formación académica y humana.

A mi director de tesis, M.C. Ricardo Ruiz Rodríguez, por su guía, sus observaciones y su disposición constante para orientar este trabajo. Su experiencia y compromiso fueron fundamentales para llevarlo a buen término.

A la Universidad Tecnológica de la Mixteca, por la excelente formación que me ha aportado para llevar a cabo este proyecto de tesis.

A las autoridades universitarias, por el apoyo económico que hizo posible mi estancia en Valencia, España, durante la *International Conference on Artificial Intelligence in Society 2026*, donde se presentó parte de este trabajo. Ese respaldo no solo cubrió el alojamiento en la sede del evento: me dio la libertad de permanecer los tres días completos y conversar con investigadores llegados de todo el mundo a exponer sus artículos. Los intercambios que surgieron de esas conversaciones —algunos en las sesiones, muchos más en los pasillos— influyeron en decisiones de este trabajo que difícilmente habría tomado desde el escritorio, y abrieron vínculos académicos que espero conservar. Pocas veces una institución le concede a un estudiante de licenciatura la oportunidad de sentarse a esa mesa; agradezco que la mía lo haya hecho.

Índice

Índice	III
Lista de tablas	X
Lista de figuras	XIII
Resumen	XVI
1 Introducción	1
1.1 Planteamiento del problema	2
1.2 Justificación	3
1.3 Delimitaciones de la tesis	4
1.4 Hipótesis del trabajo	4
1.5 Objetivos del trabajo	5
1.6 Cronograma general del trabajo	5
1.7 Metodología	6
1.8 Declaración sobre el uso de herramientas de IA generativa	9
2 Marco Teórico	10
2.1 Métodos de aprendizaje en inteligencia artificial	10
2.2 Aplicaciones de la IA en la computación	12
2.3 Contexto Histórico y Evolución de la IA	14
2.3.1 Nacimiento de la disciplina (1943–1956)	14
2.3.2 El Test de Turing	15
2.3.3 Las tres oleadas de la IA	18
2.3.4 Hitos que transformaron la IA	23
2.3.5 Problema de la caja negra	25
2.3.6 Desafíos éticos, sociales y ambientales	26
2.3.7 Analogía: el rascacielos de la IA	27
2.3.8 Los LLM en el panorama actual	27

2.4	Sumario del capítulo	35
3	Diseño e Implementación del Sistema	36
3.1	Tecnologías y áreas de conocimiento utilizadas en LIMA	36
3.1.1	Teoría de agentes inteligentes y posicionamiento de LIMA	36
3.1.2	Procesamiento de lenguaje natural y reconocimiento de voz	40
3.1.3	Interacción humano-computadora e inteligencia artificial centrada en el humano	41
3.2	Consideraciones adicionales de diseño	44
3.2.1	La interfaz de línea de comandos como barrera de accesibilidad	44
3.2.2	Seguridad en sistemas asistidos por inteligencia artificial	44
3.3	Trabajos relacionados	45
3.3.1	Consideraciones finales	48
3.4	Visión general de la arquitectura	48
3.5	Orquestador principal: <code>lima.pl</code>	50
3.5.1	Inicialización y seguridad de entorno	50
3.5.2	Diccionario de <i>callbacks</i>	50
3.5.3	Función <code>process_text</code> : el <i>pipeline</i> de tres niveles	50
3.5.4	Función <code>execute_command</code> : despacho de acciones	52
3.5.5	Manejo de errores	52
3.6	Módulo de reconocimiento de voz: <code>VoiceRecognition.pm</code>	53
3.6.1	Captura de audio basada en procesos	53
3.6.2	Transcripción con OpenAI Whisper	54
3.6.3	Transcripción con Vosk (modo fuera de línea)	55
3.6.4	Subrutinas del módulo	55
3.7	Módulo de detección de palabra clave: <code>WakeWordDetector.pm</code>	55
3.7.1	Arquitectura de comunicación interproceso	56
3.7.2	Algoritmo de detección de silencio	57
3.7.3	Verificación de dependencias	57
3.8	Módulo de detección local de intenciones: <code>IntentDetector.pm</code>	58
3.8.1	Categorías de intención	58
3.9	Módulo de aprendizaje de tareas: <code>TaskLearner.pm</code>	59
3.9.1	Posición en el <i>pipeline</i> de tres niveles	60
3.9.2	Algoritmo TF-IDF con similitud coseno	61
3.9.3	Mecanismo de aprendizaje e ingeniería inversa de variables	62
3.9.4	Extracción por anclas léxicas (<i>slot-filling</i> adaptativo)	62

3.9.5	Gestión del ciclo de vida de los patrones	65
3.9.6	API pública del módulo	65
3.10	Módulo de interpretación de comandos: <code>CommandParser.pm</code>	66
3.10.1	Esquema JSON de comandos	66
3.10.2	Diseño del <i>prompt</i> del sistema	67
3.10.3	Historial de conversación	67
3.10.4	Modos de conexión al LLM	68
3.10.5	Optimizaciones para el modo fuera de línea	69
3.10.6	Limpieza de la respuesta JSON	70
3.11	Módulo de gestión de archivos: <code>FileManager.pm</code>	70
3.11.1	Validación de rutas y seguridad	70
3.11.2	Operaciones de creación	71
3.11.3	Búsqueda avanzada con filtros	71
3.11.4	Operaciones de eliminación	73
3.11.5	Información del sistema	73
3.11.6	Creación de proyectos a partir de plantillas	73
3.11.7	Búsqueda web	73
3.11.8	Análisis de frecuencia de palabras	74
3.11.9	Editor de texto integrado	74
3.12	Módulo de síntesis de voz: <code>TTS.pm</code>	75
3.12.1	Perfiles de voz por categoría de acción	75
3.12.2	Motor OpenAI TTS	75
3.12.3	Motor eSpeak (fuera de línea)	76
3.12.4	Método no bloqueante: <code>speak_async()</code>	76
3.13	Módulo de detección de distribución: <code>DistroDetector.pm</code>	77
3.13.1	Familias de distribuciones soportadas	77
3.13.2	Traducción de paquetes	77
3.13.3	Caché estática	78
3.13.4	Validación empírica del soporte multidistribución	78
3.14	Interfaz gráfica de usuario: <code>GUI_GTK3.pm</code>	81
3.14.1	Sistema de temas con CSS	81
3.14.2	Indicador visual de estado con animación	81
3.14.3	Retroalimentación sonora	82
3.14.4	Detección de micrófonos	82
3.14.5	Diálogo de configuración	82
3.14.6	Tutorial interactivo	83

3.14.7	Visor de historial de peticiones	83
3.14.8	Diálogo de confirmación	83
3.14.9	Editor de texto integrado	84
3.14.10	Visor de resultados de búsqueda enriquecido	84
3.15	Sistema de seguridad multicapa	85
3.16	Configuración del sistema	86
3.17	Plantillas de proyectos	87
3.18	Flujo completo de una interacción	87
3.19	Modo en línea y modo fuera de línea	90
3.20	Dependencias externas del sistema	91
3.21	Evolución posterior a la evaluación	91
3.21.1	Distribución del sistema: instalador multidistribución y versión portable	93
3.21.2	Interacción conversacional continua	94
3.21.3	Códigos de acceso y proxy en el borde	98
3.22	Diseño prospectivo: planificación de tareas extensas	101
3.22.1	La limitación actual	102
3.22.2	Por qué no basta con encadenar acciones	102
3.22.3	Diseño propuesto	102
3.22.4	Encaje con lo existente	103
3.22.5	Riesgos previsibles	104
4	Evaluación	105
4.1	Primera ronda de pruebas de usabilidad	105
4.1.1	Hallazgos de la primera ronda	105
4.2	Pruebas locales: conjunto de datos de evaluación	106
4.2.1	Composición del conjunto de datos	107
4.2.2	Metodología de evaluación	107
4.2.3	Conjunto reducido: viabilidad con un modelo local	108
4.3	Segunda ronda de pruebas de usabilidad	111
4.3.1	Preguntas de investigación	111
4.3.2	Diseño metodológico	111
4.3.3	Tareas de evaluación	113
4.3.4	Instrumentos de recolección de datos	114
4.3.5	Procedimiento de cada sesión	117
4.3.6	Tratamiento y análisis de los datos	118
4.3.7	Consideraciones éticas	120

4.3.8	Cronograma de la evaluación	120
4.4	Prueba complementaria: búsqueda manual de comandos frente a LIMA	120
4.4.1	Propósito y encuadre	120
4.4.2	Diseño experimental	121
4.4.3	La tarea	122
4.4.4	Participantes	122
4.4.5	Instrumento de registro	123
4.4.6	Procedimiento	124
4.4.7	Plan de análisis	124
4.4.8	Delimitación de lo que esta prueba mide	126
5	Resultados	127
5.1	Resultados del desarrollo de software	127
5.1.1	Interfaces del sistema	127
5.1.2	Ventana principal en reposo	128
5.1.3	Modo Atención activado	128
5.1.4	Grabación de un comando	130
5.1.5	Ventana de aclaración	130
5.1.6	Diálogo de configuración: micrófono y modo de operación	130
5.1.7	Diálogo de configuración: voz, seguridad y aprendizaje	132
5.1.8	Mejoras de interfaz implementadas tras la evaluación de usabilidad . . .	132
5.1.9	Arquitectura final y módulos	133
5.1.10	Funcionalidades implementadas	134
5.2	Resultados de las pruebas locales del conjunto de datos	134
5.2.1	Precisión de generación de comandos	134
5.2.2	Detección de operaciones riesgosas	137
5.2.3	Cobertura del detector local de intenciones	138
5.2.4	Comparación entre modelos: GPT-4o frente a DeepSeek	139
5.2.5	Conjunto reducido con modelo local	140
5.3	Resultados de la segunda ronda de pruebas de usabilidad	143
5.3.1	Muestra final de participantes	143
5.3.2	Desarrollo de las sesiones	144
5.3.3	Hallazgos cualitativos	144
5.3.4	Casos particulares e incidencias técnicas	145
5.3.5	Contraste con la primera ronda	146
5.4	Métricas y estadísticas de uso de LIMA	146

5.4.1	Eficacia: tasa de éxito y casos de éxito	146
5.4.2	Eficiencia: tiempos de ejecución	147
5.4.3	Satisfacción	148
5.4.4	Comparación por condición de modelo: DeepSeek frente a GPT-4o . . .	152
5.5	Resultados de la prueba complementaria: voz frente a búsqueda manual de co- mandos	153
5.5.1	Composición de la muestra	153
5.5.2	Tiempos de completitud	154
5.5.3	Contraste estadístico	156
5.5.4	Efecto del orden y control del contrabalanceo	156
5.5.5	Esfuerzo y completitud	157
5.5.6	Quién se beneficia más	158
5.5.7	Observaciones cualitativas	159
5.5.8	Veredicto sobre la condición C3	159
5.6	Resultados generales de la investigación	160
5.6.1	Respuesta a las preguntas de investigación	160
5.6.2	Cumplimiento de los objetivos y la hipótesis	161
5.7	Discusión	162
5.7.1	La eficiencia depende de con qué se compare	162
5.7.2	Contraste con estudios de usabilidad de asistentes de voz	163
5.7.3	Limitaciones del estudio	164
6	Conclusiones y trabajo futuro	165
	Referencias	174
A	Interfaces obsoletas	182
B	Conjunto de datos de evaluación	188
B.1	Gestión de archivos	188
B.2	Información del sistema	191
B.3	Control de procesos	193
B.4	Redes	194
B.5	Permisos	196
B.6	Búsqueda web	198
B.7	Creación de proyectos	199
B.8	Apertura de aplicaciones	200

C Instrumentos de evaluación cualitativa	202
C.1 Formato de consentimiento informado — Segunda ronda	202
C.2 Guión de prueba de usabilidad — Segunda ronda	205
C.3 Formato de consentimiento informado — Primera ronda	216
C.4 Guión de prueba de usabilidad — Primera ronda	217
D Código fuente de módulos principales	222

Lista de tablas

1.1	Cronograma general del trabajo de tesis	6
1.2	Cronograma de trabajo en formato Gantt simplificado (octubre 2025 – julio 2026). Celda coloreada = mes de actividad activa; el color identifica la etapa.	7
2.1	Comparación entre modelos de lenguaje local y remoto	35
3.1	Evaluación de LIMA frente a los criterios de agencia (Wooldridge y Jennings, 1995)	40
3.2	Las 18 directrices para la interacción humano-IA (Amershi et al., 2019)	43
3.3	Comparativa de proyectos relacionados.	48
3.4	Inventario de componentes de LIMA	49
3.5	Diccionario de <i>callbacks</i> del orquestador	51
3.6	Maapeo de acciones a métodos de <code>FileManager</code>	53
3.7	Subrutinas de <code>VoiceRecognition.pm</code>	55
3.8	Protocolo IPC del detector de palabra clave	57
3.9	Categorías de intención local y ejemplos de patrones	59
3.10	Comparación de los tres niveles del <i>pipeline</i> de interpretación	61
3.11	Roles semánticos y anclas léxicas asociadas en <code>TaskLearner.pm</code>	63
3.12	Subrutinas públicas de <code>TaskLearner.pm</code>	65
3.13	Esquema JSON de un comando interpretado	67
3.14	Filtros de búsqueda soportados por <code>FileManager</code>	72
3.15	Comandos del sistema ejecutados por <code>system_info()</code>	73
3.16	Categorías de perfiles de voz	76
3.17	Familias de distribuciones GNU/Linux soportadas y sus gestores de paquetes	78
3.18	Validación del instalador universal sobre máquinas virtuales limpias	79
3.19	Estados del indicador visual de LIMA	82
3.20	Secciones de <code>config.json</code> : conexión, rutas y seguridad	87
3.21	Secciones de <code>config.json</code> : audio, voz, interfaz y aprendizaje	88
3.22	Plantillas de proyectos incluidas en LIMA	88
3.23	Comparación de los modos de operación de LIMA	91

3.24	Dependencias externas de LIMA	92
3.25	Estado de las capacidades del sistema respecto del corte de la evaluación	93
3.26	Rutas expuestas por el proxy en el borde	100
4.1	Composición del conjunto de datos de evaluación	107
4.2	Distribución de participantes por condición experimental de LLM	113
4.3	Tareas de la evaluación cualitativa — Segunda Ronda	114
4.4	Dimensiones de análisis por tarea	119
4.5	Cronograma planificado para la segunda ronda de pruebas de usabilidad	120
4.6	Diseño de la prueba complementaria de la condición C3.	121
5.1	Módulos finales de LIMA y tamaño aproximado en líneas de código (Perl 5). .	133
5.2	Relación entre funcionalidades planificadas e implementadas.	135
5.3	Concordancia de acción por categoría (<i>in-scope</i> , <i>deepseek-chat</i>)	136
5.4	Cobertura y precisión efectiva sobre las 250 tareas (subconjunto <i>in-scope</i> , $n = 127$).	137
5.5	Confirmación exigida frente al riesgo esperado (<i>deepseek-chat</i>)	138
5.6	Cobertura del detector local por categoría (250 tareas).	139
5.7	Comparación de modelos sobre el subconjunto <i>in-scope</i>	139
5.8	Concordancia de acción sobre el conjunto reducido de 100 tareas (subconjunto <i>in-scope</i> , modelo local).	140
5.9	Comparación de los tres modelos sobre las mismas tareas	141
5.10	Detección de riesgo sobre el conjunto reducido (modelo local).	142
5.11	Contraste entre el conjunto completo y el reducido.	142
5.12	Características agregadas de la muestra final ($n = 14$).	143
5.13	Preferencia de modalidad declarada por los participantes al terminar cada tarea. .	145
5.14	Eficacia por tarea y modalidad. Sí = éxito completo, Parc. = éxito parcial, No = sin éxito; la tasa pondera $w \in \{1.0; 0.5; 0\}$	147
5.15	Tiempos de ejecución en segundos por tarea y modalidad. Se reportan media ($\bar{x}$), desviación estándar (DE), mínimo y máximo.	147
5.16	Tiempos medios (s) de la voz y del entorno gráfico por condición de modelo ($n_A = 7$, $n_B = 7$).	148
5.17	Facilidad de uso percibida post-tarea (escala 1–5). Media y DE.	149
5.18	Cuestionario final de satisfacción. Calificación en escala 1–10; el resto en escala 1–5. Media (DE) y desglose por condición.	150
5.19	Comparación exploratoria por condición de modelo.	152
5.20	Perfil de la muestra de la prueba complementaria ($n = 7$).	153

5.21	Registro por participante. Tiempos en segundos; Δ = manual – LIMA; reducción = Δ /manual.	154
5.22	Estadísticos descriptivos de la prueba complementaria ($n = 7$). Tiempos en segundos.	154
5.23	Contraste estadístico de la diferencia de tiempos ($n = 7$ pares).	156
5.24	Tiempos medios (s) por orden de presentación de las condiciones.	157
5.25	Esfuerzo y completitud por condición ($n = 7$).	157
6.1	Contraste de la hipótesis con los resultados.	166
6.2	Grado de cumplimiento de los objetivos específicos.	168
B.1	Tareas de evaluación — Gestión de archivos (50 tareas)	188
B.2	Tareas de evaluación — Información del sistema (45 tareas)	191
B.3	Tareas de evaluación — Control de procesos (37 tareas)	193
B.4	Tareas de evaluación — Redes (37 tareas)	195
B.5	Tareas de evaluación — Permisos (31 tareas)	196
B.6	Tareas de evaluación — Búsqueda web (20 tareas)	198
B.7	Tareas de evaluación — Creación de proyectos (20 tareas)	199
B.8	Tareas de evaluación — Apertura de aplicaciones (10 tareas)	201

Lista de figuras

1.1	Flujo metodológico iterativo del trabajo de tesis.	8
2.1	Métodos fundamentales de aprendizaje en IA: supervisado, no supervisado y por refuerzo.	11
2.2	Métodos híbridos y avanzados: semi-supervisado y auto-supervisado.	11
2.3	Línea temporal de la IA - Parte 1: Orígenes y fundamentos teóricos (1943–1956).	19
2.4	Línea temporal de la IA - Parte 2: Primera oleada simbólica y el primer invierno.	21
2.5	Línea temporal de la IA - Parte 3: Conexionismo, modelos estadísticos y el segundo invierno.	23
2.6	Línea temporal de la IA - Parte 4: Era del Deep Learning y los Transformers.	24
2.7	Línea temporal de la IA - Parte 5: Era generativa y proyecciones futuras.	25
2.8	Arquitectura Mixture of Experts (MoE) de DeepSeek con activación selectiva de parámetros. Solo 2–3 expertos se activan por token, reduciendo el cómputo en ~80% mientras mantiene 671B parámetros totales. Fuente: elaboración propia.	29
2.9	Arquitectura simplificada de GPT-4 (OpenAI). El modelo utiliza un <i>Transformer decoder-only</i> con atención multi-cabeza y alineación mediante RLHF para generar respuestas seguras y útiles. Fuente: elaboración propia.	31
2.10	Arquitectura conceptual de Grok (xAI). El modelo integra información contextual de la plataforma X para generar respuestas personalizadas con menor moderación algorítmica. Fuente: elaboración propia.	33
2.11	Arquitectura multimodal nativa de Gemini (Google DeepMind). El modelo procesa múltiples modalidades mediante encoders especializados que convergen en una representación unificada. Fuente: elaboración propia.	34
3.1	Ciclo percepción-acción del agente inteligente (Russell y Norvig, 2021). El agente percibe su entorno a través de sensores y actúa sobre él mediante actuadores.	37
3.2	Clasificación de los tipos de agente según Russell y Norvig (2021), de menor a mayor complejidad, con el posicionamiento de LIMA.	38

3.3	Posicionamiento de LIMA en la taxonomía de Sánchez (1997). LIMA se sitúa en la intersección de las tres categorías, con énfasis en la función de agente de interfaz.	41
3.4	Pipeline conceptual de LIMA: desde la entrada de voz hasta la respuesta al usuario. La fila superior muestra la etapa de entrada y análisis; la inferior, la ejecución y retroalimentación.	42
3.5	Marco HCAI de Shneiderman (2022). LIMA se posiciona en el cuadrante de IA centrada en el humano: alta automatización con alto control del usuario.	43
3.6	Arquitectura general de LIMA. Las flechas sólidas indican flujo de control; las flechas punteadas muestran las dos escalaciones del <i>pipeline</i> de interpretación: IntentDetector → TaskLearner → CommandParser (LLM).	49
3.7	Flujo de datos del módulo VoiceRecognition.pm.	54
3.8	Flujo del módulo WakeWordDetector.pm. El bucle de escucha opera continuamente en un proceso Python separado.	56
3.9	Flujo de decisión del módulo IntentDetector.pm.	58
3.10	Flujo dual de TaskLearner.pm.	60
3.11	Flujo de datos del módulo CommandParser.pm.	66
3.12	Estructura del módulo FileManager.pm. Toda operación que involucra rutas pasa primero por <code>_validate_path()</code>	70
3.13	Flujo del módulo TTS.pm. Si OpenAI TTS falla, el sistema degrada automáticamente a eSpeak.	75
3.14	Flujo del módulo DistroDetector.pm.	77
3.15	Traducción de nombres de paquete según la distribución	79
3.16	Estructura de la interfaz gráfica GUI_GTK3.pm. Las flechas punteadas indican diálogos modales que se abren desde la ventana principal.	81
3.17	Capas del sistema de seguridad de LIMA. Cada capa opera independientemente del LLM.	86
3.18	Flujo completo de una interacción con LIMA. El paso 6 muestra el <i>pipeline</i> de tres niveles: detector local (IntentDetector), patrones aprendidos (TaskLearner) y LLM (CommandParser). El paso 10 cierra el ciclo de aprendizaje para solicitudes resueltas por el LLM.	89
3.19	Turno en el modo de conversación continua	97
3.20	Flujo de una petición a través del proxy en el borde	99
5.1	Ventana principal (interfaz actual)	129
5.2	Modo Atención activado (interfaz actual)	129
5.3	Captura de un comando (interfaz actual)	130

5.4	Ventana de aclaración	131
5.5	Configuración: micrófono y modo	131
5.6	Configuración: voz, seguridad y aprendizaje	132
5.7	Nivel de frustración por tarea y condición de modelo.	145
5.8	Tasa de completitud (eficacia) por tarea, modalidad y condición de modelo. . .	147
5.9	Tiempo promedio en T1 (organizar carpetas) por modalidad y condición ($\pm$ error estándar).	149
5.10	Tiempo promedio en T2 (mover PDF) por modalidad y condición ($\pm$ error estándar).	149
5.11	Tiempo promedio en T3 (búsqueda web) por modalidad y condición ($\pm$ error estándar).	150
5.12	Calificación general de LIMA por participante (escala 1–10).	151
5.13	Dimensiones del cuestionario final por condición de modelo ($\pm$ error estándar). .	151
5.14	Perfil de experiencia de usuario por condición de modelo.	152
5.15	Tiempo de completitud por participante en ambas condiciones. Cada línea gris une las dos mediciones de una misma persona; la línea negra une las medias. . .	155
5.16	Tiempo medio por condición ($\pm$ error estándar) y reducción del tiempo por participante.	155
A.1	Ventana principal (interfaz previa)	183
A.2	Modo Atención activado (interfaz previa)	184
A.3	Captura de un comando (interfaz previa)	184
A.4	Diálogo de confirmación previa (interfaz previa)	185
A.5	Visor de resultados de búsqueda (interfaz previa)	185
A.6	Visor de historial de peticiones (interfaz previa)	186
A.7	Tutorial de primer uso, paso 1 (interfaz previa)	186
A.8	Tutorial de primer uso, paso 2 (interfaz previa)	187
A.9	Tutorial de primer uso, paso 3 (interfaz previa)	187

Resumen

El presente trabajo de tesis describe el diseño, desarrollo y evaluación de **LIMA** (*Linux Intelligent Middleware Assistant*), un *software* de asistencia por voz que permite a usuarios hispanohablantes interactuar con el sistema operativo GNU/Linux mediante comandos de voz en español.

La problemática que motivó este trabajo radica en que la interfaz de línea de comandos de GNU/Linux, pese a su potencia, constituye una barrera significativa para usuarios no especializados. Operaciones cotidianas de administración de archivos exigen memorizar y componer comandos cuya sintaxis resulta críptica, lo que limita la adopción de este sistema operativo en contextos donde sus ventajas —costo nulo, seguridad, personalización— serían especialmente valiosas.

El sistema integra herramientas de reconocimiento automático del habla, modelos de lenguaje de gran escala para la interpretación de intenciones, síntesis de voz para la retroalimentación y una interfaz gráfica que presenta de forma transparente las operaciones a ejecutar. El desarrollo se fundamentó en los principios de Inteligencia Artificial Centrada en el Humano (HCAI), priorizando la transparencia, la seguridad del usuario y la facilidad de uso.

La evaluación del sistema se realizó mediante un conjunto de datos de 250 tareas representativas de interacción con GNU/Linux —gestión de archivos, información del sistema, control de procesos, redes, permisos, búsqueda web, creación de proyectos y apertura de aplicaciones—, pruebas de usabilidad con usuarios de la comunidad universitaria y una prueba de tiempo de completitud que contrasta la interacción por voz con la búsqueda manual de comandos, en la que los participantes resolvieron la tarea un 66.6 % más rápido con el asistente.

Capítulo 1

Introducción

Este capítulo contextualiza el problema que motiva el presente trabajo: la brecha entre la potencia de GNU/Linux y la complejidad de su interfaz, que limita su accesibilidad para usuarios no especializados. Se describe el panorama actual de la inteligencia artificial y los asistentes de voz, se delimita el problema de investigación, se enuncian la hipótesis y los objetivos, se justifica la pertinencia del trabajo, se presentan el alcance y las limitaciones, y se concluye con el cronograma de actividades y la metodología de desarrollo.

Los últimos años han sido testigos de una transformación acelerada en el campo de la inteligencia artificial. La arquitectura *Transformer* (Vaswani et al., 2017) desencadenó una sucesión de modelos de lenguaje de gran escala —GPT-3 (Brown et al., 2020), GPT-4 (OpenAI, 2023), DeepSeek (DeepSeek-AI, 2024)— que demostraron capacidades sin precedentes en comprensión y generación de texto, programación asistida (Chen et al., 2021) y razonamiento general. Paralelamente, los sistemas de reconocimiento automático del habla alcanzaron niveles de precisión equiparables a los humanos en múltiples idiomas, como evidencia el modelo Whisper de OpenAI (OpenAI, 2022).

En el ámbito comercial, los asistentes virtuales de voz se han convertido en productos de consumo masivo. Siri (Apple, 2011), Google Assistant (2016) y Alexa (Amazon, 2014) acumulan más de mil millones de usuarios activos en conjunto (Juniper Research, 2023) y se integran en teléfonos inteligentes, altavoces, automóviles y electrodomésticos. Estos asistentes permiten realizar tareas cotidianas —reproducir música, consultar el clima, controlar dispositivos del hogar— mediante instrucciones de voz en lenguaje natural (Luger y Sellen, 2016).

GNU/Linux es el sistema operativo predominante en la infraestructura tecnológica mundial: impulsa aproximadamente el 96% de los servidores *web* más utilizados del planeta, la totalidad de las 500 supercomputadoras más potentes y más del 70% de los dispositivos móviles a través de Android (Top500.org, 2023; W3Techs, 2023). Su adopción en entornos académicos, empresariales y gubernamentales crece constantemente, impulsada por su naturaleza de código

abierto, su robustez en materia de seguridad y su costo nulo de licenciamiento.

Sin embargo, la interacción principal con GNU/Linux sigue dependiendo de la *interfaz de línea de comandos* (CLI, por sus siglas en inglés: *Command-Line Interface*), un paradigma diseñado en la década de 1970 que privilegia la eficiencia del usuario experto por encima de la facilidad de aprendizaje del usuario novato (Norman, 1988; Raymond, 2003). Operaciones relativamente sencillas —como buscar archivos por tamaño, organizar documentos por tipo o crear estructuras de directorios— exigen memorizar y componer comandos cuya sintaxis resulta críptica para la mayoría de los usuarios no especializados.

No obstante, el avance de la IA conversacional no ha cerrado la brecha entre las capacidades de los asistentes comerciales y las necesidades de administración de sistemas operativos. Los asistentes de voz actuales operan dentro de ecosistemas cerrados orientados al consumo; ninguno ofrece la posibilidad de ejecutar operaciones reales sobre el sistema de archivos de GNU/Linux. Por otra parte, las herramientas de generación de código basadas en modelos de lenguaje —como Codex (Chen et al., 2021) o sistemas de traducción de lenguaje natural a comandos de terminal (Lin et al., 2018)— demuestran la viabilidad técnica de la conversión instrucción–comando, pero carecen de integración con la voz, de mecanismos de seguridad que clasifiquen el riesgo de las operaciones y de soporte nativo para el español.

Esta convergencia de factores —la madurez de los modelos de lenguaje, la precisión del reconocimiento de voz, la predominancia de GNU/Linux en la infraestructura tecnológica y la ausencia de un asistente de voz para su administración— configura tanto una oportunidad como una necesidad. El enfoque de Inteligencia Artificial Centrada en el Humano (HCAI) (Shneiderman, 2022) proporciona el marco teórico para abordarla: un sistema que no solo traduzca voz a comandos, sino que garantice transparencia, seguridad y control del usuario en todo momento.

1.1. Planteamiento del problema

La brecha entre la potencia de GNU/Linux y la complejidad de su interfaz genera un problema de accesibilidad tecnológica con varias dimensiones:

1. **Barrera de aprendizaje.** Investigaciones previas reportan que usuarios novatos invierten, en promedio, entre 7 y 8 minutos buscando el comando apropiado para tareas que un experto resuelve en menos de un minuto (Lin et al., 2018).
2. **Riesgo operativo.** La CLI no distingue visualmente entre operaciones inocuas y destructivas; un error menor en un comando como `rm -rf` puede provocar la pérdida irrecuperable de información.
3. **Exclusión lingüística.** La práctica totalidad de la documentación técnica, las páginas de manual y las herramientas de asistencia existentes se encuentran en idioma inglés, lo que

agrava la barrera para la comunidad hispanohablante.

4. **Limitaciones de las soluciones actuales.** Los administradores de archivos gráficos sacrifican la potencia de la CLI; los asistentes de propósito general como ChatGPT sugieren comandos pero no los ejecutan; y los asistentes de voz comerciales (Alexa, Siri, Google Assistant) carecen de capacidades reales de administración del sistema.

A partir de la problemática descrita, la pregunta central que guió esta investigación es:

¿Es viable el desarrollo de un middleware con interacción por voz en español, con base en los principios de HCAI y componentes de IA, en particular modelos de lenguaje, integrado con un mecanismo de detección de riesgo por capas, que permita la gestión de archivos, las consultas del sistema y otras tareas cotidianas dentro de GNU/Linux con una precisión de al menos 85% respecto a las intenciones del usuario y la ejecución?

1.2. Justificación

La facilidad de acceso a herramientas computacionales potentes constituye un imperativo en la sociedad del conocimiento. GNU/Linux, pese a sus ventajas en costo, seguridad y personalización, permanece inaccesible para millones de usuarios potenciales debido a la complejidad de su interfaz de comandos (Warschauer, 2004). Un *software* que permita interactuar con el sistema de archivos mediante comandos de voz en español contribuye directamente a reducir la brecha digital —una preocupación central de la Interacción Humano-Computadora orientada a los Objetivos de Desarrollo Sostenible (Kumar et al., 2020)—, especialmente en contextos educativos y de administración pública donde la adopción de *software* libre es estratégica.

Este trabajo se inscribe además en el marco de la Inteligencia Artificial Centrada en el Humano (HCAI), un paradigma que postula que los sistemas de IA deben diseñarse para *aumentar* las capacidades humanas, no para reemplazarlas, garantizando fiabilidad, seguridad y transparencia (Shneiderman, 2020, 2022). La investigación contribuye al cuerpo teórico de HCAI al explorar cómo integrar modelos de lenguaje de gran escala en una herramienta de uso cotidiano manteniendo el control y la comprensión del usuario en todo momento.

LIMA ofrece:

- Un conjunto de capacidades funcionales que demuestra la viabilidad de la traducción voz → instrucción verbal → acción ejecutable, incluyendo gestión de archivos y directorios, consultas de información del sistema, apertura de archivos o carpetas con la aplicación predeterminada y búsquedas web básicas.

- Soporte tanto para modelos en la nube como para modelos locales, lo que permite operar sin conexión a internet y con total privacidad de los datos.
- Un sistema de seguridad que clasifica los comandos en tres niveles de riesgo y exige confirmación explícita para operaciones destructivas.
- Generación automatizada de estructuras completas de proyectos de *software* mediante plantillas activadas por voz (por ejemplo, *frontend*, *backend* o *fullstack*), funcionalidad particularmente útil para estudiantes de desarrollo de *software* que necesitan iniciar proyectos con una estructura estándar de forma rápida y sin errores.

1.3. Delimitaciones de la tesis

El presente trabajo se delimita en los siguientes aspectos:

1. El *software* se centra en la gestión de archivos y directorios, consultas de información del sistema, control básico de procesos, operaciones de red y permisos, así como búsquedas web básicas; no abarca la administración completa del sistema operativo (usuarios, servicios o configuración avanzada).
2. La interacción por voz se limita al español hablado en México, con énfasis en el contexto lingüístico de Oaxaca.
3. La evaluación cuantitativa se realiza sobre un conjunto de 250 tareas representativas; la evaluación cualitativa se conduce con participantes de la comunidad universitaria de la UTM.
4. El sistema requiere un entorno GNU/Linux funcional con acceso a terminal.

1.4. Hipótesis del trabajo

La integración de herramientas de IA, en particular modelos de lenguaje, con un enfoque basado en los principios de HCAI habilitará a usuarios no expertos en GNU/Linux a realizar actividades de gestión de archivos, consultas del sistema y otras tareas cotidianas dentro del alcance definido con una precisión de al menos el 85%.

1. La traducción automática de instrucciones verbales en español a operaciones de gestión de archivos, consultas del sistema y otras tareas cotidianas dentro del alcance definido alcanzará un nivel de precisión que permita su uso práctico ($\geq 85\%$ de comandos correctos).
2. El mecanismo de clasificación de riesgo identificará operaciones potencialmente destructivas antes de su ejecución con una tasa $\geq 95\%$.

3. Los usuarios no especializados completarán tareas cotidianas representativas dentro del alcance del sistema en menor tiempo que mediante la búsqueda manual de comandos.
4. Los modelos de lenguaje ejecutados localmente ofrecerán resultados funcionales para un subconjunto de tareas cotidianas, especialmente las relacionadas con gestión de archivos, constituyendo una alternativa viable que preserve la privacidad.

1.5. Objetivos del trabajo

Diseñar, desarrollar y evaluar un *software* de asistencia por voz que, fundamentado en los principios de Inteligencia Artificial Centrada en el Humano, permita a usuarios no especializados gestionar archivos y directorios, consultar información del sistema y realizar otras tareas cotidianas dentro del alcance definido en GNU/Linux mediante instrucciones verbales en español.

1. Potenciar el acercamiento de usuarios neófitos al uso de GNU/Linux, con la finalidad de que puedan realizar tareas cotidianas y la gestión básica del sistema de archivos mediante interacción por voz.
2. Investigar y sintetizar los fundamentos teóricos de la Interacción Humano-Computadora, los principios de HCAI y las tecnologías de procesamiento de lenguaje natural aplicables al dominio de la asistencia por voz en sistemas operativos.
3. Diseñar e implementar una arquitectura por capas que integre las herramientas de IA — reconocimiento de voz, interpretación semántica mediante modelos de lenguaje, validación de seguridad y retroalimentación multimodal— en un *middleware* funcional.
4. Implementar un *software* funcional (*LIMA*) que materialice la arquitectura propuesta, incluyendo detección de palabra clave, transcripción de voz, generación de comandos e interfaz gráfica.
5. Evaluar cuantitativamente el *software* mediante un conjunto de datos de 250 tareas representativas distribuidas en ocho categorías funcionales, midiendo precisión de generación, detección de operaciones riesgosas y tiempo de completitud.
6. Evaluar cualitativamente el producto de *software* mediante pruebas de usabilidad con usuarios de la comunidad universitaria, observando facilidad de uso, comprensión de las respuestas y satisfacción general.

1.6. Cronograma general del trabajo

El desarrollo de la tesis se organizó en etapas sucesivas. En una fase intermedia se contó con un *producto mínimo viable* (MVP, por sus siglas en inglés: *Minimum Viable Product*), en-

tendido como una primera versión funcional del sistema sobre la cual se realizaron iteraciones posteriores de evaluación y mejora.

Tabla 1.1: Cronograma general del trabajo de tesis

Etapa	Actividad principal	Resultado
1	Investigación bibliográfica y construcción del marco teórico.	Fundamentación conceptual del problema, del enfoque HCAI y de las tecnologías empleadas.
2	Diseño arquitectónico y desarrollo de una primera versión funcional (MVP).	Base operativa de LIMA con reconocimiento de voz, interpretación de intenciones, ejecución y retroalimentación.
3	Evaluación inicial y retroalimentación académica.	Identificación de ajustes conceptuales, técnicos y metodológicos.
4	Corrección de errores e implementación de mejoras.	Integración de <i>wake word</i> , interfaz GTK3, modelos locales, empaquetado y refinamientos de seguridad.
5	Evaluación final, redacción y preparación de la defensa.	Análisis cuantitativo y cualitativo, consolidación del documento de tesis, versión definitiva y presentación ante el jurado.

La Tabla 1.2 presenta el mismo cronograma en formato Gantt simplificado: cada celda coloreada indica un mes de actividad y el color identifica la etapa. Las actividades activas en los mismos meses se ejecutan en paralelo; la lectura horizontal revela las dependencias entre etapas sucesivas.

1.7. Metodología

El presente trabajo se imbuje en el contexto de **investigación aplicada** y adopta un enfoque **mixto** en su evaluación (cuantitativo y cualitativo). Metodológicamente, se articula como un proceso de **investigación-desarrollo** orientado a la construcción, evaluación y refinamiento iterativo de un artefacto tecnológico.

La Figura 1.1 presenta el flujo metodológico general del trabajo.

El desarrollo de LIMA siguió un modelo iterativo e incremental articulado en ciclos de diseño, evaluación y mejora, con tres momentos principales:

1. **Fase de construcción:** Diseño y desarrollo de una primera versión funcional siguiendo una arquitectura modular de cinco capas (interfaz, reconocimiento de voz, detección de *wake word*, interpretación de intenciones y gestión de archivos).

Tabla 1.2: Cronograma de trabajo en formato Gantt simplificado (octubre 2025 – julio 2026). Celda coloreada = mes de actividad activa; el color identifica la etapa.

Actividad	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun	Jul
Etapa 1 — Investigación bibliográfica y marco teórico										
Revisión bibliográfica										
Redacción del marco teórico										
Etapa 2 — Diseño y desarrollo del MVP										
Diseño de arquitectura del sistema										
Desarrollo del MVP (v1)										
Pruebas unitarias y de usabilidad 1										
Etapa 3 — Protocolo y refinamiento del sistema										
Redacción del protocolo de tesis										
Refinamiento del sistema (v2)										
Evaluación y retroalimentación académica										
Redacción de la tesis (actividad transversal)										
Redacción del documento de tesis										
Etapa 4 — Maduración del sistema y evaluación final										
Consolidación del sistema (v3)										
Pruebas de usabilidad 2 (producto final)										
Etapa 5 — Cierre y defensa de tesis										
Entrega del documento a revisores										
Elaboración de presentación										
Defensa de tesis										

■ Etapa 1
 ■ Etapa 2
 ■ Etapa 3
 ■ Redacción
 ■ Etapa 4
 ■ Etapa 5

- Fase de retroalimentación:** Evaluación inicial, corrección de errores e implementación de mejoras derivadas de las observaciones técnicas y académicas.
- Fase de evaluación final:** Medición cuantitativa del desempeño y valoración cualitativa de la usabilidad.

Se emplea un conjunto de datos de **250 tareas** de interacción con GNU/Linux expresadas como instrucciones verbales en español, distribuidas en ocho categorías: gestión de archivos (50), información del sistema (45), control de procesos (37), redes (37), permisos (31), búsqueda web (20), creación de proyectos (20) y apertura de aplicaciones (10). Cada tarea incluye la acción o comando esperado y su nivel de riesgo (*seguro*, *advertencia*, *peligro*). El conjunto de datos completo se presenta en el Anexo B. Se miden: precisión de generación de acciones, detección de operaciones riesgosas y tiempo de completitud.

Las pruebas de usabilidad se realizan con una muestra de **al menos 15 participantes** (1–

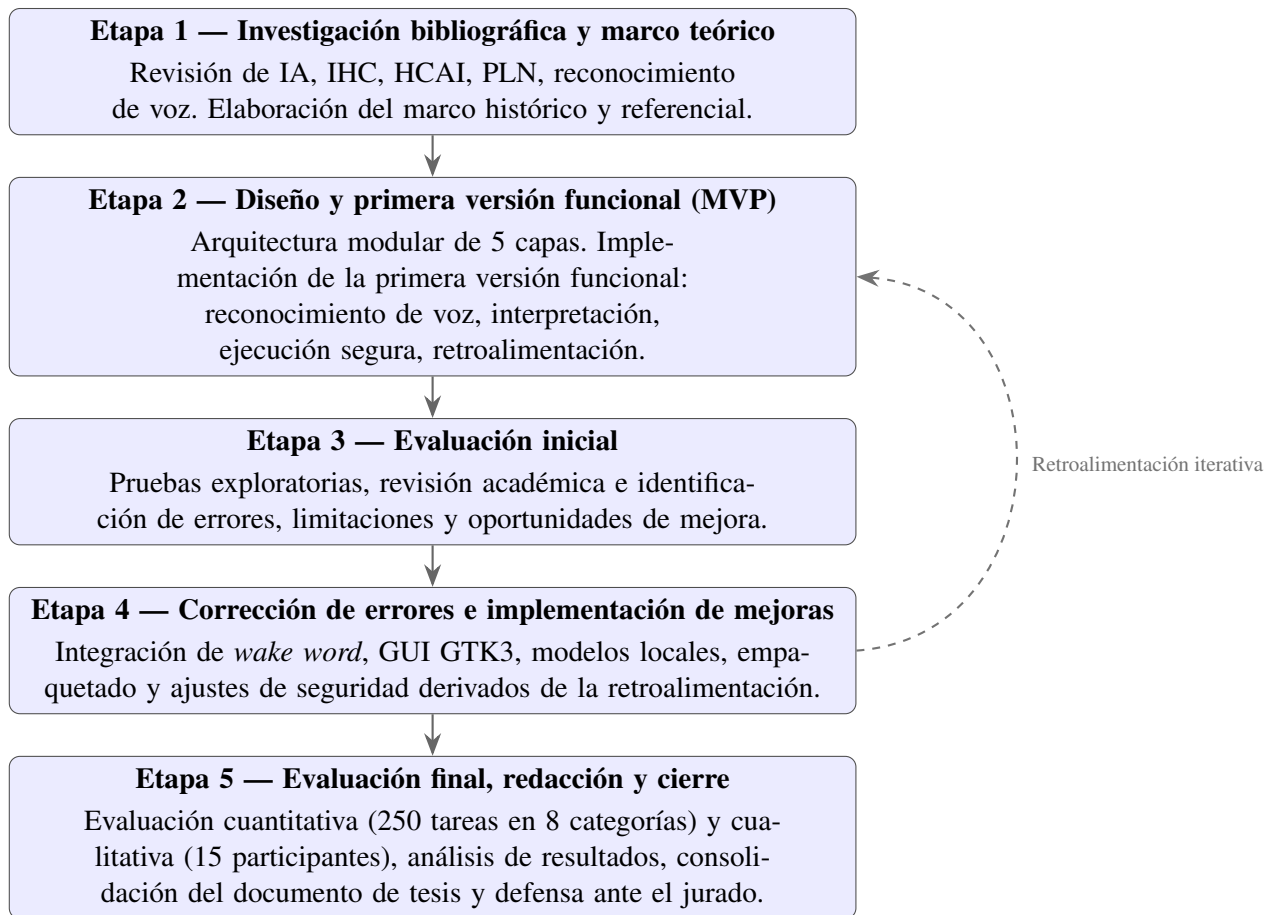


Figura 1.1: Flujo metodológico iterativo del trabajo de tesis.

2 pruebas piloto + 13–14 pruebas reales), pertenecientes a la comunidad universitaria de la UTM y procurando incluir perfiles de distintas áreas académicas. Los participantes varían en su nivel de experiencia tecnológica (básico, intermedio, avanzado). Cada sesión contempla 3 tareas prácticas con registro de tiempo, tasa de completitud y percepción subjetiva.

1. **Conjunto de datos de 250 tareas:** Archivo CSV con identificador, categoría, instrucción verbal, acción o comando esperado y nivel de riesgo (véase Anexo B).
2. **Guión de pruebas de usabilidad:** Protocolo estructurado que incluye instrucciones para el participante, escenarios de las tareas, protocolo de observación y preguntas de control (véase Anexo C.4).
3. **Formato de consentimiento informado:** Documento que garantiza la participación voluntaria, el derecho a retirarse en cualquier momento y la confidencialidad de los datos (véase Anexo C.3).
4. **Registros automáticos del sistema:** *Logs* de actividad generados por LIMA que incluyen la transcripción de voz, el comando interpretado, el resultado de la ejecución y las marcas

temporales.

- **Precisión de generación de comandos:** Porcentaje de comandos generados correctamente respecto al total de tareas, desglosado por categoría.
- **Detección de operaciones peligrosas:** Proporción de comandos destructivos identificados correctamente por el sistema.
- **Tiempo de completitud:** Comparación del tiempo promedio por tarea entre el uso de LIMA y la búsqueda manual de comandos.
- **Usabilidad:** Análisis cualitativo de las observaciones durante las pruebas y de las respuestas a las preguntas de control.

1.8. Declaración sobre el uso de herramientas de IA generativa

En el desarrollo de este trabajo de tesis se emplearon herramientas de inteligencia artificial generativa como apoyo al proceso de investigación y redacción. En particular:

- **Asistentes de redacción (ChatGPT, GitHub Copilot):** Se utilizaron para sugerir redacciones, revisar ortografía y gramática, y generar borradores iniciales de secciones técnicas. Todo el contenido generado fue revisado, verificado y, en su caso, reformulado por el autor.
- **Búsqueda bibliográfica asistida:** Se emplearon herramientas de IA para localizar referencias académicas relevantes. Cada referencia fue verificada manualmente contra la fuente original antes de su inclusión.
- **Generación de código y *scripts*:** GitHub Copilot se utilizó como apoyo en la escritura de módulos del sistema LIMA. Cada fragmento fue revisado, probado y adaptado por el autor.

El autor asume plena responsabilidad por el contenido, la originalidad y la veracidad de la información presentada en este documento. Las herramientas de IA se emplearon como instrumentos de apoyo, no como sustitutos del criterio académico ni del proceso de investigación.

Capítulo 2

Marco Teórico

Este capítulo inicia con una breve descripción de los conceptos fundamentales asociados a la Inteligencia Artificial en relación directa con los términos y acepciones comúnmente asociados en el uso de los Modelos Grandes de Lenguaje (LLM, por sus siglas en inglés: *Large Language Model*), aspectos que resultan esenciales para contextualizar el desarrollo de este proyecto. Desde los paradigmas de aprendizaje automático hasta las arquitecturas neuronales que sustentan estos sistemas, cada elemento teórico contribuye a desentrañar el funcionamiento y las capacidades de herramientas que han transformado radicalmente la interacción entre humanos y máquinas.

2.1. Métodos de aprendizaje en inteligencia artificial

Los métodos de aprendizaje empleados en inteligencia artificial constituyen la base sobre la cual los modelos adquieren conocimiento a partir de los datos. Cada enfoque difiere en la forma en que se estructuran los datos de entrenamiento y en el tipo de supervisión aplicada durante el proceso de aprendizaje (Russell y Norvig, 2021; Mitchell, 1997). Las Figuras 2.1 y 2.2 sintetizan visualmente las cinco categorías principales de aprendizaje, que se describen a continuación.

1. **Aprendizaje supervisado.** Se basa en el uso de datos etiquetados, donde el modelo aprende a asociar entradas con salidas previamente conocidas. Este método es ampliamente utilizado en tareas de clasificación y regresión. *Ejemplo representativo:* diagnóstico médico a partir de imágenes etiquetadas, como el sistema *CheXNet*, desarrollado por investigadores de Stanford en 2017, capaz de detectar neumonía en radiografías de tórax con precisión comparable a radiólogos expertos.
2. **Aprendizaje no supervisado.** En este enfoque, el modelo analiza datos sin etiquetas para descubrir patrones, agrupamientos o estructuras subyacentes. Es común en la segmenta-

Métodos de Aprendizaje en IA - Parte 1: Enfoques Fundamentales

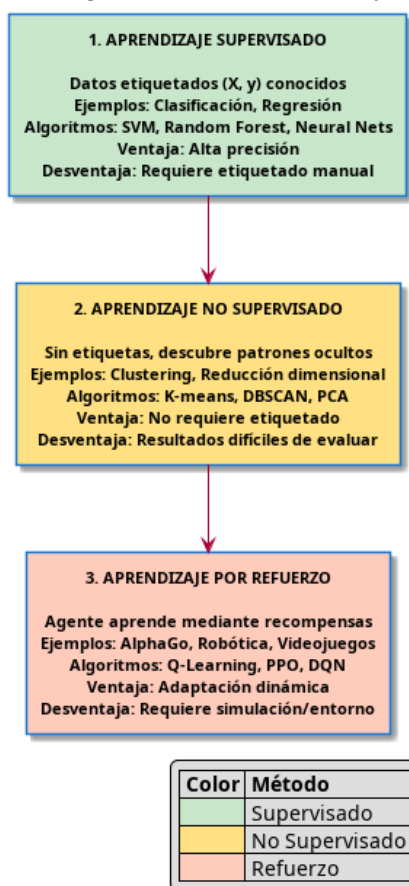


Figura 2.1: Métodos fundamentales de aprendizaje en IA: supervisado, no supervisado y por refuerzo.

Métodos de Aprendizaje en IA - Parte 2: Enfoques Híbridos y Avanzados

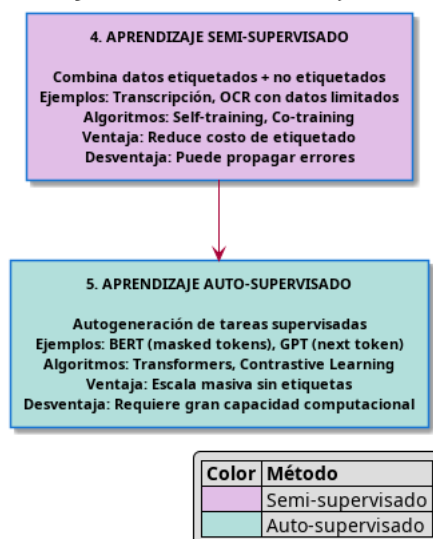


Figura 2.2: Métodos híbridos y avanzados: semi-supervisado y auto-supervisado.

ción de clientes o la reducción de dimensionalidad. *Ejemplo representativo*: segmentación de clientes mediante el algoritmo *K-means* aplicado a bases de datos de compras.

3. **Aprendizaje por refuerzo.** En este paradigma, un agente aprende a través de la interacción con un entorno, recibiendo recompensas o penalizaciones según sus acciones. Se utiliza con éxito en entornos de simulación y juegos complejos. *Ejemplo representativo*: el sistema *AlphaGo*, desarrollado por DeepMind, que logró dominar el juego de Go mediante autoaprendizaje continuo (véase Sección 2.3.4 para más detalles).
4. **Aprendizaje semi-supervisado.** Combina un conjunto reducido de datos etiquetados con una gran cantidad de datos sin etiquetar, mejorando la precisión sin depender de grandes volúmenes de anotaciones manuales. *Ejemplo representativo*: modelos de reconocimiento de voz que aprovechan grabaciones parcialmente etiquetadas.
5. **Aprendizaje auto-supervisado.** Este método permite al modelo generar sus propias etiquetas a partir de la estructura interna de los datos, sin requerir supervisión externa. *Ejemplo representativo*: modelos de lenguaje como BERT (Devlin et al., 2019), que aprenden a predecir palabras ocultas dentro de secuencias textuales (Russell y Norvig, 2021; Sutton y Barto, 2018; He et al., 2022).

2.2. Aplicaciones de la inteligencia artificial en ramas de la computación

La inteligencia artificial ha revolucionado diversas áreas de la computación, introduciendo nuevas formas de optimizar procesos, automatizar análisis y desarrollar soluciones innovadoras. A continuación, se describen las principales aplicaciones documentadas en distintos campos (Russell y Norvig, 2021).

1. **Ciencias de la Computación (Algoritmos y Teoría).** Los algoritmos genéticos se aplican a la resolución de problemas de optimización complejos, como el **Problema del Agente Viajero** (también conocido como *El Viajante* o TSP por sus siglas en inglés: *Travelling Salesman Problem*), que consiste en encontrar la ruta más corta que permita a un vendedor visitar un conjunto de ciudades exactamente una vez y regresar al punto de origen —un problema NP-completo clásico cuya complejidad crece factorialmente con el número de ciudades (Goldberg, 1989). Asimismo, en las pruebas de software, el aprendizaje automático permite generar casos de prueba de manera automatizada, reduciendo significativamente el esfuerzo humano.

2. **Ingeniería de Software.** Los sistemas de generación de código asistidos por IA sugieren fragmentos de código en diversos lenguajes mediante análisis contextual (Chen et al., 2021). Además, modelos especializados permiten identificar patrones de errores o vulnerabilidades en repositorios de software.
3. **Bases de Datos.** El aprendizaje automático contribuye a la optimización de consultas y a la elección de planes de ejecución más eficientes (Marcus et al., 2019). Algunos sistemas incorporan mecanismos predictivos para el mantenimiento automatizado y la detección de anomalías.
4. **Redes de Computadoras.** En las redes definidas por software, los algoritmos inteligentes optimizan el enrutamiento dinámico, mejorando la latencia y la estabilidad en redes 5G (Mao et al., 2016). En materia de seguridad, los modelos de IA identifican patrones anómalos que indican posibles intrusiones.
5. **Sistemas Embebidos e Internet de las Cosas (IoT).** La computación en el borde (*edge computing*) posibilita la ejecución de modelos de visión por computadora en dispositivos con recursos limitados (Howard et al., 2017). En entornos industriales, la robótica inteligente emplea aprendizaje automático para ajustar trayectorias con alta precisión.
6. **Computación Gráfica y Visión por Computadora.** Las redes neuronales convolucionales mejoran la calidad visual en tiempo real y facilitan la reconstrucción de imágenes (Karras et al., 2020). Los modelos generativos, por su parte, son capaces de sintetizar imágenes coherentes a partir de descripciones textuales detalladas.
7. **Seguridad en Cómputo.** Los sistemas modernos de detección de malware se apoyan en clasificadores entrenados con aprendizaje supervisado (Ahmadian et al., 2020). De igual forma, los mecanismos de autenticación biométrica utilizan redes neuronales para el reconocimiento facial en dispositivos personales.
8. **Computación Cuántica.** En el ámbito de la computación cuántica, la IA se aplica a la corrección de errores en circuitos cuánticos, incrementando la estabilidad de los qubits (Biamonte et al., 2017). Estudios recientes muestran el potencial del aprendizaje automático para optimizar operaciones y minimizar la decoherencia cuántica.

2.3. Contexto Histórico y Evolución de la Inteligencia Artificial

La comprensión cabal de los Modelos Grandes de Lenguaje exige situar su desarrollo dentro del devenir histórico de la Inteligencia Artificial como disciplina científica. Los LLM no constituyen un fenómeno aislado, sino la tecnología de vanguardia que emerge tras más de seis décadas de investigación, fracasos, reinventaciones y avances que han transformado la relación entre computación e inteligencia.

2.3.1. El nacimiento formal de la disciplina (1943–1956)

La Inteligencia Artificial surge formalmente en 1956 durante el célebre **Dartmouth Workshop**, organizado por **John McCarthy** —quien acuñó el término “Artificial Intelligence”—, **Marvin Minsky**, **Nathaniel Rochester** y **Claude Shannon**. La premisa fundacional, expresada en la propuesta original del taller, establecía que “cualquier aspecto del aprendizaje o cualquier otra característica de la inteligencia puede, en principio, describirse con tal precisión que una máquina puede simularlo” (McCarthy et al., 1955).

Precursores intelectuales. Antes de Dartmouth, figuras seminales habían sentado las bases matemáticas y teóricas que harían posible la IA:

- **Alan Turing** (1912–1954): En su artículo *Computing Machinery and Intelligence* (1950), propuso el célebre **Test de Turing** como criterio operacional para evaluar la inteligencia de una máquina. Su trabajo previo sobre la máquina universal (1936) estableció los fundamentos de la computabilidad (Turing, 1950).
- **Claude Shannon** (1916–2001): Su *Teoría Matemática de la Comunicación* (1948) formalizó conceptos como entropía, información y codificación, esenciales para el procesamiento de señales y, eventualmente, del lenguaje natural (Shannon, 1948).
- **Norbert Wiener** (1894–1964): Fundador de la **Cibernética**, su obra de 1948 exploró los mecanismos de retroalimentación y control en sistemas biológicos y artificiales, anticipando conceptos centrales en el aprendizaje automático (Wiener, 1948).
- **Warren McCulloch y Walter Pitts** (1943): Propusieron el primer modelo matemático de una neurona artificial, demostrando que redes de unidades simples podían, en principio, computar cualquier función lógica (McCulloch y Pitts, 1943).

2.3.2. El Test de Turing: Fundamento filosófico de la IA conversacional

El artículo *Computing Machinery and Intelligence*, publicado por Alan Turing en la revista *Mind* en octubre de 1950, representa uno de los textos fundacionales de la inteligencia artificial y, específicamente, de la IA conversacional que culminaría décadas después en los Modelos Grandes de Lenguaje (Turing, 1950).

Contexto histórico y motivación

En 1950, las computadoras digitales apenas existían como concepto práctico. La supercomputadora **E.N.I.A.C.** (por sus siglas en inglés: *Electronic Numerical Integrator and Computer*), construida en 1945, había demostrado la viabilidad del cálculo electrónico, pero la idea de que una máquina pudiera “pensar” parecía ciencia ficción. Turing, consciente de la carga filosófica de la pregunta “¿Pueden las máquinas pensar?”, decidió reformularla en términos operacionales que evitaran debates interminables sobre la naturaleza de la consciencia.

El Juego de la Imitación

Turing propuso un experimento denominado **Juego de la Imitación** (*Imitation Game*), estructurado de la siguiente manera:

1. **Participantes:** Un interrogador humano (C), un humano (B) y una máquina (A), todos en habitaciones separadas.
2. **Comunicación:** Exclusivamente mediante texto escrito (teletipos en la época; hoy, terminales de chat).
3. **Objetivo del interrogador:** Determinar cuál participante es la máquina y cuál es humano.
4. **Objetivo de la máquina:** Convencer al interrogador de que es humana.
5. **Criterio de éxito:** Si el interrogador no puede distinguir consistentemente entre A y B, la máquina “pasa” el test.

La elección del texto escrito como medio de comunicación fue deliberada: eliminaba señales no verbales (tono de voz, apariencia física) que podrían revelar la naturaleza artificial del participante, enfocando la evaluación exclusivamente en la **competencia lingüística y cognitiva**.

Objeciones anticipadas por Turing

En su artículo original, Turing anticipó y refutó nueve objeciones potenciales a su propuesta:

1. **Objeción teológica:** “El pensamiento es función del alma inmortal, que las máquinas no poseen.” Turing respondió que Dios podría, si quisiera, otorgar alma a cualquier entidad.
2. **Objeción “cabeza en la arena”:** “Las consecuencias de máquinas pensantes serían terribles, así que no pueden existir.” Turing la descartó como irracional.
3. **Objeción matemática:** Basada en los teoremas de incompletitud de Gödel, sugiere límites a lo que las máquinas pueden demostrar. Turing argumentó que los humanos también tienen limitaciones demostrables.
4. **Argumento de la consciencia:** “Solo quien experimenta consciencia puede pensar.” Turing señaló que, llevado al extremo, este argumento conduce al solipsismo.
5. **Argumento de las discapacidades:** “Las máquinas nunca podrán X” (enamorarse, escribir poesía, etc.). Turing cuestionó la base empírica de tales afirmaciones.
6. **Objeción de Lady Lovelace:** Ada Lovelace afirmó que las máquinas “solo hacen lo que se les ordena”. Turing argumentó que las máquinas podrían sorprendernos mediante comportamiento emergente.
7. **Argumento de la continuidad del sistema nervioso:** El cerebro es analógico, las computadoras digitales. Turing demostró que sistemas digitales pueden aproximar comportamiento continuo arbitrariamente.
8. **Argumento de la informalidad del comportamiento:** “Las reglas no pueden capturar toda la conducta humana.” Turing distinguió entre reglas de conducta y leyes de comportamiento.
9. **Argumento de la percepción extrasensorial:** Turing, sorprendentemente, tomó en serio la posibilidad de telepatía, sugiriendo blindaje telepático para el test.

Implicaciones filosóficas: Funcionalismo vs. Consciencia

El Test de Turing encarna una posición filosófica conocida como **funcionalismo**: lo que importa para atribuir inteligencia no es el sustrato material (neuronas vs. transistores), sino la **organización funcional** del sistema y su capacidad para producir comportamiento apropiado.

Esta postura contrasta con enfoques que exigen:

- **Consciencia fenomenológica:** Experiencia subjetiva (“sentir algo”)
- **Intencionalidad genuina:** Estados mentales “sobre” algo (creencias, deseos)
- **Comprensión semántica:** No solo manipulación sintáctica de símbolos

El debate entre estas posiciones permanece abierto. Los Modelos Grandes de Lenguaje actuales exhiben comportamiento lingüístico sofisticado, pero si “comprenden” genuinamente o solo realizan predicción estadística de tokens sigue siendo una cuestión filosóficamente controvertida (McCarthy et al., 1955; Turing, 1950).

Críticas posteriores al Test

A lo largo de las décadas, el Test de Turing ha recibido críticas sustanciales:

- **Argumento de la Habitación China** (John Searle, 1980): Searle propuso un experimento mental donde una persona, sin saber chino, manipula símbolos siguiendo reglas y produce respuestas correctas en chino. Aunque “pasa el test”, claramente no comprende chino. Conclusión: la simulación del comportamiento no implica comprensión genuina (Searle, 1980).
- **Superficialidad conductual:** Programas como ELIZA (1966) y chatbots posteriores demostraron que trucos conversacionales pueden engañar a evaluadores sin inteligencia genuina.
- **Antropocentrismo:** El test privilegia inteligencia de tipo humano, excluyendo formas de inteligencia alternativas, especializadas o superiores en dominios específicos.
- **Dependencia del evaluador:** Los resultados varían según la sofisticación del interrogador y el tiempo disponible para la evaluación.

El Premio Loebner y competencias modernas

En 1990, Hugh Loebner estableció el **Premio Loebner**, una competencia anual basada en el Test de Turing. Durante décadas, ningún programa logró convencer consistentemente a los jueces. Programas ganadores como **Eugene Goostman** (2014) —que simulaba ser un adolescente ucraniano con inglés limitado— generaron controversia sobre si representaban avances genuinos o simplemente explotaban expectativas reducidas.

La llegada de ChatGPT (2022) y modelos posteriores transformó radicalmente el panorama: por primera vez, sistemas de IA pueden mantener conversaciones extensas, coherentes y contextualmente apropiadas sobre prácticamente cualquier tema, aproximándose significativamente al criterio original de Turing (OpenAI, 2023).

Relevancia para los LLM contemporáneos

El Test de Turing anticipó con notable precisión el desarrollo actual:

- **Conversación como benchmark:** La capacidad conversacional sigue siendo el criterio primario de evaluación de chatbots y asistentes virtuales.
- **Texto como interfaz:** Turing eligió comunicación textual; los LLM operan fundamentalmente con texto.
- **Imitación como estrategia:** Los LLM aprenden a “imitar” texto humano mediante predicción del siguiente token.
- **Debates filosóficos persistentes:** ¿Comprenden los LLM o solo simulan comprensión? La pregunta de Turing sigue sin respuesta definitiva.

El legado de Turing trasciende el test específico: estableció que la **conversación en lenguaje natural** constituye un desafío central y un criterio de evaluación fundamental para la inteligencia artificial (Turing, 1950; Crevier, 1993).

2.3.3. Las tres oleadas de la Inteligencia Artificial

El desarrollo de la IA no ha sido lineal, sino cíclico, caracterizado por períodos de entusiasmo desmedido seguidos de “inviernos” de desencanto y recortes de financiamiento. La comprensión de estos ciclos resulta esencial para contextualizar el estado actual de la disciplina. Las Figuras 2.3 a 2.7 presentan una línea temporal que abarca desde los orígenes teóricos de la IA hasta las proyecciones futuras.

Primera oleada: IA Simbólica (1956–1974)

La primera etapa de la IA estuvo dominada por el paradigma **simbólico** o **GOF AI** (*Good Old-Fashioned AI*, término acuñado por el filósofo John Haugeland en 1985), fundamentado en la **Hipótesis de los Símbolos Físicos** de Newell y Simon (1976) (Newell y Simon, 1976): la manipulación de símbolos mediante reglas lógicas constituye condición necesaria y suficiente para la inteligencia general.

Logros representativos.

- **STUDENT** (1964, Daniel Bobrow): Sistema capaz de resolver problemas algebraicos expresados en lenguaje natural, pionero en comprensión de lenguaje.

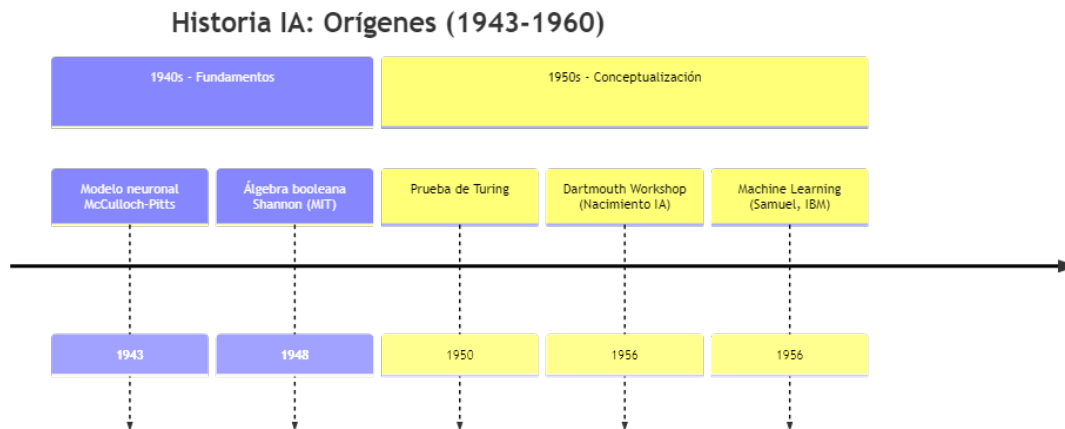


Figura 2.3: Línea temporal de la IA - Parte 1: Orígenes y fundamentos teóricos (1943–1956).

- **SHRDLU** (1970, Terry Winograd): Demostró comprensión y razonamiento sobre un “mundo de bloques” virtual mediante procesamiento de lenguaje natural (Searle, 1980).

ELIZA: El primer chatbot de la historia

En 1966, Joseph Weizenbaum, investigador del Massachusetts Institute of Technology (MIT), desarrolló **ELIZA** (Weizenbaum, 1966), considerado el primer programa informático capaz de simular una conversación en lenguaje natural. El sistema representó un hito en la historia de la interacción humano-computadora y sentó las bases de lo que hoy conocemos como *chatbots*.

Arquitectura y funcionamiento. ELIZA operaba mediante un mecanismo relativamente simple de **coincidencia de patrones** (*pattern matching*) y **sustitución de palabras clave**. El programa identificaba palabras o frases específicas en la entrada del usuario y aplicaba reglas de transformación predefinidas para generar respuestas. Por ejemplo:

- Si el usuario escribía “*I am sad*”, ELIZA podía responder “*Why do you say you are sad?*”
- Frases con “*my mother*” activaban preguntas como “*Tell me more about your family*”
- Cuando no encontraba patrones, recurría a respuestas genéricas: “*Please go on*” o “*That’s very interesting*”

El script DOCTOR. El módulo más famoso de ELIZA fue **DOCTOR**, que simulaba el estilo conversacional de un psicoterapeuta **rogeriano** (basado en la terapia centrada en el cliente de Carl Rogers). Este enfoque terapéutico se caracteriza por:

- **Escucha activa** mediante reformulación de las palabras del paciente
- **Preguntas abiertas** que invitan a la reflexión
- **Empatía no directiva** sin ofrecer consejos explícitos

El Efecto ELIZA. Weizenbaum observó un fenómeno inesperado: muchos usuarios desarrollaban **vínculos emocionales genuinos** con el programa, a pesar de saber que interactuaban con una máquina. Algunos solicitaban privacidad durante sus “sesiones” y atribuían comprensión y empatía al sistema. Este fenómeno, denominado **Efecto ELIZA** (Weizenbaum, 1976), demostró:

- La tendencia humana a **antropomorfizar** sistemas que exhiben comportamiento conversacional
- El poder de la **ilusión de comprensión** generada por respuestas contextualmente apropiadas
- Las implicaciones **éticas** de sistemas que pueden manipular emocionalmente a usuarios vulnerables

Legado para los LLM modernos. ELIZA anticipó debates que permanecen vigentes en la era de los Modelos Grandes de Lenguaje:

- ¿Los chatbots modernos “comprenden” o solo generan respuestas estadísticamente probables?
- ¿Cómo proteger a usuarios emocionalmente vulnerables de sistemas persuasivos?
- ¿Dónde trazar la línea entre asistencia computacional y sustitución de interacción humana genuina?

El Primer Invierno (1974–1980). El optimismo inicial colapsó cuando los sistemas simbólicos fallaron al escalar hacia problemas del mundo real. El informe **Lighthill** (1973) (Lighthill, 1973) en Reino Unido criticó devastadoramente las promesas incumplidas, provocando recortes masivos de financiamiento (véase la Figura 2.4).

Segunda oleada: Conexionismo y Redes Neuronales (1980–1995)

El resurgimiento de la IA en los años 1980 provino de un paradigma radicalmente distinto: el **conexionismo**, que proponía modelar la inteligencia mediante redes de unidades simples interconectadas, inspiradas en la arquitectura neuronal biológica.

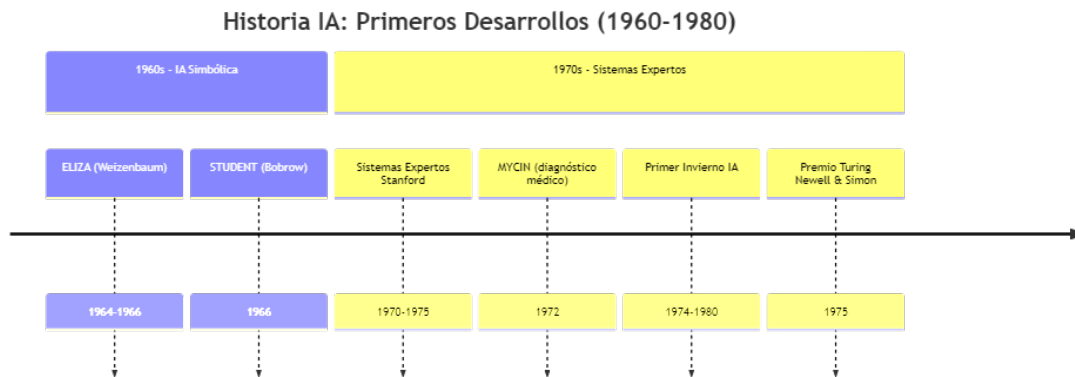


Figura 2.4: Línea temporal de la IA - Parte 2: Primera oleada simbólica y el primer invierno.

Contribuciones fundamentales.

- **John Hopfield (1982):** Físico que aplicó conceptos de materiales magnéticos a las redes neuronales, demostrando que podían funcionar como memorias asociativas estables. Su trabajo le valdría el **Premio Nobel de Física 2024** (Hopfield, 1982).
- **Backpropagation (1986, Rumelhart, Hinton y Williams):** Popularizaron el algoritmo de retropropagación del error, que permitía entrenar redes multicapa de manera eficiente. Este método sigue siendo la base del aprendizaje profundo moderno (Rumelhart et al., 1986).
- **Sistemas Expertos (1965–1990):** Paralelamente, florecieron los sistemas basados en conocimiento (MYCIN, DENDRAL, R1/XCON), que codificaban la sabiduría humana en forma de reglas, alcanzando éxito comercial significativo.

El Segundo Invierno (1987–1993). Las expectativas comerciales desmedidas generaron una nueva crisis cuando las promesas no se materializaron. Las redes neuronales de la época, aunque teóricamente prometedoras, carecían de datos suficientes y poder de cómputo para demostrar ventajas prácticas claras.

Modelos de Lenguaje Estadístico: Precursores de los LLM

Durante el segundo invierno de la IA y en los años posteriores, un desarrollo paralelo sentaría las bases conceptuales para los Modelos Grandes de Lenguaje: los **modelos de lenguaje estadístico**.

El enfoque probabilístico del lenguaje. A diferencia de los sistemas simbólicos que intentaban codificar gramáticas formales, el enfoque estadístico modelaba el lenguaje como una se-

cuencia probabilística. La idea central, anticipada por Claude Shannon en 1948, era predecir la probabilidad de una palabra dada las palabras anteriores:

$$P(w_n | w_1, w_2, \dots, w_{n-1}) \quad (2.1)$$

N-gramas (1980s–2000s). Los modelos de **n-gramas** simplificaban el problema asumiendo que la probabilidad de una palabra depende solo de las $n - 1$ palabras anteriores (aproximación de Markov). Aunque limitados, estos modelos dominaron el procesamiento de lenguaje natural durante décadas:

- **Reconocimiento de voz:** IBM y Dragon Systems utilizaron n-gramas para desambiguar pronunciaciones.
- **Traducción automática:** El enfoque estadístico de IBM (1990) superó a los sistemas basados en reglas.
- **Corrección ortográfica:** Microsoft Word y otros procesadores implementaron sugerencias basadas en frecuencias de n-gramas.

Word2Vec (2013): El punto de inflexión. En 2013, Tomas Mikolov y su equipo en Google publicaron **Word2Vec** (Mikolov et al., 2013), un método para aprender **representaciones vectoriales de palabras** (*word embeddings*), que son vectores numéricos densos que capturan el significado semántico de cada palabra a partir de grandes corpus de texto. La innovación fue revolucionaria:

- **Semántica geométrica:** Palabras con significados similares tenían vectores cercanos en el espacio de embeddings.
- **Aritmética semántica:** Surgieron relaciones algebraicas sorprendentes:

$$\vec{r\acute{e}y} - \vec{hombre} + \vec{mujer} \approx \vec{re\acute{i}na} \quad (2.2)$$

- **Transferencia de conocimiento:** Los embeddings preentrenados podían reutilizarse en múltiples tareas, prefigurando el paradigma de *pretraining* + *fine-tuning* de los LLM actuales.
- **Escalabilidad:** El algoritmo era suficientemente eficiente para procesar miles de millones de palabras en horas.

Extensiones posteriores. Word2Vec fue seguido por mejoras significativas:

- **GloVe** (2014, Stanford) (Pennington et al., 2014): Combinaba estadísticas globales de corpus con aprendizaje local.
- **FastText** (2016, Facebook): Incorporaba información de subpalabras, manejando mejor palabras desconocidas.
- **ELMo** (2018, Allen AI): Introdujo embeddings *contextuales* que variaban según el contexto de uso, precursor directo de BERT.

La Figura 2.5 ilustra la evolución de estos avances a lo largo de la segunda oleada.

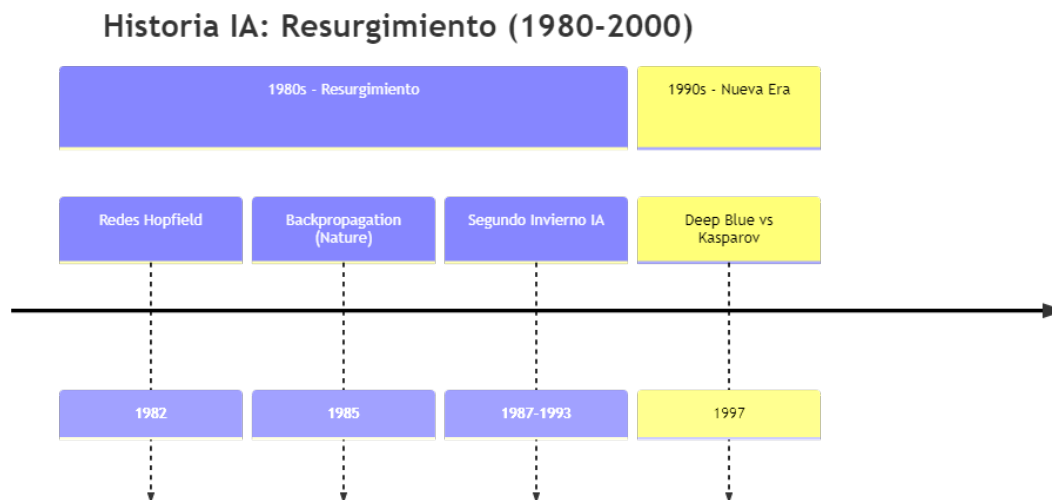


Figura 2.5: Línea temporal de la IA - Parte 3: Conexionismo, modelos estadísticos y el segundo invierno.

Tercera oleada: Deep Learning y la Era Moderna (2011–presente)

El renacimiento definitivo de la IA comenzó hacia 2011, impulsado por tres factores convergentes: **Big Data** (disponibilidad masiva de datos etiquetados), **GPUs** (poder de cómputo paralelo accesible) y **avances algorítmicos** (nuevas arquitecturas y técnicas de regularización), como muestra la Figura 2.6.

2.3.4. Hitos que transformaron la percepción de la IA

Ciertos eventos marcaron puntos de inflexión que alteraron tanto la investigación científica como la percepción pública de las capacidades de la IA.

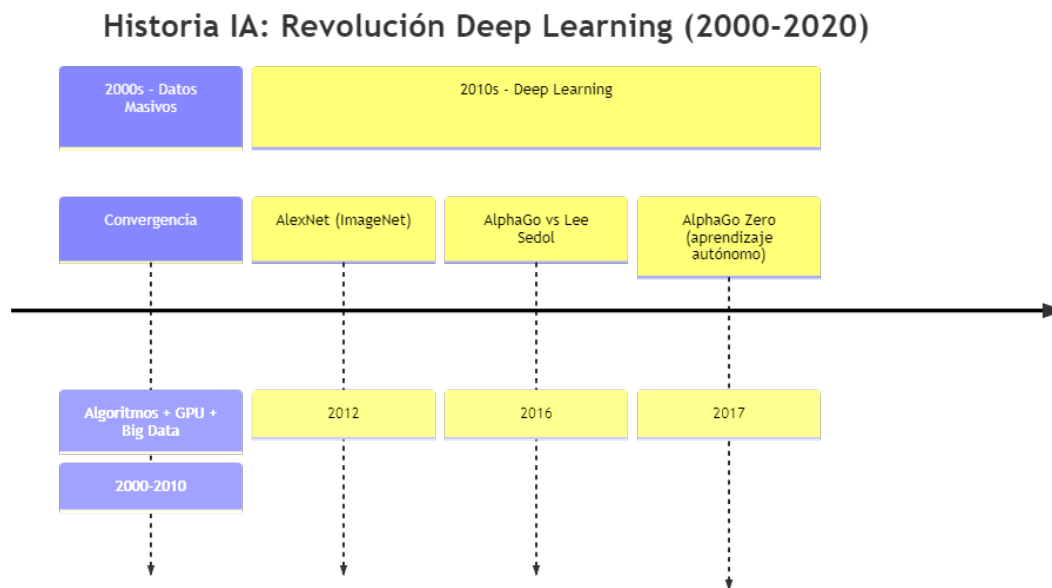


Figura 2.6: Línea temporal de la IA - Parte 4: Era del Deep Learning y los Transformers.

Deep Blue (1997). La victoria del sistema de IBM sobre el campeón mundial de ajedrez Garry Kasparov capturó la imaginación pública, aunque su funcionamiento difería radicalmente de la “inteligencia” humana: empleaba principalmente **fuerza bruta** (evaluación de 200 millones de posiciones por segundo) y búsqueda en árbol con heurísticas especializadas.

AlexNet (2012). El momento decisivo para el *Deep Learning* llegó cuando la red neuronal convolucional **AlexNet**, desarrollada por Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton (Krizhevsky et al., 2012), ganó el concurso ImageNet de reconocimiento visual con un margen sin precedentes (error del 15.3% vs. 26.2% del segundo lugar). Este resultado demostró definitivamente la superioridad de las redes profundas para tareas de percepción.

Attention Is All You Need (2017). El artículo de Vaswani et al. (2017) introdujo la arquitectura **Transformer**, eliminando las recurrencias en favor de mecanismos de *self-attention* que permitían procesar secuencias en paralelo. Esta innovación posibilitó el entrenamiento de modelos masivos y constituye la base de GPT, BERT, LLaMA y prácticamente todos los LLM contemporáneos.

AlphaGo y AlphaZero (2016–2017). **AlphaGo** (DeepMind) derrotó al campeón mundial de Go, Lee Sedol, en un juego considerado intratable para la fuerza bruta debido a su espacio de búsqueda astronómico (10^{170} posiciones posibles). Su sucesor, **AlphaZero** (Silver et al., 2017),

demonstró algo aún más extraordinario: partiendo de *tabula rasa* (sin conocimiento humano previo), aprendió Go, ajedrez y shogi jugando contra sí mismo, superando en apenas 4 horas al mejor motor de ajedrez existente (**Stockfish 8**).

AlphaFold 2 (2020–2021). DeepMind resolvió el **problema del plegamiento de proteínas**, uno de los grandes desafíos de la biología estructural durante 50 años. AlphaFold 2 predijo con precisión experimental la estructura 3D de proteínas a partir de su secuencia de aminoácidos. Este logro fue reconocido con el **Premio Nobel de Química 2024** para Demis Hassabis y John Jumper (Jumper et al., 2021).

ChatGPT y la democratización de los LLM (2022). El lanzamiento público de **ChatGPT** (30 de noviembre de 2022) marcó el punto de inflexión en la percepción pública de la IA. Cabe destacar que el primer modelo de la serie GPT (GPT-1) fue publicado por OpenAI en junio de 2018, seguido por GPT-2 (2019) y GPT-3 (2020) (Brown et al., 2020). ChatGPT, basado en GPT-3.5, alcanzó 100 millones de usuarios en dos meses (OpenAI, 2023). La Figura 2.7 cierra la línea temporal con la era generativa y las proyecciones a futuro.

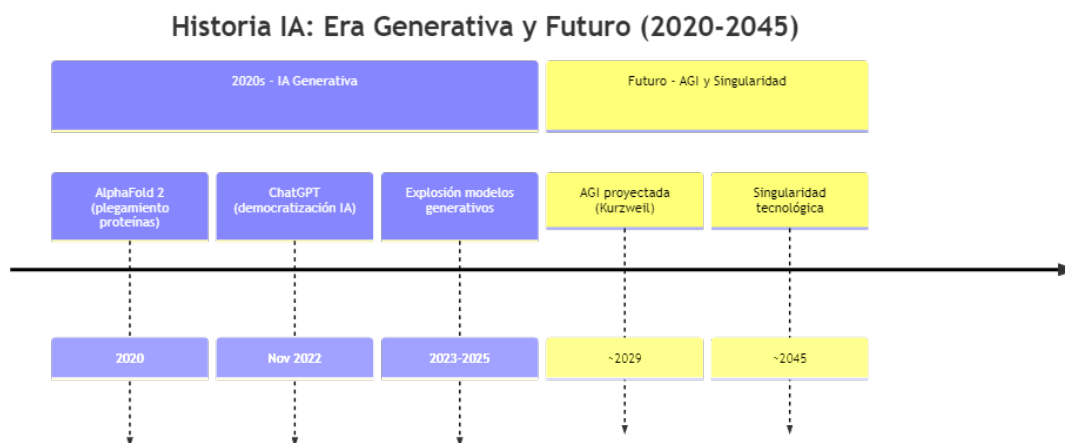


Figura 2.7: Línea temporal de la IA - Parte 5: Era generativa y proyecciones futuras.

2.3.5. El problema de la caja negra y los desafíos epistemológicos

El éxito práctico de las redes neuronales profundas plantea un dilema epistemológico fundamental: estos sistemas alcanzan respuestas correctas mediante procesos internos que resultan opacos incluso para sus creadores.

Explicabilidad versus resultados. La ciencia tradicional exige no solo resultados reproducibles, sino comprensión del mecanismo causal. Las redes neuronales, en contraste, operan como **cajas negras**: millones de parámetros interactúan de maneras que no admiten interpretación simbólica directa.

IA explicable (XAI). El campo de la *Explainable AI* (IA Explicable o XAI) busca desarrollar técnicas para interpretar las decisiones de modelos complejos: mapas de atención, atribución de características, modelos sustitutos interpretables. Este área cobra especial importancia en aplicaciones críticas como medicina, justicia y finanzas, donde las decisiones algorítmicas deben ser auditables y justificables (Lipton, 2018; Rudin, 2019).

2.3.6. Desafíos éticos, sociales y ambientales

El poder creciente de los sistemas de IA conlleva responsabilidades y riesgos que trascienden lo técnico.

Privacidad y vigilancia. Los modelos de lenguaje se entrenan con cantidades masivas de datos, frecuentemente extraídos de la web sin consentimiento explícito. Su capacidad para generar perfiles, analizar comunicaciones y predecir comportamientos plantea amenazas significativas a la privacidad.

Sesgos algorítmicos. Los modelos reproducen y amplifican los sesgos presentes en sus datos de entrenamiento. Casos documentados incluyen sistemas de contratación que discriminaban a mujeres (Amazon, 2018), algoritmos de riesgo crediticio que penalizaban minorías raciales, y modelos de lenguaje que perpetuaban estereotipos de género y etnia (Bender et al., 2021).

Consumo energético y huella de carbono. El entrenamiento de modelos masivos requiere recursos computacionales extraordinarios. Se estima que entrenar GPT-3 consumió aproximadamente 1,287 MW/h de electricidad, equivalente a las emisiones de CO₂ de 5 automóviles durante su vida útil (Strubell et al., 2019).

Desinformación y contenido sintético. La capacidad de generar texto, imagen, audio y video indistinguibles de producciones humanas facilita la creación de desinformación a escala industrial. Los *deepfakes* —contenidos audiovisuales hiperrealistas generados mediante redes neuronales— plantean desafíos sin precedentes para la verificación de información.

Proyecciones futuristas. Figuras como **Ray Kurzweil** (Director de Ingeniería de Google) predicen que la IA igualará la inteligencia humana general hacia 2029, y que la **Singularidad** ocurrirá alrededor de 2045 (Kurzweil, 2005). Aunque controvertidas, estas predicciones influyen en políticas públicas, inversiones y narrativas culturales sobre el futuro de la humanidad.

2.3.7. Analogía integradora: el rascacielos de la IA

Para comprender por qué la IA funciona hoy cuando fracasó antes, resulta útil una analogía arquitectónica:

Analogía del Rascacielos

La historia de la IA puede entenderse como sucesivos intentos de construir un rascacielos:

- **Primera oleada (1956–1974):** Se intentó construir usando solo *ladrillos lógicos* y planos perfectos (símbolos, reglas explícitas). La estructura colapsaba al intentar hacerla muy alta.
- **Segunda oleada (1980–1995):** Gracias a la física (Hopfield) y las matemáticas (Hinton), se descubrió el “acero” y el “concreto” de la construcción neural. Sin embargo, faltaban maquinaria y materia prima.
- **Tercera oleada (2011–presente):** Finalmente se dispone de la maquinaria pesada —**GPUs** y **TPUs**— y la materia prima masiva (**Big Data**, **Internet**) para levantar estructuras gigantescas como **ChatGPT**, **AlphaFold** y **DeepSeek**.

Los materiales (algoritmos) existían desde los 80; lo que faltaba era la escala industrial para aplicarlos.

2.3.8. Los Modelos Grandes de Lenguaje en el panorama actual

Los **Modelos Grandes de Lenguaje** representan la tecnología de vanguardia que ha emergido de esta trayectoria histórica. Entrenados con volúmenes masivos de texto (billones de tokens), estos modelos han demostrado capacidades emergentes que no fueron explícitamente programadas: razonamiento en cadena, seguimiento de instrucciones complejas, generación de código, y comprensión multilingüe (Brown et al., 2020).

El análisis que sigue examina cuatro exponentes representativos del panorama actual:

- **DeepSeek-R1:** Exponente de apertura y ejecución local con arquitectura *Mixture of Experts* (MoE) optimizada (ver sección 2.3.8).

- **GPT-4 (OpenAI)**: Referente comercial multimodal con alineación humana robusta.
- **Grok (xAI)**: Propuesta de integración con redes sociales y menor moderación algorítmica.
- **Gemini (Google DeepMind)**: Modelo multimodal nativo de Google con integración en su ecosistema.

DeepSeek

DeepSeek emerge como un modelo de lenguaje que pone énfasis en dos desafíos cardinales de los LLM modernos: la *eficiencia computacional* y la *accesibilidad*. Basado en una arquitectura *Mixture of Experts* (MoE) optimizada y en técnicas de entrenamiento con alto grado de automatización, se presenta como un caso relevante para comprender la evolución hacia modelos escalables y pragmáticos, aptos para entornos con recursos limitados (DeepSeek-AI, 2024). La Figura 2.8 ilustra su esquema de activación selectiva de expertos.

Arquitectura *Mixture of Experts* (MoE). DeepSeek emplea una variante de MoE en la que se activa sólo una fracción de los parámetros por consulta, combinando precisión mixta —FP16¹ y FP8— para acelerar operaciones matriciales con estrategias de paralelización en la generación de tokens (Zhou et al., 2022; Finn et al., 2017).

Entrenamiento automatizado. El ciclo de entrenamiento incorpora componentes de aprendizaje por refuerzo de forma autónoma, con currículos progresivos, autoevaluación continua contra *benchmarks* adaptativos y ajuste dinámico de hiperparámetros mediante técnicas de meta-aprendizaje (Cobbe et al., 2021).

Rendimiento en *benchmarks*. En evaluaciones estandarizadas orientadas al razonamiento matemático (GSM8K), a la generación de código (HumanEval) y al razonamiento general (MM-LU —*Massive Multitask Language Understanding*—), se reporta un rendimiento competitivo cercano a modelos comerciales de última generación (Liu et al., 2023).

Modelo de accesibilidad y adopción. DeepSeek-R1 adopta una política de acceso abierto controlado: proporciona API con umbrales gratuitos para instituciones académicas y distribuye pesos preentrenados bajo licencias no comerciales orientadas a investigación (DeepSeek-AI, 2024).

¹**FP16** (*half precision*): formato de punto flotante de 16 bits que reduce a la mitad la memoria respecto al estándar FP32; **FP8** (*quarter precision*): formato de 8 bits que reduce aún más el cómputo matricial a costa de una ligera pérdida de precisión numérica.

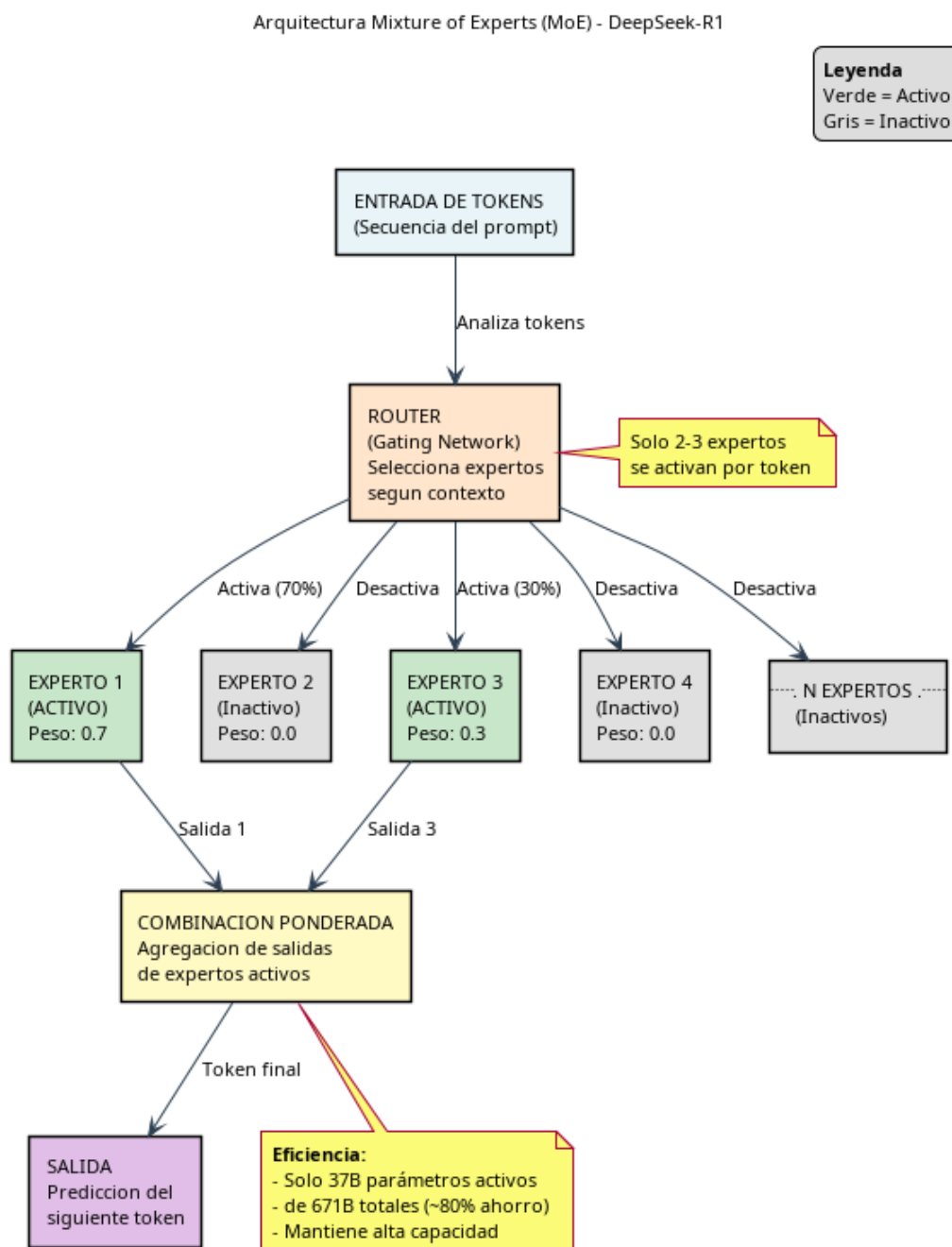


Figura 2.8: Arquitectura Mixture of Experts (MoE) de DeepSeek con activación selectiva de parámetros. Solo 2–3 expertos se activan por token, reduciendo el cómputo en ~80% mientras mantiene 671B parámetros totales. Fuente: elaboración propia.

Seguridad y privacidad mediante ejecución local. La posibilidad de ejecutar el modelo localmente constituye un valor añadido para escenarios sensibles (salud, banca, educación personalizada o análisis jurídico), al evitar la transferencia de datos a terceros. Este enfoque facilita el cumplimiento normativo y posibilita auditoría y adaptación del comportamiento del modelo bajo el principio de *privacy by design* (DeepSeek-AI, 2024).

OpenAI (GPT-4/ChatGPT)

OpenAI se ha consolidado como uno de los actores determinantes en Modelos Grandes de Lenguaje a través de la familia *Generative Pretrained Transformer* (GPT). **GPT-4** y sus variantes optimizadas representan un punto de madurez técnica y comercial, con capacidades multimodales que integran texto e imagen en un mismo contexto de interacción (OpenAI, 2023).

Arquitectura y capacidades multimodales. GPT-4 se apoya en una arquitectura *Transformer* densa a gran escala, sin recurrir a MoE. Su diseño apunta a comprensión contextual extendida, razonamiento lógico y manejo multilingüe, además de análisis de imágenes en contexto (Brown et al., 2020). La Figura 2.9 presenta una vista simplificada de esta arquitectura.

Entrenamiento y alineación con valores humanos. El proceso de ajuste incorpora *Reinforcement Learning from Human Feedback* (RLHF), mediante el cual la retroalimentación humana guía el comportamiento del sistema hacia respuestas útiles, seguras y consistentes con pautas éticas (Ouyang et al., 2022).

Desempeño en *benchmarks* y razonamiento. GPT-4 mantiene resultados de referencia en múltiples *benchmarks* (MMLU, HumanEval), mostrando fortaleza en razonamiento general, generación de código y comprensión narrativa (Hendrycks et al., 2021).

Infraestructura y escalabilidad. El servicio se ofrece vía API y a través de la interfaz ChatGPT, soportado por infraestructura distribuida de alta disponibilidad (Narayanan et al., 2021). Si bien no se distribuye como *open-source*, su plataforma permite integración empresarial a gran escala.

Grok (xAI)

Grok es el modelo de lenguaje desarrollado por xAI, organización fundada en 2023. Su posicionamiento estratégico enfatiza una interacción conversacional con menor fricción en la moderación algorítmica, promoviendo respuestas directas sobre temas sensibles dentro de los límites legales aplicables (xAI, 2024). Su arquitectura conceptual se detalla en la Figura 2.10.

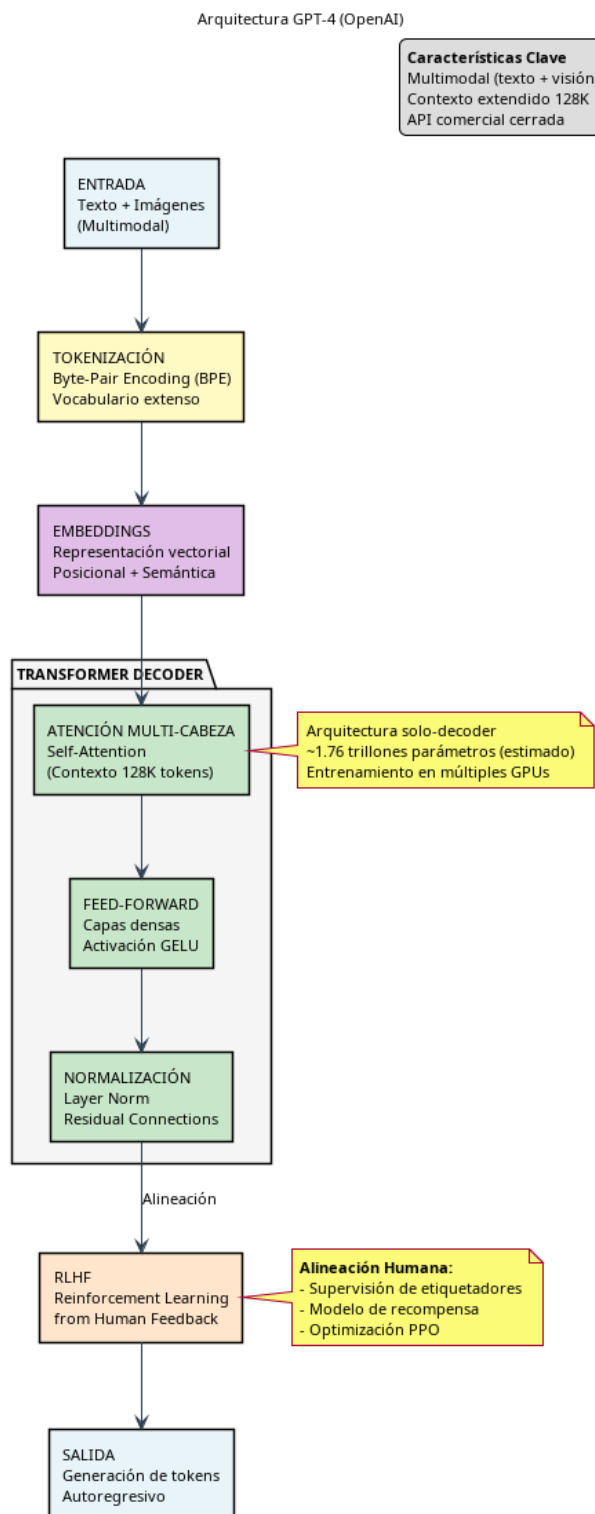


Figura 2.9: Arquitectura simplificada de GPT-4 (OpenAI). El modelo utiliza un *Transformer decoder-only* con atención multi-cabeza y alineación mediante RLHF para generar respuestas seguras y útiles. Fuente: elaboración propia.

Integración contextual. Grok se integra de forma nativa con la plataforma X, accediendo al contexto social público (publicaciones, tendencias) para generar respuestas personalizadas y sensibles a la dinámica de la red.

Gemini (Google DeepMind)

Gemini es la familia de modelos de lenguaje multimodales desarrollada por **Google DeepMind**, presentada oficialmente en diciembre de 2023. Representa la convergencia de las capacidades de investigación de Google Brain y DeepMind.

Arquitectura multimodal nativa. A diferencia de otros modelos que añaden capacidades multimodales como extensiones, Gemini fue diseñado desde su concepción para procesar y razonar sobre múltiples tipos de entrada: texto, imágenes, audio, video y código. La Figura 2.11 muestra cómo los encoders especializados convergen en una representación unificada.

Familia de modelos. Google ha estructurado Gemini en tres variantes principales:

- **Gemini Ultra:** El modelo más capaz, diseñado para tareas altamente complejas.
- **Gemini Pro:** Equilibrio entre rendimiento y eficiencia, adecuado para aplicaciones empresariales.
- **Gemini Nano:** Versión optimizada para dispositivos móviles, permitiendo inferencia local sin conexión a la nube.

Integración en el ecosistema Google. Gemini se despliega de forma nativa en productos como Google Workspace, Google Cloud, Android y el buscador de Google. Destaca particularmente en la comprensión y generación sobre contextos largos (hasta 1 millón de tokens en versiones avanzadas) y en tareas que requieren síntesis de información visual y textual.

Análisis comparativo de LLM: ejecución local vs. remota

La elección entre modelos locales y remotos constituye una decisión arquitectónica fundamental. Aragon Toledo (2026) documentó una evaluación comparativa entre DeepSeek-R1-Distill-Qwen-7B (local, vía LM Studio) y GPT-4 (remoto, vía API), cuyos resultados se resumen en la Tabla 2.1:

La conclusión de dicho análisis fundamenta la arquitectura híbrida de LIMA, donde el usuario puede elegir entre un *backend* local para garantizar privacidad u operación sin conexión, o un *backend* remoto cuando se requiere máxima precisión.

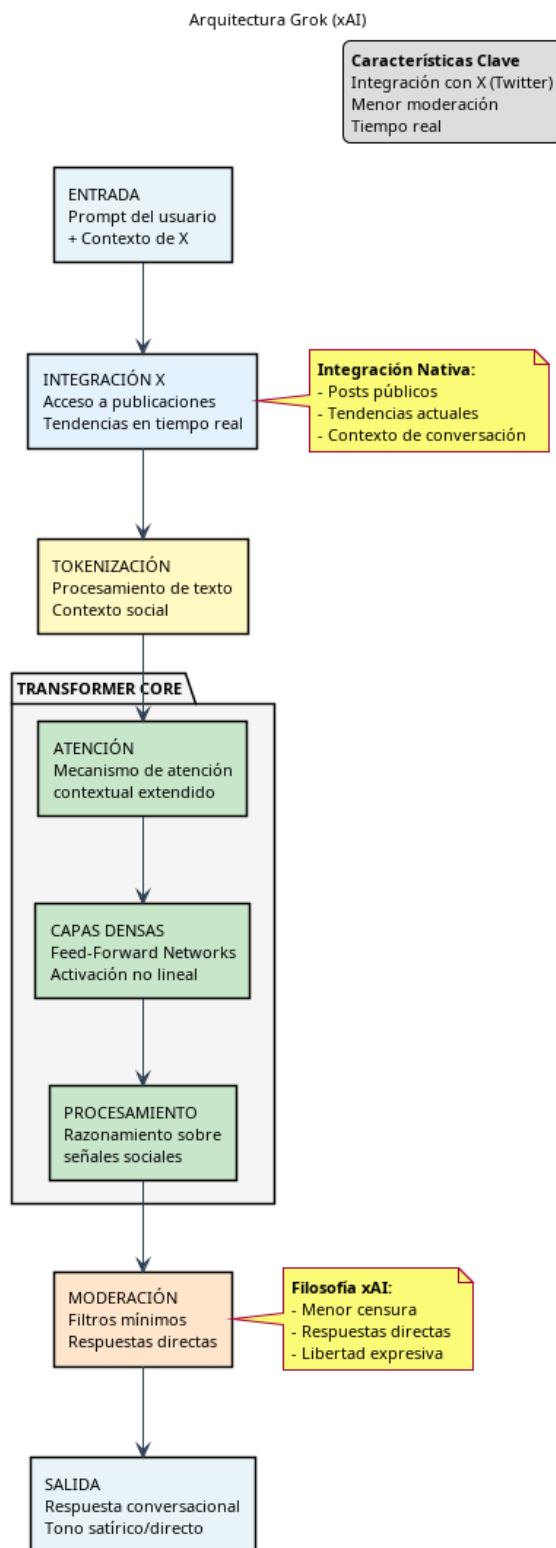


Figura 2.10: Arquitectura conceptual de Grok (xAI). El modelo integra información contextual de la plataforma X para generar respuestas personalizadas con menor moderación algorítmica. Fuente: elaboración propia.

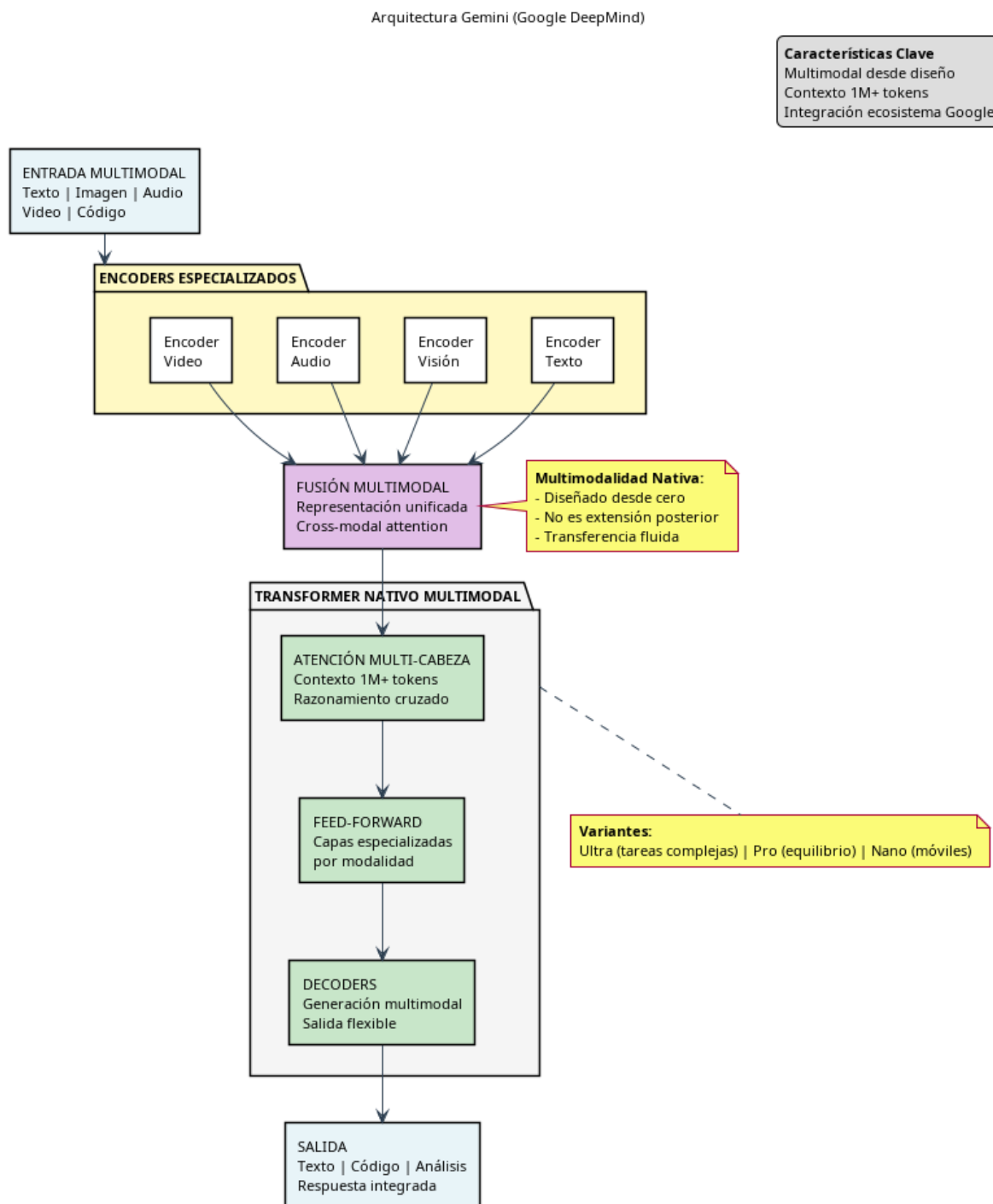


Figura 2.11: Arquitectura multimodal nativa de Gemini (Google DeepMind). El modelo procesa múltiples modalidades mediante encoders especializados que convergen en una representación unificada. Fuente: elaboración propia.

Tabla 2.1: Comparación entre modelos de lenguaje local y remoto

Dimensión	DeepSeek-R1 (local)	GPT-4o (remoto)
Latencia de inferencia	3.8–4.2 s	~1.9 s
Ventana de contexto	8 192 <i>tokens</i>	128K <i>tokens</i>
Concurrencia	Secuencial	Paralelo
Precisión semántica	>95% adecuada	Superior
Costo operativo	Nulo (tras HW)	\$0.005–0.03/llamada
Privacidad	Total	Datos enviados a servidor
Conectividad requerida	No	Sí

2.4. Sumario del capítulo

El recorrido histórico presentado evidencia que los Modelos Grandes de Lenguaje constituyen el resultado de una evolución acumulativa que abarca más de seis décadas de investigación en Inteligencia Artificial. Desde los fundamentos teóricos establecidos por Turing y Shannon, pasando por los “inviernos” de la IA y el resurgimiento conexionista, hasta la revolución del *deep learning* y la arquitectura *Transformer*, cada etapa ha contribuido elementos fundamentales al estado actual del campo.

Los cuatro modelos analizados —DeepSeek, GPT-4, Grok y Gemini— ilustran la diversidad de aproximaciones que coexisten en el ecosistema contemporáneo: desde la eficiencia computacional mediante arquitecturas MoE hasta la multimodalidad nativa, pasando por la alineación ética y la integración con plataformas sociales. El marco conceptual aquí establecido sienta las bases para comprender las decisiones técnicas del sistema LIMA.

Los fundamentos tecnológicos, el marco de Interacción Humano-Computadora, las barreras de la interfaz de línea de comandos, las consideraciones de seguridad en sistemas asistidos por IA y los trabajos relacionados se presentan en el capítulo siguiente como preámbulo al diseño de LIMA.

Capítulo 3

Diseño e Implementación del Sistema

El presente capítulo se organiza en dos partes complementarias. La primera — Secciones 3.1, 3.2 y 3.3— expone las tecnologías y áreas de conocimiento que sustentan el sistema, las consideraciones adicionales de diseño —la interfaz de línea de comandos como barrera de accesibilidad y la seguridad en sistemas asistidos por IA— y los trabajos relacionados; estos elementos constituyeron las consideraciones guía del diseño de LIMA (*Linux Intelligent Middleware Assistant*). La segunda parte —Sección 3.4 en adelante— describe la arquitectura, los módulos, los algoritmos y las decisiones de diseño del sistema. LIMA está implementado en aproximadamente 9 500 líneas de código Perl 5, con un módulo auxiliar embebido de 250 líneas en Python 3; se compone de 9 módulos funcionales, un orquestador principal (`lima.pl`), un archivo de configuración JSON y tres plantillas de proyectos.¹

3.1. Tecnologías y áreas de conocimiento utilizadas en LIMA

3.1.1. Teoría de agentes inteligentes y posicionamiento de LIMA

El concepto de *agente inteligente* ocupa un lugar central en la Inteligencia Artificial y resulta pertinente para determinar la naturaleza del sistema desarrollado en este trabajo. Esta sección examina la teoría de agentes y la taxonomía propuesta en (Sánchez, 1997), y concluye con una caracterización formal de LIMA dentro de dicho marco conceptual.

¹A lo largo de este capítulo el sistema, su ejecutable (`lima.pl`), su palabra de activación («LIMA») y su directorio de datos (`~/ .lima/`) se nombran de forma uniforme como **LIMA**, conforme a la decisión de unificación de identidad adoptada tras la segunda ronda de pruebas de usabilidad (Sección 5.1.8). La unificación se aplicó también al código: la palabra de activación configurada, los rótulos de la interfaz, los diálogos y los mensajes de registro nombran hoy al sistema como LIMA. Persisten únicamente identificadores de infraestructura ya desplegada —el nombre del servicio en el borde (`botas-proxy`), el de su base de datos y el prefijo de instalación del paquete—, que no pueden renombrarse sin romper las instalaciones distribuidas y que en ningún caso son visibles para el usuario.

Definición de agente

Russell y Norvig (2021) definen un **agente** como *cualquier entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores*. Esta definición, deliberadamente amplia, abarca desde un termostato simple hasta un robot autónomo, pasando por sistemas de software. Lo que distingue a un *agente inteligente* es que su comportamiento está orientado a maximizar una función de rendimiento en relación con su entorno y con base en su historial de percepciones.

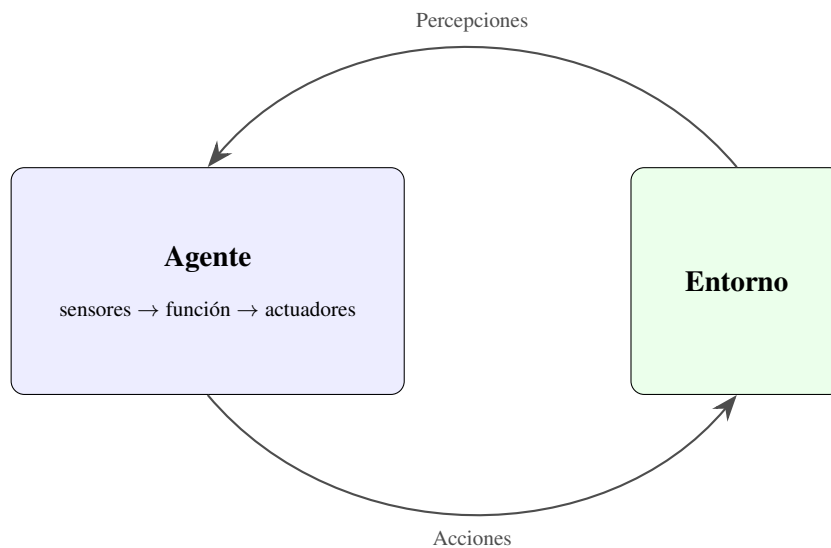


Figura 3.1: Ciclo percepción-acción del agente inteligente (Russell y Norvig, 2021). El agente percibe su entorno a través de sensores y actúa sobre él mediante actuadores.

Wooldridge y Jennings (1995) refinan esta noción y distinguen entre la **concepción débil** y la **concepción fuerte** de agencia. Según la concepción débil, un agente debe exhibir cuatro propiedades fundamentales:

- **Autonomía:** opera sin intervención humana directa, ejerciendo control sobre sus propias acciones y estado interno.
- **Reactividad:** percibe su entorno y responde a los cambios de forma oportuna.
- **Pro-actividad:** no se limita a responder a estímulos; toma la iniciativa y persigue objetivos de manera proactiva.
- **Habilidad social:** interactúa con otros agentes o con humanos a través de algún mecanismo de comunicación.

La concepción **fuerte** extiende estas propiedades con estados mentales explícitos: creencias (*beliefs*), deseos (*desires*) e intenciones (*intentions*), formalizados en el modelo BDI (Wooldrid-

ge y Jennings, 1995). Los sistemas que implementan arquitecturas BDI —como PRS o JACK— son capaces de razonar sobre sus propias metas y modificarlas dinámicamente.

Clasificación de agentes según Russell y Norvig

Russell y Norvig (2021) proponen cinco tipos arquitectónicos de agentes, ordenados por complejidad creciente:

1. **Agente reflejo simple:** actúa únicamente con base en la percepción actual, sin memoria de estados pasados. Implementa reglas del tipo *si condición, entonces acción*.
2. **Agente reflejo basado en modelo:** mantiene un estado interno que representa partes del entorno no directamente observables, permitiéndole operar en entornos parcialmente observables.
3. **Agente basado en metas:** incorpora información sobre situaciones deseables y puede planear secuencias de acciones para alcanzar sus objetivos.
4. **Agente basado en utilidad:** dispone de una función de utilidad que le permite comparar el desempeño relativo de distintas situaciones y elegir la acción que maximiza la utilidad esperada.
5. **Agente que aprende:** incluye un componente de aprendizaje que mejora su rendimiento con la experiencia, modificando su comportamiento a partir de la retroalimentación del entorno.

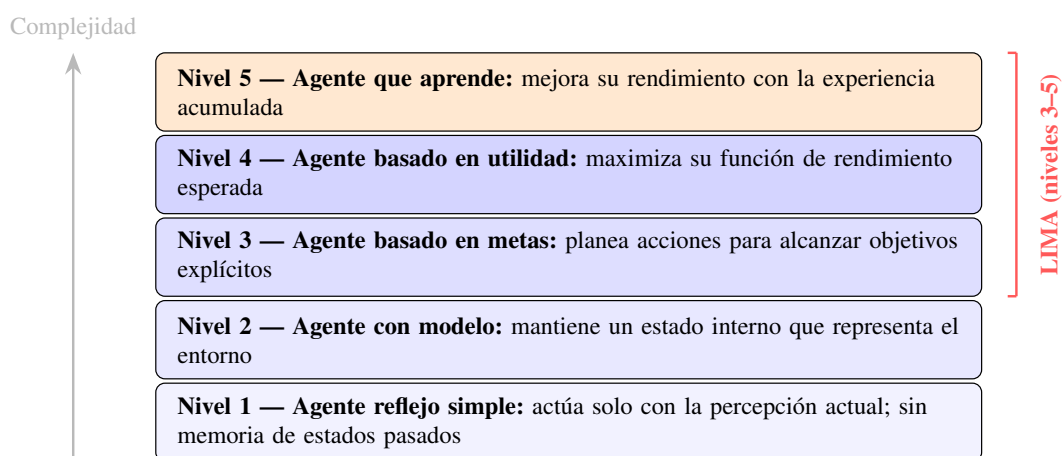


Figura 3.2: Clasificación de los tipos de agente según Russell y Norvig (2021), de menor a mayor complejidad, con el posicionamiento de LIMA.

Taxonomía de agentes de Sánchez

En complemento a lo anterior, y como trabajo pionero en el área, Sánchez (1997) publicó el reporte técnico *A taxonomy of agents*, derivado de su investigación doctoral (Sánchez, 1996). Esta taxonomía propone tres grandes categorías de agentes clasificadas según la función que desempeñan en relación con el usuario y el sistema:

1. **Agentes de interfaz (*interface agents*)**: actúan como intermediarios entre el usuario y los recursos del sistema. Observan las acciones del usuario, aprenden de sus preferencias y ejecutan tareas en su nombre, con el objetivo de reducir la carga cognitiva de la interacción.
2. **Agentes de información (*information agents*)**: se especializan en la recuperación, filtrado, síntesis y gestión de información proveniente de múltiples fuentes, potencialmente distribuidas. Incorporan conocimiento del dominio para seleccionar y presentar únicamente la información relevante.
3. **Agentes de tarea (*task agents*)**: ejecutan secuencias de acciones concretas de forma autónoma en nombre del usuario. Su objetivo primario es completar tareas bien definidas, coordinando los recursos necesarios del sistema.

Sánchez y Leggett (1997) aplicaron esta taxonomía al diseño de bibliotecas digitales, donde los *agentes de usuario* son servicios que ofrecen personalización del acceso a los recursos de información. La taxonomía de Sánchez enfatiza la *relación funcional* del agente con el usuario, en contraste con las clasificaciones puramente arquitectónicas de Russell y Norvig.

Contexto de LIMA

La Tabla 3.1 evalúa LIMA frente a los cuatro criterios de la concepción débil de agencia de Wooldridge y Jennings (1995).

LIMA cumple los cuatro criterios de la **concepción débil**, por lo que puede decirse, con base en estos criterios, que es un **agente inteligente**. No satisface, sin embargo, la **concepción fuerte**: no implementa un modelo formal BDI ni un motor de planificación deliberativa de propósito general. El razonamiento sobre acciones complejas se delega al modelo de lenguaje (GPT-4o o DeepSeek-R1), que actúa como componente deliberativo externo.

En la taxonomía de Sánchez (Sánchez, 1997), LIMA se sitúa principalmente en la categoría de **agente de interfaz**: su función primaria es mediar entre el usuario y los recursos del sistema operativo GNU/Linux, interpretando intenciones expresadas en lenguaje natural. Adicionalmente, incorpora comportamientos propios de un **agente de información** —a través de las subrutinas de búsqueda web, análisis de frecuencia léxica (`analyze_text`) y consulta de

Tabla 3.1: Evaluación de LIMA frente a los criterios de agencia (Wooldridge y Jennings, 1995)

Propiedad	Evidencia en LIMA
Autonomía	El orquestador <code>lima.pl</code> selecciona de forma independiente el módulo de resolución apropiado (local, LLM remoto o LLM local), ejecuta comandos del sistema operativo, gestiona su ciclo de vida y mantiene el historial de sesión sin intervención del usuario durante la ejecución.
Reactividad	El módulo <code>WakeWordDetector</code> detecta continuamente la palabra clave de activación; al percibirla, el sistema transita de estado inactivo a activo y responde al entorno sonoro en tiempo real actualizando la GUI.
Pro-actividad	Cuando la intención es ambigua, LIMA genera proactivamente una solicitud de aclaración (<code>action = clarify</code>) sin esperar a que el usuario reformule. El módulo <code>TaskLearner</code> acumula y generaliza automáticamente patrones del historial para anticipar necesidades recurrentes.
Habilidad social	LIMA se comunica con el usuario mediante voz (TTS) y GUI, y se comunica con agentes externos —modelos de lenguaje de OpenAI y LM Studio— mediante protocolos API estandarizados (HTTP/JSON).

estado del sistema— y de un **agente de tarea** —al ejecutar de forma autónoma operaciones sobre el sistema de archivos, procesos y red. LIMA opera, pues, en la intersección de las tres categorías de la taxonomía.

Desde la perspectiva de Russell y Norvig (Russell y Norvig, 2021), LIMA corresponde a un **agente basado en metas con modelo interno**: mantiene un estado que representa el contexto de la interacción (historial de diez turnos, configuración persistente, sesión activa) y dispone de metas explícitas formuladas en el prompt del sistema. El módulo `TaskLearner` añade la dimensión de **agente que aprende**: LIMA mejora su rendimiento con cada interacción, generalizando patrones de comandos a partir del historial del usuario. Esta caracterización implica que los mecanismos de seguridad multicapa descritos en la Sección 3.15 no son añadidos opcionales, sino componentes esenciales de la arquitectura de agente.

3.1.2. Procesamiento de lenguaje natural y reconocimiento de voz

Como se expuso en el Capítulo 2, los Modelos Grandes de Lenguaje (LLM, *Large Language Models*) constituyen la tecnología que hace posible el núcleo funcional de LIMA. El sistema emplea un LLM como motor de interpretación semántica de intenciones: traduce las solicitudes del usuario, expresadas en lenguaje natural, a un plan de acción estructurado en formato JSON. Esta capacidad de traducir lenguaje natural a comandos ejecutables fue explorada inicialmente por el trabajo pionero de Lin et al. (2018) con NL2Bash, que sin embargo se limitó a la generación de

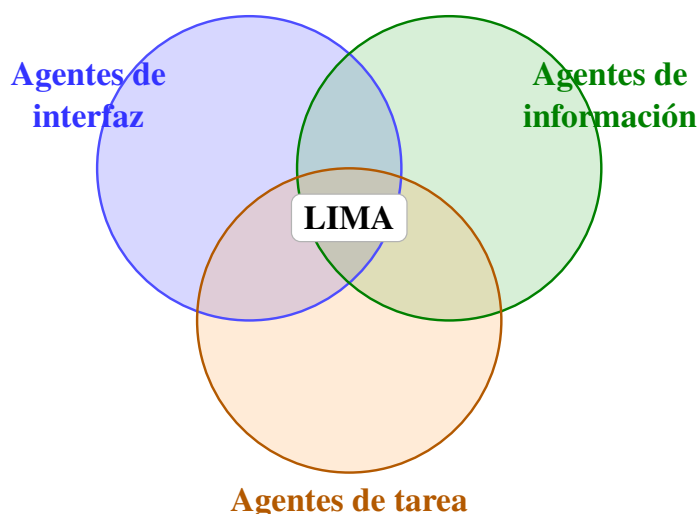


Figura 3.3: Posicionamiento de LIMA en la taxonomía de Sánchez (1997). LIMA se sitúa en la intersección de las tres categorías, con énfasis en la función de agente de interfaz.

comandos sin contemplar su ejecución, la seguridad ni la interacción por voz —precisamente las dimensiones que LIMA integra—. Para ello, LIMA admite tanto proveedores remotos (GPT-4o) como locales (por ejemplo, `deepseek-coder-6.7b-instruct` mediante LM Studio), lo que le permite operar en línea o fuera de línea según las necesidades de privacidad y conectividad del usuario.

El reconocimiento automático del habla (ASR, *Automatic Speech Recognition*) ha experimentado avances transformadores con la arquitectura *Transformer* (Vaswani et al., 2017). OpenAI Whisper (OpenAI, 2022) logró precisión cercana a la humana en múltiples idiomas, incluido el español, mediante entrenamiento sobre 680 000 horas de audio. Los asistentes virtuales comerciales —Amazon Alexa, Apple Siri, Google Assistant— han normalizado la interacción por voz, pero ofrecen capacidades limitadas de administración del sistema operativo (Luger y Sellen, 2016). Alternativas de código abierto como Vosk proporcionan reconocimiento sin conexión a internet. Estos avances determinaron la elección de dos motores complementarios en LIMA: Whisper para transcripción en modo en línea y Vosk para el modo fuera de línea y para la detección de la palabra clave de activación (*wake word*).

3.1.3. Interacción humano-computadora e inteligencia artificial centrada en el humano

La Interacción Humano-Computadora (IHC) estudia el diseño, la evaluación y la implementación de sistemas informáticos interactivos (Dix et al., 2004). Un principio central es que la tecnología debe adaptarse al usuario y no a la inversa (Norman, 1988). Shneiderman y Plai-

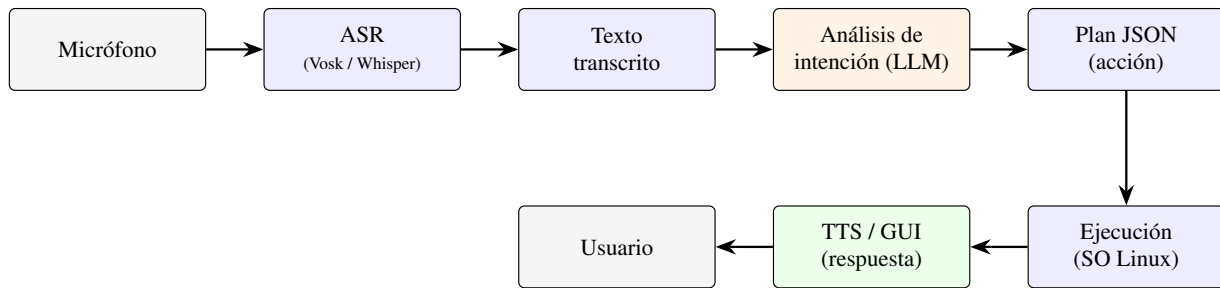


Figura 3.4: Pipeline conceptual de LIMA: desde la entrada de voz hasta la respuesta al usuario. La fila superior muestra la etapa de entrada y análisis; la inferior, la ejecución y retroalimentación.

sant (2010) establecen ocho reglas de oro para el diseño de interfaces, entre las que destacan la consistencia, la retroalimentación informativa, la prevención de errores y el soporte para la reversibilidad. Nielsen (1994) complementa con heurísticas de usabilidad: visibilidad del estado del sistema, correspondencia con el mundo real y control del usuario.

Por otro lado, el paradigma de la Inteligencia Artificial Centrada en el Humano (HCAI, *Human-Centered AI*), formalizado por Shneiderman (2020, 2022), propone que los sistemas de IA deben ser fiables, seguros y dignos de confianza, priorizando el empoderamiento del usuario sobre la automatización opaca. En este sentido, los principios que guiaron el diseño de LIMA son:

- **Transparencia:** el usuario debe comprender qué hará el sistema y por qué.
- **Control humano:** el usuario debe poder supervisar, modificar o rechazar las decisiones de la IA.
- **Seguridad:** el sistema debe prevenir activamente acciones dañinas.
- **Empoderamiento:** la IA debe amplificar las capacidades humanas, no sustituirlas.

Amershi et al. (2019) complementan este marco con 18 directrices prácticas para la interacción humano-IA, agrupadas en cuatro fases temporales de la experiencia del usuario, como se detalla en la Tabla 3.2. LIMA implementa directamente varias de estas directrices: G1 mediante el tutorial interactivo de siete pasos; G2 al distinguir visualmente entre resolución local y resolución por LLM; G6 con el *pipeline* de tres niveles que prioriza el análisis determinista; G8 mediante el diálogo de aclaración (`clarify`); G9 con el botón de cancelación y el comando verbal «cancela»; G11 a través del indicador visual de estado; G12 con el historial de conversación de diez turnos; G15 mediante el registro auditable de peticiones; y G17 mediante el diálogo de configuración.

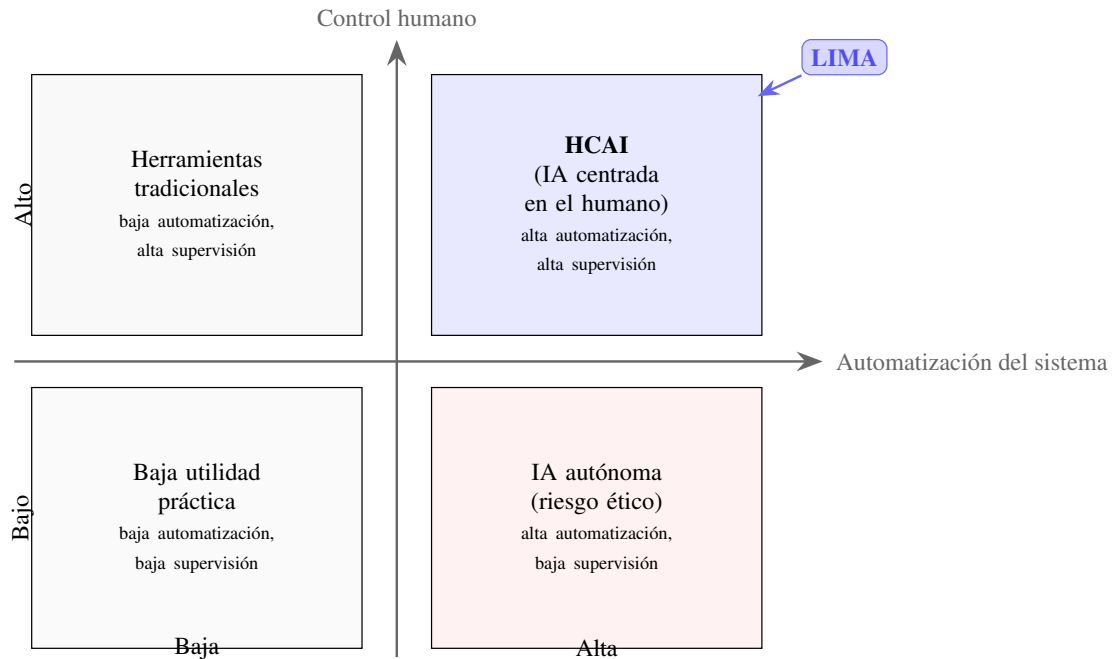


Figura 3.5: Marco HCAI de Shneiderman (2022). LIMA se posiciona en el cuadrante de IA centrada en el humano: alta automatización con alto control del usuario.

Tabla 3.2: Las 18 directrices para la interacción humano-IA (Amershi et al., 2019)

#	Fase	Directriz
G1	Inicialmente	Aclarar qué puede hacer el sistema.
G2		Aclarar qué tan bien puede hacer lo que hace.
G3	Durante la interacción	Gestionar la interacción según el contexto del tiempo.
G4		Mostrar información contextualmente relevante.
G5		Coincidir con las convenciones de diseño social relevantes.
G6		Mitigar las imprecisiones del servicio de IA.
G7		Soportar la invocación eficiente.
G8		Soportar la corrección eficiente.
G9		Soportar la descartación eficiente.
G10		Delimitar los servicios cuando hay incertidumbre.
G11	Cuando algo sale mal	Hacer claro por qué el sistema hizo lo que hizo.
G12		Recordar interacciones recientes.
G13		Aprender del comportamiento del usuario.
G14		Actualizar y adaptarse con cautela.
G15		Fomentar la retroalimentación granular.
G16		Comunicar los cambios al servicio de IA.
G17	A lo largo del tiempo	Proporcionar controles globales.
G18		Notificar al usuario sobre cambios.

3.2. Consideraciones adicionales de diseño

3.2.1. La interfaz de línea de comandos como barrera de accesibilidad

La interfaz de línea de comandos (CLI) de Unix, concebida por Thompson y Ritchie en los laboratorios Bell a principios de la década de 1970, se diseñó bajo el paradigma de herramientas pequeñas y combinables mediante tuberías (*pipes*) (Raymond, 2003). Pese a su potencia expresiva, la CLI presenta barreras que dificultan su adopción por usuarios no especializados: la necesidad de memorizar nombres de comandos y sus opciones, la ausencia de descubrimiento visual de funcionalidades, la carencia de mecanismos de deshacer integrados y la naturaleza críptica de los mensajes de error.

Investigaciones previas reportan que usuarios novatos invierten, en promedio, entre 7 y 8 minutos buscando el comando apropiado para tareas que un experto resuelve en menos de un minuto (Lin et al., 2018). Asimismo, la CLI no distingue visualmente entre operaciones inocuas y destructivas, lo que representa un riesgo operativo significativo. Estas barreras definen los requisitos que LIMA debe satisfacer: eliminar la necesidad de memorizar comandos, hacer las operaciones descubribles mediante instrucciones verbales en lenguaje natural, y prevenir las acciones de alto riesgo mediante confirmación explícita del usuario.

3.2.2. Seguridad en sistemas asistidos por inteligencia artificial

La seguridad en sistemas que ejecutan acciones a partir de instrucciones generadas por IA constituye un área de investigación crítica (Amodei et al., 2016). Los riesgos incluyen la generación de comandos incorrectos, la ejecución de operaciones no intencionadas y la posibilidad de inyección de instrucciones maliciosas (*prompt injection*). Pu00e9rez et al. (2022) han demostrado la importancia del *red teaming* para identificar vulnerabilidades en sistemas basados en LLM.

Estas consideraciones determinaron que el diseño de LIMA incorporara seguridad como preocupación transversal, implementada en múltiples capas independientes del comportamiento del modelo de lenguaje: saneamiento del entorno de ejecución, validación de rutas de archivos contra una lista blanca, clasificación determinista del nivel de riesgo de cada operación (seguro, advertencia, peligro), confirmación explícita del usuario para operaciones destructivas, límite en el número de archivos afectados por una sola instrucción, y registro auditable de todas las interacciones. Ninguna de estas capas depende de la evaluación del LLM, lo que garantiza su efectividad incluso ante respuestas inesperadas del modelo.

3.3. Trabajos relacionados

Con base en los fundamentos expuestos en las secciones anteriores, se revisan a continuación los sistemas existentes más relevantes para la traducción de lenguaje natural a comandos de sistema y la asistencia por voz en entornos GNU/Linux. El análisis permite identificar las brechas específicas que ningún proyecto existente cubre de forma simultánea y que motivaron el diseño de LIMA.

Jasper: plataforma de voz de código abierto

Jasper fue una de las primeras plataformas de código abierto (licencia MIT) para el desarrollo de aplicaciones controladas por voz (Saha et al., 2014). Desarrollado en 2014 por Shubhro Saha, Charles Marsh y Jan Holthuis, el proyecto estaba orientado a Raspberry Pi y sistemas GNU/Linux. Jasper implementaba un flujo *siempre activo* (*always-on*): escuchaba continuamente una palabra clave, convertía la voz a texto mediante PocketSphinx o Google STT, y ejecutaba módulos (*modules*) escritos en Python que respondían a patrones de intenciones. El usuario podía consultar el clima, leer noticias, reproducir música o ejecutar comandos personalizados. Aunque fue pionero en demostrar la viabilidad de un asistente de voz de código abierto sobre GNU/Linux, Jasper carecía de mecanismos de seguridad para la ejecución de comandos del sistema, no incluía un modelo de riesgo y su desarrollo cesó en 2015.

Mycroft: asistente de voz de código abierto para Linux

Mycroft fue un asistente virtual de código abierto (licencia Apache 2.0) desarrollado por Mycroft AI, Inc. desde 2015, concebido como una alternativa de código abierto a Siri, Alexa y Google Assistant (Bhartiya, 2016). Escrito en Python, Mycroft se ejecutaba de forma nativa en GNU/Linux y realizaba todo su procesamiento localmente en el dispositivo del usuario, sin depender de servidores en la nube para el procesamiento central —un principio compartido con LIMA—.

Su arquitectura modular incluía:

- **Detección de palabra clave:** mediante el motor Precise, una red neuronal recurrente (RNN) entrenada para distinguir la palabra de activación (Bhartiya, 2016).
- **Reconocimiento de voz:** integración con Mozilla DeepSpeech para transcripción local.
- **Interpretación de intenciones:** dos analizadores —Adapt (coincidencia de palabras clave en secuencia) y Padatious (inferencia basada en ejemplos)—.
- **Síntesis de voz:** motor Mimic, basado en Festival Lite.
- **Habilidades (*skills*):** sistema extensible con más de 100 habilidades comunitarias.

Mycroft también produjo *hardware* de código abierto (Mark I y Mark II), altavoces inteligentes basados en Raspberry Pi. En febrero de 2023, la empresa cesó su desarrollo debido a una demanda por patentes (Dobberstein, 2023). La comunidad continuó el proyecto a través de OpenVoiceOS.

Aunque Mycroft fue el asistente de voz de código abierto más maduro para GNU/Linux, se enfocaba en tareas de domótica y consulta de información general, no en la gestión de archivos del sistema operativo. Además, no implementaba un modelo de clasificación de riesgo por capas ni un enfoque explícito de HCAI.

OpenVoiceOS (OVOS): continuación comunitaria de Mycroft

Tras el cese de Mycroft AI, la comunidad *fork* OpenVoiceOS (OVOS) se convirtió en el principal sucesor del proyecto (OpenVoiceOS, 2024). OVOS mantiene la arquitectura modular de Mycroft pero añade mejoras significativas: soporte para LLM a través del módulo *ovos-persona* (que permite conectar modelos de lenguaje como respaldo cuando las habilidades no pueden resolver una consulta), instalador automatizado para Raspberry Pi (RaspOVOS), un sistema de distribución de imágenes personalizado (*ovos-buildroot*) y soporte multilingüe mejorado a través de portales de traducción comunitarios. OVOS se distribuye bajo licencia Apache 2.0 y se instala mediante `pip install ovos-core`.

Al igual que Mycroft, OVOS se orienta a tareas de asistencia general (clima, noticias, reproducción de medios, domótica) y no contempla la gestión de archivos del sistema operativo como caso de uso principal, ni implementa un modelo de seguridad semántico por capas.

NL2Bash: traducción de lenguaje natural a comandos Bash

Lin et al. (2018) construyeron un corpus paralelo de más de 9 000 pares lenguaje natural–comando Bash y entrenaron modelos de traducción secuencia a secuencia. El sistema Tellina permitía a usuarios describir tareas en inglés y recibir el comando Bash correspondiente. El sistema demostró la viabilidad de la traducción NL→Bash, pero se limitó a la generación de comandos sin contemplar ejecución, seguridad, retroalimentación ni interacción por voz.

Codex: generación de código a partir de lenguaje natural

Chen et al. (2021) presentaron Codex, un modelo basado en GPT-3 afinado sobre repositorios de código público. El modelo recibe *docstrings* o descripciones y genera fragmentos de código en múltiples lenguajes de programación. Codex alcanzó un 28.8% de precisión en el *benchmark* HumanEval, demostrando capacidades emergentes de generación de código. Sin embargo, no

aborda la ejecución directa sobre el sistema operativo ni la interacción por voz.

El sistema BOTAS: antecedente directo

BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*) fue desarrollado como proyecto de servicio social y constituye el antecedente directo de LIMA (Aragon Toledo, 2026). Implementó cuatro capas funcionales: STT (*Speech-to-Text*), NLP (interpretación mediante GPT-4), ejecución segura y retroalimentación GUI + TTS. BOTAS validó la viabilidad técnica de un asistente de voz para tareas de GNU/Linux, pero presentó limitaciones: interfaz gráfica básica, ausencia de detección de palabra clave, dependencia exclusiva del modelo remoto y falta de mecanismos de auditoría. LIMA extiende este trabajo incorporando: interfaz GTK3 modernizada, detección de palabra clave (*wake word*) mediante Vosk, motor TTS alternativo con OpenAI, creación automatizada de proyectos a partir de plantillas JSON, detección automática de distribución Linux, módulo de detección local de intenciones, indicador visual de estado, registro de peticiones para auditoría, y empaquetado para distribución multiplataforma.

ClawdBot (OpenClaw): asistente personal de IA autohospedado

ClawdBot —también conocido como OpenClaw— es un proyecto de código abierto (licencia MIT) creado por Peter Steinberger y la comunidad (Steinberger, 2025). El sistema se ejecuta localmente y actúa como un *gateway* que conecta canales de mensajería (WhatsApp, Telegram, Slack, Discord, Signal, iMessage, Microsoft Teams, entre otros) con un agente de IA. Su arquitectura se organiza en torno a un *Gateway WebSocket* como plano de control único para sesiones, enrutamiento y conexiones de canal.

Entre sus características se encuentran: ejecución local sin dependencia de la nube, atención simultánea a múltiples canales de mensajería, modo de voz con detección de palabra clave (*Voice Wake*) y conversación continua (*Talk Mode*), ejecución de comandos de *shell* en el sistema anfitrión, control de navegador, trabajos programados, y seguridad por aislamiento en contenedores Docker.

ClawdBot y LIMA comparten principios de autohospedaje, privacidad, reconocimiento de voz y ejecución de comandos. Sin embargo, ClawdBot es un asistente de propósito general orientado a desarrolladores avanzados y opera en inglés; LIMA se enfoca en usuarios no especializados de GNU/Linux, en español, con énfasis en gestión de archivos, consultas del sistema y búsquedas web básicas, además de un modelo de seguridad semántico de tres niveles y un enfoque HCAI.

3.3.1. Consideraciones finales

La Tabla 3.3 resume las características de los proyectos revisados en relación con las dimensiones que LIMA integra.

Tabla 3.3: Comparativa de proyectos relacionados.

Característica	<i>Jasper</i>	<i>Mycroft/OVOS</i>	<i>NL2Bash</i>	<i>Codex</i>	<i>BOTAS</i>	<i>ClawdBot</i>
Código abierto	✓	✓			✓	✓
Interacción por voz	✓	✓			✓	✓
Ejecución en GNU/Linux	✓	✓			✓	Parcial
Gestión de archivos del SO					✓	
Idioma español					✓	
Seguridad por capas						
Enfoque HCAI						

Ninguno de los trabajos revisados integra de forma completa: (a) ejecución directa sobre el sistema operativo, (b) interacción por voz en español, (c) un modelo de seguridad semántico por capas y (d) un enfoque explícito de Inteligencia Artificial Centrada en el Humano. LIMA se posiciona en la intersección de estas cuatro dimensiones.

La Tabla 3.4 presenta el inventario completo de módulos que conforman LIMA; el resto de este capítulo describe la arquitectura del sistema y sus componentes.

3.4. Visión general de la arquitectura

LIMA se diseña como un sistema modular organizado en capas, cada una con una responsabilidad claramente definida. El orquestador `lima.pl` instancia todos los módulos en tiempo de arranque e interconecta sus funciones mediante un diccionario de 12 funciones *callback*. Esta decisión arquitectónica —un patrón mediador— permite que los módulos no se conozcan entre sí: toda la comunicación pasa por el orquestador, lo que facilita la sustitución de cualquier componente sin afectar a los demás. La Figura 3.6 presenta una vista general de esta organización por capas.

La arquitectura se organiza en seis capas lógicas:

1. **Capa de interfaz:** `GUI_GTK3.pm` y los dispositivos de entrada (micrófono, teclado).
2. **Capa de voz:** `VoiceRecognition.pm` (captura y transcripción) y `WakeWordDetector.pm` (activación por palabra clave).

Tabla 3.4: Inventario de componentes de LIMA

Componente	Lenguaje	Líneas	Subrutinas
lima.pl	Perl 5	~480	8
CommandParser.pm	Perl 5	~700	8
FileManager.pm	Perl 5	~870	20
GUI_GTK3.pm	Perl 5	~2 500	35
IntentDetector.pm	Perl 5	235	3
TTS.pm	Perl 5	200	8
VoiceRecognition.pm	Perl 5	280	10
WakeWordDetector.pm	Perl 5 + Python 3	~530	12 + 5
DistroDetector.pm	Perl 5	180	9
TaskLearner.pm	Perl 5	~850	23
config.json	JSON	62	—
Plantillas de proyecto (3)	JSON	~170	—
Total		~8 025	~151

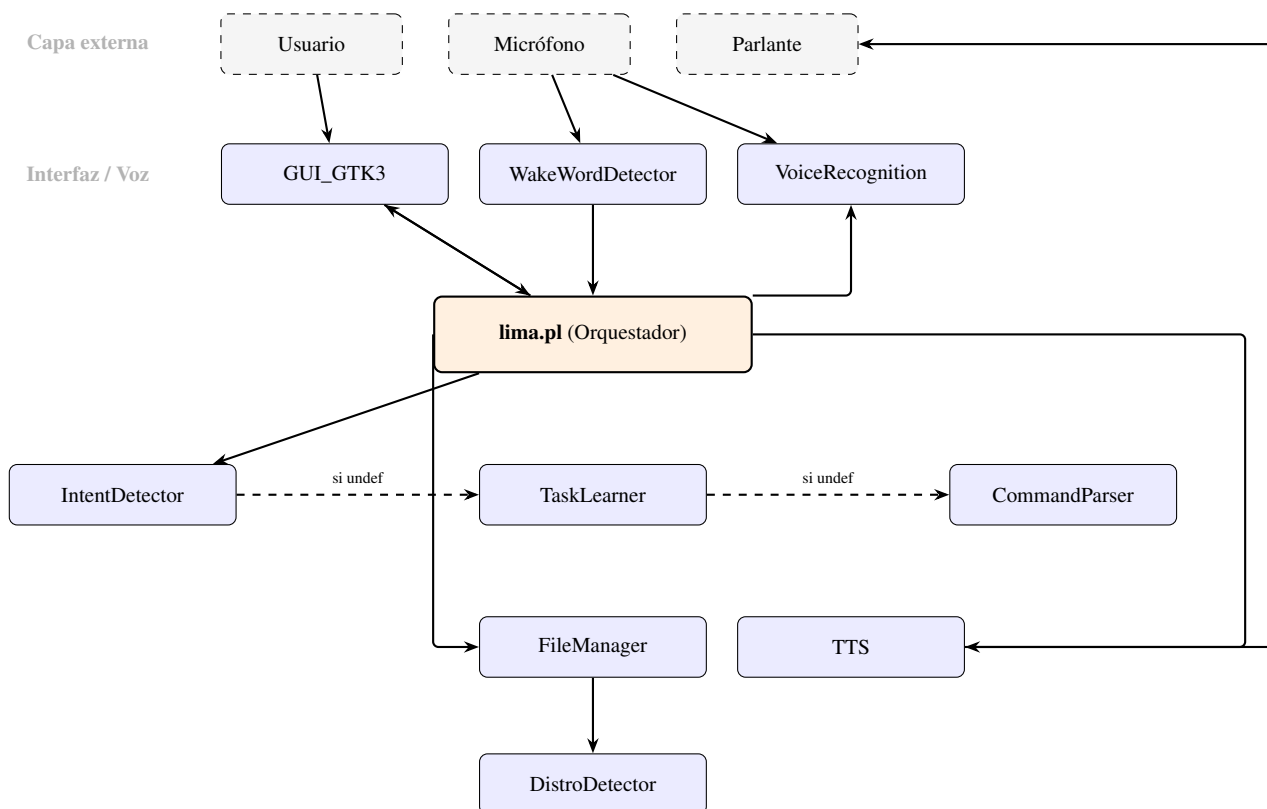


Figura 3.6: Arquitectura general de LIMA. Las flechas sólidas indican flujo de control; las flechas punteadas muestran las dos escalaciones del *pipeline* de interpretación: IntentDetector → TaskLearner → CommandParser (LLM).

3. **Capa de interpretación:** `IntentDetector.pm` (regex local), `TaskLearner.pm` (patrones TF-IDF aprendidos) y `CommandParser.pm` (LLM remoto o local).
4. **Capa de ejecución:** `FileManager.pm` (operaciones de sistema de archivos validadas).
5. **Capa de retroalimentación:** `TTS.pm` (síntesis de voz) y la interfaz gráfica (resultados visuales).
6. **Capa de soporte:** `DistroDetector.pm` (adaptación multiplataforma) y `config.json` (configuración centralizada).

3.5. Orquestador principal: `lima.pl`

El archivo `lima.pl` (~480 líneas) es el punto de entrada del sistema. Su responsabilidad es instanciar los módulos, definir las funciones *callback* e implementar el ciclo principal de eventos mediante el bucle de GTK3/Glib.

3.5.1. Inicialización y seguridad de entorno

Al arrancar, un bloque `BEGIN` realiza dos operaciones de saneamiento del entorno:

1. Elimina `LD_LIBRARY_PATH` si contiene la cadena `/snap/`, para evitar conflictos con bibliotecas compartidas de paquetes Snap que interfieren con GTK3.
2. Elimina `LD_PRELOAD` incondicionalmente, previniendo la inyección de bibliotecas maliciosas.

Después, `lima.pl` carga la configuración desde `config/config.json` mediante `File::Slurp` y la deserializa con el módulo `JSON`. Esta estructura se pasa como parámetro al constructor de cada módulo.

3.5.2. Diccionario de *callbacks*

La comunicación entre módulos se realiza exclusivamente mediante un diccionario (*hashref*) de 14 funciones *callback* que el orquestador construye como clausuras (*closures*) Perl. Este diccionario se pasa a `GUI_GTK3` para que la interfaz pueda invocar las acciones del sistema sin conocer los módulos internos. La Tabla 3.5 enumera las funciones registradas.

3.5.3. Función `process_text`: el *pipeline* de tres niveles

La función central del orquestador es `process_text($text)`, que implementa el *pipeline* de interpretación de tres niveles en cascada. Cada nivel es significativamente más costoso

Tabla 3.5: Diccionario de *callbacks* del orquestador

<i>Callback</i>	Función
<code>on_listen</code>	Inicia la grabación de audio mediante <code>VoiceRecognition</code> .
<code>on_stop</code>	Detiene la grabación, transcribe, procesa el texto y ejecuta el comando.
<code>on_confirm</code>	Confirma la ejecución de un comando que requiere aprobación del usuario.
<code>on_cancel</code>	Cancela un comando pendiente de confirmación.
<code>on_mic_change</code>	Propaga el cambio de dispositivo de audio a <code>VoiceRecognition</code> y <code>WakeWordDetector</code> .
<code>on_new_session</code>	Limpia el historial de conversación de <code>CommandParser</code> .
<code>on_attention_start</code>	Activa el detector de palabra clave e inicia un temporizador Glib de 100 ms para sondear eventos.
<code>on_attention_stop</code>	Detiene el detector de palabra clave y elimina el temporizador.
<code>get_parser</code>	Retorna la instancia del <code>CommandParser</code> para acceso directo.
<code>speak</code>	Invoca <code>TTS->speak_async()</code> con el texto y la categoría de acción proporcionados.
<code>test_openai_voice</code>	Establece temporalmente la voz de OpenAI especificada, reproduce un texto de prueba con <code>TTS->speak_async()</code> y restaura la voz anterior. Se usa para la vista previa de voces en el diálogo de configuración.
<code>on_tts_engine_change</code>	Cambia el motor de síntesis de voz (<code>OpenAI TTS</code> ↔ <code>eSpeak</code>).
<code>on_espeak_params_change</code>	Actualiza los parámetros de <code>eSpeak</code> (velocidad, tono, espacio entre palabras).
<code>on_mode_change</code>	Cambia entre modo en línea y fuera de línea, propagando a todos los módulos.

que el anterior, por lo que el sistema escala solo cuando los niveles anteriores no producen coincidencia:

1. **Nivel 1 — Detector local (sin costo):** Invoca `IntentDetector->detect($text)`. Si retorna una estructura definida, la solicitud se resuelve mediante expresiones regulares en español y se marca con el indicador `_local =>1`. Latencia < 1 ms.
2. **Nivel 2 — Patrones aprendidos (sin costo):** Si el nivel 1 retorna `undef`, invoca `TaskLearner->match($text)`, que busca en los patrones almacenados mediante TF-IDF con similitud coseno. Si la similitud supera el umbral de confianza (0.60 por defecto), retorna el comando reconstituido con el indicador `_learned =>1`. Latencia < 5 ms.
3. **Nivel 3 — LLM (costoso):** Si ambos niveles anteriores retornan `undef`, invoca `CommandParser->parse($text)`, que envía la solicitud al LLM (GPT-4o en modo en línea o DeepSeek-R1 en modo fuera de línea). Tras una ejecución exitosa, el orquestador llama a `TaskLearner->learn()` para almacenar el patrón y evitar consultas futuras al LLM para solicitudes similares.

En los tres casos, el resultado es un *hashref* con estructura JSON idéntica (acción, parámetros, respuesta TTS), lo que permite que el resto del sistema —ejecución, registro, retroalimentación— opere de manera uniforme independientemente del origen del comando.

3.5.4. Función `execute_command`: despacho de acciones

El despacho de acciones se realiza mediante un bloque condicional que mapea el campo `action` del JSON a métodos de `FileManager`, como se resume en la Tabla 3.6:

Tras cada ejecución exitosa, el orquestador invoca `CommandParser->update_last_result()`, que inyecta anotaciones del resultado (`[RESULTADO DE EJECUCION]`, `[DETALLE]`, `[LISTA]`) en el historial de conversación para que el LLM pueda responder preguntas de seguimiento con contexto.

3.5.5. Manejo de errores

Toda la cadena de transcripción, procesamiento y ejecución se envuelve en un bloque `eval`. Si ocurre una excepción, el orquestador analiza el mensaje de error mediante coincidencia de patrones:

- Errores que contienen `/ruta|path/`: se genera un mensaje TTS indicando «No tengo acceso a esa ubicación».

Tabla 3.6: Mapeo de acciones a métodos de FileManager

Acción JSON	Método invocado
create	FileManager->create(\$cmd)
list	FileManager->list(\$cmd)
search	FileManager->search(\$cmd)
move	FileManager->move(\$cmd)
rename	FileManager->rename(\$cmd)
delete	FileManager->delete(\$cmd)
info	FileManager->info(\$cmd)
edit	FileManager->edit(\$cmd)
create_project	FileManager->create_project(\$cmd)
system_info	FileManager->system_info(\$cmd)
analyze_text	FileManager->analyze_text(\$cmd)
web_search	FileManager->web_search(\$cmd)
answer	Respuesta directa vía TTS (sin operación)
clarify	Solicitud de aclaración al usuario

- Errores que contienen /API|GPT|conexión/: se genera «Hay un problema de conexión».
- Otros errores: se genera «Ocurrió un error inesperado».

Todos los errores se registran en `request_history.log` con la fuente 'error'.

3.6. Módulo de reconocimiento de voz: VoiceRecognition.pm

Este módulo (280 líneas, 10 subrutinas) gestiona la captura de audio y la transcripción a texto. Soporta dos motores de transcripción: OpenAI Whisper (en línea) y Vosk (fuera de línea). La Figura 3.7 muestra el flujo de datos desde el micrófono hasta la obtención del texto transcrito.

3.6.1. Captura de audio basada en procesos

La grabación de audio utiliza la bifurcación de procesos de POSIX (`fork()` + `exec()`) para ejecutar el comando `rec` de SoX en un proceso hijo:

Listing 3.1: Lanzamiento del proceso de grabación

```
rec -q -c $channels -r $sample_rate $filename
trim 0 $max_duration
```

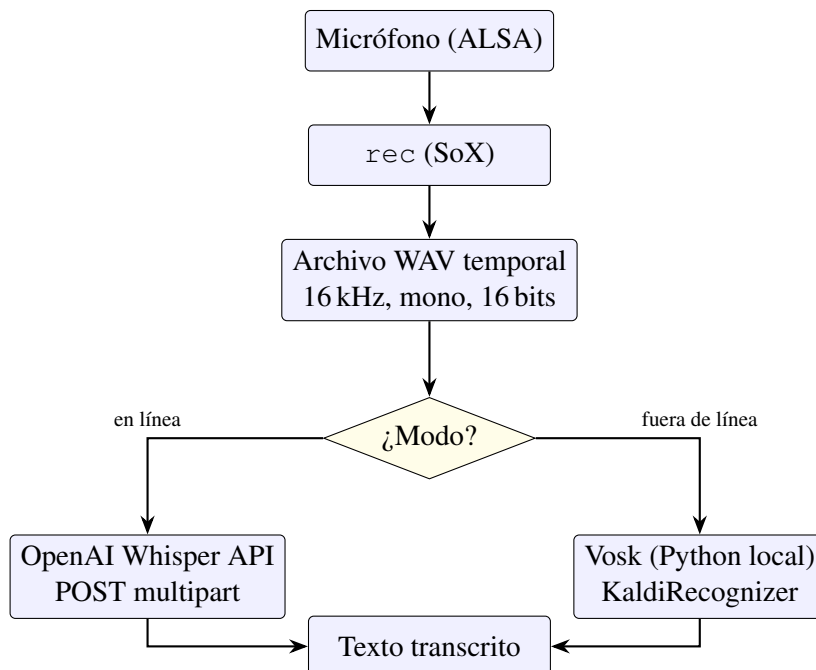


Figura 3.7: Flujo de datos del módulo `VoiceRecognition.pm`.

El proceso padre almacena el PID del hijo. Al invocar `stop_recording()`, el módulo envía una señal `SIGINT` al proceso hijo, espera hasta 3 segundos, y si el proceso no termina, envía `SIGKILL`. Finalmente, verifica que el archivo resultante supere los 500 bytes para descartar grabaciones vacías.

Los parámetros de audio se configuran en `config.json`:

- Formato: WAV (sin compresión).
- Frecuencia de muestreo: 16 000 Hz (óptima para Whisper).
- Canales: 1 (mono).
- Duración máxima: 30 segundos.

3.6.2. Transcripción con OpenAI Whisper

En modo en línea, la transcripción se realiza mediante una solicitud HTTP POST multi-parte al *endpoint* `/v1/audio/transcriptions` de la API de OpenAI, utilizando el módulo `LWP::UserAgent`. El modelo utilizado es `whisper-1` y el idioma se fija en español (`language=es`).

3.6.3. Transcripción con Vosk (modo fuera de línea)

En modo fuera de línea, el módulo genera dinámicamente un *script* Python que utiliza la biblioteca Vosk con un modelo preentrenado en español. El *script* se escribe en un archivo temporal, se ejecuta mediante `python3` y su salida estándar se captura como texto transcrito. Esta estrategia de interoperabilidad Perl–Python permite aprovechar el ecosistema de modelos de Vosk sin requerir *bindings* nativos Perl.

El módulo utiliza la variable de entorno `PLURIX_PYTHON` si está definida, lo que permite especificar un intérprete Python de un entorno virtual (*virtualenv*) dedicado.

3.6.4. Subrutinas del módulo

La Tabla 3.7 detalla las subrutinas que conforman el módulo.

Tabla 3.7: Subrutinas de `VoiceRecognition.pm`

Subrutina	Descripción
<code>new(\$config)</code>	Constructor. Configura clave API, modelo, directorio temporal y parámetros de audio.
<code>_init()</code>	Crea el directorio temporal; si no es escribible, recurre a <code>/tmp/lima_audio_\$<</code> .
<code>set_device(\$dev)</code>	Establece el dispositivo ALSA de captura (por ejemplo, <code>hw:1,0</code>).
<code>get_device()</code>	Retorna el dispositivo actualmente configurado.
<code>is_recording()</code>	Retorna verdadero si hay un proceso de grabación activo.
<code>start_recording()</code>	Bifurca un proceso hijo con <code>rec</code> ; retorna la ruta del archivo temporal.
<code>stop_recording()</code>	Envía <code>SIGINT/SIGKILL</code> al hijo; valida tamaño del archivo (<code>>500</code> bytes).
<code>transcribe(\$file)</code>	Despacha a <code>_transcribe_whisper()</code> o <code>_transcribe_vosk()</code> según el modo.
<code>_transcribe_whisper(\$f)</code>	POST multiparte a la API de OpenAI Whisper.
<code>_transcribe_vosk(\$f)</code>	Genera y ejecuta <i>script</i> Python con Vosk KaldiRecognizer.

3.7. Módulo de detección de palabra clave: `WakeWordDetector.pm`

Este módulo (~ 530 líneas: 280 en Perl y 250 en Python embebido) implementa la detección de la palabra clave «LIMA» para activar el sistema sin intervención manual. A diferencia del

reconocimiento de voz principal, este módulo opera completamente fuera de línea utilizando el motor Vosk, lo que garantiza que la escucha pasiva no genere tráfico de red ni costos de API. El flujo de operación se presenta en la Figura 3.8.

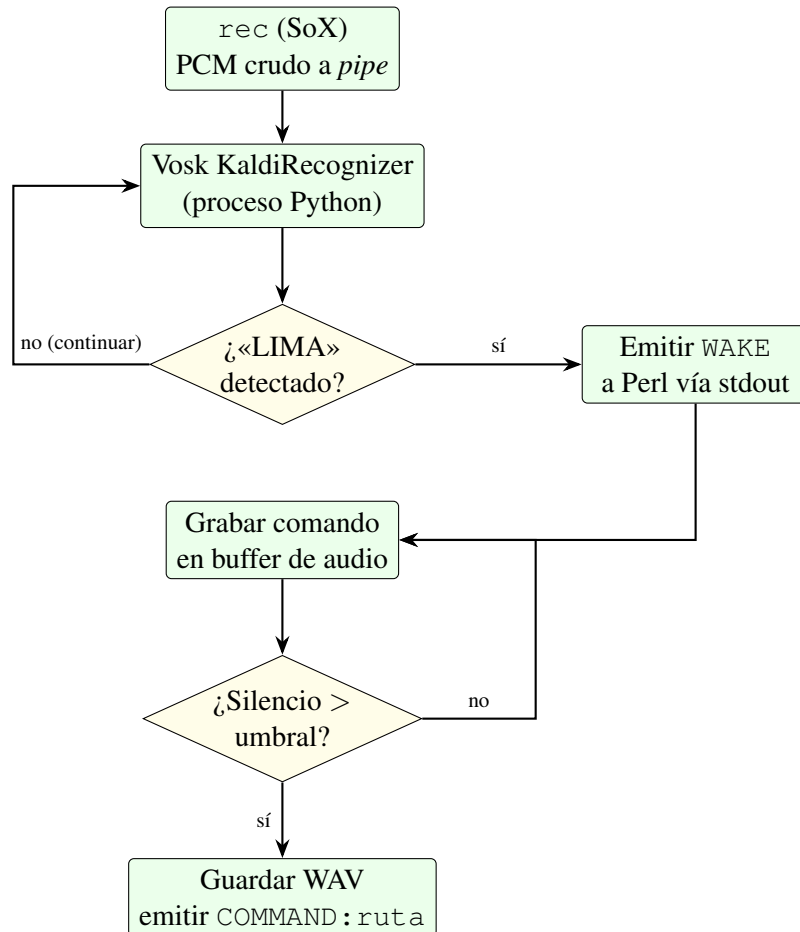


Figura 3.8: Flujo del módulo `WakeWordDetector.pm`. El bucle de escucha opera continuamente en un proceso Python separado.

3.7.1. Arquitectura de comunicación interproceso

El diseño utiliza un puente Perl–Python mediante `IPC::Open3`: el módulo Perl genera dinámicamente un *script* Python de ~250 líneas, lo escribe en un archivo temporal y lo lanza como proceso hijo. La comunicación se realiza mediante un protocolo de líneas en la salida estándar del proceso Python, descrito en la Tabla 3.8:

Para no bloquear el bucle de eventos de GTK3, el *handle* de lectura del proceso hijo se configura en modo no bloqueante mediante `fcntl(F_SETFL, O_NONBLOCK)`. El orquestador sondea el detector cada 100 ms mediante un temporizador `Glib::Timeout`, invocando

Tabla 3.8: Protocolo IPC del detector de palabra clave

Mensaje	Significado
READY	El detector se ha inicializado correctamente y está escuchando.
WAKE	Se ha detectado la palabra clave «LIMA» en el flujo de audio.
COMMAND:/ruta	La grabación del comando ha finalizado (silencio detectado); el archivo WAV está en la ruta indicada.
STOPPED	El detector se ha detenido limpiamente.

`check_events()` que intenta un `sysread()` y retorna inmediatamente si no hay datos disponibles.

3.7.2. Algoritmo de detección de silencio

El proceso Python calcula la energía RMS (*Root Mean Square*) de cada fragmento de audio:

$$\text{RMS} = \frac{\sqrt{\frac{1}{n} \sum_{i=1}^n s_i^2}}{32768}$$

donde s_i son las muestras de audio con signo de 16 bits y 32768 es el valor máximo para normalizar al rango $[0, 1]$. Si la energía RMS permanece por debajo del umbral configurable (`silence_threshold`, por defecto 0.02) durante el periodo de silencio configurado (`silence_duration`, por defecto 8 segundos), el detector considera que el usuario ha terminado de hablar y guarda el audio acumulado como archivo WAV.

Como medida de seguridad, existe un límite máximo de duración (`max_command_duration`, 30 segundos) que fuerza el guardado del audio independientemente del nivel de silencio.

3.7.3. Verificación de dependencias

Al inicializarse, el módulo verifica la disponibilidad de tres dependencias:

1. `sox`: verificado mediante `which rec`.
2. `vosk` (Python): verificado ejecutando `python3 -c "import vosk"`.
3. Modelo Vosk: verificado comprobando la existencia del directorio configurado en `config.wake_word.vosk_model`.

3.8. Módulo de detección local de intenciones: `IntentDetector.pm`

Este módulo (235 líneas, 3 subrutinas) resuelve solicitudes frecuentes sin recurrir al LLM, reduciendo la latencia y el consumo de tokens de la API. Implementa diez categorías de intención, cada una con un conjunto de expresiones regulares en español y una función manejadora que genera la respuesta. La Figura 3.9 esquematiza el flujo de decisión del módulo.

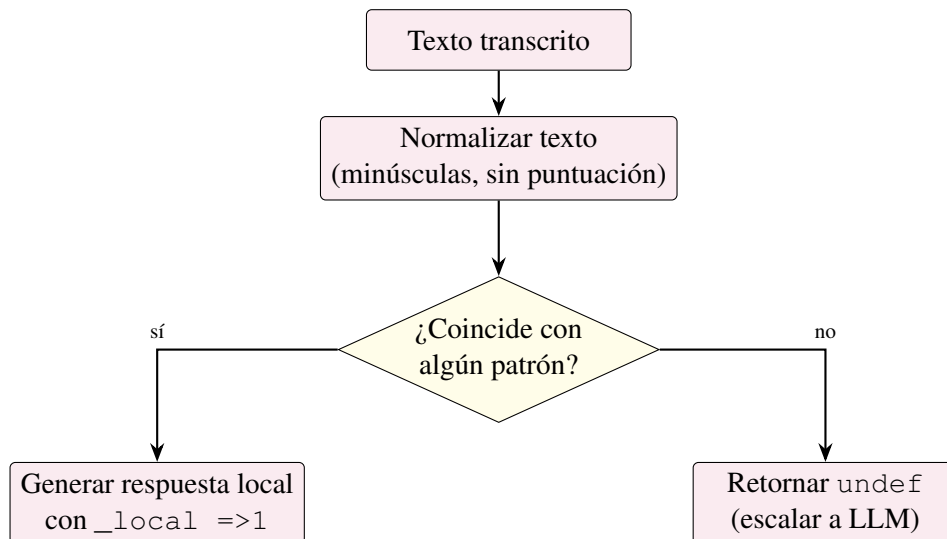


Figura 3.9: Flujo de decisión del módulo `IntentDetector.pm`. Si el texto transcrito coincide con algún patrón, el módulo genera la respuesta localmente; en caso contrario retorna `undef` y la solicitud escala al siguiente nivel del *pipeline* (`TaskLearner` y, finalmente, el LLM).

3.8.1. Categorías de intención

La Tabla 3.9 resume las diez categorías implementadas, junto con ejemplos de los patrones que las activan y el tipo de respuesta generada.

Un aspecto destacable del diseño es que las expresiones regulares utilizan clases de caracteres Unicode para aceptar tanto la forma con acento como sin acento: por ejemplo, `[eé]` coincide con «e» y con «é». Esto aumenta la robustez frente a variaciones en la transcripción del motor de voz.

Cuando una categoría coincide, la función manejadora retorna una estructura JSON idéntica a la que produciría `CommandParser`. Esta uniformidad de interfaz es una decisión de diseño deliberada: el orquestador, el módulo de ejecución y el sistema de registro no necesitan distinguir entre un comando resuelto localmente y uno interpretado por el LLM.

Tabla 3.9: Categorías de intención local y ejemplos de patrones

Categoría	Patrones (ejemplo)	Respuesta
Hora del sistema	qu[eé] hora es, dime la hora	Hora formateada con <code>strftime (%H:%M)</code>
Fecha del sistema	qu[eé] fecha, qu[eé] d[ií]a es	Fecha completa con día de la semana en español
Saludo	hola, buenos d[ií]as, qu[eé] tal	Saludo contextual según hora (mañana/tarde/noche)
Identidad	qui[eé]n eres, c[oó]mo te llamas	Presentación fija del asistente
Agradecimiento	gracias, muchas gracias	Respuesta aleatoria de 3 opciones
Información de RAM	cu[aá]nta memoria, cu[aá]nta ram	<code>system_info</code> con <code>info_type</code> =>'memory'
Espacio en disco	espacio en disco, cu[aá]nto espacio	<code>system_info</code> con <code>info_type</code> =>'disk'
Procesos	procesos.*cpu, qu[eé] procesos	<code>system_info</code> con <code>info_type</code> =>'processes'
Puertos	puertos abiertos, puertos en uso	<code>system_info</code> con <code>info_type</code> =>'ports'
Cancelación	nada, cancela, olv[ií]dalo	Respuesta de despedida cortés

3.9. Módulo de aprendizaje de tareas: **TaskLearner.pm**

Este módulo (~1 100 líneas, 30 subrutinas) es el componente más distintivo de LIMA y el núcleo de su capacidad de aprendizaje: constituye la capa de memoria a largo plazo del sistema. Conviene distinguirlo del `IntentDetector` (sección 3.8), con el que a veces se confunde: mientras que el `IntentDetector` solo reconoce, mediante reglas fijas, un repertorio cerrado de intenciones conversacionales, el `TaskLearner` permite que el asistente *aprenda* tareas nuevas a partir de las que el modelo de lenguaje resuelve con éxito y las reproduzca después sin volver a consultarlo. Actúa como intermediario entre `IntentDetector` y `CommandParser`: cuando el LLM resuelve con éxito una solicitud, `TaskLearner` vectoriza esa interacción y la almacena como patrón. En invocaciones futuras con formulaciones similares, el módulo recupera el patrón más cercano, extrae los valores actuales mediante un mecanismo de *slot-filling* adaptativo basado en anclas léxicas, y retorna el comando reconstituido —sin necesidad de con-

sultar el LLM—. La Figura 3.10 ilustra el flujo de aprendizaje e inferencia.

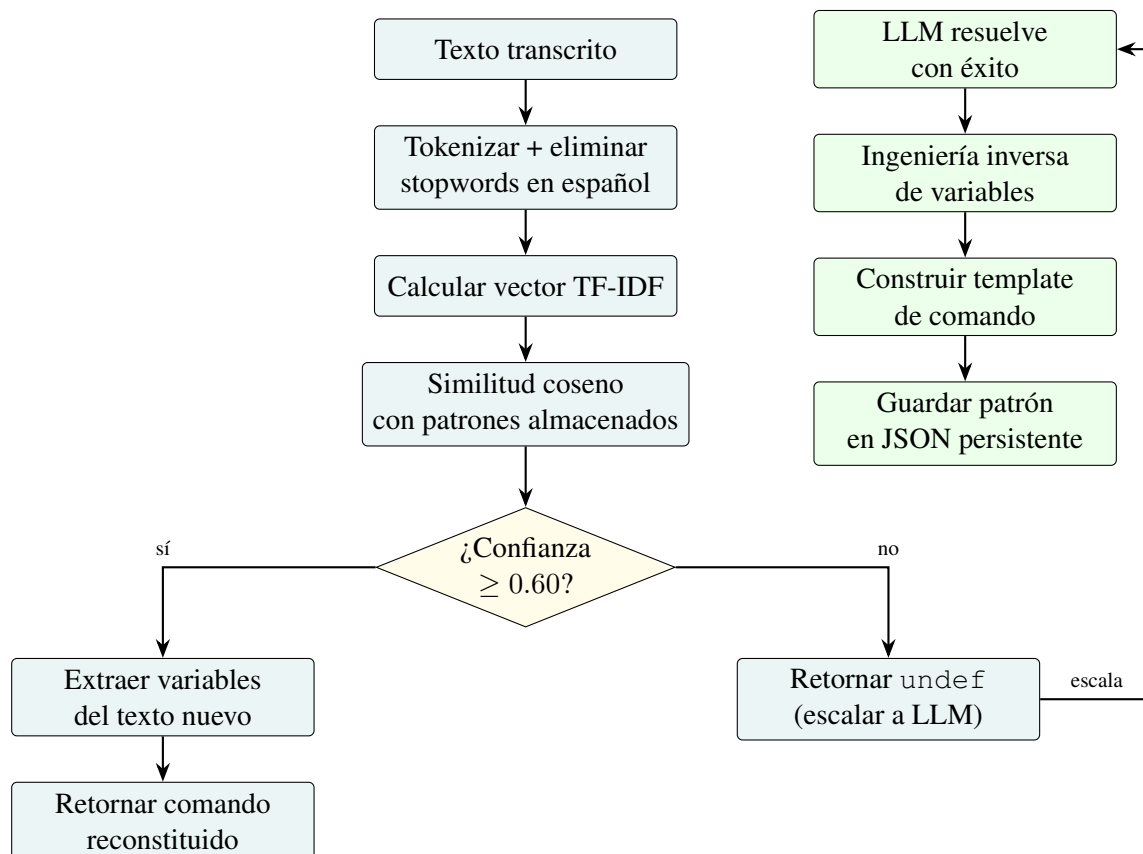


Figura 3.10: Flujo dual de TaskLearner.pm: inferencia (izquierda) cuando existe un patrón cuya similitud coseno supera el umbral de confianza configurable (`confidence_threshold`, por defecto 0.60), y aprendizaje (derecha) cuando ningún patrón lo supera y el LLM resuelve la solicitud nueva.

3.9.1. Posición en el *pipeline* de tres niveles

TaskLearner ocupa el **Nivel 2** del *pipeline* de interpretación de `process_text`. Esta posición es intencional: se sitúa después de `IntentDetector` (que cubre saludos, fecha/hora y consultas de sistema mediante regex) y antes de `CommandParser` (que invoca al LLM). De este modo, el módulo solo aprende tareas que el LLM pudo resolver y que no se cubren por reglas fijas, que son precisamente las solicitudes de mayor costo y mayor frecuencia de repetición.

La Tabla 3.10 compara los tres niveles en términos de costo, latencia y capacidad:

Tabla 3.10: Comparación de los tres niveles del *pipeline* de interpretación

Nivel	Módulo	Mecanismo	Latencia / Costo
1	IntentDetector	Expresiones regulares en español	<1 ms / ninguno
2	TaskLearner	TF-IDF + similitud coseno	<5 ms / ninguno
3	CommandParser	LLM (GPT-4o o DeepSeek-R1)	2–30 s / tokens de API

3.9.2. Algoritmo TF-IDF con similitud coseno

El motor de búsqueda de similitud está implementado en Perl puro, sin dependencias externas. El proceso se divide en tres pasos:

1. **Tokenización:** El texto se convierte a minúsculas, se elimina la puntuación y se normalizan los acentos ($\acute{a} \rightarrow a$, $\acute{e} \rightarrow e$, etc.). Las palabras con longitud ≤ 1 y aquellas presentes en una lista de 70 *stopwords* en español se descartan. El resultado es una lista de *tokens* significativos.
2. **Vectorización TF-IDF:** Para cada *token* se calcula su frecuencia de término (TF) normalizada por la longitud del documento y su frecuencia inversa de documento (IDF) suavizada:

$$\text{IDF}(t) = \log_2 \left(\frac{N}{1 + \text{df}(t)} \right) + 1$$

donde N es el número de patrones almacenados y $\text{df}(t)$ es el número de patrones que contienen el término t . El vector TF-IDF resultante es disperso (*sparse*), almacenado como un *hash* Perl.

3. **Similitud coseno:** La similitud entre el vector de la nueva solicitud $\vec{q}$ y cada patrón almacenado $\vec{p}_i$ se calcula como:

$$\text{sim}(\vec{q}, \vec{p}_i) = \frac{\vec{q} \cdot \vec{p}_i}{\|\vec{q}\| \|\vec{p}_i\|}$$

El patrón con la mayor similitud que supere el umbral de confianza (campo `confidence_threshold` en `config.json`, por defecto 0.60) se considera coincidente. Este valor se ajustó de forma empírica como compromiso entre precisión y cobertura: un umbral menor aumentaría las coincidencias falsas —con el riesgo de ejecutar un comando incorrecto—, mientras que uno mayor exigiría reformulaciones casi idénticas a la original y desaprovecharía el aprendizaje. Las solicitudes que no alcanzan este umbral escalan al LLM.

Los vectores de los patrones se calculan de forma *lazy* y se almacenan en caché. El caché se

invalida automáticamente cuando se agrega o elimina un patrón (campo `idf_dirty`), lo que garantiza que el IDF siempre refleje el corpus actual.

3.9.3. Mecanismo de aprendizaje e ingeniería inversa de variables

Cuando `CommandParser` resuelve exitosamente una solicitud, el orquestador invoca `TaskLearner->learn($input, $command)`. El módulo ejecuta los siguientes pasos antes de almacenar el patrón:

1. **Verificación de duplicados:** Calcula la similitud coseno de la nueva solicitud contra todos los patrones existentes. Si alguno supera 0.95, solo actualiza el contador de uso (`use_count`) del patrón existente y no almacena uno nuevo.
2. **Extracción de variables:** La función `_reverse_engineer_variables()` compara los valores concretos del JSON devuelto por el LLM (p. ej., el nombre del archivo en `target_path`) con el texto de la solicitud. Cada valor encontrado se sustituye por un *placeholder* con nombre semántico (`{NAME}`, `{QUERY}`, `{EXTENSION}`, etc.) y se construye una expresión regular de extracción para invocaciones futuras.
3. **Construcción del template:** La estructura JSON del comando se clona y los valores concretos se reemplazan por los *placeholders* identificados, generando un *template* reutilizable.
4. **Persistencia:** El patrón, junto con sus metadatos (`id`, `created_at`, `use_count`, `success_count`, `original_input`), se serializa en `~/.lima/learned_patterns.json`.

Cuando en una invocación futura `match()` encuentra un patrón con confianza suficiente, la función `_fill_template()` debe inyectar en el *template* los valores actuales extraídos del nuevo texto. Para tolerar variaciones de redacción y de orden de los argumentos, esta función encadena tres estrategias de extracción —de la más flexible a la más rígida—: (i) extracción por **anclas léxicas** (*slot-filling*), descrita en la Sección 3.9.4; (ii) la **expresión regular** construida al aprender el patrón, válida cuando la redacción es idéntica a la original; y (iii) una **heurística posicional** de respaldo. Si las tres fracasan, `match()` retorna `undef` y el orquestador delega al LLM, que aprenderá la nueva variante.

3.9.4. Extracción por anclas léxicas (*slot-filling* adaptativo)

La extracción de variables del texto nuevo no puede depender únicamente de la expresión regular del patrón original, dado que el usuario rara vez repite la misma frase. Considérese, por ejemplo, un patrón aprendido a partir de la frase “Crea una carpeta llamada Matemáticas en

Documentos”. Una invocación posterior podría formularse como “Quiero que crees una carpeta llamada Backup dentro de la carpeta Documentos”, con orden invertido respecto al patrón original, palabras adicionales (*Quiero que*) y una frase preposicional alternativa (*dentro de la carpeta* en lugar de *en*). Una expresión regular rígida fracasa en estos casos y obligaría a invocar al LLM repetidamente, anulando el propósito del aprendizaje. Peor aún, una extracción heurística por posición puede capturar fragmentos espurios como “la carpeta Documentos” y producir rutas inválidas.

Para resolver este problema se incorporó un mecanismo de extracción por **anclas léxicas**, también denominado *slot-filling* adaptativo. La idea consiste en asociar cada variable a un **rol semántico** (ubicación, nombre, fuente, consulta, etc.) y reconocer ese rol por las palabras o frases que lo introducen en español, sin importar la posición ni la redacción exacta.

Diccionario de anclas por rol. El módulo define un mapa estático `%ROLE_ANCHORS` que asocia cada rol con una lista de frases-ancla, evaluadas de la más larga (más específica) a la más corta. La Tabla 3.11 muestra los roles definidos y un extracto representativo de sus anclas.

Tabla 3.11: Roles semánticos y anclas léxicas asociadas en `TaskLearner.pm`

Rol	Anclas representativas (ordenadas por especificidad)
location	<i>dentro de la carpeta, en la carpeta, en el directorio, dentro de, dentro, en, sobre</i>
name	<i>que se llame, con el nombre, llamada, nombrada, la carpeta, una carpeta, el archivo, un archivo</i>
new_name	<i>cámbiale el nombre a, renómbrala a, cambiar a, nuevo nombre</i>
source	<i>el archivo, la carpeta, mueve, copia</i>
query	<i>busca sobre, información sobre, acerca de, busca</i>
pattern	<i>que contengan, con extensión, de tipo, extensión</i>

Algoritmo de extracción. La subrutina `_anchor_extract()` procesa el texto en seis pasos:

1. **Normalización preservando posiciones.** El texto se convierte a minúsculas, se quitan acentos y la puntuación se reemplaza por espacios. La sustitución uno-a-uno preserva los índices, de forma que las posiciones encontradas en la versión normalizada pueden usarse para tomar `substr` del texto original, manteniendo intactas mayúsculas y acentos en los valores extraídos.
2. **Búsqueda exhaustiva de anclas.** Para cada rol y cada ancla del diccionario se localizan *todas* sus ocurrencias en el texto normalizado.

3. **Resolución de solapamientos.** Dos anclas pueden compartir *tokens*; por ejemplo, *dentro de la carpeta* contiene *dentro de*. El algoritmo ordena los aciertos por posición ascendente y, en empate, por longitud descendente, y descarta cualquier ancla que comience dentro del rango ya ocupado por una previa. De este modo la frase más específica gana sobre las cortas embebidas.
4. **Extracción del valor.** Para cada ancla seleccionada, el valor es la subcadena del texto original comprendida entre el final del ancla y el inicio de la siguiente (o el fin del texto, si es la última).
5. **Limpieza de bordes.** Se eliminan *stopwords* y palabras-ruido de dominio (*carpeta, directorio, haz, crear*, etc.) que pudieran aparecer en los extremos del valor capturado. Una lista `%EXTRACTION_NOISE` declara estas palabras como removibles únicamente desde los bordes, nunca del centro del valor.
6. **Asignación variable–rol.** Cada variable del patrón consulta su rol (campo `role` almacenado al aprender, o inferido en tiempo de extracción mediante `_detect_var_role()` para patrones *legacy* sin este campo) y toma el siguiente valor disponible de la lista correspondiente. Si una variable no encuentra un valor para su rol, el algoritmo retorna vacío y el control pasa al siguiente mecanismo de *fallback*.

Almacenamiento del rol al aprender. La función `_reverse_engineer_variables()`, además de identificar las variables, infiere el rol semántico de cada una observando qué ancla precede al valor en el texto original del usuario. Este rol se persiste en el patrón aprendido, de modo que en consultas futuras la asignación variable–rol es directa, sin necesidad de re-inferir.

Ejemplo trazado. Para el caso planteado al inicio de esta sección:

- **Entrada:** “Quiero que crees una carpeta llamada Backup dentro de la carpeta Documentos”.
- **Anclas detectadas:** *llamada* (rol *name*) y *dentro de la carpeta* (rol *location*). La frase corta *dentro de* embebida en la larga se descarta por la resolución de solapamientos.
- **Valores capturados:** entre *llamada* y *dentro de la carpeta* se extrae “Backup”; entre *dentro de la carpeta* y el final del texto se extrae “Documentos”.
- **Resultado:** `target_path = ~/Documentos/Backup`, sin invocar al LLM.

Esta extensión convierte al `TaskLearner` en un componente verdaderamente adaptativo: una vez que el LLM resuelve la versión canónica de un comando, las variaciones léxicas posteriores —reordenamientos, sinónimos y frases preposicionales alternativas— se resuelven localmente, lo que reduce la latencia percibida, el costo de *tokens* de API y, sobre todo, la dependencia de la conectividad para tareas que el sistema ya ha aprendido a interpretar.

3.9.5. Gestión del ciclo de vida de los patrones

El módulo implementa tres mecanismos para mantener el conjunto de patrones manejable y de alta calidad:

- **Límite de capacidad:** El parámetro `max_patterns` (500 por defecto) limita el número de patrones. Cuando se alcanza el límite, la función `_evict_least_used()` elimina el patrón con el menor índice de utilidad, calculado como $\text{usos} + (\text{éxitos}/\text{usos}) \times 10$.
- **Retroalimentación de éxito:** Tras ejecutar un comando aprendido, el orquestador invoca `update_success($pattern_id, $success)` para actualizar `success_count`, lo que influye en la métrica de utilidad y en la prioridad de retención.
- **Reinicio de caché por sesión:** Al iniciar una nueva sesión (`on_new_session()`), los vectores TF-IDF cacheados se eliminan y el IDF se recalcula en la siguiente consulta, reflejando cualquier patrón nuevo agregado durante la sesión anterior.

3.9.6. API pública del módulo

La Tabla 3.12 resume las subrutinas públicas de `TaskLearner.pm`:

Tabla 3.12: Subrutinas públicas de `TaskLearner.pm`

Subrutina	Descripción
<code>new(\$config)</code>	Constructor. Carga los parámetros de la sección <code>learning</code> del JSON de configuración y los patrones persistidos.
<code>match(\$text)</code>	Busca el patrón más similar mediante TF-IDF + coseno. Retorna el comando reconstituido o <code>undef</code> .
<code>learn(\$input, \$command)</code>	Aprende de una tarea resuelta por el LLM. Realiza deduplicación, extrae variables y persiste el patrón.
<code>update_success(\$id, \$ok)</code>	Registra si la ejecución de un patrón aprendido fue exitosa.
<code>forget(\$id)</code>	Elimina un patrón por su identificador.
<code>get_stats()</code>	Retorna estadísticas agregadas: total de patrones, usos, éxitos y tasa de éxito.
<code>on_new_session()</code>	Invalida los vectores TF-IDF cacheados al inicio de una nueva sesión.
<code>export_patterns()</code>	Serializa todos los patrones a JSON para exportación.
<code>import_patterns(\$json)</code>	Importa patrones desde JSON evitando duplicados por identificador.

3.10. Módulo de interpretación de comandos: `CommandParser.pm`

Este módulo (~700 líneas, 8 subrutinas) constituye el puente entre el lenguaje natural del usuario y las operaciones estructuradas del sistema. Envía la solicitud del usuario a un modelo de lenguaje grande (LLM) y espera una respuesta en formato JSON con la acción a ejecutar. La Figura 3.11 esquematiza este flujo de datos.

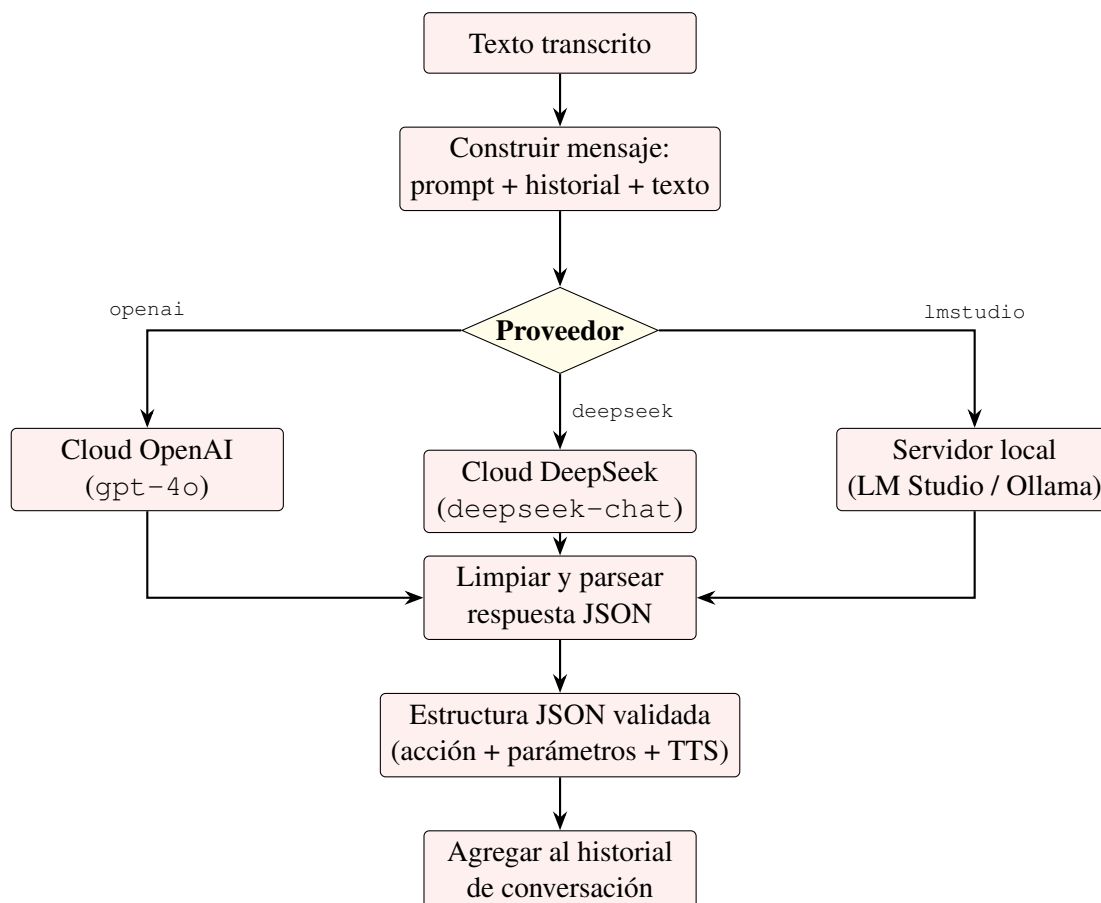


Figura 3.11: Flujo de datos del módulo `CommandParser.pm`.

3.10.1. Esquema JSON de comandos

Cada solicitud del usuario se traduce a una estructura JSON con los campos descritos en la Tabla 3.13:

Tabla 3.13: Esquema JSON de un comando interpretado

Campo	Tipo	Descripción
<code>action</code>	cadena	Acción a ejecutar: <code>create</code> , <code>list</code> , <code>search</code> , <code>edit</code> , <code>move</code> , <code>rename</code> , <code>delete</code> , <code>info</code> , <code>analyze_text</code> , <code>answer</code> , <code>create_project</code> , <code>system_info</code> , <code>web_search</code> o <code>clarify</code> .
<code>target_type</code>	cadena	Tipo de objetivo: <code>file</code> , <code>directory</code> , <code>system</code> o <code>web</code> .
<code>target_path</code>	cadena	Ruta del archivo o directorio objetivo, con soporte para <code>~</code> .
<code>source_path</code>	cadena	Ruta de origen (para operaciones de mover/renombrar).
<code>parameters</code>	objeto	Parámetros adicionales específicos de cada acción (extensión, patrón, contenido, filtros de fecha/tamaño, etc.).
<code>requires_confirmation</code>	booleano	Indica si la operación requiere confirmación explícita del usuario.
<code>tts_response</code>	cadena	Mensaje breve (máximo 12 palabras) para lectura por TTS.

3.10.2. Diseño del *prompt* del sistema

El módulo incluye dos versiones del *prompt* del sistema:

1. `_get_system_prompt()` (~400 líneas): *prompt* completo para modelos grandes (GPT-4o). Incluye la documentación de las 14 acciones, reglas de memoria contextual, expansión de rutas relativas, documentación de parámetros de filtro por fecha y tamaño, creación de archivos con contenido, búsqueda web en 5 plataformas, protocolo de aclaración y aproximadamente 15 ejemplos completos de pares solicitud–JSON.
2. `_get_local_system_prompt()` (~50 líneas): *prompt* compacto para modelos pequeños (DeepSeek-R1 8B en LM Studio), optimizado para caber en una ventana de contexto reducida.

Por su extensión —en particular la versión completa para GPT-4o—, ambos *prompts* se reproducen íntegros en el Apéndice D (Listados D.1 y D.2).

3.10.3. Historial de conversación

El módulo mantiene un historial de conversación como un arreglo de pares `user/assistant`, con un límite máximo de 10 intercambios (20 mensajes). Este historial se

envía al LLM en cada solicitud, lo que permite que el usuario realice preguntas de seguimiento como «¿cuántos de esos son archivos PDF?» sin necesidad de repetir el contexto.

Para enriquecer este historial, tras cada ejecución el orquestador invoca `update_last_result()`, que inyecta las siguientes anotaciones en la última respuesta del asistente:

- `[RUTA_ACTUAL]`: directorio de trabajo actual.
- `[RESULTADO DE EJECUCION]`: estado (éxito/error).
- `[DETALLE: X archivos, Y carpetas]`: resumen cuantitativo.
- `[LISTA: archivov1, archivov2, ...]`: listado de elementos afectados.

3.10.4. Modos de conexión al LLM

El módulo `CommandParser` soporta tres modos de conexión al LLM, seleccionables mediante el campo `provider` en la sección `nlp` de `config.json`: dos en la nube (Cloud OpenAI y Cloud DeepSeek) y uno con servidor local.

Modo 1: Cloud OpenAI (`openai`)

Utiliza la API oficial de OpenAI en `api.openai.com`. Requiere una clave API (`api_key`) activa configurada en `config.json`. El modelo predeterminado es `gpt-4o` con el *prompt* completo del sistema (`_get_system_prompt()`, ~400 líneas). Este es el modo de mayor precisión y el empleado en los experimentos de la primera y la segunda ronda de pruebas de usabilidad (Condición B).

Modo 2: Cloud DeepSeek (`deepseek`)

Utiliza la API oficial de DeepSeek en `api.deepseek.com`. La interfaz HTTP de DeepSeek es compatible con el protocolo de OpenAI (mismos campos `messages`, `model`, `temperature` y formato de respuesta), por lo que la integración se reduce a sustituir la URL base y la clave API. Requiere una clave API activa de DeepSeek configurada en `config.json`; el modelo predeterminado es `deepseek-chat`. Este es el proveedor empleado en la Condición A de la segunda ronda de pruebas de usabilidad.

Modo 3: Servidor local (`lmstudio`)

Utiliza un servidor LLM autoalojado compatible con la API de OpenAI. Las herramientas recomendadas son **LM Studio** (interfaz gráfica, `lmstudio.ai`) u **Ollama** (línea de comandos, `ollama.com`). Para autoalojar un modelo DeepSeek:

1. Instalar LM Studio u Ollama según preferencia.
2. Descargar el modelo deseado (p. ej., `deepseek-r1:8b` en Ollama o `DeepSeek-R1-Distill-Qwen-8B` en LM Studio).
3. Iniciar el servidor local: LM Studio escucha en `localhost:1234`; Ollama en `localhost:11434`.
4. Configurar `config.json`: establecer `mode = "offline"` (o `nlp.provider = "lmstudio"`), `nlp.lmstudio.api_base` con la URL del servidor y `nlp.lmstudio.model` con el identificador del modelo.

Este modo no requiere conexión a internet ni clave API, preservando la privacidad del usuario. La velocidad de inferencia depende del hardware disponible; en equipos con GPU dedicada los tiempos son aceptables para uso interactivo.

Configuración del proveedor desde la GUI

LIMA proporciona una ventana de configuración en su interfaz gráfica —descrita en la Sección 3.14— desde la cual el usuario conmuta entre el modo en línea (OpenAI) y el modo fuera de línea (servidor local), elige el modelo de OpenAI y ajusta los parámetros del servidor local sin necesidad de editar manualmente `config.json`. La selección directa del proveedor activo y la captura de la clave API desde esta ventana —en lugar de definirla en `config.json`— se contemplan como una ampliación futura.

3.10.5. Optimizaciones para el modo fuera de línea

Cuando el proveedor es un servidor local (modo `lmstudio`), el módulo aplica las siguientes adaptaciones para acomodar las limitaciones de los modelos pequeños:

- El historial se limita a los últimos 2 intercambios (en vez de 10) para no exceder la ventana de contexto del modelo 8B.
- El límite de tokens se establece en 2 048 (frente al predeterminado de OpenAI).
- El tiempo de espera HTTP se incrementa a 300 segundos (frente a 60 s para OpenAI), dado que la inferencia local es más lenta.
- Para modelos DeepSeek-R1, si el campo `content` de la respuesta está vacío pero existe `reasoning_content`, el módulo extrae el JSON del contenido de razonamiento mediante expresiones regulares.

3.10.6. Limpieza de la respuesta JSON

La respuesta del LLM puede contener texto adicional antes o después del JSON. El módulo implementa un proceso de limpieza que:

1. Busca la primera llave de apertura { y la última llave de cierre }.
2. Extrae únicamente el contenido entre ambas.
3. Descodifica el JSON resultante con `JSON::decode_json()`.

Adicionalmente, se realiza una manipulación temporal de la localización numérica (`LC_NUMERIC = "C"`) antes de codificar JSON, para garantizar que los números decimales utilicen el punto como separador y no la coma (comportamiento predeterminado en localización española).

3.11. Módulo de gestión de archivos: **FileManager.pm**

Este módulo (~1 120 líneas, 28 subrutinas) es el motor de ejecución del sistema. Implementa todas las operaciones sobre el sistema de archivos —creación, listado, búsqueda, movimiento, renombrado, eliminación, inspección, edición y creación de proyectos— así como consultas de información del sistema y apertura de búsquedas web. La Figura 3.12 presenta la estructura del módulo y sus subrutinas principales.

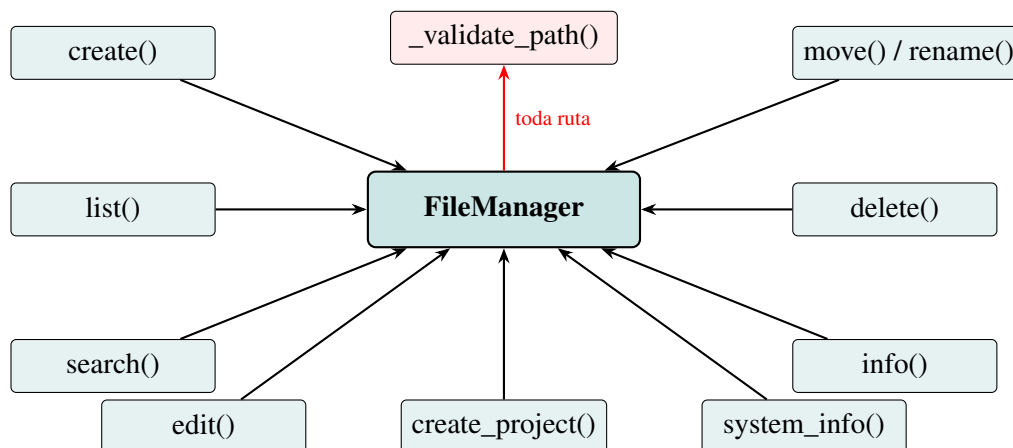


Figura 3.12: Estructura del módulo `FileManager.pm`. Toda operación que involucra rutas pasa primero por `_validate_path()`.

3.11.1. Validación de rutas y seguridad

La subrutina `_validate_path()` es la primera línea de defensa del sistema. Cada ruta proporcionada por el LLM pasa por el siguiente proceso:

1. Expansión de la virgulilla (`~`) y la variable `$HOME` a la ruta absoluta del directorio personal del usuario.
2. Resolución de la ruta mediante `Cwd::abs_path()` para eliminar componentes relativos (`..`, `.`) y enlaces simbólicos.
3. Verificación de que la ruta resuelta se encuentra dentro de al menos uno de los directorios permitidos definidos en `config.paths.allowed_dirs`.
4. Si la ruta no se encuentra en ningún directorio permitido, la función lanza una excepción (`die`).

Los directorios permitidos por defecto son: `$HOME`, `$HOME/Documentos`, `$HOME/Descargas`, `$HOME/Proyectos`, `$HOME/Escritorio`, `$HOME/Desktop`, `$HOME/Música`, `$HOME/Videos` y `$HOME/Imágenes`. Esta lista blanca previene ataques de *path traversal* que podrían originarse si el LLM generara rutas maliciosas como `/etc/passwd o ../../root/`.

3.11.2. Operaciones de creación

La subrutina `create()` maneja tanto la creación de archivos como de directorios:

- **Archivos:** Crea el archivo en la ruta indicada. Si el campo `parameters.content` está presente, escribe ese contenido en el archivo; en caso contrario, crea un archivo vacío. También crea los directorios intermedios si no existen, mediante `File::Path::make_path()`.
- **Directorios:** Crea el directorio y, opcionalmente, una estructura de subdirectorios especificada en `parameters.subdirectories`.

3.11.3. Búsqueda avanzada con filtros

La subrutina `search()` es una de las más complejas del sistema. Utiliza `File::Find::Rule` para recorrer el sistema de archivos y soporta los criterios de filtro enumerados en la Tabla 3.14:

Análisis de fechas en lenguaje natural

La subrutina `_parse_date()` acepta los siguientes formatos de fecha en español:

- **Fechas relativas:** «hoy», «ayer», «hace 3 días».
- **Fechas absolutas con nombres de meses:** «15 de enero», «15 de enero de 2026».
- **Formato numérico:** «DD/MM/YYYY».

Tabla 3.14: Filtros de búsqueda soportados por FileManager

Parámetro	Tipo	Descripción
<code>pattern</code>	cadena	Patrón de nombre (coincidencia parcial, insensible a mayúsculas).
<code>extension</code>	cadena	Extensión de archivo (por ejemplo, <code>.py</code> , <code>.pdf</code>).
<code>date_from</code>	cadena	Fecha mínima de modificación (lenguaje natural en español).
<code>date_to</code>	cadena	Fecha máxima de modificación.
<code>date_exact</code>	cadena	Fecha exacta de modificación (ventana de 24 horas).
<code>size_min</code>	cadena	Tamaño mínimo (por ejemplo, «100MB»).
<code>size_max</code>	cadena	Tamaño máximo.
<code>size_exact</code>	cadena	Tamaño aproximado ($\pm 10\%$ de tolerancia).
<code>content_search</code>	cadena	Texto a buscar dentro del contenido de los archivos.
<code>content_min_count</code>	entero	Número mínimo de ocurrencias que debe tener el texto buscado en el archivo.

Los nombres de los meses en español se mapean a índices numéricos (`enero` $\rightarrow$ 0, `febrero` $\rightarrow$ 1, ..., `diciembre` $\rightarrow$ 11) y se convierten a marcas de tiempo Unix mediante `Time::Local::timelocal()`.

Análisis de tamaños

La subrutina `_parse_size()` convierte cadenas como «100MB» o «1.5GB» a bytes, soportando las unidades B, KB, MB, GB y TB con valores decimales.

Búsqueda flexible con normalización

Si la búsqueda exacta no produce resultados, el módulo activa automáticamente la **búsqueda flexible**: la subrutina `_flexible_search()` recorre nuevamente los archivos aplicando normalización a los nombres (conversión a minúsculas, eliminación de acentos, espacios, guiones y guiones bajos). Esta estrategia compensa las discrepancias entre lo que el usuario dicta verbalmente y los nombres reales de los archivos.

Búsqueda por contenido

La subrutina `_filter_by_content()` abre cada archivo candidato y busca la cadena especificada dentro de su texto. Como medidas de protección, omite archivos binarios (detectados por extensión y, en caso de duda, por `file -mime-type`) y archivos mayores a 50 MB.

3.11.4. Operaciones de eliminación

La subrutina `delete()` soporta dos modos de operación:

1. **Eliminación individual:** Si el objetivo es un archivo, utiliza `unlink()`; si es un directorio, utiliza `File::Path::remove_tree()`.
2. **Eliminación masiva por patrón:** Si se especifica un patrón, busca todos los archivos coincidentes (limitados a `max_file_operations`) y los elimina secuencialmente.

3.11.5. Información del sistema

La subrutina `system_info()` ejecuta comandos del sistema operativo para obtener información en tiempo real. La Tabla 3.15 detalla los comandos disponibles:

Tabla 3.15: Comandos del sistema ejecutados por `system_info()`

Tipo de información	Comando ejecutado
Procesos	<code>ps aux -sort=-%cpu head -15</code>
Memoria	<code>free -h</code>
Disco	<code>df -h</code>
Red	<code>ss -tuln</code>
Puertos	<code>ss -tlnp</code>
General	<code>hostname, uptime -p, uname -r</code>

3.11.6. Creación de proyectos a partir de plantillas

La subrutina `create_project()` lee una plantilla JSON (sección 3.17) y genera automáticamente una estructura de directorios y archivos. Las plantillas definen directorios base, archivos base y *features* (módulos funcionales) que pueden activarse selectivamente.

3.11.7. Búsqueda web

La subrutina `web_search()` construye URLs de búsqueda correctamente codificadas y las abre en el navegador del usuario. Soporta cinco plataformas: Google, YouTube, GitHub, Stack Overflow y Wikipedia. La detección del navegador se realiza probando secuencialmente `firefox`, `google-chrome`, `chromium` y, como último recurso, `xdg-open`.

Como mejora adicional, la subrutina invoca internamente `_fetch_web_context()` antes de abrir el navegador. Esta función auxiliar envía una solicitud HTTP GET a la

URL de búsqueda con un tiempo de espera de 8 segundos y extrae un fragmento de contexto relevante de la respuesta HTML mediante un conjunto de analizadores específicos por plataforma (`_parse_google_context()`, `_parse_youtube_context()`, `_parse_github_context()`, `_parse_wikipedia_context()`, `_parse_stackoverflow_context()`). Si el análisis tiene éxito, el fragmento se incluye en el campo `message` del resultado, permitiendo que el sistema lea en voz alta un adelanto del resultado antes de que el usuario cambie al navegador.

3.11.8. Análisis de frecuencia de palabras

La subrutina `analyze_text($command)` implementa un análisis estadístico de frecuencia léxica sobre archivos de texto. Opera de manera equivalente al *pipeline* clásico de Unix `tr '[:punct:]' ' ' | tr 'A-Z' 'a-z' | sort | uniq -c | sort -rn | head -N`, pero integrado directamente en el sistema como una acción propia (`action = "analyze_text"`).

El flujo de procesamiento es el siguiente:

1. **Validación de seguridad:** La ruta del archivo pasa por `_validate_path()` y se verifica que sea un archivo de texto existente y de tamaño no mayor a 50 MB.
2. **Lectura y normalización:** Se elimina la puntuación de cada línea y se convierte a minúsculas.
3. **Conteo de tokens:** Cada token (subcadena separada por espacios) se acumula en un mapa de frecuencias.
4. **Resultado:** Si el parámetro `word` está presente, retorna solo la frecuencia de esa palabra específica. En caso contrario, ordena el mapa por frecuencia descendente y retorna las `top_n` palabras más frecuentes (por defecto 20) junto con el número total de palabras y de palabras únicas.

3.11.9. Editor de texto integrado

La subrutina `edit()` distingue dos casos: si el objetivo es un archivo de texto, retorna un indicador (`use_internal_editor => 1`) para que la GUI active su editor integrado (un `GtkTextView` modal con carga y guardado UTF-8); si es un directorio o un archivo binario, delega al gestor de archivos del sistema mediante `xdg-open`.

3.12. Módulo de síntesis de voz: **TTS .pm**

Este módulo (200 líneas, 8 subrutinas) gestiona la retroalimentación auditiva del sistema. Soporta dos motores de síntesis de voz: OpenAI TTS (en línea) y eSpeak (fuera de línea). La Figura 3.13 presenta el flujo de degradación automática entre motores.

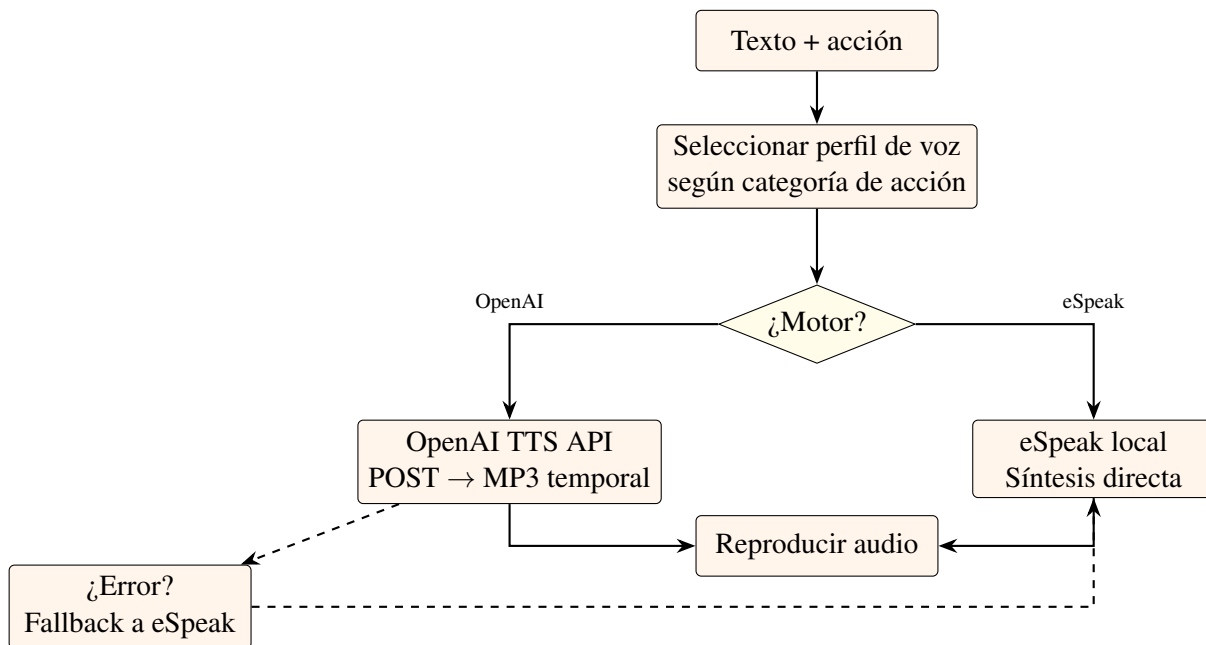


Figura 3.13: Flujo del módulo **TTS .pm**. Si OpenAI TTS falla, el sistema degrada automáticamente a eSpeak.

3.12.1. Perfiles de voz por categoría de acción

El módulo implementa un sistema de perfiles de voz que asigna diferentes voces de OpenAI según la categoría de la acción ejecutada, como se muestra en la Tabla 3.16. Esto permite, por ejemplo, utilizar una voz neutra para operaciones de archivos y una voz con tono diferente para mensajes de error.

3.12.2. Motor OpenAI TTS

La subrutina `_speak_openai()` realiza una solicitud POST al *endpoint* `/v1/audio/speech` de OpenAI con los parámetros: modelo `tts-1`, voz seleccionada según el perfil, formato de salida `mp3`. El audio resultante se escribe en un archivo temporal y se reproduce mediante `mpg123` o `ffplay`.

Tabla 3.16: Categorías de perfiles de voz

Categoría	Acciones asociadas
files	create, list, move, rename, delete, info, edit
search	search
system	system_info
web	web_search
error	Respuestas de error
success	Respuestas de éxito general

Si la llamada a la API falla (error de red, clave inválida, etc.), el módulo degrada automáticamente a eSpeak, garantizando que el usuario siempre reciba retroalimentación auditiva.

3.12.3. Motor eSpeak (fuera de línea)

La subrutina `_speak_espeak()` construye y ejecuta el comando:

```
espeak -v $lang -s $speed -p $pitch -g $gap -a 100 "$text" &
```

Los parámetros de velocidad (`-s`), tono (`-p`) y espacio entre palabras (`-g`) son configurables desde la interfaz de ajustes del sistema.

3.12.4. Método no bloqueante: `speak_async()`

El orquestador llama exclusivamente a `speak_async()` en lugar de `speak()`, dado que el bucle de eventos de GTK3 no debe bloquearse durante la síntesis de audio. La diferencia de implementación es la siguiente:

- **`speak()`**: Ejecuta la síntesis de forma síncrona y bloquea el hilo de la interfaz hasta que el audio finaliza. Se conserva principalmente para uso interno y pruebas.
- **`speak_async()`**: Para el motor eSpeak, lanza el proceso con `nohup . . . &` mediante `system()`, de modo que el audio se reproduce en segundo plano sin bloquear. Para el motor OpenAI, descarga y reproduce el archivo MP3 del mismo modo.

El uso de `speak_async()` garantiza que la interfaz permanezca responsiva durante la retroalimentación auditiva: el usuario puede presionar botones, ver actualizaciones del indicador de estado y recibir nuevos resultados mientras el TTS sigue hablando.

3.13. Módulo de detección de distribución: `DistroDetector.pm`

Este módulo (180 líneas, 9 subrutinas) identifica la distribución GNU/Linux del usuario y proporciona funciones de traducción de paquetes y generación de comandos de instalación. Es un módulo puramente Perl sin dependencias externas. La Figura 3.14 muestra el flujo de detección.

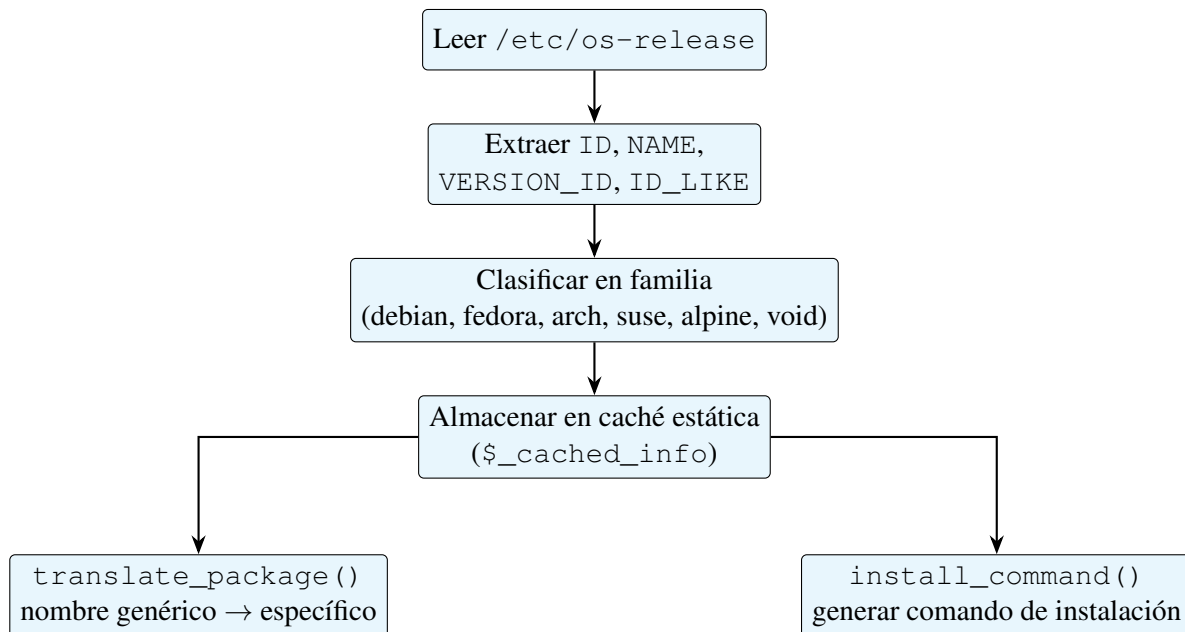


Figura 3.14: Flujo del módulo `DistroDetector.pm`.

3.13.1. Familias de distribuciones soportadas

La Tabla 3.17 enumera las familias reconocidas por el módulo y sus gestores de paquetes correspondientes.

3.13.2. Traducción de paquetes

El módulo mantiene un mapa de traducción de 12 paquetes genéricos (nombrados según la convención Debian) a sus equivalentes en cada familia. Ejemplo: `libgtk3-perl` en Debian corresponde a `perl-gtk3` en Arch y a `perl-Gtk3` en Fedora.

El diseño responde a un problema concreto de la fragmentación de GNU/Linux: una misma biblioteca recibe un nombre distinto en cada distribución, y no existe convención común que permita deducir uno a partir de otro. `libgtk3-perl`, `perl-gtk3` y `perl-Gtk3` designan

Tabla 3.17: Familias de distribuciones GNU/Linux soportadas y sus gestores de paquetes

Familia	Gestor	Distribuciones incluidas
debian	apt-get	Ubuntu, Linux Mint, Pop!_OS, Elementary, Zorin, Kali, Raspbian, MX Linux
fedora	dnf	Fedora, RHEL, CentOS, Rocky Linux, AlmaLinux, Nobara
arch	pacman	Arch Linux, Manjaro, EndeavourOS, Garuda, Artix
suse	zypper	openSUSE, SLES
alpine	apk	Alpine Linux
void	xbps-install	Void Linux

el mismo paquete, pero difieren en el orden de los términos, en el separador y hasta en el uso de mayúsculas. Ninguna transformación mecánica los relaciona; la correspondencia hay que declararla.

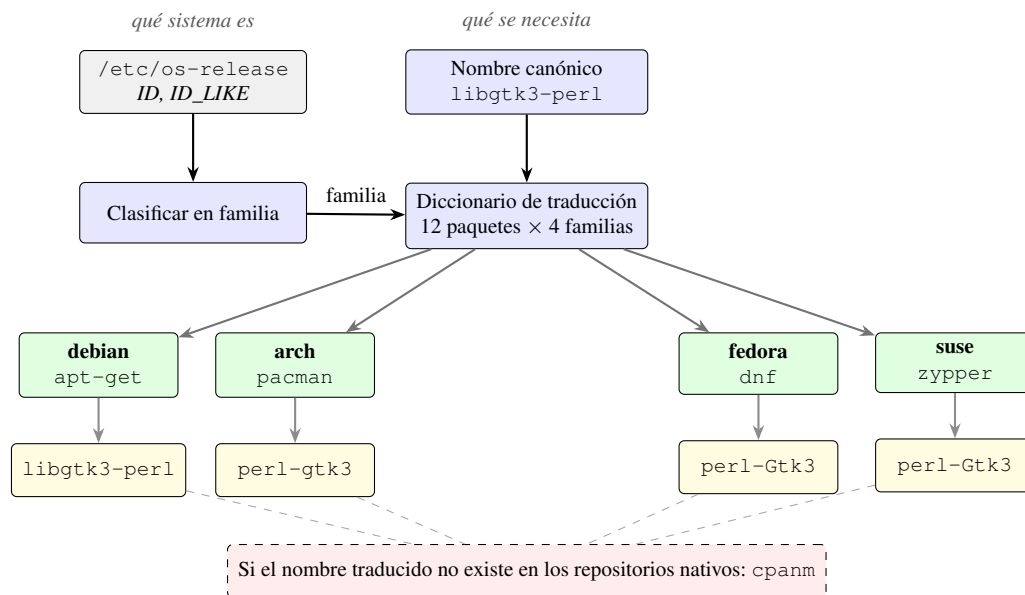
La alternativa ingenua —escribir una rama condicional por distribución en cada punto donde se instale algo— multiplicaría el mismo condicional por todo el instalador y obligaría a tocar decenas de lugares para dar soporte a una familia nueva. La solución adoptada invierte esa relación: el resto del sistema nombra los paquetes en una sola convención, la de Debian, que actúa como **lengua franca**, y un único diccionario traduce ese nombre canónico al de la familia detectada. Añadir una familia es añadir una columna al diccionario, no reescribir el instalador. La Figura 3.15 representa el mecanismo.

3.13.3. Caché estática

La detección se ejecuta una única vez y el resultado se almacena en una variable de paquete (`$_cached_info`). Las invocaciones subsiguientes al constructor retornan inmediatamente el resultado cacheado, evitando lecturas repetidas del sistema de archivos.

3.13.4. Validación empírica del soporte multidistribución

El módulo `DistroDetector.pm` (sección 3.13) y el instalador universal `install.sh` traducen los nombres de paquete y los comandos de instalación a las seis familias de distribuciones de la Tabla 3.17. No obstante, la ejecución sistemática de las pruebas descritas en el Capítulo 4 se realizó únicamente sobre Ubuntu 22.04: el resto de las familias quedaba respaldado por la lógica del módulo, pero no por una comprobación en un entorno real. Para cerrar esa distancia entre la afirmación de diseño y la evidencia, se validó la portabilidad ejecutando el mismo instalador, sin modificación alguna, sobre máquinas virtuales recién instaladas de tres familias adicionales.



Un solo nombre canónico en todo el instalador; el diccionario absorbe la diferencia. Dar soporte a una familia nueva es añadir una columna, no reescribir el instalador.

Figura 3.15: Mecanismo de traducción de paquetes. El instalador nombra cada dependencia una sola vez, en la convención de Debian; el diccionario resuelve el nombre real y el gestor correspondiente a partir de la familia detectada.

El procedimiento fue idéntico en todos los casos y consistió en un único comando —un `git clone` del repositorio seguido de `sudo bash install.sh`—, sin intervención manual posterior. En cada entorno se verificó que el instalador: (i) detectara correctamente la familia a partir de `/etc/os-release`; (ii) seleccionara el gestor de paquetes y los nombres de paquete adecuados; (iii) resolviera las dependencias de Perl, recurriendo a `cpanm` cuando alguna no estuviera disponible en los repositorios nativos; y (iv) dejara operativa la aplicación. Como criterio más exigente se comprobó, además, la instalación de la palabra clave de activación fuera de línea (Vosk y su modelo acústico en español), por ser el componente con más dependencias de sistema. La Tabla 3.18 resume los resultados.

Tabla 3.18: Validación del instalador universal sobre máquinas virtuales limpias

Familia	Distribución	Instalador	Aplicación	Palabra clave
debian	Ubuntu 22.04 (base)	✓	✓	✓
debian	Kali (rolling)	✓	✓	Parcial
fedora	Fedora 44 Workstation	✓	✓	✓
arch	EndeavourOS / Manjaro	✓	✓	✓

El instalador adaptó el entorno de forma autónoma en las cuatro familias y la aplicación quedó operativa en todas ellas. El resultado más destacable se obtuvo en Fedora 44, donde la

instalación completa —traducción del gestor de paquetes, dependencias de Perl y palabra clave de activación fuera de línea— se ejecutó de forma totalmente desatendida, sin un solo paso manual. En Kali la aplicación quedó plenamente funcional y la única tarea que no se completó fue la descarga del modelo acústico fuera de línea, bloqueada por la red del entorno de prueba y no por el instalador; de ahí la marca «Parcial» de la última columna.

Esta ronda de validación no se limitó a confirmar el comportamiento esperado, sino que reveló y motivó varias medidas de robustez que se incorporaron al instalador:

- **Caché de metadatos en Fedora.** En una instalación reciente la caché de `dnf` está fría, por lo que la primera invocación de instalación tardaba varios minutos y podía abortar el lote de paquetes. Se antepuso una llamada explícita a `dnf makecache`, con lo que la instalación se resuelve a la primera y por la vía nativa, sin recurrir innecesariamente a `cpanm`.
- **Rutas de usuario bajo sudo.** Al ejecutarse el instalador con privilegios de superusuario, la variable `HOME` apunta a `/root`, mientras que la aplicación se ejecuta como el usuario real. El entorno virtual de Vosk, el modelo acústico y el archivo de configuración se crean por tanto en el directorio personal del usuario invocador —obtenido de `SUDO_USER`— y se restituye su propiedad con `chown`, de modo que la aplicación los localice al arrancar.
- **Instalación por lotes con reintento individual.** Algunos *bindings* de Perl no existen con el mismo nombre —o no existen en absoluto— en los repositorios de todas las familias. El instalador intenta primero el lote completo y, si falla, reintenta paquete por paquete, instalando los disponibles y delegando el resto en `cpanm` como red de seguridad.
- **Soporte HTTPS de Perl.** El módulo `LWP::Protocol::https`, imprescindible para las conexiones cifradas con las APIs remotas, no forma parte de los paquetes base en varias distribuciones; se añadió de forma explícita a la lista de dependencias de cada familia.

Con estas comprobaciones, la afirmación «LIMA soporta seis familias de distribuciones» deja de ser una propiedad de diseño para convertirse en un resultado verificado empíricamente sobre cuatro de esas seis familias, cubriendo los tres gestores de paquetes de mayor difusión (`apt`, `dnf` y `pacman`). La ejecución de la batería completa de tareas en cada familia —más allá de la puesta en marcha del sistema— queda planteada como trabajo futuro y se recoge entre las limitaciones del Capítulo 6.

3.14. Interfaz gráfica de usuario: GUI_GTK3.pm

Este módulo (~2 800 líneas, ~50 subrutinas) implementa la interfaz gráfica completa del sistema utilizando el *toolkit* GTK3 con *bindings* Perl. Es el módulo más extenso del sistema y gestiona todos los aspectos de la presentación visual y la interacción con el usuario. La Figura 3.16 ilustra la estructura jerárquica de la interfaz.

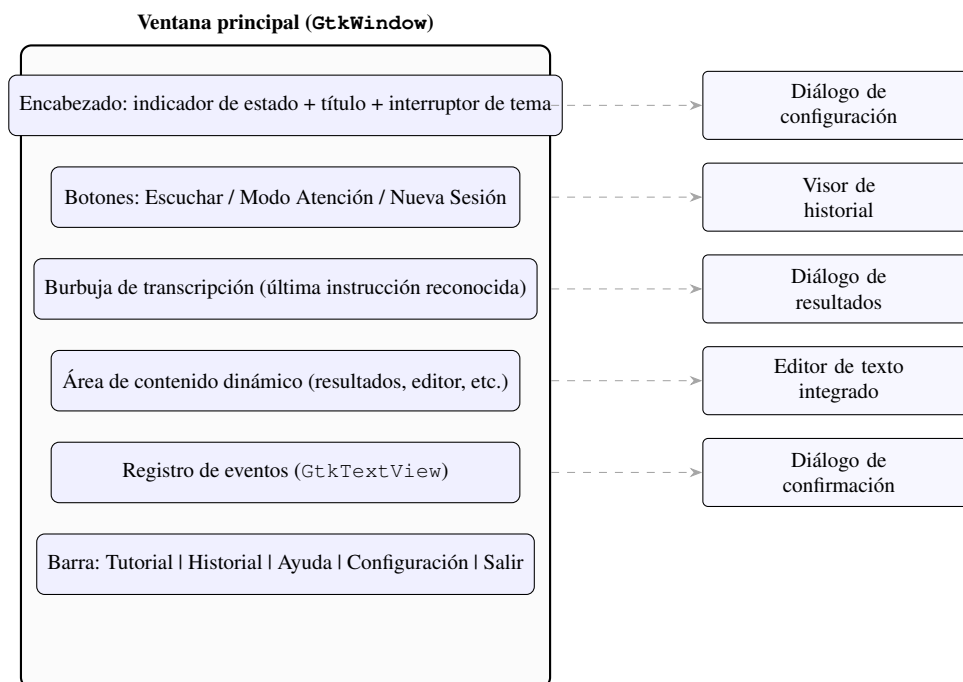


Figura 3.16: Estructura de la interfaz gráfica GUI_GTK3.pm. Las flechas punteadas indican diálogos modales que se abren desde la ventana principal.

3.14.1. Sistema de temas con CSS

La interfaz soporta dos temas visuales —claro y oscuro— implementados mediante hojas de estilo CSS aplicadas a través de `Gtk3::CssProvider` a nivel de pantalla completa. Cada tema define aproximadamente 140 líneas de CSS que estilizan todos los componentes: botones, etiquetas, áreas de texto, barras de desplazamiento, diálogos y el indicador de estado. El usuario puede alternar entre temas mediante un interruptor en el encabezado de la ventana.

3.14.2. Indicador visual de estado con animación

El indicador de estado es un `GtkDrawingArea` personalizado que utiliza Cairo para renderizar un círculo coloreado. La Tabla 3.19 describe los colores y sus significados:

Tabla 3.19: Estados del indicador visual de LIMA

Color	Estado	Significado
Gris (estático)	Inactivo	El sistema espera; detector de palabra clave activo en segundo plano.
Azul (pulsante)	Escuchando	Palabra clave detectada; el modo de atención está activo.
Rojo (estático)	Grabando	Capturando audio del micrófono para transcripción.
Naranja (pulsante)	Procesando	El comando está siendo interpretado o ejecutado.

La animación de pulsación se implementa mediante un temporizador `Glib::Timeout` de 600 ms que alterna la opacidad del círculo entre 1.0 y 0.3, creando un efecto de «latido» visual.

3.14.3. Retroalimentación sonora

La interfaz reproduce sonidos de inicio (`start.mp3`), detención (`stop.mp3`) y error (`error.mp3`) mediante el comando `play` de SoX, con velocidad duplicada y volumen al 75% para que sean breves y no invasivos. El método privado `_play_sound($type)` gestiona la reproducción interna, mientras que `play_error_sound()` es una interfaz pública que el orquestador puede invocar directamente para señalar errores de ejecución sin acceso a los métodos privados del módulo.

3.14.4. Detección de micrófonos

La subrutina `_detect_microphones()` analiza la salida de `arecord -l` para enumerar los dispositivos de captura ALSA disponibles. El análisis soporta tanto la salida en inglés (`card, device`) como en español (`tarjeta, dispositivo`). Para cada dispositivo encontrado, genera las variantes `hw:X,Y` (acceso directo) y `plughw:X,Y` (con conversión automática de formato).

3.14.5. Diálogo de configuración

El diálogo de configuración (`_show_settings()`) permite ajustar los siguientes parámetros:

- **Micrófono:** Selección del dispositivo ALSA, botón de prueba (graba 2 segundos y verifica >500 bytes) y botón de actualización.

- **Motor TTS:** Selección entre OpenAI y eSpeak, con configuración de voz, velocidad, tono y espacio entre palabras para eSpeak.
- **Modo:** Conmutación entre modo en línea (OpenAI) y fuera de línea (Vosk + LM Studio + eSpeak).
- **Modelo GPT:** Selector del modelo de OpenAI.
- **LM Studio:** URL del servidor y nombre del modelo local.
- **Duración máxima de grabación:** Control numérico.

3.14.6. Tutorial interactivo

El módulo incluye un tutorial de 7 pasos que guía al usuario a través de las capacidades del sistema:

1. Bienvenida y presentación.
2. Capacidades generales.
3. Gestión de archivos.
4. Búsqueda avanzada.
5. Información del sistema.
6. Búsqueda en internet.
7. Modo de atención (palabra clave).

Cada paso incluye narración por TTS, lo que permite al usuario escuchar la explicación mientras observa la interfaz.

3.14.7. Visor de historial de peticiones

La subrutina `_show_request_history()` muestra un diálogo con un `GtkTreeView` que lista todas las interacciones registradas. Incluye columnas ordenables (fecha, entrada, intención, resultado, fuente), un campo de búsqueda, un panel de detalle y estadísticas agregadas: total de peticiones, peticiones locales vs. LLM, tasa de éxito/error y porcentaje de resolución local.

3.14.8. Diálogo de confirmación

El diálogo de confirmación (`show_confirmation_dialog()`) se activa para operaciones que requieren aprobación del usuario (por ejemplo, eliminación de archivos). Muestra una descripción legible de la acción generada por `_get_action_description()`, que traduce la estructura JSON a texto en español.

Para la acción especial `clarify`, el diálogo muestra la pregunta del sistema y, al aceptar, reactiva automáticamente el micrófono para que el usuario pueda responder verbalmente.

3.14.9. Editor de texto integrado

La subrutina `show_text_editor()` presenta un diálogo modal con un `GtkTextView` que permite editar archivos de texto directamente dentro de LIMA. El archivo se carga como UTF-8 y se guarda al confirmar.

3.14.10. Visor de resultados de búsqueda enriquecido

La subrutina `show_search_results()` presenta los resultados de búsqueda en un diálogo modal con clasificación tipográfica de archivos, iconos por categoría y botones de acción contextual por elemento. El sistema de visualización se apoya en un conjunto de subrutinas auxiliares organizadas en tres grupos:

Clasificación y presentación de archivos

- **`_get_file_category($path)`**: Clasifica un archivo en una de ocho categorías (audio, video, imagen, documento, pdf, archivo comprimido, código, texto) mediante su extensión. Los directorios se clasifican como categoría `directory`.
- **`_get_category_icon($category)`**: Retorna el emoji correspondiente a cada categoría (p.ej., un ícono de nota musical para audio, de claqueta para video, de monitor para código).
- **`_get_file_extension_label($path)`**: Retorna la extensión del archivo en mayúsculas para mostrar junto al nombre.
- **`_format_size($bytes)`**: Convierte un tamaño en bytes a representación legible (B, KB, MB, GB, TB) con una cifra decimal.
- **`_sort_search_items($items, $sort_by, $ascending)`**: Ordena los resultados por nombre, tipo o tamaño en orden ascendente o descendente.

Botones de acción por elemento

La subrutina `_create_action_buttons_for_item()` genera un conjunto de botones contextuales por cada resultado en función de su categoría:

- **Directorios**: Botón «Abrir» con `xdg-open`.
- **Audio/Video**: Botones «Reproducir» y «VLC» (si está instalado).
- **Imágenes**: Botones «Ver» y «Editar con GIMP» (si está disponible).

- **Documentos:** Botón «LibreOffice».
- **PDF:** Botones «Navegador» y «Evince» (si está instalado).
- **Archivos comprimidos:** Botones «Abrir» y «Extraer aquí» con soporte para zip, tar.gz, tar.bz2, tar.xz, 7z y rar.
- **Texto/Código:** Botones «Editor integrado» y «Editor externo».
- **Todos:** Botón universal «Abrir carpeta contenedora».

La disponibilidad de aplicaciones externas (VLC, GIMP, Evince, 7z, unrar) se verifica en tiempo de ejecución mediante `_command_exists()`, que comprueba la presencia del ejecutable con `which`.

Dimensionado adaptable

La subrutina `_clamp_size($w, $h)` ajusta las dimensiones de diálogos y ventanas para que nunca excedan el 92% del ancho ni el 85% del alto de la pantalla disponible. Esto garantiza que los diálogos de resultados y el editor de texto sean siempre completamente visibles, independientemente de la resolución del monitor.

3.15. Sistema de seguridad multicapa

La seguridad es una preocupación transversal en LIMA, ya que el sistema ejecuta operaciones sobre el sistema de archivos a partir de instrucciones generadas por un LLM, lo que introduce un riesgo inherente. El diseño implementa múltiples capas de protección, ninguna de las cuales depende del comportamiento del LLM, tal como se esquematiza en la Figura 3.17.

1. **Saneamiento del entorno:** El bloque `BEGIN` de `lima.pl` elimina las variables `LD_LIBRARY_PATH` (si contiene `/snap/`) y `LD_PRELOAD` para prevenir la carga de bibliotecas no autorizadas.
2. **Validación de rutas:** La función `_validate_path()` de `FileManager` verifica que toda ruta resuelta pertenezca a un directorio de la lista blanca. Las rutas fuera de este perímetro provocan una excepción inmediata. Esto previene ataques de *path traversal* (`../../../../etc/passwd`).
3. **Clasificación de riesgo:** Los comandos se clasifican en tres niveles —*seguro*, *advertencia* y *peligro*— mediante reglas deterministas basadas en el tipo de acción y los parámetros, sin confiar en la evaluación del LLM.
4. **Confirmación explícita:** Las operaciones clasificadas como *advertencia* o *peligro* requieren que el usuario confirme su ejecución mediante un diálogo modal en la GUI. La opción `confirm_destructive` en `config.json` controla este comportamiento.

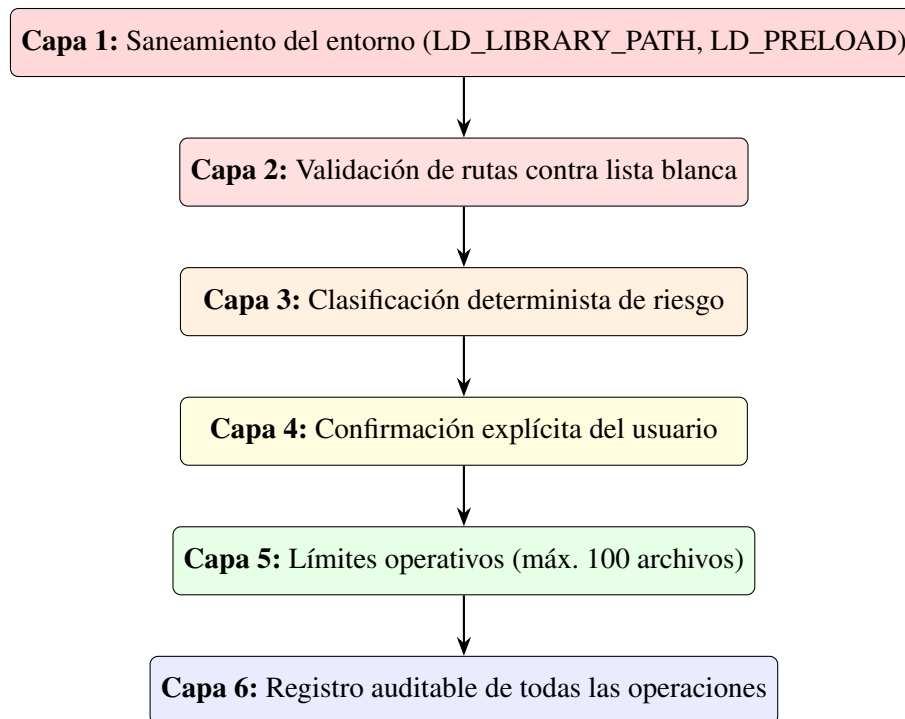


Figura 3.17: Capas del sistema de seguridad de LIMA. Cada capa opera independientemente del LLM.

5. **Límites operativos:** El parámetro `max_file_operations` (100) limita el número de archivos que pueden verse afectados por una sola instrucción, previniendo operaciones masivas accidentales.
6. **Registro auditable:** Cada interacción —exitosa o fallida— se registra en `logs/request_history.log` con marca temporal, texto original, intención detectada, resultado y fuente (local/LLM/error).

3.16. Configuración del sistema

El comportamiento de LIMA se controla mediante un archivo JSON centralizado (`config/config.json`, 88 líneas) que se carga al iniciar el sistema y se pasa a todos los módulos. La Tabla 3.20 resume las secciones relativas a la conexión con el LLM, las rutas y la seguridad, y la Tabla 3.21, las relativas al audio, la síntesis de voz, la interfaz y el aprendizaje. Los valores citados corresponden al archivo de configuración vigente; las secciones `proxy` y `chat` se incorporaron con posterioridad a la evaluación de usabilidad y se describen en detalle en la Sección 3.21.

Tabla 3.20: Secciones de `config.json`: conexión, rutas y seguridad

Sección	Contenido
<code>mode</code>	Modo de operación: "online" (proveedor en la nube) u "offline" (servidor local).
<code>proxy</code>	Encaminamiento a través del proxy en el borde: habilitación (<code>enabled</code>), URL base del <i>Worker</i> (<code>api_base</code>) y código de acceso del usuario (<code>access_code</code>). Cuando está activo, sustituye en tiempo de arranque la URL y la credencial de <code>openai</code> , de modo que los tres módulos que consumen APIs remotas lo usan sin modificación (Sección 3.21.3).
<code>openai</code>	Clave API de OpenAI (<code>api_key</code>), modelo de transcripción (<code>gpt-4o-transcribe</code>), modelo GPT (<code>gpt-4o</code>), URL base. Con el proxy activo, <code>api_key</code> queda vacía.
<code>deepseek</code>	Clave API de DeepSeek (<code>api_key</code>), modelo (<code>deepseek-chat</code>), URL base (<code>api.deepseek.com</code>).
<code>nlp</code>	Proveedor activo del LLM (<code>provider</code>): <code>openai</code> (GPT-4o), <code>deepseek</code> (DeepSeek API) o <code>lmstudio</code> (servidor local); proveedor de nube por defecto (<code>cloud_provider</code> : <code>openai</code> o <code>deepseek</code>); y parámetros del servidor local (<code>lmstudio</code>): URL, modelo, temperatura.
<code>paths</code>	Directorio <code>home</code> , lista de directorios permitidos (9 rutas), directorio de audio temporal.
<code>security</code>	Confirmación de operaciones destructivas, modo <i>dry run</i> , máximo de operaciones de archivo.

3.17. Plantillas de proyectos

LIMA incluye tres plantillas JSON que permiten generar automáticamente la estructura de directorios y archivos de proyectos de desarrollo. La Tabla 3.22 describe las plantillas disponibles:

Cada plantilla define tres secciones: `directories` (directorios base), `base_files` (archivos que siempre se crean) y `features` (módulos funcionales con sus propios archivos). El usuario solicita la creación verbalmente —por ejemplo, «crea un proyecto fullstack en mi escritorio»— y LIMA genera toda la estructura con un solo comando.

3.18. Flujo completo de una interacción

Para sintetizar las interacciones entre todos los módulos, la Figura 3.18 muestra el flujo de datos completo desde la activación del usuario hasta la retroalimentación final.

Los pasos del flujo completo son:

1. **Activación:** El usuario pronuncia la palabra clave «LIMA» (detectada por

Tabla 3.21: Secciones de `config.json`: audio, voz, interfaz y aprendizaje

Sección	Contenido
audio	Formato (WAV), frecuencia de muestreo (16 000 Hz), canales (1), duración máxima (90 s), dispositivo.
tts	Motor (openai o espeak), idioma, velocidad, tono, espacio entre palabras, voz de OpenAI, modelo TTS, perfiles de voz (6 categorías).
wake_word	Habilitación, palabra clave, umbral de silencio (0.01), duración de silencio (5 s), duración máxima (0, es decir, sin límite: el turno lo cierra únicamente el silencio) y ruta del modelo Vosk. Los tres parámetros temporales son los que gobiernan el cierre de turno de la interacción continua y siguen sujetos a ajuste empírico (Sección 3.21.2).
gui chat	Dimensiones iniciales de la ventana (800×600), tema visual. Habilitación del diálogo conversacional, profundidad del historial que se envía al LLM (<code>max_history</code> : 8 turnos), estado inicial del interruptor de ejecución (<code>unlock_default</code> : desactivado) y minimización a la bandeja del sistema al activarlo.
learning	Habilitación, umbral de confianza (0.60), aprendizaje automático, máximo de patrones (500), ruta del archivo de patrones (<code>~/lima/learned_patterns.json</code>), lista de acciones aprendibles (12).

Tabla 3.22: Plantillas de proyectos incluidas en LIMA

Plantilla	Nombre	Dirs.	Descripción
<code>fullstack.json</code>	Fullstack - Aplicación Completa	14	<i>Frontend + backend + directorio compartido + Docker.</i> Características: autenticación, panel de control, administración.
<code>backend_api.json</code>	Backend - API REST	11	Estructura Node.js/Express con controladores, modelos, rutas, <i>middleware</i> , servicios y pruebas unitarias e integración.
<code>frontend_pos.json</code>	Frontend - Punto de Venta	14	Aplicación React con componentes para inicio de sesión, inventario, ventas, reportes y usuarios.

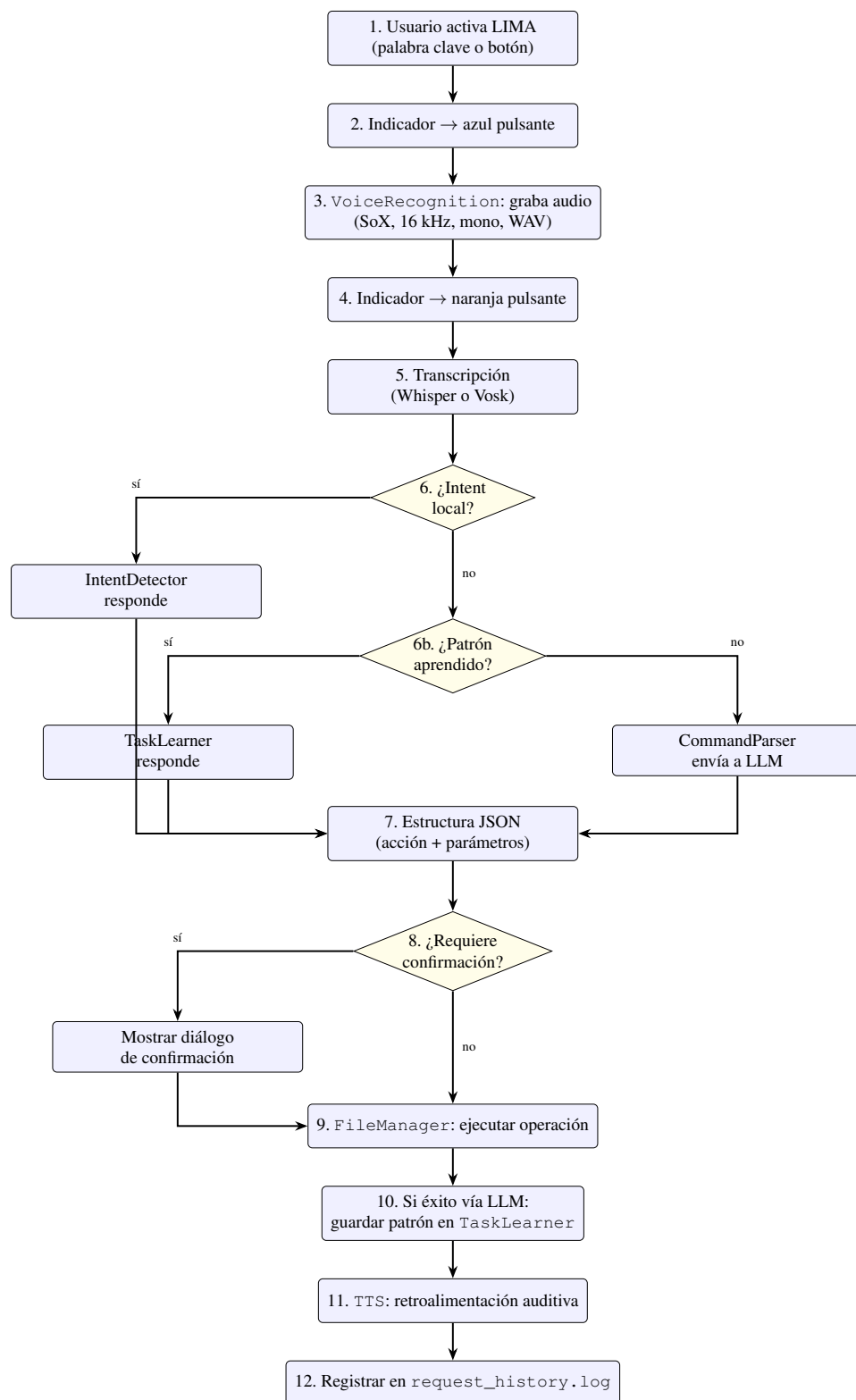


Figura 3.18: Flujo completo de una interacción con LIMA. El paso 6 muestra el *pipeline* de tres niveles: detector local (IntentDetector), patrones aprendidos (TaskLearner) y LLM (CommandParser). El paso 10 cierra el ciclo de aprendizaje para solicitudes resueltas por el LLM.

WakeWordDetector mediante Vosk) o presiona el botón de micrófono en la GUI.

2. **Indicador de escucha:** El indicador visual cambia a azul pulsante, informando al usuario que el sistema está atento.
3. **Grabación:** VoiceRecognition bifurca un proceso `rec` (SoX) que captura audio a 16 kHz, mono, formato WAV.
4. **Indicador de procesamiento:** Tras detener la grabación, el indicador cambia a naranja pulsante.
5. **Transcripción:** El audio se envía a OpenAI Whisper (en línea) o se procesa con Vosk (fuera de línea) para obtener el texto.
6. **Interpretación de tres niveles (Nivel 1):** IntentDetector evalúa el texto mediante expresiones regulares. Si hay coincidencia, la solicitud se resuelve localmente en < 1 ms.
Nivel 2: Si el nivel 1 no produce coincidencia, TaskLearner busca el patrón más similar en la base de conocimiento aprendida usando TF-IDF con similitud coseno. Si la confianza supera 0.60, reconstituye el comando extrayendo las variables del texto actual.
Nivel 3: Si ambos niveles anteriores fallan, CommandParser envía la solicitud al LLM para su interpretación semántica.
7. **Estructura JSON:** Independientemente del origen, el resultado es un JSON con acción, parámetros, nivel de riesgo y mensaje TTS.
8. **Confirmación condicional:** Si la operación es destructiva (`requires_confirmation = true`), se muestra un diálogo modal al usuario.
9. **Ejecución:** FileManager ejecuta la operación tras validar las rutas.
10. **Aprendizaje:** Si la tarea fue resuelta por el LLM y su ejecución fue exitosa, TaskLearner->learn() almacena el patrón, cerrando el ciclo de aprendizaje.
11. **Retroalimentación:** TTS reproduce la respuesta auditiva; la GUI muestra el resultado visual.
12. **Registro:** La interacción completa se escribe en `logs/request_history.log`.

3.19. Modo en línea y modo fuera de línea

LIMA soporta dos modos de operación que pueden conmutarse en tiempo de ejecución desde la interfaz de configuración, como se compara en la Tabla 3.23:

La conmutación se propaga a los tres módulos afectados (VoiceRecognition, CommandParser y TTS) mediante el *callback* `on_mode_change` del orquestador, sin necesidad de reiniciar la aplicación.

Tabla 3.23: Comparación de los modos de operación de LIMA

Componente	Modo en línea	Modo fuera de línea
Transcripción	OpenAI Whisper (<i>cloud</i>)	Vosk (local, modelo español)
Interpretación	GPT-4o (OpenAI API)	DeepSeek-R1 8B (LM Studio local)
Síntesis de voz	OpenAI TTS (<i>cloud</i>)	eSpeak (local)
Latencia	~2–5 s (dependiente de red)	~5–30 s (dependiente de hardware)
Costo	Consumo de tokens de API	Sin costo monetario
Privacidad	Audio enviado a servidores de OpenAI	Todo el procesamiento es local
Precisión	Alta	Variable (menor en modelos 8B)

3.20. Dependencias externas del sistema

Finalmente, la Tabla 3.24 resume todas las dependencias externas de LIMA, tanto a nivel de bibliotecas Perl como de programas del sistema operativo.

Con la descripción de estas dependencias concluye la exposición del sistema tal como fue evaluado. El artefacto resultante —sus módulos, su *pipeline* de interpretación de tres niveles, sus mecanismos de seguridad multicapa y su configuración— constituye aquel cuyo desempeño y usabilidad se valoran en el Capítulo 4. La sección siguiente documenta tres capacidades incorporadas con posterioridad al cierre de esa evaluación.

3.21. Evolución posterior a la evaluación

Las secciones anteriores describen LIMA en el estado en que fue sometido a las pruebas locales sobre el conjunto de datos y a la segunda ronda de pruebas de usabilidad. El desarrollo, sin embargo, no se detuvo al cerrar la recolección de datos: entre esa fecha y la redacción final de esta memoria se incorporaron tres capacidades que modifican de manera sustantiva la forma en que el sistema se distribuye, se paga y conversa con el usuario.

Se documentan aquí, en una sección propia y claramente delimitada, por una razón metodológica: **ninguna de las tres fue sometida a la evaluación reportada en los Capítulos 4 y 5**. Presentarlas junto al resto del diseño podría inducir a atribuirles un respaldo empírico que no poseen. Las métricas de eficacia, eficiencia y satisfacción reportadas más adelante corresponden exclusivamente al sistema descrito en las Secciones 3.4 a 3.20. La Tabla 3.25 sitúa cada capacidad respecto de ese corte.

Tabla 3.24: Dependencias externas de LIMA

Categoría	Dependencia	Uso
Módulos Perl (CPAN)	JSON	Codificación y decodificación de estructuras JSON.
	LWP::UserAgent	Cliente HTTP para las APIs de OpenAI y LM Studio.
	File::Slurp	Lectura de archivos completos en una sola operación.
	File::Find::Rule	Búsqueda recursiva de archivos con filtros.
	Gtk3	<i>Bindings</i> Perl para el <i>toolkit</i> gráfico GTK3.
	Glib	Bucle de eventos, temporizadores y constantes booleanas.
Programas del SO	sox (rec, play)	Grabación y reproducción de audio.
	espeak	Síntesis de voz fuera de línea.
	mpg123 / ffmpeg	Reproducción de archivos MP3 (TTS de OpenAI).
	arecord	Enumeración de dispositivos de captura ALSA.
	xdg-open	Apertura de archivos y URLs con la aplicación predeterminada.
Python 3	vosk	Reconocimiento de voz fuera de línea (transcripción y palabra clave).
	wave, struct	Manipulación de archivos de audio WAV.
APIs externas	OpenAI Whisper API	Transcripción de audio a texto (modo en línea).
	OpenAI Chat Completions API	Interpretación de comandos mediante GPT-4o (modo en línea).
	OpenAI TTS API	Síntesis de voz de alta calidad (modo en línea).

Tabla 3.25: Estado de las capacidades del sistema respecto del corte de la evaluación

Capacidad	Evaluada	Actual	Observaciones
Detección de distribución (<code>DistroDetector.pm</code>)	✓	✓	Presente durante la evaluación; su validación empírica en cuatro familias (Sección 3.13.4) es posterior.
Instalador multidistribución desatendido	Parcial	✓	Durante la evaluación solo se ejercitó sobre Debian y derivadas.
Versión portable y empaquetado <code>.deb</code>	—	✓	Sección 3.21.1.
Interacción conversacional continua	Parcial	Parcial	El encadenamiento de aclaraciones ya existía; el cierre de turno por silencio adaptativo es posterior y sigue en ajuste. Sección 3.21.2.
Códigos de acceso y proxy en el borde	—	✓	Sección 3.21.3. Durante la evaluación cada equipo de prueba usaba una clave API propia.
Planificación de tareas extensas	—	—	No implementada; su diseño prospectivo se expone en la Sección 3.22.

3.21.1. Distribución del sistema: instalador multidistribución y versión portable

El módulo `DistroDetector.pm` (Sección 3.13) resuelve el problema de *reconocer* el entorno, pero reconocerlo no basta: para que un usuario sin conocimientos de administración pueda instalar LIMA, la detección debe estar al servicio de un procedimiento de instalación que no le pida ninguna decisión técnica. Esa es la función del instalador `install.sh` y de la versión portable.

Justificación del calendario. El soporte multidistribución figuraba entre los objetivos del proyecto desde febrero de 2026, pero su implementación se pospuso deliberadamente hasta julio. La razón es de orden arquitectónico y merece explicitarse, porque ilustra una dependencia real del desarrollo incremental: un paquete portable es, por definición, una fotografía congelada del árbol de módulos, de sus dependencias externas y de la estructura del archivo de configuración. Mientras esos tres elementos siguieran cambiando —y cambiaron de forma sustancial: el módulo `AsyncHTTP.pm` apareció a mitad del desarrollo, el número de dependencias de sistema creció con la incorporación del TTS de OpenAI y el esquema de configuración se reorganizó dos veces—, cualquier paquete construido habría quedado obsoleto antes de poder probarse. Empaquetar prematuramente habría significado mantener en paralelo dos artefactos divergentes: el repositorio de trabajo y una versión distribuable perpetuamente desactualizada. Se optó, por tanto, por congelar primero la arquitectura, validarla con usuarios, y solo entonces construir

la distribución.

El instalador universal. `install.sh` implementa un procedimiento de un solo paso —`git clone` seguido de `sudo bash install.sh`— que encadena las siguientes etapas: (i) consulta a `DistroDetector.pm` para clasificar el sistema en una de las seis familias soportadas; (ii) selecciona el gestor de paquetes correspondiente y traduce los nombres genéricos de las dependencias mediante el mapa descrito en la Sección 3.13; (iii) instala las dependencias por la vía nativa y, solo cuando alguna no está disponible en los repositorios de la distribución, recurre a `cpanm` como mecanismo de respaldo; (iv) genera un `config/config.json` inicial a partir de la plantilla de ejemplo; y (v) registra la aplicación en el menú del escritorio mediante un archivo `.desktop`. Un instalador auxiliar, `install_wakeword.sh`, atiende por separado la instalación de Vosk y de su modelo acústico en español, por ser el componente con más dependencias de sistema y el único que requiere descarga de datos.

La versión portable. El directorio `Portable/` contiene un árbol autocontenido —orquestador, los diez módulos, plantillas de proyectos, recursos de sonido y configuración— acompañado de `build_deb.sh`, que lo empaqueta como `lima_2.0.0_all.deb`. La elección de un paquete *de arquitectura independiente* (`all`) no es accidental: al estar el sistema escrito íntegramente en Perl 5 y Python 3, ambos interpretados, no hay binarios que compilar para cada arquitectura, de modo que un único paquete sirve tanto para `amd64` como para `arm64`. El guion `packaging/build_package.sh` generaliza esta construcción a los formatos `.deb` y `.rpm`, cubriendo las dos familias de mayor cuota de instalación.

La relación entre ambos mecanismos es de complementariedad, no de redundancia. El instalador desde el repositorio está pensado para quien quiere seguir el desarrollo o modificar el sistema; el paquete portable, para quien solo quiere ejecutarlo. El primero resuelve las dependencias en el momento de la instalación y se adapta a lo que encuentre; el segundo las declara en sus metadatos y delega su resolución al gestor de paquetes del sistema. Sostener las dos vías obliga a mantener una única lista canónica de dependencias —la del mapa de `DistroDetector.pm`— de la que ambos derivan, lo que evita que se desincronicen.

3.21.2. Interacción conversacional continua

El Modo Atención descrito en la Sección 3.7 resuelve la activación manos libres, pero por sí solo no produce una *conversación*: cada petición es un episodio aislado que empieza con la palabra de activación y termina con la ejecución. Un diálogo real exige dos cosas más: que el sistema sepa cuándo el interlocutor ha terminado de hablar sin que este tenga que anunciarlo,

y que pueda encadenar turnos cuando una tarea no se resuelve en uno solo. Esta subsección documenta ambos mecanismos.

Cierre de turno por detección de pausa

El problema es el siguiente: una vez detectada la palabra de activación, el sistema empieza a grabar, pero nada le indica cuándo detenerse. Las dos soluciones triviales son insatisfactorias. Fijar una duración constante obliga al usuario a rellenar el silencio o le corta la frase a la mitad. Exigir una pulsación para terminar destruye la interacción manos libres, que es precisamente lo que el Modo Atención pretende ofrecer.

La solución adoptada es un detector de actividad vocal por energía. El proceso Python embebido en `WakeWordDetector.pm` lee el flujo de audio en fragmentos de 4 000 muestras —aproximadamente 0.25 s a 16 kHz— y calcula para cada uno la raíz cuadrática media (RMS) normalizada al intervalo $[0, 1]$:

$$\text{RMS} = \frac{1}{32768} \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (3.1)$$

donde x_i son las muestras del fragmento en formato entero de 16 bits con signo y N su número. Un fragmento cuyo RMS cae por debajo del umbral `SILENCE_THRESHOLD` se considera silencio; cuando el silencio se sostiene durante `SILENCE_DURATION` segundos, el turno se cierra, el audio acumulado se escribe en un archivo WAV y el proceso lo notifica al orquestador. El parámetro `MAX_DURATION` vale 0 en la configuración vigente, lo que desactiva el corte por duración: el único criterio de cierre es la pausa del interlocutor, de modo que una petición larga no se trunca por haber excedido un cronómetro.

Calibración adaptativa del umbral

Un umbral fijo funciona en el laboratorio y falla en cualquier otra parte. El ruido de fondo de una oficina, un ventilador o una conversación lejana pueden situar el nivel ambiente por encima del umbral configurado; en ese caso ningún fragmento se clasifica nunca como silencio y la grabación no termina jamás. Para evitarlo, el detector mide el ruido ambiente durante sus primeros 0.75 s de vida y eleva el umbral a tres veces el piso de ruido observado.

Esta corrección, sin embargo, introdujo un fallo simétrico y más dañino, que conviene documentar porque motivó la última pieza del algoritmo. Si durante esos 0.75 s de calibración sonaba algo fuerte —en particular, el propio anuncio por voz del asistente saliendo por el altavoz—, el piso de ruido medido resultaba altísimo y el umbral adaptado quedaba *por encima del volumen de la voz del usuario*. El efecto era que todo se clasificaba como silencio: la grabación se cortaba

a los `SILENCE_DURATION` segundos y las peticiones llegaban truncadas a mitad de frase. Se introdujeron tres salvaguardas:

1. **Techo de calibración.** El umbral adaptado nunca puede superar cuatro veces el valor configurado, por muy alto que resulte el piso de ruido medido. Acota el daño de una calibración contaminada.
2. **Guarda de voz observada.** El turno solo se cierra por silencio si antes se registró al menos un fragmento por encima del umbral. Si el umbral quedara mal calibrado, cerrar en ese punto truncaría la petición; en su lugar el sistema aguanta hasta un temporizador de respaldo (`NO_SPEECH_TIMEOUT`), que evita grabar indefinidamente cuando el usuario de verdad no habla.
3. **Desplazamiento del anuncio.** El aviso hablado de «Modo Atención activado» se emite al recibir el evento `READY` del detector, es decir, *después* de la ventana de calibración, y no al pulsar el conmutador. Así el detector no se calibra oyéndose a sí mismo.

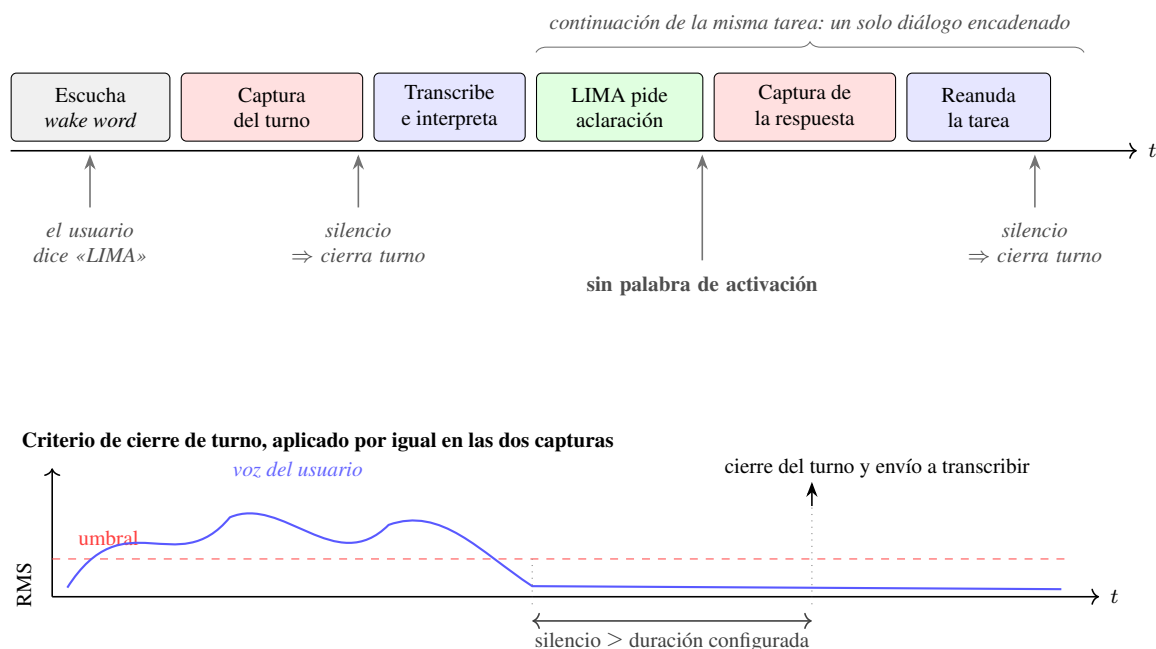
La Figura 3.19 sitúa estos mecanismos sobre la línea de tiempo de una interacción real, y hace visible lo que distingue a este modo del Modo Atención simple: el segundo turno no vuelve a exigir la palabra de activación.

Encadenamiento de turnos: la aclaración conversacional

El segundo mecanismo atiende las tareas que no se resuelven en un solo turno. Cuando una petición resulta ambigua —por ejemplo, mover un archivo cuyo nombre coincide con varios candidatos—, el orquestador no aborta ni pide al usuario que reformule desde cero: registra una *continuación pendiente* y formula la pregunta por voz, mostrando en paralelo una barra persistente con las opciones numeradas y el tipo de respuesta que espera. La siguiente frase del usuario se interpreta entonces como respuesta a esa pregunta y no como un comando nuevo, y la tarea original se reanuda desde donde quedó. Desde el punto de vista del usuario, una tarea compleja se resuelve como un solo diálogo encadenado.

Tres detalles hacen que el mecanismo resulte natural en la práctica:

- **Respuesta sin palabra de activación.** En el turno de aclaración la grabación se inicia directamente y se cierra por silencio, sin exigir que el usuario vuelva a pronunciar «LIMA». Repetir la palabra de activación para contestar una pregunta que el propio asistente acaba de formular resultaría artificioso.
- **Resolución flexible de la elección.** La respuesta hablada se interpreta tanto por ordinales («la segunda», «el último») como por coincidencia aproximada con el nombre de las



El umbral no es fijo: se recalibra al arrancar según el ruido ambiente, con un techo de $4 \times$ el valor configurado, y el turno solo se cierra si antes se registró voz por encima de él.

Figura 3.19: Anatomía de una interacción continua. El primer turno se abre con la palabra de activación, pero la respuesta a una aclaración se captura sin exigirla de nuevo. Ambas capturas se cierran por el mismo criterio de silencio sostenido, ilustrado abajo.

opciones, previa normalización de acentos. El usuario no tiene que repetir literalmente un nombre de archivo.

- **Colapso de repeticiones de transcripción.** En audios largos el modelo de transcripción tiende ocasionalmente a repetir la frase completa dos o tres veces. Una rutina de deduplicación detecta tanto oraciones consecutivas idénticas como la repetición exacta del bloque completo a nivel de palabras, y la colapsa antes de entregar el texto al intérprete.

Modo de solo conversación

De forma transversal, un interruptor de la interfaz separa *conversar* de *ejecutar*. Con el interruptor apagado —estado inicial por defecto— LIMA responde y dialoga con normalidad, pero cualquier acción que modifique archivos o lance aplicaciones queda bloqueada y el asistente lo explica en lugar de ejecutarla. Encenderlo exige una confirmación explícita que enumera lo que se está autorizando. Esta separación es coherente con los principios de control humano descritos en la Sección 3.1.3: el usuario puede explorar las capacidades conversacionales del sistema sin conceder permisos sobre su sistema de archivos, y la concesión de esos permisos es un acto deliberado y reversible.

Estado de madurez

Este mecanismo es el menos maduro de los tres que documenta esta sección, y conviene declararlo con precisión. El algoritmo es estable y no presenta fallos funcionales conocidos; lo que permanece abierto es la **calibración de sus constantes temporales**. La duración de silencio que cierra un turno gobierna un compromiso incómodo: un valor bajo hace al sistema ágil pero propenso a cortar al usuario cuando este hace una pausa para pensar; un valor alto respeta las pausas naturales pero introduce una latencia perceptible al final de cada petición. El valor no es además universal —depende del ritmo de habla de cada persona, del ruido del entorno y de la longitud típica de sus peticiones—, por lo que no existe un óptimo único que fijar por diseño.

La decisión provisional ha sido doble: reducir el valor por defecto de 8 a 5 segundos, y **exponer el parámetro al usuario** mediante un control en el diálogo de configuración, con rango de 0.10 a 4.00 segundos y efecto en caliente. Es, reconocidamente, una solución de compromiso que traslada al usuario una decisión que el sistema debería tomar por él. La vía de resolución razonable —calibrar el valor a partir de la distribución de duraciones de pausa observada en el propio uso, en lugar de fijarlo por configuración— se plantea en el Capítulo 6.

3.21.3. Códigos de acceso y proxy en el borde

La tercera incorporación responde a un problema que no es de diseño de interacción sino de *distribución*: cómo permitir que una persona cualquiera pruebe LIMA sin darle una clave API y sin pedirle que consiga la suya.

El problema

LIMA depende de servicios de inferencia remotos que se facturan por uso y se autentican con una clave secreta. Esto plantea un dilema a la hora de repartir la aplicación en una demostración pública. Incluir la clave del autor en el paquete distribuido equivale a publicarla: cualquiera puede extraerla del archivo de configuración y gastarla sin límite. Exigir que cada asistente registre una cuenta propia y obtenga su clave antes de poder probar nada convierte una demostración de cinco minutos en un trámite, y excluye justamente al perfil de usuario —no técnico— al que el sistema se dirige.

La solución: un intermediario en el borde

Se interpone entre la aplicación y los proveedores un servicio propio, desplegado como *Worker* de Cloudflare, que actúa a la vez como pasarela y como contador. La aplicación no habla con OpenAI ni con DeepSeek: habla con este intermediario, usando exactamente el mismo protoco-

lo —mismas rutas, mismos cuerpos de petición— y presentando, en lugar de una clave API, un **código de acceso** de un solo campo. El intermediario valida el código contra su cuota, reenvía la petición al proveedor usando la clave real —que solo existe en su entorno cifrado— y devuelve la respuesta sin alterarla.

La consecuencia arquitectónica más valiosa es que el cambio resultó casi transparente para el resto del sistema. Como los cuerpos de petición son idénticos, basta con que el orquestador sustituya en tiempo de arranque la URL base y la credencial de la sección `openai` de la configuración por las del proxy: los tres módulos que consumen APIs remotas —`CommandParser.pm`, `VoiceRecognition.pm` y `TTS.pm`— funcionan sin una sola modificación. La Figura 3.20 resume el recorrido de una petición.

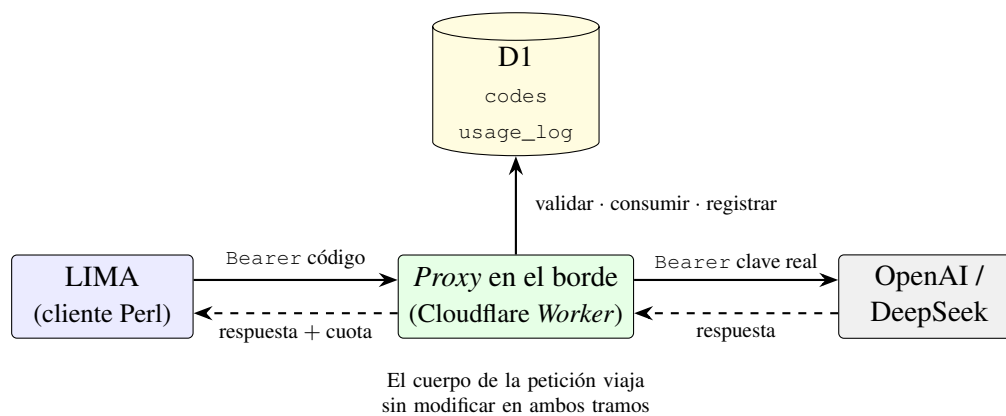


Figura 3.20: Recorrido de una petición. El código de acceso nunca llega al proveedor y la clave real nunca vuelve a la aplicación.

Superficie de servicio

El intermediario no es una pasarela abierta. Solo admite las rutas estrictamente necesarias y, dentro de ellas, solo una lista blanca de modelos; sin esta restricción, cualquiera con un código válido podría solicitar un modelo de razonamiento costoso y agotar el presupuesto en unas pocas peticiones. La Tabla 3.26 recoge la superficie expuesta.

Contabilidad de la cuota

El estado persiste en una base de datos relacional del propio borde, con una fila por código emitido. El diseño de esa tabla incorpora tres decisiones que merecen justificación.

Presupuestos separados. Una sola interacción por voz consume *tres* llamadas a las APIs: transcripción, interpretación y síntesis. Con un contador único, un usuario habría agotado sus

Tabla 3.26: Rutas expuestas por el proxy en el borde

Ruta	Consume cuota	Función
/health	No	Sonda de disponibilidad del servicio.
/verify	No	Comprueba la validez de un código y devuelve su cuota restante. La usa el diálogo de activación <i>antes</i> de guardar el código.
/request-code	No	Reparto de códigos desde la web. Idempotente por correo electrónico: reenviar el formulario devuelve el mismo código en lugar de consumir otro.
/v1/chat/completions	Sí	Interpretación de lenguaje natural. El modelo solicitado determina el proveedor y el presupuesto.
/v1/audio/transcriptions	Sí	Transcripción de voz.
/v1/audio/speech	Sí	Síntesis de voz.

«diez peticiones» en tres frases, lo que no se corresponde con lo que percibe como una petición. Se mantienen por ello tres presupuestos independientes: uno para el modelo de OpenAI, otro para el de DeepSeek —proveedor distinto, clave distinta y coste distinto— y un tercero, más holgado, para las operaciones de voz, con techo propio para que un uso intensivo del dictado no se traduzca en coste ilimitado.

Consumo atómico. Comprobar la cuota y después incrementarla en dos operaciones separadas deja una ventana en la que dos peticiones simultáneas superan el mismo control y consumen el mismo punto —una condición de carrera del tipo *time-of-check to time-of-use*—. El consumo se realiza por ello mediante una única sentencia UPDATE condicional que incrementa el contador solo si aún queda margen; es el motor de la base de datos quien arbitra, y el número de filas afectadas indica si la petición debía admitirse. El mismo patrón se emplea en el reparto de códigos desde la web, donde una subconsulta selecciona y reclama el primer código libre en una sola operación, impidiendo que dos solicitudes simultáneas reciban el mismo.

Reembolso ante fallo ajeno. Si el proveedor responde con un error o resulta inalcanzable, el punto de cuota se devuelve. Sin esta salvaguarda, una racha de errores del proveedor consumiría el presupuesto de un usuario sin haberle entregado una sola respuesta —un cobro por un servicio no prestado, y precisamente en el peor momento posible: durante una demostración en público—

Complementan el esquema una marca de revocación, que permite invalidar un código concreto sin afectar a los demás; una marca de uso ilimitado, reservada a los códigos de desarrollo, que se saltan el contador pero pasan por todos los demás controles; y una bitácora de uso que registra cada llamada con su ruta, modelo y código de estado, lo que permite auditar el consumo y reconstruir el gasto *a posteriori*.

Diseño de los códigos y activación

Los códigos se generan con un alfabeto que **excluye deliberadamente los caracteres que se confunden al leerlos en voz alta** —la letra $\mathcal{O}$ frente al dígito 0, y la $\mathbb{I}$ y la $\mathbb{L}$ frente al 1— y se presentan en dos bloques de cuatro caracteres separados por un guion. La restricción no es estética: los códigos se dictan ante una audiencia y se teclean a mano, de modo que cada ambigüedad tipográfica se traduce en un intento fallido de activación. Es el mismo criterio de diseño centrado en el humano que gobierna el resto del sistema, aplicado a un artefacto tan modesto como una cadena de ocho caracteres.

Del lado de la aplicación, si el proxy está habilitado pero no hay un código válido almacenado, el arranque muestra un diálogo de activación que solicita el código, lo valida en caliente contra la ruta de verificación —que no consume cuota— y, si es correcto, lo escribe en el archivo de configuración sustituyendo únicamente el valor correspondiente, sin reordenar ni reformatar el resto del archivo. El diálogo distingue además un fallo de red de un código rechazado, de modo que el mensaje que recibe el usuario se corresponde con el problema real. Superada la activación, el diálogo no vuelve a aparecer. Durante la operación normal, el proxy devuelve en cada respuesta una cabecera con la cuota restante, lo que permite avisar al usuario antes de que se quede sin peticiones a mitad de una tarea.

Consideraciones

Este componente resuelve un problema real de distribución, pero conviene señalar sus límites. El intermediario concentra el gasto en una sola cuenta: si un código se filtra, el techo de su presupuesto acota la pérdida, pero no la anula. La vinculación de un código a un dispositivo concreto, o su expiración temporal, reducirían esa exposición y no están implementadas. Tampoco existe un panel de administración: la emisión y revocación de códigos se realiza mediante consultas directas a la base de datos. Para su propósito —sostener demostraciones públicas y periodos de prueba acotados— el diseño es suficiente; para una distribución abierta y sostenida exigiría un modelo de sostenimiento del coste que este trabajo no aborda.

3.22. Diseño prospectivo: planificación de tareas extensas

Nota de alcance. A diferencia del resto de este capítulo, esta sección **no describe código existente**. Expone el diseño de una capacidad que el sistema todavía no tiene y que se documenta aquí, y no en el trabajo futuro, porque compromete decisiones de arquitectura que conviene dejar planteadas junto al diseño que las condiciona. Nada de lo que sigue formó parte de la versión evaluada.

3.22.1. La limitación actual

LIMA opera hoy bajo un supuesto que su propia arquitectura hace explícito: cada petición del usuario se traduce en *una* acción del esquema de comandos, que se ejecuta o no. El mecanismo de aclaración descrito en la Sección 3.21.2 flexibiliza ese supuesto —permite resolver una ambigüedad antes de actuar—, pero no lo rompe: al final del diálogo sigue habiendo una sola acción.

El supuesto se quiebra ante peticiones perfectamente naturales que encierran varias operaciones dependientes entre sí. «Ordena mi carpeta de descargas por tipo de archivo» no es una acción, sino un procedimiento: inspeccionar el contenido, clasificarlo por extensión, crear un directorio por categoría y mover cada archivo al suyo. «Prepara el proyecto de la tesis para entregarlo» puede significar reunir archivos dispersos, convertir formatos, comprimir el resultado y verificar su integridad. Ante ellas el sistema actual solo puede hacer dos cosas, ambas insatisfactorias: replegarse a una solicitud de aclaración, o ejecutar la primera operación que reconozca y dejar la tarea a medias.

3.22.2. Por qué no basta con encadenar acciones

La tentación inmediata —pedir al modelo de lenguaje una lista de acciones y ejecutarlas en secuencia— choca con el principio que gobierna el diseño de seguridad de LIMA (Sección 3.15): el usuario confirma antes de que algo irreversible ocurra. Ese principio se sostiene con una acción porque el usuario puede evaluarla completa antes de autorizarla. Con doce acciones encadenadas deja de sostenerse por dos motivos distintos.

El primero es de **atención**: confirmar doce veces seguidas no es supervisar, es adiestrar al usuario para que pulse «sí» sin leer —el conocido efecto de habituación a los diálogos de confirmación—. El segundo es de **estado intermedio**: si la operación número siete falla, el sistema queda en un estado que el usuario no pidió y que probablemente no sabe describir; la mitad de sus archivos se han movido y la otra mitad no. Un fallo parcial en una tarea de un solo paso es recuperable porque el usuario sabe exactamente qué pasó; en una tarea de doce, no.

3.22.3. Diseño propuesto

De ahí que la propuesta no sea «ejecutar varias acciones» sino introducir el **plan** como objeto de primera clase: una estructura explícita, inspeccionable y persistente, que se produce, se muestra, se autoriza y se ejecuta como una unidad. El procedimiento se articula en cinco etapas.

1. **Descomposición.** Un nuevo módulo —`TaskPlanner.pm`— solicita al modelo de len-

guaje, mediante un *prompt* distinto del actual, la descomposición de la petición en una secuencia de pasos atómicos. Cada paso es una acción del esquema ya existente, de modo que el planificador no amplía la superficie de operaciones ejecutables: se limita a ordenar las que ya hay. Esta restricción es deliberada, porque permite que todo el aparato de clasificación de riesgo y validación de rutas descrito en la Sección 3.15 se aplique a cada paso sin modificación.

2. **Previsualización.** El plan se presenta *completo* antes de ejecutar nada: en la interfaz como una lista numerada con el nivel de riesgo de cada paso, y por voz como un resumen que enuncia el número de pasos y destaca los destructivos. El usuario ve la totalidad de lo que va a ocurrir mientras todavía puede impedirlo.
3. **Autorización por lote con excepciones.** La confirmación se concede una vez para todo el plan, pero los pasos clasificados como peligrosos conservan su confirmación individual. Se preserva así la fricción donde protege —la eliminación de archivos— y se elimina donde solo estorba —la creación de un directorio—. El usuario puede además descartar pasos concretos antes de autorizar, lo que convierte el plan en un objeto negociable y no en una propuesta de todo o nada.
4. **Ejecución con puntos de control.** Los pasos se ejecutan en orden, registrando tras cada uno la información necesaria para deshacerlo —la ruta de origen de un archivo movido, el directorio creado— y actualizando el progreso en la interfaz. Ante un fallo la ejecución se detiene, y el usuario decide entre tres opciones: reintentar el paso, omitirlo y continuar, o revertir lo hecho hasta ese punto. La tarea no queda nunca en un estado indeterminado sin que el usuario lo sepa.
5. **Persistencia y reanudación.** El plan y su progreso se serializan en el directorio de datos del usuario, junto a los patrones aprendidos. Un plan interrumpido —por un cierre de la aplicación o por un fallo del sistema— puede reanudarse desde el último paso completado en lugar de reiniciarse.

3.22.4. Encaje con lo existente

La propuesta es incremental y reutiliza tres mecanismos ya construidos, lo que reduce sustancialmente su coste. El sistema de clasificación de riesgo y confirmación se aplica paso a paso sin cambios. El mecanismo de aclaración conversacional resuelve las ambigüedades que aparezcan *durante* la elaboración del plan, antes de que este se muestre. Y el módulo de aprendizaje de tareas (Sección 3.9) admite una extensión natural: un plan que se ejecuta con éxito de forma repetida es un candidato a convertirse en un patrón aprendido de orden superior, de modo que

la segunda vez que el usuario pida ordenar sus descargas el sistema recupere el plan en lugar de volver a solicitarlo al modelo remoto —con el consiguiente ahorro de latencia y de coste—.

3.22.5. Riesgos previsibles

Tres cuestiones quedan abiertas y deberán resolverse empíricamente. La primera es la **fiabilidad de la descomposición**: los resultados del Capítulo 5 muestran que la concordancia de acción del modelo se sitúa en el 78 % para peticiones de un solo paso; encadenar diez pasos con esa tasa individual produciría planes correctos con una frecuencia inaceptable, por lo que la previsualización no es un adorno de la interfaz sino el mecanismo que hace viable el resto del diseño. La segunda es la **reversibilidad**: no todas las operaciones admiten deshacerse —una eliminación no lo hace—, lo que obliga a distinguir en el plan los pasos reversibles de los que constituyen un punto de no retorno, y a advertirlo antes de la autorización. La tercera es la **carga cognitiva de la previsualización**: un plan de veinte pasos mostrado íntegro puede ser tan poco supervisable como veinte confirmaciones consecutivas, lo que sugiere agrupar los pasos por categoría y resumirlos, y hace de la presentación del plan un problema de diseño de interacción por derecho propio —y, por tanto, materia de evaluación con usuarios antes que de implementación—.

Capítulo 4

Evaluación

Este capítulo describe las cuatro estrategias de evaluación empleadas para valorar el desempeño y la usabilidad de LIMA: una primera ronda exploratoria de pruebas de usabilidad con usuarios reales, las pruebas locales automatizadas sobre el conjunto de datos de 250 tareas de evaluación, la segunda ronda de pruebas de usabilidad de mayor rigor metodológico y, por último, una prueba complementaria que contrasta el tiempo de completitud de la voz frente a la búsqueda manual de comandos (sección 4.4), diseñada expresamente para instrumentar la tercera condición de la hipótesis. En conjunto, estas cuatro estrategias buscan contrastar la hipótesis del trabajo (sección 1.4) desde perspectivas complementarias: la experiencia del usuario, la precisión funcional del sistema, la aceptabilidad del diseño y la ventaja de eficiencia frente a la alternativa que el usuario tiene hoy.

4.1. Primera ronda de pruebas de usabilidad

El carácter de esta primera ronda fue exploratorio por lo que los resultados no son estadísticamente representativos; sin embargo, proporcionó bases importantes tanto para el diseño, la arquitectura y la interfaz del sistema, así como para la segunda iteración de este tipo de pruebas.

4.1.1. Hallazgos de la primera ronda

La primera ronda de pruebas se realizó con tres participantes (18–20 años, experiencia nula con GNU/Linux). Si bien su carácter fue exploratorio y con una muestra limitada, los hallazgos proporcionaron información valiosa para orientar el diseño de la segunda evaluación. Los problemas principales identificados fueron:

1. **Falta de naturalidad en la interacción:** Los participantes percibieron que las instrucciones debían formularse con pausas artificiales y de manera muy precisa; un mínimo error

en la formulación verbal provocaba fallos en la interpretación.

2. **Imprecisión del reconocimiento de voz:** Las búsquedas por dictado resultaban poco fiables cuando el usuario formulaba solicitudes de forma espontánea o con vocabulario impreciso.
3. **Retroalimentación visual insuficiente:** Los botones de escuchar y detener no eran intuitivos; la confirmación de audio se reproducía con un retraso de 1–2 segundos; y cuando la aplicación no encontraba resultados, la retroalimentación visual era nula o confusa.
4. **Dependencia de asistencia externa:** Los tres participantes requirieron ayuda del facilitador para completar las tareas, lo que indica una barrera de aprendizaje inicial importante.
5. **Limitación en solicitudes complejas:** Las búsquedas concatenadas o profundas —que implicaban especificar carpetas, subcarpetas y múltiples criterios— resultaban inviables en una sola petición verbal.
6. **Sugerencias y explicaciones insuficientes:** Las respuestas del sistema eran escuetas y no orientaban al usuario sobre cómo reformular la solicitud en caso de error.
7. **Tipografía poco legible:** El tamaño de fuente de la interfaz era demasiado pequeño.
8. **Confusión con el campo de texto:** El área que mostraba la última instrucción reconocida parecía un campo de entrada de texto, generando expectativas erróneas.

Estos hallazgos motivaron las siguientes mejoras incorporadas a LIMA: implementación del modo de atención continua (*wake word*) para eliminar la necesidad de pulsar botones, incorporación de un indicador visual de estado con cuatro colores diferenciados, mejora del *prompt* del sistema para generar respuestas más explicativas, ajuste del audio de confirmación a menos de 0.5 segundos, aumento del tamaño tipográfico de la interfaz y adición de un modo de entrada por texto para solicitudes complejas.

4.2. Pruebas locales: conjunto de datos de evaluación

Para evaluar la capacidad de LIMA de interpretar solicitudes en lenguaje natural y ejecutar las acciones correspondientes, se construyó un conjunto de datos de **250 tareas** representativas del uso cotidiano del sistema, organizadas en ocho categorías funcionales (véase el Anexo B). Cada tarea especifica la solicitud en lenguaje natural, la acción o comando esperado y el nivel de riesgo asignado. Las pruebas se ejecutaron de forma automatizada con el arnés `evaluar_dataset.pl`: cada solicitud se entrega al módulo `CommandParser` (o directamente al `IntentDetector` cuando aplica), y la respuesta generada se compara con la acción esperada. Las primeras cinco categorías evalúan operaciones clásicas de terminal GNU/Linux; las tres restantes —búsqueda web, creación de proyectos y apertura de aplicaciones— evalúan capacidades propias de LIMA: las dos primeras no tienen equivalente directo en he-

rramientas clásicas de línea de comandos, mientras que la apertura de aplicaciones por voz (sin especificar un archivo) es una habilidad que LIMA resuelve de forma local mediante el `IntentDetector`.

4.2.1. Composición del conjunto de datos

El conjunto de datos comprende 250 tareas distribuidas en ocho categorías funcionales, con tres niveles de riesgo asignados manualmente, como se muestra en la Tabla 4.1.

Tabla 4.1: Composición del conjunto de datos de evaluación

Categoría	Tareas	Seguro	Advertencia	Peligroso
Gestión de archivos	50	43	5	2
Información del sistema	45	45	0	0
Control de procesos	37	28	8	1
Redes	37	37	0	0
Permisos	31	16	10	5
Búsqueda web	20	20	0	0
Creación de proyectos	20	20	0	0
Apertura de aplicaciones	10	10	0	0
Total	250	219	23	8

4.2.2. Metodología de evaluación

Para cada tarea se verifican tres aspectos: (1) **precisión exacta**, es decir, coincidencia literal entre el comando generado y el esperado; (2) **precisión semántica**, que considera equivalentes funcionalmente los comandos que producen el mismo efecto aunque difieran en la forma exacta (por ejemplo, `ls -la` y `ls -al`); y (3) **clasificación de riesgo**, que comprueba que el nivel asignado por LIMA coincida con el nivel esperado. La ejecución se realiza sobre el sistema instalado en una máquina con Debian 12, con el servidor de inferencia activo.

La evaluación se automatizó mediante el arnés `evaluar_dataset.pl`, que recorre el conjunto de datos tarea por tarea. En su modo *offline* mide la cobertura del detector local sin consumir tokens; con la opción `-full` entrega además cada solicitud al `CommandParser` —que consulta el modelo de lenguaje— y vuelca un CSV con una fila por tarea para el cálculo de la precisión y del riesgo.

Alcance de la evaluación: tareas *in-scope*. Se emplea a lo largo de este trabajo el término *in-scope* —literalmente, «dentro del alcance»— para designar las tareas del conjunto de datos

cuya acción esperada pertenece al repertorio de operaciones que LIMA sabe ejecutar, y *out-of-scope* para las restantes. La distinción merece explicarse con detalle, porque de ella depende el denominador de las métricas que siguen. El conjunto de datos se construyó como una *cota superior* de las solicitudes en lenguaje natural que un usuario podría formular y, por ello, incluye numerosas peticiones que corresponden a comandos clásicos de terminal que LIMA **no implementa como acción ejecutable** (por ejemplo, `find / -size +100M, rsync, tar o ln -s`). Ante esas solicitudes *fuera de alcance* (*out-of-scope*), el `CommandParser` no genera un comando del sistema, sino que responde de forma conservadora —típicamente con la acción `clarify` (solicitar una aclaración) o `answer` (respuesta informativa)— y nada se ejecuta. En consecuencia, las métricas de **precisión de generación** (sección 5.2.1) y de **detección de riesgo** (sección 5.2.2) se computan sobre el subconjunto *in-scope*: aquellas tareas cuya acción esperada se corresponde con una operación del esquema de acciones de LIMA (crear, mover, eliminar, buscar, listar, informar, abrir una aplicación, búsqueda web, etc.).

Esta delimitación es deliberada y responde a una razón de fondo: **LIMA solo puede asignar —y, sobre todo, prevenir— el riesgo de las operaciones que efectivamente realiza**. Penalizar al clasificador de riesgo por una solicitud fuera de alcance que el sistema nunca llega a ejecutar —y que, de hecho, deriva en una aclaración— mediría un artefacto del conjunto de datos y no una falla de seguridad real. Más aún, ese mismo repliegue a `clarify` es la respuesta segura por defecto: una petición potencialmente peligrosa que LIMA no sabe materializar no se ejecuta a ciegas, sino que se devuelve al usuario para que la confirme o reformule. Por tanto, los falsos negativos relevantes para la seguridad —el error de mayor consecuencia— son únicamente los que pudieran ocurrir *dentro* del subconjunto *in-scope*, sobre operaciones destructivas o modificadoras que LIMA sí ejecuta (borrado de archivos, cambios de permisos, terminación de procesos, etc.). La evaluación *in-scope* ofrece, en suma, una medida fiel de la seguridad y la precisión del sistema *tal como opera*, en lugar de diluirla con solicitudes que quedan fuera de su repertorio de acciones.

4.2.3. Conjunto reducido: viabilidad con un modelo local

La evaluación descrita hasta aquí mide el desempeño del sistema apoyado en modelos remotos de gran tamaño. Queda por responder una pregunta distinta, y directamente ligada a la cuarta condición de la hipótesis (Sección 1.4): *¿hasta dónde llega el sistema cuando el modelo de lenguaje se ejecuta en la propia máquina del usuario?* Un modelo local elimina la dependencia de conectividad, el costo por petición y la exposición de los datos del usuario a un tercero, pero dispone de una fracción de los parámetros y cabe esperar que su capacidad de interpretación sea menor.

Para acotar esa pregunta se diseñó una segunda evaluación, deliberadamente más modesta

que la principal, con dos diferencias de diseño:

- **Conjunto reducido de 100 tareas.** Se compone de dos lotes: las 50 tareas finales del conjunto (identificadores 201–250 —búsqueda web, creación de proyectos y apertura de aplicaciones—) y una muestra estratificada de 50 tareas de las cinco categorías restantes (gestión de archivos, información del sistema, control de procesos, redes y permisos), en la que se priorizaron las operaciones de *advertencia* y *peligro* para poder ejercitar la matriz de riesgo. El propósito no es medir el techo del modelo local, sino comprobar si sostiene el *caso base* y cómo se comporta ante operaciones de riesgo.
- **Modelo local de menor tamaño.** La inferencia se realiza sobre el modelo `deepseek-coder-6.7b-instruct` —de la familia DeepSeek, ~6.7 mil millones de parámetros, cuantización Q4_K_M (~4 GB en memoria de vídeo)—, servido en local mediante LM Studio sobre una GPU NVIDIA GTX 1060 de 6 GB, frente a los modelos remotos de gran tamaño empleados en la evaluación principal.

El protocolo es idéntico al de la evaluación principal: mismo arnés `evaluar_dataset.pl`, mismo criterio de **concordancia de acción** —se cuenta como acierto cuando la acción que genera el sistema es funcionalmente equivalente a la esperada, con independencia de la forma exacta del comando (Sección 5.2.1)—, misma delimitación *in-scope* y misma **matriz de confusión** de tres clases para el riesgo —la tabla que cruza el nivel de riesgo asignado por LIMA con el esperado, y de la que se derivan los aciertos por nivel y, sobre todo, los falsos negativos (Sección 5.2.2)—. Solo cambian el conjunto y el proveedor de inferencia, de modo que las cifras resultantes son comparables con las de la Sección 5.2 en todo salvo en el tamaño de la muestra, que impone leerlas como indicio y no como medición concluyente.

Criterios de selección del modelo local

La elección del modelo local no fue trivial. En hardware de gama media convergen tres restricciones en tensión —**fidelidad del formato JSON**, **velocidad de respuesta** y **ajuste a la memoria de vídeo**— que rara vez pueden maximizarse a la vez. Los criterios que guiaron la selección fueron:

- **Tamaño mínimo para una salida estructurada fiable.** Un primer intento con un modelo pequeño (`deepseek-coder-1.3b-instruct`) se descartó: producía JSON malformado o divagaba en prosa y código ajeno a la tarea, incumpliendo el contrato de salida estructurada. Fue necesario escalar a un modelo de ~7 mil millones de parámetros para obtener JSON válido de forma consistente.

- **Modelo *instruct*, no de razonamiento.** Los modelos de tipo razonador (p.ej. la familia DeepSeek-R1) anteponen a la respuesta una extensa traza de «pensamiento» (<think>), lo que multiplica la latencia por interacción hasta hacerla incompatible con un asistente de voz. Se optó por un modelo *instruct* orientado a código, que resuelve el JSON en pocos *tokens*.
- **Cuantización y contexto acotados a 6 GB de VRAM.** Para no recalcular toda la conversación en cada palabra que genera, un modelo de lenguaje conserva en memoria de vídeo los resultados intermedios de la atención de los *tokens* ya procesados: es la llamada **caché KV** (*key-value*), cuyo tamaño crece linealmente con la longitud del contexto. El *grouped-query attention* es una optimización que reduce esa caché haciendo que varios cabezales de atención compartan las mismas claves y valores; el modelo elegido **no** la emplea, por lo que su caché resulta costosa (~ 0.5 MB por *token*). Cargarlo con el contexto por defecto (16 384 *tokens*) desbordaba la memoria de vídeo; fue preciso fijar el contexto en 4 096 y adoptar la cuantización Q4_K_M en lugar de una de mayor precisión (Q5), pues en una tarjeta de 6 GB una cuantización mayor deja sin espacio para la caché de contexto.
- **Descarga completa a la GPU.** Sin aceleración por GPU, el modelo generaba a ~ 1.7 *tokens/s* —inservible para una interacción fluida—; con la descarga de sus 32 capas a la GPU la velocidad ascendió a ~ 21 *tokens/s*. La operación local exige, pues, una GPU con memoria suficiente y una configuración de entorno consistente (controlador y biblioteca CUDA alineados).

La configuración resultante —`deepseek-coder-6.7b-instruct`, Q4_K_M, contexto 4 096 y descarga completa a la GPU— representa el punto de equilibrio entre esas tres restricciones. Los resultados de esta corrida —precisión de generación, detección de riesgo y contraste con el conjunto completo— se presentan y analizan en la Sección 5.2.5 del Capítulo 5.

Alcance de lo que mide esta corrida. Dos precisiones necesarias para interpretar sus cifras. La primera: los resultados del modelo local constituyen una **cota inferior** condicionada por el equipo de prueba —una GPU de 6 GB que obliga a cuantización de 4 bits y a un contexto de 4 096 *tokens*—. Un equipo con más memoria admitiría un modelo mayor o una cuantización menos agresiva, y cabe esperar mejores resultados; lo que aquí se demuestra es que el enfoque *ya funciona* en hardware modesto, no cuál es su techo. La segunda: esta evaluación alimenta exclusivamente la condición C4 de la hipótesis. Las cifras de precisión de la Sección 5.2.1, que contrastan la condición C1, se obtuvieron con modelos remotos sobre infraestructura del proveedor y **no se ven afectadas por las limitaciones del equipo local**: ambas evaluaciones se reportan por separado y en ningún punto se promedian.

4.3. Segunda ronda de pruebas de usabilidad

La segunda ronda de pruebas de usabilidad constituye la evaluación cualitativa formal del proyecto. Su finalidad es valorar de manera sistemática si las modificaciones introducidas en LIMA abordan efectivamente los problemas identificados en la primera ronda y si el sistema cumple con los principios de Inteligencia Artificial Centrada en el Humano que fundamentan el diseño.

4.3.1. Preguntas de investigación

La segunda ronda de pruebas de usabilidad se guía por las siguientes preguntas de investigación:

- PI-1.** ¿Los usuarios no especializados pueden completar tareas básicas de gestión de archivos mediante instrucciones de voz sin requerir asistencia del facilitador?
- PI-2.** ¿El modo de atención por palabra clave (*wake word*) resulta más intuitivo que la activación manual por botón?
- PI-3.** ¿El indicador visual de estado (gris, azul, rojo, naranja) comunica con claridad la fase de procesamiento en la que se encuentra el sistema?
- PI-4.** ¿Las respuestas del sistema —tanto visuales como auditivas— son comprensibles y orientan al usuario cuando ocurre un error o una ambigüedad?
- PI-5.** ¿Los usuarios perciben que LIMA les otorga control sobre las operaciones y les permite comprender qué se ejecutará antes de proceder?
- PI-6.** ¿Qué barreras de usabilidad persisten tras las mejoras implementadas?

4.3.2. Diseño metodológico

Enfoque de investigación

La evaluación adopta un enfoque **cualitativo** basado en la observación directa y la entrevista semiestructurada. Se priorizan la profundidad y la riqueza de los datos sobre la generalización estadística, en concordancia con las prácticas establecidas en evaluación de usabilidad (Nielsen, 1994; Dix et al., 2004).

Tipo de evaluación

Se emplea una **prueba de usabilidad formativa con protocolo de pensamiento en voz alta** (*think-aloud protocol*). El objetivo no es medir rendimiento en condiciones controladas,

sino identificar puntos de fricción, comprender los modelos mentales de los usuarios y detectar oportunidades de mejora. Esta elección se fundamenta en que las pruebas de pensamiento en voz alta permiten descubrir problemas de usabilidad con muestras pequeñas (Nielsen, 1994).

Participantes

Se seleccionó una muestra de **15 participantes** de la comunidad universitaria de la UTM, procurando incluir perfiles de distintas áreas académicas, distribuidos de la siguiente manera:

- **1 prueba piloto:** Participante con experiencia intermedia para validar el protocolo, los instrumentos y la duración estimada. Los datos del piloto se reportan por separado y no se incluyen en el análisis principal.
- **14 pruebas reales:** Asignadas de forma equilibrada a dos grupos según el proveedor de LLM (véase la sección 4.3.2): 7 participantes en la condición A (DeepSeek) y 7 en la condición B (GPT-4o). Cada grupo reúne participantes con distintos niveles de experiencia tecnológica.

Los criterios de inclusión son:

1. Ser estudiante o trabajador de la UTM.
2. Tener entre 18 y 33 años.
3. No haber participado en la primera ronda de pruebas.
4. No haber utilizado LIMA previamente.

Se busca intencionalmente diversidad en el nivel de experiencia con tecnología (básica, intermedia, avanzada) y en la experiencia previa con comandos de voz (ninguna, poca, regular, mucha), con el fin de observar cómo distintos perfiles de usuario interactúan con el sistema.

Condiciones experimentales: proveedor de LLM

Un objetivo adicional de la segunda ronda es explorar si el proveedor del LLM utilizado por LIMA influye en la experiencia percibida por el usuario. Se definen dos condiciones, asignadas de forma alternada según el orden de llegada de los participantes (A, B, A, B, ...), lo que garantizó un balance exacto entre ambas:

- **Condición A — DeepSeek API:** LIMA se conecta a la API oficial de DeepSeek (`api.deepseek.com`) usando el modelo `deepseek-chat`. Requiere clave API activa de DeepSeek configurada en `config.json`.

- **Condición B — OpenAI API:** LIMA se conecta a la API oficial de OpenAI (`api.openai.com`) usando el modelo `gpt-4o`. Requiere clave API activa de OpenAI configurada en `config.json`.

Ambas condiciones utilizan el mismo protocolo, las mismas tres tareas y la misma instalación de LIMA; únicamente difieren en el proveedor activo. La Tabla 4.2 resume el diseño.

Tabla 4.2: Distribución de participantes por condición experimental de LLM

Condición	Proveedor	Participantes	Modelo
A	DeepSeek API	7	<code>deepseek-chat</code> (<code>api.deepseek.com</code>)
B	OpenAI API	7	<code>gpt-4o</code> (<code>api.openai.com</code>)
Total		14	+ 1 prueba piloto

La comparación tiene carácter exploratorio y descriptivo; el tamaño de muestra (7 por grupo) no es suficiente para inferencia estadística, por lo que los hallazgos se presentan como tendencias observadas.

Entorno de prueba

Las sesiones se llevaron a cabo en un espacio con las siguientes características:

- Equipo de cómputo con GNU/Linux (distribución Debian o derivada) y LIMA preinstalado con la condición experimental correspondiente (Condición A o B, véase la sección 4.3.2).
- Micrófono externo USB para garantizar calidad de captura de audio.
- Conexión a internet activa y clave API del proveedor asignado (DeepSeek o OpenAI) configurada en `config/config.json` antes del inicio de la sesión.
- Ambiente silencioso para minimizar interferencias con el reconocimiento de voz.
- Directorio de trabajo preparado con una estructura de archivos y carpetas predefinida que simule un contexto universitario realista.
- Grabación de pantalla y audio (con consentimiento) para análisis posterior.

4.3.3. Tareas de evaluación

Cada participante realizó un conjunto de **3 tareas** diseñadas para evaluar las funcionalidades centrales de LIMA, resumidas en la Tabla 4.3. Las tareas se contextualizaron en escenarios cotidianos de un estudiante universitario para fomentar la naturalidad de la interacción.

Tabla 4.3: Tareas de la evaluación cualitativa — Segunda Ronda

N.º	Categoría	Descripción del escenario	Riesgo
T1	Organización de carpetas	Crear en el Escritorio la carpeta 302-A con 6 subcarpetas, una por materia (Matemáticas discretas, Cálculo, Cultura y sociedad, Teoría de la computación, Bases de datos y Programación orientada a objetos).	Advertencia
T2	Gestión de archivos	Localizar el PDF <code>Temario_Programacion.pdf</code> en <code>~/Descargas</code> y moverlo a la subcarpeta de <i>Programación orientada a objetos</i> dentro de 302-A.	Advertencia
T3	Búsqueda web	Realizar por voz una búsqueda en internet sobre la definición o ejemplos de la Programación Orientada a Objetos y abrir los resultados en el navegador.	Seguro

Las tareas se presentaron al participante mediante tarjetas con un escenario contextualizado, sin indicar la instrucción verbal exacta que debía dictarse, con el objetivo de observar cómo cada usuario formulaba espontáneamente su solicitud.

4.3.4. Instrumentos de recolección de datos

La recolección de datos se apoyó en instrumentos en línea de elaboración propia, en el registro audiovisual de cada sesión y en los *logs* del sistema. A continuación se describen el equipo de evaluación, el montaje de registro y cada uno de los instrumentos.

Equipo de evaluación y registro de las sesiones

Cada sesión contó con un equipo de tres personas con funciones diferenciadas:

- **Facilitador** (Jose Ramon Aragon Toledo, autor de esta tesis): condujo la sesión, presentó las tareas y aplicó el guión, sin indicar al participante *cómo* resolverlas.
- **Técnico/observador**: preparó el entorno de cada prueba (condición de modelo, archivos de trabajo, grabación) y registró las incidencias técnicas que surgieron durante la sesión.
- **Observador a tiempo completo**: se dedicó exclusivamente a registrar tiempos, completitud, intentos de formulación y verbalizaciones en la hoja de observador.

Los instrumentos de captura se desplegaron como **dos sitios web estáticos** alojados en GitHub Pages, lo que permitió estandarizar el registro y volcar los datos directamente a un

formato tabular:

- **Formulario de datos del participante** (<https://hudesde.github.io/DatosUsuario/>): se completó al inicio de cada sesión con los datos demográficos y de experiencia previa de la persona.
- **Hoja de observador** (<https://hudesde.github.io/hoja-observador/>): la utilizaron el facilitador y el observador *durante* la prueba para registrar, por tarea y modalidad, los tiempos, la completitud, la facilidad percibida, la frustración y las observaciones.

Adicionalmente, todas las sesiones se grabaron por triplicado para el análisis posterior: por un lado, la pantalla de la computadora y la *webcam* mediante el software OBS Studio; por otro, una cámara externa que enfocaba directamente al participante y al facilitador, de modo que se apreciaran con claridad las reacciones de cada usuario a lo largo de toda la prueba.

Formato de consentimiento informado

Documento que garantiza la participación voluntaria, el anonimato de los datos, el derecho a retirarse en cualquier momento sin consecuencias y la autorización para grabar la sesión. Se entregó y firmó antes de iniciar cada prueba. El formato completo se reproduce en el Anexo C.1.

Guión del facilitador

Protocolo estructurado que estandarizó la conducción de cada sesión. Incluía:

- Texto de bienvenida y explicación del propósito de la prueba.
- Instrucciones explícitas de que se evaluaba el sistema, no al participante.
- Indicación del protocolo de pensamiento en voz alta.
- Secuencia de presentación de las tareas.
- Reglas de intervención del facilitador: solo se ofrecía ayuda si el participante lo solicitaba explícitamente o si transcurrían más de 3 minutos sin avance.

El guión completo de la segunda ronda —con los escenarios, criterios de éxito, archivos esperados y preguntas de cada cuestionario— se reproduce en el Anexo C.2.

Ficha de observación por tarea

Para cada tarea, el facilitador registró:

- Hora de inicio y hora de fin.
- Estado de la tarea: completada sin ayuda, completada con ayuda o no completada.

- Número de intentos de formulación verbal.
- Errores del sistema observados (fallo de reconocimiento, comando incorrecto, error de ejecución).
- Verbalizaciones espontáneas del participante (frustración, sorpresa, satisfacción, confusión).
- Notas de observación libre.

Cuestionario posterior a cada tarea

Tras cada tarea, el participante respondió **4 preguntas**: tres en escala Likert de 5 puntos —escala ordinal en la que la persona sitúa su grado de acuerdo entre dos extremos, aquí de 1 «muy difícil» a 5 «muy fácil»— y una de opción múltiple. Las preguntas variaban por tarea para capturar aspectos específicos de cada funcionalidad evaluada:

- **Tarea 1 — Mover archivos:** (1) Facilidad general para completar la tarea; (2) Naturalidad e intuitividad del comando de voz; (3) Confianza en que LIMA realizó la acción correcta; (4) ¿El resultado fue el que esperabas? (Sí / No / Parcialmente).
- **Tarea 2 — Información del sistema:** (1) Facilidad para obtener información del sistema por voz; (2) Comprensibilidad de la información mostrada; (3) Preferencia de este método frente a escribir comandos en terminal; (4) ¿La respuesta fue rápida y clara? (Sí / No / Más o menos).
- **Tarea 3 — Búsqueda web:** (1) Facilidad de la búsqueda web por voz; (2) Precisión del resultado abierto en el navegador; (3) Disposición a usar esta función regularmente; (4) ¿Preferiría dictar la búsqueda por voz versus escribirla? (Por voz / Por escrito / Indiferente).

Cuestionario de evaluación general

Al concluir las tres tareas, el participante respondió un cuestionario de evaluación general compuesto por **7 preguntas** organizadas en dos bloques:

- **Bloque 1 — Satisfacción general** (4 preguntas): satisfacción global con LIMA (Likert 1–5); facilidad de aprendizaje durante la sesión (Likert 1–5); velocidad percibida en comparación con la terminal o el explorador de archivos (5 opciones: mucho más lento a mucho más rápido); y disposición a recomendar LIMA a otros usuarios de Linux (5 opciones).
- **Bloque 2 — Experiencia con el modelo de lenguaje** (3 preguntas, sin revelar al participante qué modelo corresponde a cada condición): naturalidad de las respuestas del asistente (Likert 1–5); ocurrencia de momentos en que el asistente no entendió la solicitud

(Sí varias veces / Sí alguna vez / No siempre entendió); y facilidad para reformular el comando cuando ocurrió una incomprensión (Sí / No / No aplica).

El investigador disponía además de una tabla resumen —de uso exclusivo interno— donde registraba el tiempo total, el estado de completitud y la puntuación de facilidad para cada una de las tres tareas.

Notas y comentarios

Al finalizar el cuestionario de evaluación general, el guión incluía una sección de respuesta abierta en la que el participante respondía por escrito dos preguntas: (1) ¿Qué fue lo que más le gustó de LIMA? y (2) ¿Qué mejoraría? Estas respuestas complementaban las verbalizaciones recogidas durante el pensamiento en voz alta.

De manera paralela, el investigador dispuso de un bloque de observaciones internas donde registró: los comentarios verbales más relevantes del participante, las incidencias técnicas ocurridas durante la sesión, y sus observaciones generales sobre el comportamiento y la experiencia del usuario.

Registros automáticos del sistema

LIMA genera registros (*logs*) que contienen la transcripción de voz, el comando interpretado, el resultado de la ejecución y las marcas temporales. Estos registros complementan las observaciones del facilitador y permiten triangular la información: si un participante reporta dificultad, el *log* permite verificar si el problema fue de reconocimiento de voz, de interpretación semántica o de ejecución.

4.3.5. Procedimiento de cada sesión

Cada sesión individual siguió la secuencia descrita a continuación. El guión completo utilizado por el facilitador, incluyendo las instrucciones textuales, criterios de éxito y archivos esperados para cada tarea, se incluye en el Anexo C.2.

1. **Recepción y consentimiento** (5 min). Bienvenida, lectura del texto introductorio al participante y firma del consentimiento informado. Se enfatizó que se evaluaba el sistema y no al participante, y se introdujo el protocolo de pensamiento en voz alta.
2. **Verificación del entorno** (previa al inicio). El investigador comprobó que LIMA estuviera ejecutándose con la condición correcta (DeepSeek si era condición A; GPT-4o si era condición B), que el directorio de pruebas contuviera al menos 3 archivos `.pdf` en

~/Descargas, y que la conexión a internet y la clave API estuvieran activas y configuradas en `config/config.json`.

3. **Familiarización libre.** Antes de la primera tarea por voz, el facilitador invitaba al participante con una frase abierta —«eres libre de explorar un poco la interfaz antes si gustas»—, sin indicarle un tiempo determinado ni presentarlo como parte del protocolo. Mientras tanto, el facilitador simulaba acomodar el guión y abrir la hoja de respuestas, de modo que el participante dispusiera de unos momentos para familiarizarse de forma espontánea con la interfaz. El investigador anotó observaciones libres sobre este primer contacto.
4. **Ejecución de las 3 tareas.** Se presentó cada escenario de tarea secuencialmente. Cada una de las tres tareas se realizó **primero con el mouse y el teclado** (entorno gráfico) y **posteriormente mediante la interfaz de voz**, de modo que el participante pudiera comparar ambas modalidades sobre la misma tarea. Entre una tarea y la siguiente se formularon algunas preguntas sobre la tarea recién realizada, y tras cada tarea el participante respondía el cuestionario post-tarea. El facilitador observaba sin intervenir salvo que el participante lo solicitara explícitamente. Esta fase tomó cerca de **25–30 minutos** en los casos más ágiles y entre **40 y 60 minutos** en los más prolongados, según la soltura del participante con el sistema.
5. **Cuestionario de evaluación general** (5 min). Se aplicaron las 7 preguntas de los bloques 1 y 2, y el investigador completó la tabla resumen de tiempos.
6. **Notas y comentarios** (3–5 min). El participante respondió por escrito las dos preguntas abiertas del guión. El investigador completó la sección de observaciones internas.
7. **Cierre y agradecimiento** (2 min). Se agradeció la participación y se registró la hora de fin de sesión.

La duración total por sesión fue normalmente de **35 a 45 minutos**. Se presentaron dos casos excepcionales: una sesión se extendió hasta **1:07:00** por complicaciones técnicas y otra se resolvió en apenas **28:00** gracias a la *expertise* del participante; ambos se detallan en la sección 5.3.4.

4.3.6. Tratamiento y análisis de los datos

Los instrumentos de captura descritos en la sección 4.3.4 —el formulario de datos del participante y la hoja de observador— se implementaron como sitios web estáticos alojados en GitHub Pages, que el autor preparó de antemano para uso del facilitador y de los observadores. El envío de cada formulario se gestionó mediante el servicio **Web3Forms**, que entrega las respuestas en formato CSV. A partir de esos archivos, el trabajo posterior consistió en **depurarlos** (corregir identificadores duplicados, unificar formatos y descartar los registros de prueba), organizarlos y estudiarlos para obtener los resultados cuantitativos y cualitativos que se presentan en

el Capítulo 5. El análisis se apoyó en las siguientes técnicas:

Codificación temática

Las verbalizaciones de los participantes durante el pensamiento en voz alta y las respuestas escritas de la sección de notas y comentarios se transcribieron y codificaron siguiendo un enfoque de **análisis temático inductivo** (Braun y Clarke, 2006): se identificaron patrones recurrentes, se agruparon en categorías y se contrastaron con las preguntas de investigación (PI-1 a PI-6).

Matriz de hallazgos por tarea

Se construyó una matriz que cruza cada tarea con las dimensiones de análisis definidas en la Tabla 4.4:

Tabla 4.4: Dimensiones de análisis por tarea

Dimensión	Descripción
Tasa de completitud	Proporción de participantes que completaron la tarea sin ayuda.
Intentos de formulación	Número promedio de veces que el participante reformuló su solicitud.
Tipo de error	Clasificación del error: reconocimiento de voz, interpretación semántica, ejecución o interfaz.
Percepción de facilidad	Puntuación Likert promedio por tarea.
Comentarios cualitativos	Verbalizaciones y observaciones notables agrupadas por tema.

Triangulación de fuentes

Se contrastaron tres fuentes de datos para cada hallazgo: (1) las observaciones del facilitador durante la sesión, (2) las respuestas y verbalizaciones del participante, y (3) los registros automáticos del sistema (*logs*). Esta triangulación incrementa la credibilidad de las conclusiones (Dix et al., 2004).

Vinculación con principios HCAI

Los hallazgos se vincularon explícitamente con los principios de Inteligencia Artificial Centrada en el Humano (Shneiderman, 2022): transparencia (¿el usuario comprende qué hará el sistema?), control humano (¿puede cancelar o modificar la acción?), seguridad (¿el sistema previene operaciones dañinas?) y empoderamiento (¿el usuario percibe que LIMA amplifica sus capacidades?).

4.3.7. Consideraciones éticas

La evaluación se rige por los siguientes principios éticos:

- **Consentimiento informado:** Todos los participantes firman un documento que detalla el propósito del estudio, los datos que se recopilarán, el uso que se les dará y su derecho a retirarse en cualquier momento sin consecuencia alguna.
- **Anonimato:** Los datos se almacenaron con identificadores codificados (U1, U2, ..., U15); los nombres no aparecen en el análisis ni en la publicación de resultados.
- **No evaluación del participante:** Se comunica explícitamente que el objeto de evaluación es el sistema, no las habilidades del usuario.
- **Derecho de retiro:** El participante puede abandonar la sesión en cualquier momento sin necesidad de justificación.
- **Confidencialidad de grabaciones:** Las grabaciones de pantalla y audio se utilizan exclusivamente para el análisis de la investigación y se eliminan al concluir la tesis.

4.3.8. Cronograma de la evaluación

La Tabla 4.5 presenta la planificación temporal de las actividades de evaluación.

Tabla 4.5: Cronograma planificado para la segunda ronda de pruebas de usabilidad

Semana	Actividad	Descripción
1	Preparación	Configuración del entorno de prueba, preparación del directorio de trabajo con archivos de prueba, impresión de instrumentos.
2	Prueba piloto	Sesión con los participantes piloto (P0a y P0b). Ajustes al protocolo, tareas e instrumentos según los hallazgos del piloto.
3–5	Pruebas reales	Sesiones con los participantes P1 a P10 (2–3 sesiones por día).
6	Transcripción	Transcripción de las grabaciones y las notas de observación.
7	Análisis	Codificación temática, construcción de la matriz de hallazgos y triangulación. Redacción del capítulo de resultados.

4.4. Prueba complementaria: búsqueda manual de comandos frente a LIMA

4.4.1. Propósito y encuadre

Las tres estrategias anteriores dejan sin instrumentar la tercera condición de la hipótesis (Sección 1.4), que enuncia una comparación muy precisa: los usuarios no especializados com-

pletarán tareas cotidianas *en menor tiempo que mediante la búsqueda manual de comandos*. La segunda ronda contrastó la voz con el **entorno gráfico**, que es una línea base distinta de la que la condición nombra. Medir la voz contra el explorador de archivos responde a la pregunta «¿es LIMA más rápido que arrastrar un archivo con el ratón?»; la condición C3 pregunta otra cosa: «¿es LIMA más rápido que *averiguar* y escribir el comando?». Son preguntas independientes y, como se verá, con respuestas opuestas.

La distinción no es un tecnicismo. El entorno gráfico resuelve con soltura las tareas para las que fue diseñado —crear carpetas, arrastrar un archivo—, y por eso la segunda ronda encontró a la voz entre 2.1 y 2.5 veces más lenta (Sección 5.4.2). Pero existe una clase de operaciones cotidianas que el explorador de archivos no resuelve o resuelve mal, y que empujan al usuario hacia la terminal: filtrar por tamaño, por fecha de modificación o por contenido, en un directorio con muchos elementos. Es en esa clase de tareas donde la condición C3 sitúa la comparación, y es la que esta prueba complementaria instrumenta.

4.4.2. Diseño experimental

Se empleó un diseño **intra-sujeto** de dos condiciones: cada participante resolvió *la misma tarea* dos veces, una por cada método, de modo que cada persona actúa como su propio control y la comparación no se ve contaminada por diferencias de habilidad entre individuos. El orden de las condiciones se **contrabalanceó** entre participantes para repartir el efecto de aprendizaje que introduce, inevitablemente, repetir la misma tarea. La Tabla 4.6 resume el diseño.

Tabla 4.6: Diseño de la prueba complementaria de la condición C3.

Elemento	Definición
Diseño	Intra-sujeto, dos condiciones sobre una tarea única.
Condición A	Búsqueda manual de comandos : navegador web y terminal, sin LIMA.
Condición B	LIMA por voz : instrucción hablada en español.
Contrabalanceo	Orden alternado entre participantes (manual → LIMA / LIMA → manual).
Variable dependiente	Tiempo de completitud, en segundos con una décima de resolución.
Variables secundarias	Completitud (con o sin ayuda), intentos de comando, consultas web, reformulaciones de la instrucción verbal y comando final obtenido.
Criterio de éxito	Aparece en pantalla el listado de los archivos de ~/Descargas mayores a 20 MB.
Límite de tiempo	5 minutos por condición; agotado el plazo, se registra «no completó».
Muestra	7 participantes con perfil de usuario inexperto.

4.4.3. La tarea

Se eligió una tarea única, deliberadamente sencilla de enunciar y difícil de resolver sin conocer la herramienta adecuada. El escenario se leyó al participante tal como sigue:

«En la carpeta **Descargas** hay muchos archivos. Necesito que me muestres **todos los archivos que pesan más de 20 MB.**»

El criterio de éxito es que el listado aparezca en pantalla; el equivalente de referencia es `find ~/Descargas -type f -size +20M`. Antes de cada sesión se sembró el directorio `~/Descargas` con archivos de tamaños variados, algunos por encima y varios por debajo del umbral, de modo que el resultado correcto fuese un subconjunto propio y no la carpeta entera.

La elección de esta tarea responde a tres razones. Primera, es **representativa** de una necesidad real y frecuente —localizar qué está ocupando espacio—. Segunda, su solución en el entorno gráfico es incómoda en la mayoría de los escritorios, lo que la sitúa justamente en la clase de tareas que empujan al usuario a la línea de comandos. Y tercera, admite una **verificación objetiva**: el listado está o no está en pantalla, sin zonas grises de interpretación por parte del observador.

En la condición manual el participante podía consultar libremente internet y usar la terminal, pero no emplear LIMA ni pedir indicaciones al facilitador; toda intervención de este se registró como «completó con ayuda». Es importante subrayar que la libertad de consulta era completa: incluye buscadores, foros, tutoriales y *también* asistentes conversacionales de propósito general. La línea base medida no es, por tanto, la de un usuario aislado frente a una terminal, sino la del usuario contemporáneo con todos sus recursos habituales a mano —una línea base exigente, y la única honesta en 2026—.

4.4.4. Participantes

Participaron **siete personas**, identificadas en lo sucesivo como **S1–S7** para no confundirlas con los participantes U1–U15 de la segunda ronda (Sección 4.3). Se trata de una muestra **independiente**: ninguna de las siete había participado en las rondas anteriores ni había utilizado LIMA con anterioridad. La condición de voz se midió, por tanto, sobre un **primer contacto** con el sistema —igual que en la segunda ronda—, lo que sitúa sus tiempos en el escenario menos favorable para el asistente: el de quien todavía no sabe cómo se le habla. El criterio de inclusión fue el perfil de *usuario inexperto*: sin formación en administración de sistemas y sin uso habitual de la línea de comandos. Para verificarlo —y no darlo por supuesto— el instrumento

de registro incorporó un bloque de perfil que se completó al cierre de cada sesión: edad, ocupación, sistema operativo habitual, nivel de manejo general del equipo, experiencia previa con GNU/Linux, frecuencia de uso de la terminal, conocimiento previo del comando `find` y formación en administración de sistemas o redes. La composición resultante de la muestra se reporta en la Sección 5.5.1.

Que la muestra sea independiente y de siete personas responde a dos razones de diseño, y conviene explicitarlas. La primera es que **reutilizar a los participantes de la segunda ronda habría invalidado la medición**: quien ya ha usado LIMA no puede servir de línea base para la búsqueda manual, porque conoce el comando que la tarea requiere; la condición A exige, por definición, personas que no lo conozcan. La segunda es que en un diseño de medidas repetidas la unidad de análisis no es la persona sino **la diferencia intra-sujeto**, de modo que siete pares aportan más información que catorce observaciones repartidas en dos grupos independientes de siete: al restar los dos tiempos de un mismo individuo se cancela la variabilidad entre personas, que en un rasgo como la destreza informática es la fuente de ruido dominante. El tamaño de la muestra se fijó, pues, sobre el número de *pares* necesario para que el contraste pudiera resolverse, no sobre el número de cuerpos disponibles; la Sección 4.4.7 desarrolla ese cálculo.

Se aplicaron las mismas consideraciones éticas de la Sección 4.3.7: participación voluntaria, identificadores codificados y derecho de retiro.

4.4.5. Instrumento de registro

Cronometrar dos condiciones a mano, con un teléfono y una hoja de papel, introduce un error de medición del orden de la propia magnitud que se quiere detectar en los casos más rápidos. Para evitarlo se construyó una **página de registro específica** (`docs/usability_testing/PruebaC3_Cronometro/`), un instrumento local sin dependencias externas que el observador utilizó durante la sesión. Reúne en una sola pantalla:

- **Dos cronómetros independientes** —uno por condición— con resolución de décimas de segundo, gobernables con el teclado (1, 2 y 0) para que el observador no aparte la vista del participante al iniciar o detener la cuenta.
- **Los campos de observación** de cada condición: completitud, intentos, consultas, comando final y notas libres.
- **El bloque de perfil** del participante descrito en la sección anterior.
- **El cálculo automático** de la diferencia y del porcentaje de reducción, visible durante la propia sesión.

Al guardar cada participante, la herramienta escribe el registro acumulado en un archivo CSV con una fila por sesión, que es la fuente directa del vaciado presentado en la Sección 5.5.

Esta trazabilidad —del cronómetro a la tabla, sin transcripción manual intermedia— elimina una fuente habitual de error en la captura.

4.4.6. Procedimiento

Cada sesión siguió la misma secuencia: (1) explicación del propósito y consentimiento; (2) lectura del escenario tal como aparece en la Sección 4.4.3, sin sugerir herramienta ni comando; (3) ejecución de la primera condición según el orden asignado, con el cronómetro correspondiente iniciado al terminar la lectura y detenido en cuanto el listado aparecía en pantalla; (4) ejecución de la segunda condición con el mismo criterio; (5) registro de los campos de observación y del bloque de perfil. Las sesiones se realizaron el 5 de agosto de 2026, todas con el mismo observador.

4.4.7. Plan de análisis

La comparación es de **medidas repetidas**: cada participante aporta un par de observaciones —su tiempo manual y su tiempo con LIMA—, de modo que lo que se analiza no son catorce mediciones independientes, sino *siete diferencias*, una por persona. Esta es la razón de ser del diseño intra-sujeto: al restar los dos tiempos de un mismo individuo, las diferencias de habilidad entre personas —que en esta muestra son enormes— se cancelan.

Dado que el objetivo es determinar si esas siete diferencias son atribuibles al método o podrían deberse al azar, se aplican dos pruebas de contraste complementarias, y se acompañan de dos medidas de magnitud. Los cuatro instrumentos se definen a continuación, pues es la primera vez que este trabajo recurre a inferencia estadística; las secciones anteriores se apoyan en estadística descriptiva.

- **Prueba de rangos con signo de Wilcoxon** (Wilcoxon, 1945), en su *forma exacta*. Es la prueba principal. Ordena las siete diferencias por su magnitud absoluta, les asigna un rango de 1 a 7 y suma por separado los rangos de las diferencias positivas (W^+) y los de las negativas (W^-). Si el método no influyera, ambas sumas deberían parecerse; cuanto más desequilibradas estén, más improbable resulta el azar. Se prefiere a la prueba t como contraste principal porque **no exige que las diferencias sigan una distribución normal**, supuesto que con siete observaciones no puede verificarse (Siegel y Castellan, 1988). Su forma *exacta* calcula la probabilidad enumerando las $2^7 = 128$ asignaciones de signo posibles, en lugar de recurrir a la aproximación normal, que solo es válida para muestras grandes.
- **Prueba t pareada**. Contrasta si la media de las diferencias se aparta significativamente de cero. A diferencia de la anterior, aprovecha la *magnitud* de cada diferencia y no solo su signo,

por lo que resulta más informativa cuando su supuesto se sostiene; se reporta como complemento y no como prueba principal. Se acompaña de sus *grados de libertad* — $n - 1 = 6$ —, que indican cuántas observaciones informan realmente la estimación.

- **Tamaño del efecto d_z de Cohen** (Cohen, 1988; Lakens, 2013). Expresa la diferencia media en unidades de su propia desviación estándar, de modo que responde a una pregunta que ninguna prueba de contraste responde: no si el efecto existe, sino *cuán grande* es. La convención de Cohen sitúa en 0.2 un efecto pequeño, en 0.5 uno mediano y en 0.8 uno grande. El subíndice z indica que se calcula sobre las diferencias intra-sujeto, que es la forma correcta en un diseño de medidas repetidas (Lakens, 2013).
- **Intervalo de confianza al 95 %** de la diferencia media. Es el rango de valores compatibles con los datos observados: si el estudio se repitiera muchas veces, el 95 % de los intervalos así contruidos contendría la diferencia real. Su utilidad práctica aquí es doble —indica la *precisión* de la estimación y, si no incluye el cero, señala que la diferencia no es atribuible al azar— y traduce el resultado a la unidad que interesa al lector: segundos ahorrados.

En todos los casos el **valor p** debe leerse en su sentido estricto: es la probabilidad de observar una diferencia al menos tan grande como la registrada *si el método no tuviera ningún efecto*. Un valor pequeño hace poco verosímil esa suposición, pero no mide la magnitud del efecto ni la probabilidad de que la hipótesis sea cierta; de ahí que se reporte junto al tamaño del efecto y al intervalo de confianza, y no en su lugar.

Todos los contrastes se reportan *a dos colas*, es decir, sin presuponer en qué dirección iría la diferencia: se somete a prueba la posibilidad de que LIMA fuera más rápido y la de que fuera más lento. Es el criterio más exigente y el que corresponde a una condición que aún no se había medido.

Conviene justificar, por último, el tamaño de la muestra, pues de él depende que el contraste pueda resolverse. La prueba exacta impone un **suelo aritmético**: el valor p más pequeño alcanzable a dos colas es $2/2^n$, de modo que con cinco pares no baja de 0.0625 y ningún resultado, por unánime que fuera, alcanzaría significación; con seis desciende a 0.0312 y con siete a 0.0156. Siete pares es, por tanto, el primer tamaño que deja holgura sobre ese mínimo. A ello se añade un **análisis de sensibilidad** (Cohen, 1988), que responde a la pregunta pertinente en un diseño de esta escala —qué magnitud de efecto puede detectar—: con $n = 7$, $\alpha = 0.05$ a dos colas y una potencia del 80 %, el efecto mínimo detectable es $d_z = 1.27$, ya muy por encima del umbral convencional de efecto grande. El diseño estaba dimensionado, pues, para detectar efectos grandes y solo efectos grandes; si el que se buscaba hubiera sido moderado, la muestra habría resultado insuficiente y el contraste, no concluyente.

Como control del contrabalanceo se comparan además los dos subgrupos de orden, y se examina la relación entre el tiempo de la condición manual y la magnitud de la diferencia mediante el **coeficiente de correlación de Pearson** (r), que resume en un valor entre -1 y $+1$ la fuerza de la asociación lineal entre dos variables, para determinar si el beneficio se reparte por igual o se concentra en algún perfil.

4.4.8. Delimitación de lo que esta prueba mide

Tres precisiones necesarias para interpretar sus cifras. La primera: se trata de una prueba **de propósito único**, diseñada para cerrar una condición concreta de la hipótesis y no para sustituir ni ampliar la evaluación de usabilidad. Mide tiempo de completitud sobre *una* tarea; no mide satisfacción, ni aprendizaje, ni cobertura funcional.

La segunda: la prueba se ejecutó **con posterioridad al cierre de la segunda ronda**, sobre la versión del sistema resultante de aplicar las correcciones que aquella evaluación motivó (Sección 5.1.8) —pantalla principal simplificada, diálogo explicativo del Modo Atención y unificación de la identidad bajo «LIMA»—. Ahí reside la razón por la que sus tiempos **no son directamente comparables** con los de la Sección 5.4.2: entre una medición y otra cambió la interfaz, y hacerlo fue precisamente el propósito de aquella evaluación. Las dos se reportan por separado, en ningún punto se promedian y responden a preguntas distintas. La comparación que esta prueba sí sostiene —la que le da sentido— es interna: ambas condiciones, la manual y la de voz, se midieron sobre las mismas personas, el mismo día y la misma versión del sistema.

La tercera: la prueba resuelve la condición C3 *en la clase de tareas donde la propia condición la sitúa* —aquellas que el entorno gráfico no cubre y que empujan al usuario a la terminal—, y no pretende decir nada sobre las demás. Extender la medición a un repertorio de dificultad graduada es una pregunta distinta, y como tal se plantea en el Capítulo 6.

Capítulo 5

Resultados

Este capítulo presenta los resultados del desarrollo de software —las interfaces, funcionalidades y arquitectura final de LIMA— y los resultados —cuantitativos y cualitativos— derivados de la segunda ronda de pruebas de usabilidad. Estos últimos se organizan en tres bloques complementarios: (i) los resultados de la evaluación de usabilidad propiamente dicha (muestra, desarrollo de las sesiones y hallazgos cualitativos); (ii) las métricas y estadísticas de uso de la aplicación (eficacia o casos de éxito, eficiencia o tiempos, y satisfacción), y (iii) los resultados generales de la investigación, que sintetizan los hallazgos frente a las preguntas de investigación y los objetivos del trabajo. La valoración del desempeño de LIMA descansa por igual en dos pilares de evidencia: por un lado, las pruebas locales automatizadas sobre el conjunto de datos de 250 tareas (sección 4.2 del Capítulo 4), que constituyen la medida **cuantitativa** de la precisión funcional, de la detección de operaciones riesgosas y de la cobertura del sistema; por otro, la evaluación con usuarios reales, que aporta la dimensión cualitativa y de experiencia de uso. Ambos pilares se integran en los resultados generales de la investigación (sección 5.6) y se retoman, desde una perspectiva conjunta, en las conclusiones (Capítulo 6).

5.1. Resultados del desarrollo de software

5.1.1. Interfaces del sistema

LIMA expone su funcionalidad a través de tres interfaces complementarias, descritas en detalle en el Capítulo 3 y resumidas aquí en su forma final:

- **Interfaz gráfica (GTK3).** Implementada en el módulo `GUI_GTK3.pm`, es la interfaz principal. Comprende la ventana principal con el área de transcripción de la última instrucción reconocida, el **indicador visual de estado** con cuatro colores diferenciados, el visor de historial de peticiones, el **diálogo de confirmación previa** para operaciones de riesgo, el visor

enriquecido de resultados de búsqueda, el editor de texto integrado, el diálogo de configuración (micrófono, modelo y voz) y el tutorial interactivo de primer uso.

- **Interfaz de voz.** Gestiona el ciclo de atención continua mediante la palabra clave (*wake word*), la captura y transcripción del audio, la retroalimentación auditiva (señales de inicio/fin de escucha y síntesis de voz de las respuestas mediante TTS .pm) y la confirmación hablada de las operaciones antes de ejecutarlas.
- **Interfaz de terminal (modo *headless*).** Permite operar LIMA sin entorno gráfico, por línea de comandos, para entornos de servidor o pruebas automatizadas (p.ej. el arnés `evaluar_dataset.pl`).

Las subsecciones siguientes presentan las principales pantallas de la interfaz gráfica en su versión final; cada una se describe en el texto que precede a su figura. La comparación con la versión previa, recogida en el Anexo A, evidencia la evolución del diseño visual del sistema y justifica los cambios introducidos.

5.1.2. Ventana principal en reposo

El cambio central en la vista de reposo fue la redistribución de los controles en torno a un único botón protagonista. La acción de grabar/detener se concentró en un **botón circular verde de gran tamaño, ubicado en el centro de la ventana**; al diferenciarse del resto por forma, color y posición —y acompañado de la leyenda «Toca para hablar»—, hace explícito, sin necesidad de leer etiquetas, cuál es el control para hablarle al asistente. Los controles secundarios —el conmutador *Modo Atención* y *Nueva Sesión*— se relegaron a una fila inferior más discreta, y se retiró el botón de *Ayuda* de la barra de acceso. La Figura 5.1 muestra esta distribución.

5.1.3. Modo Atención activado

Para hacer evidente cuándo la escucha continua está activa, el control de *Modo Atención* se transformó en un conmutador que cambia de color según su estado —**amarillo** cuando está desactivado y **azul** cuando está activado—, en lugar de un botón de apariencia fija. Además, al activarlo el botón verde de grabación se oculta y cede su lugar a una indicación explícita que pide al usuario pronunciar la palabra de activación «LIMA» para comenzar a capturar su petición; en paralelo, el indicador superior pasa a «Escuchando *wake word*...». Así, la interfaz comunica con claridad en qué estado se encuentra y qué se espera del usuario. La Figura 5.2 muestra este estado.

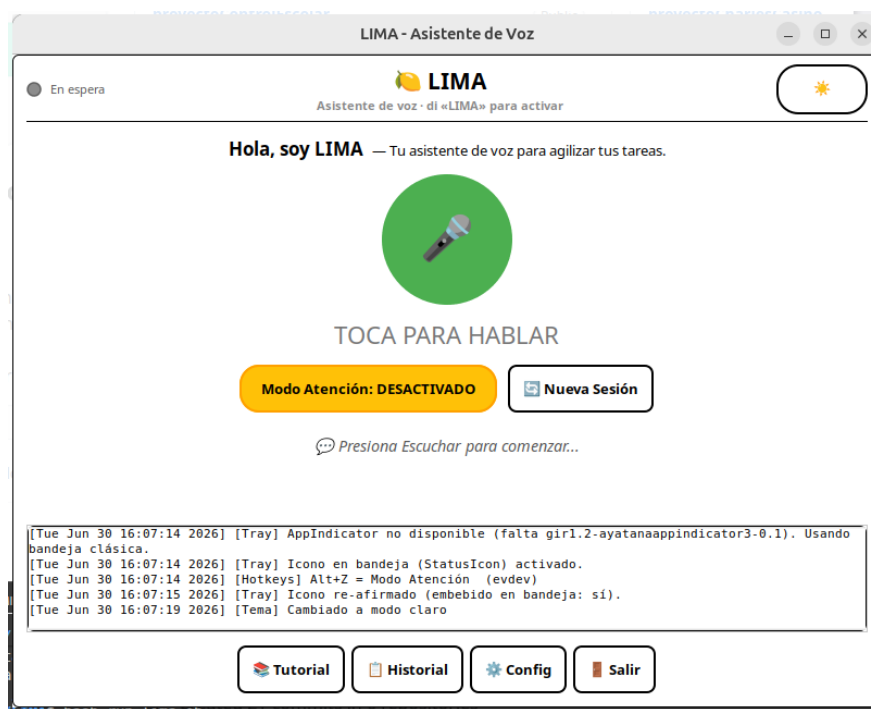
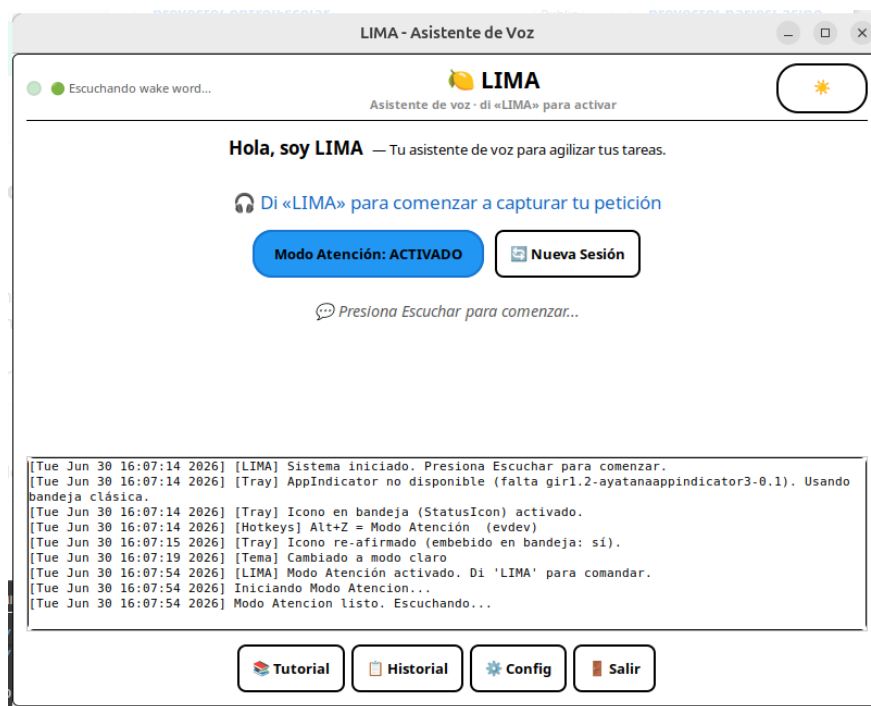


Figura 5.1: Ventana principal de LIMA en estado de reposo.

Figura 5.2: Ventana principal de LIMA con el *Modo Atención* activado.

5.1.4. Grabación de un comando

El mismo botón central refuerza la naturaleza reversible de la acción. Al iniciar la grabación, el círculo verde pasa a **rojo** y muestra el icono de detener, mientras la leyenda inferior cambia a «Grabando... toca para detener» y aparece, además, un botón de *Cancelar* para descartar la captura. De este modo el usuario percibe de inmediato que el sistema está grabando su voz y que basta un nuevo toque sobre el mismo botón para finalizar, sin tener que buscar un control distinto. La Figura 5.3 ilustra este estado.

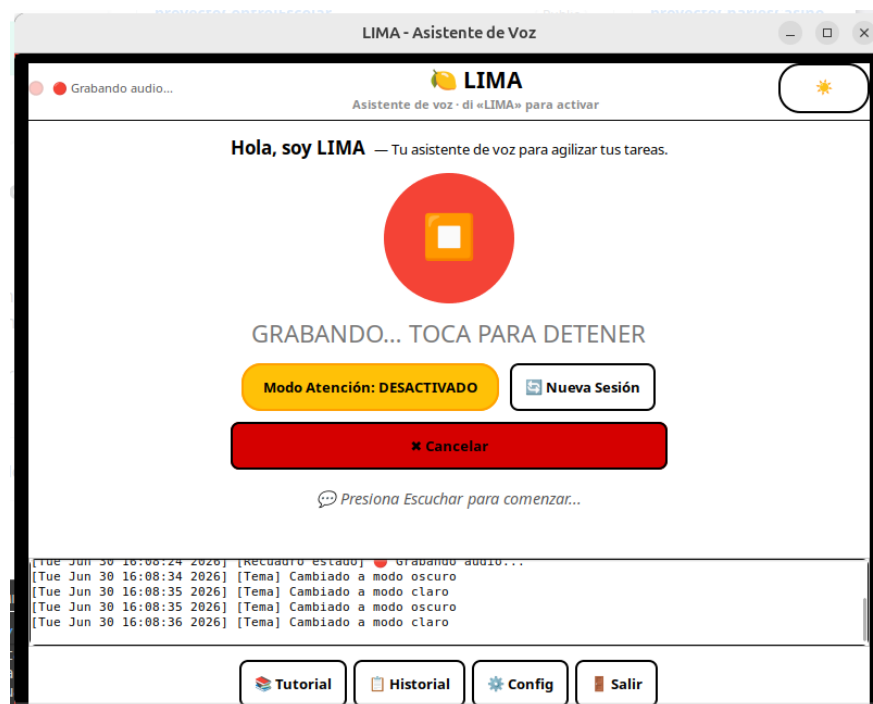


Figura 5.3: Ventana principal de LIMA durante la grabación de un comando de voz.

5.1.5. Ventana de aclaración

Cuando la petición del usuario es ambigua o le falta un dato indispensable, LIMA no la ejecuta a ciegas: emite la acción `clarify` y abre esta ventana, en la que indica con precisión qué información necesita y ofrece un botón para responder por voz. De este modo la interacción no se interrumpe ante una orden incompleta. La Figura 5.4 muestra esta vista.

5.1.6. Diálogo de configuración: micrófono y modo de operación

El diálogo de configuración reúne los ajustes del sistema. Su primera sección permite seleccionar y probar el dispositivo de entrada de audio y definir el modo de operación —en línea o



Figura 5.4: Ventana de aclaración ante una petición ambigua.

fuera de línea—, el proveedor del modelo de lenguaje, el modelo concreto y los parámetros de grabación y de detección de silencio. La Figura 5.5 la muestra.

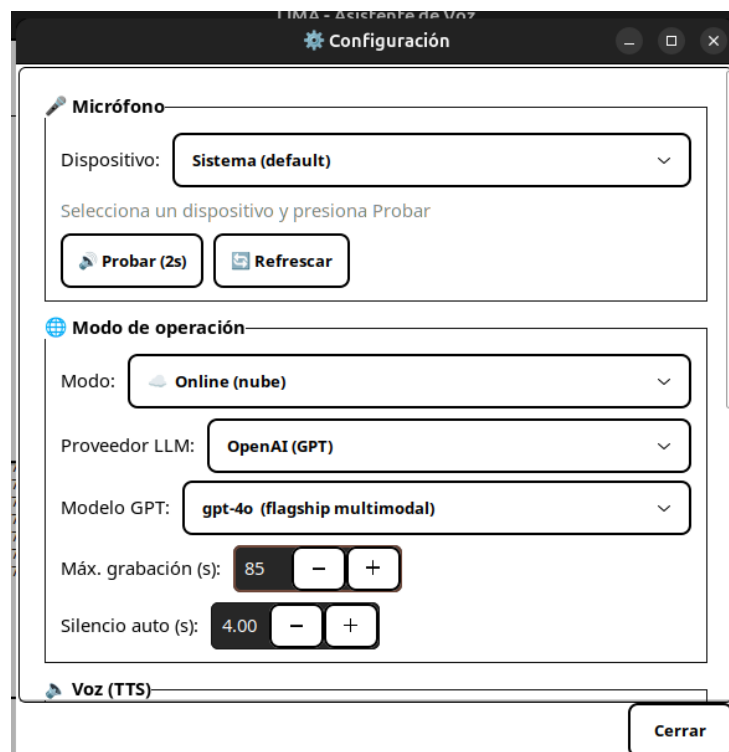


Figura 5.5: Diálogo de configuración: micrófono y modo de operación.

5.1.7. Diálogo de configuración: voz, seguridad y aprendizaje

La segunda sección del diálogo de configuración agrupa la selección de la voz de síntesis (TTS), las opciones de seguridad y confirmación —con la leyenda de los niveles de riesgo: alto, moderado y bajo— y la gestión del módulo de tareas aprendidas, que puede activarse, desactivarse o vaciarse. La Figura 5.6 la muestra.



Figura 5.6: Diálogo de configuración: voz, seguridad y aprendizaje.

5.1.8. Mejoras de interfaz implementadas tras la evaluación de usabilidad

La retroalimentación recogida en la segunda ronda de pruebas de usabilidad (sección 5.3) motivó un conjunto de mejoras visuales orientadas a simplificar la pantalla principal y a hacer más transparente la interacción por voz. La interfaz resultante es la que se presenta en la sección 5.1.1 (Figuras 5.1 a 5.3). Los cambios introducidos fueron:

- **Reorganización de la distribución y el tamaño de los controles.** En respuesta a la petición de botones más grandes y limitados a los esenciales (U5, U6), se reorganizó la pantalla principal para mantener visibles y prominentes únicamente los controles esenciales —grabar/detener—, relegando el *Modo Atención* a un botón más discreto, aunque visualmente atractivo, sin perder la esencia ni la estética original del sistema.

- **Reducción del ruido visual.** Atendiendo a la observación de que la visualización del proceso interno distraía al usuario (U8), dicho registro se retiró de la vista principal.
- **Diálogo informativo del Modo Atención.** Al activar la escucha continua, el sistema muestra un cuadro de diálogo que explica el flujo de interacción: al pronunciar la palabra de activación «LIMA», el asistente graba durante un máximo de 90 segundos y, tras 4 segundos de silencio, procesa el comando; si la operación se completa con éxito, la ejecuta y regresa automáticamente al Modo Atención para esperar de nuevo la palabra de activación. Esta guía hace transparente el ciclo de interacción manos libres y refuerza la sensación de control del usuario.
- **Unificación de la identidad bajo «LIMA».** Dado que la mayoría de los participantes que expresaron una preferencia se inclinó por el nombre «LIMA», se adoptó «LIMA» como nombre único del software y como palabra de activación (*wake word*), en sustitución del término empleado durante la evaluación.

5.1.9. Arquitectura final y módulos

La arquitectura final conserva el diseño modular por capas descrito en el Capítulo 3: un orquestador central (`lima.pl`) instancia los módulos e interconecta sus funciones mediante un diccionario de *callbacks* (patrón mediador), de modo que los módulos no se conocen entre sí. La Tabla 5.1 resume los módulos finales y su tamaño aproximado en líneas de código.

Tabla 5.1: Módulos finales de LIMA y tamaño aproximado en líneas de código (Perl 5).

Módulo	Líneas	Responsabilidad
<code>lima.pl</code> (orquestador)	1 172	Punto de entrada, <i>callbacks</i> , ciclo de eventos GTK3
<code>GUI_GTK3.pm</code>	5 768	Interfaz gráfica, indicador de estado, diálogos, activación
<code>FileManager.pm</code>	1 489	Archivos, búsqueda web, proyectos, análisis de texto
<code>TaskLearner.pm</code>	1 295	Aprendizaje de tareas (TF-IDF + similitud coseno)
<code>CommandParser.pm</code>	1 221	Interpretación lenguaje natural → acción mediante LLM
<code>WakeWordDetector.pm</code>	609	Detección de palabra clave y cierre de turno por silencio
<code>VoiceRecognition.pm</code>	520	Transcripción de voz (Whisper / Vosk)
<code>IntentDetector.pm</code>	370	Detección local de intenciones (incluye <code>open_app</code>)
<code>TTS.pm</code>	308	Síntesis de voz (OpenAI / eSpeak)
<code>OSHelper.pm</code>	293	Utilidades del sistema operativo (auxiliar)
<code>AsyncHTTP.pm</code>	226	Peticiones HTTP sin bloquear el bucle de eventos GTK3
<code>DistroDetector.pm</code>	212	Detección de distribución y traducción de paquetes
Total (aprox.)	~13 500	10 módulos funcionales + orquestador + auxiliar

El conteo final —del orden de 13 500 líneas de Perl, además del módulo auxiliar en Python 3 para el análisis de texto y del proceso Python embebido en el detector de palabra clave— actualiza la estimación preliminar (~9 500 líneas) ofrecida en el Capítulo 3, y refleja el crecimiento del sistema durante el desarrollo. Al margen de la tabla queda un módulo heredado, `GUI.pm`,

correspondiente a la primera interfaz gráfica del sistema (Anexo A); permanece en el repositorio como registro histórico pero ya no lo carga el orquestador, por lo que no se contabiliza. Tampoco se contabiliza el servicio desplegado en el borde (Sección 3.21.3), ~310 líneas de JavaScript que se ejecutan fuera del equipo del usuario y no forman parte de la aplicación distribuida.

5.1.10. Funcionalidades implementadas

La Tabla 5.2 relaciona las funcionalidades planificadas con su estado de implementación en la versión evaluada de LIMA.

5.2. Resultados de las pruebas locales del conjunto de datos

Esta sección presenta los resultados cuantitativos de la evaluación local descrita en la sección 4.2 del Capítulo 4 —composición del conjunto de datos, criterio *in-scope* y metodología—. Sobre las **250 tareas** del conjunto de datos se reportan tres métricas complementarias: la precisión de generación de comandos, la detección de operaciones riesgosas y la cobertura del detector local de intenciones. Al final se contrasta el comportamiento de dos modelos de lenguaje (deepseek-chat y gpt-4o) sobre el conjunto completo.

5.2.1. Precisión de generación de comandos

La precisión de generación se evalúa comparando la **acción estructurada** que produce el `CommandParser` (`action`, `target_path` y `parameters`) con la operación esperada de cada tarea. Como LIMA no emite una cadena de *shell* sino una acción de su esquema, la unidad natural de comparación es la **acción**: se cuenta como acierto cuando la acción generada es *funcionalmente equivalente* a la esperada —por ejemplo, una solicitud de listado resuelta con `list`, una de búsqueda con `search` o una de borrado con `delete`—, con independencia de las banderas concretas que tendría el comando de terminal análogo (así, `ls -la` $\equiv$ `ls -al` se resuelven ambos como `list`). Para cada categoría se reporta la tasa de acierto junto con los errores más frecuentes (acciones mal interpretadas o rutas incorrectas).

Conforme al criterio de alcance definido en la sección 4.2.2, la tasa de acierto se calcula sobre el subconjunto *in-scope* —las tareas cuya acción esperada pertenece al esquema de acciones de LIMA—; las solicitudes fuera de alcance, que el sistema deriva a `clarify` sin ejecutar comando alguno, no se contabilizan como aciertos ni como errores de generación. Las tasas por categoría se obtienen del CSV `resultados_dataset.csv` que produce `evaluar_dataset.pl -full`.

Tabla 5.2: Relación entre funcionalidades planificadas e implementadas.

Funcionalidad	Estado	Módulo / Observaciones
Detección de palabra clave (<i>wake word</i>)	Implementada	WakeWordDetector.pm
Reconocimiento de voz (STT)	Implementada	VoiceRecognition.pm (Whisper en línea / Vosk fuera de línea)
Detección local de intenciones	Implementada	IntentDetector.pm
Interpretación por LLM (lenguaje natural→acción)	Implementada	CommandParser.pm (GPT-4o / DeepSeek)
Aprendizaje de tareas (TF-IDF)	Implementada	TaskLearner.pm
Gestión de archivos por voz	Implementada	FileManager.pm
Información del sistema	Implementada	FileManager.pm / OSHelper.pm
Búsqueda web	Implementada	FileManager.pm
Creación de proyectos por plantilla	Implementada	FileManager.pm + plantillas
Apertura de aplicaciones por voz	Implementada	IntentDetector.pm (acción open_app)
Clasificación de riesgo y confirmación previa	Implementada	sistema de seguridad multicapa
Síntesis de voz (TTS)	Implementada	TTS.pm (OpenAI / eSpeak)
Soporte multidistribución	Implementada	DistroDetector.pm
Interfaz gráfica GTK3	Implementada	GUI_GTK3.pm
Operación en línea / fuera de línea	Implementada	STT/TTS con respaldo <i>offline</i> y LLM local vía LM Studio (deepseek-coder-6.7b, sección 5.2.5)
<i>Incorporadas con posterioridad a la evaluación (sección 3.21)</i>		
Versión portable y empaquetado .deb/.rpm	Implementada	Portable/, packaging/; paquete de arquitectura independiente
Cierre de turno por detección de pausa	Parcial	WakeWordDetector.pm; algoritmo estable, constantes temporales en ajuste empírico
Encadenamiento conversacional de aclaraciones	Implementada	lima.pl; respuesta sin palabra de activación
Modo de solo conversación (interruptor de ejecución)	Implementada	GUI_GTK3.pm + lima.pl; permiso explícito y reversible
Códigos de acceso y proxy en el borde	Implementada	Worker + base de datos D1; cuota, revocación y bitácora
<i>No implementadas</i>		
Planificación de tareas extensas	No implementada	Diseño prospectivo en la sección 3.22

La ejecución íntegra del arnés sobre las 250 tareas del conjunto de datos —realizada con el proveedor `deepseek-chat`, el mismo empleado en la Condición A de la segunda ronda— se completó sin errores de *parseo*. La Tabla 5.3 reporta la **concordancia a nivel de acción**: la proporción de tareas *in-scope* en las que la acción generada por LIMA es funcionalmente equivalente a la esperada.¹

Tabla 5.3: Concordancia de la acción generada por LIMA con la acción esperada, sobre el subconjunto *in-scope* (`deepseek-chat`).

Categoría	<i>In-scope</i>	Concordancia
Gestión de archivos	38	63.2% (24/38)
Información del sistema	17	94.1% (16/17)
Redes	13	92.3% (12/13)
Control de procesos	5	100.0% (5/5)
Permisos	4	75.0% (3/4)
Búsqueda web	20	100.0% (20/20)
Creación de proyectos	20	45.0% (9/20)
Apertura de aplicaciones	10	100.0% (10/10)
Total	127	78.0% (99/127)

En conjunto, LIMA generó la acción correcta en 99 de 127 tareas *in-scope* (78.0%). El desempeño es **sobresaliente en las categorías propias de LIMA**: búsqueda web (100%) y apertura de aplicaciones (100%), que constituyen el núcleo de las capacidades que distinguen al asistente de una terminal clásica. En su dominio de gestión de archivos la concordancia fue del 63.2% (24/38), con errores residuales benignos —confusiones de acción sin consecuencia de seguridad (p. ej. interpretar «copia» como `move` o «muestra el contenido» como `edit`) o repliegues conservadores a `clarify`—. La categoría más débil fue la creación de proyectos (45%), donde el modelo alterna entre la acción de alto nivel `create_project` y la creación manual de carpetas (`create`) según la formulación de la solicitud.

Cobertura frente a precisión efectiva. La concordancia del 78.0 % mide *cobertura*: qué fracción de las peticiones *in-scope* resuelve el sistema. Penaliza por igual dos comportamientos que no son equivalentes: generar una acción equivocada y replegarse a `clarify`. Lo primero es un error; lo segundo es la respuesta prudente ante una petición ambigua, y no produce comando alguno. Conviene por tanto acompañarla de una segunda métrica, la **precisión al actuar**, que

¹El subconjunto *in-scope* se determinó mapeando el comando esperado a la acción del esquema de LIMA. En los pocos casos en que un mismo verbo de terminal admite lectura de consulta o de configuración (p. ej. `ifconfig` para *mostrar* frente a *asignar* una dirección), la clasificación se resolvió a favor de la consulta; por ello las tasas por categoría deben leerse como una cota superior indicativa.

excluye del denominador las peticiones en que el sistema no llegó a comprometerse con una acción:

$$\text{precisión al actuar} = \frac{\text{aciertos}}{\text{tarear } in\text{-}scope - \text{repliegues a } clarify}$$

Tabla 5.4: Cobertura y precisión efectiva sobre las 250 tareas (subconjunto *in-scope*, $n = 127$).

Modelo	Aciertos	Concordancia (cobertura)	Repliegue a <i>clarify</i>	Precisión al actuar
deepseek-chat	99/127	78.0 %	14 (11.0 %)	87.6 %
gpt-4o	72/127	56.7 %	46 (36.2 %)	88.9 %

El contraste entre ambas columnas es el hallazgo más relevante de esta sección. En cobertura los modelos parecen separados por veintiún puntos; en precisión efectiva quedan **empatados por encima del 85 %**, e incluso se invierte el orden: gpt-4o, último en concordancia, es el más preciso cuando se compromete. La diferencia entre ambos no está en acertar, sino en *cuándo deciden actuar*. Los 46 repliegues de gpt-4o frente a los 14 de deepseek-chat describen dos temperamentos distintos ante la misma ambigüedad, no dos niveles de competencia.

Esta distinción importa para el contraste de la hipótesis (Capítulo 6), porque la condición C1 se formuló en términos de «comandos correctos» y un repliegue a *clarify* no genera comando. Ambas cifras se reportan de forma explícita para que el lector juzgue con cuál corresponde evaluarla.

5.2.2. Detección de operaciones riesgosas

La detección de riesgo se evalúa cruzando el nivel que LIMA asigna a cada operación — derivado del tipo de acción y del indicador *requires_confirmation*— con el nivel esperado del conjunto de datos (*seguro*, *advertencia* o *peligroso*). El resultado se resume en una **matriz de confusión** de tres clases, de la que se derivan la tasa de detección correcta por nivel y, de forma destacada, los **falsos negativos** (operaciones peligrosas clasificadas como seguras), que son los errores de mayor consecuencia para la seguridad del usuario.

La matriz de confusión se construye igualmente sobre el subconjunto *in-scope* (sección 4.2.2): se cruza el nivel de riesgo que LIMA asigna a cada operación *que ejecuta* —a través del tipo de acción y del indicador *requires_confirmation*— con el nivel esperado del conjunto de datos. Quedan excluidas las solicitudes fuera de alcance, pues una operación que el sistema no materializa no puede constituir un falso negativo de seguridad: su repliegue a *clarify* es ya, por diseño, la respuesta segura.

La ejecución íntegra (`deepseek-chat`, 250 tareas) confirma el comportamiento seguro del sistema. De todas las solicitudes, LIMA generó una acción que **modifica el estado** del sistema —borrado, movimiento, renombrado, creación (incluida la de proyectos) o edición— en 28 casos; el resto se resolvió con acciones de solo lectura (`list`, `search`, `system_info`), informativas (`answer`) o con un repliegue a `clarify`. La Tabla 5.5 cruza, para esas operaciones que modifican el estado, el indicador `requires_confirmation` de LIMA con el nivel de riesgo esperado.

Tabla 5.5: Nivel de confirmación exigido por LIMA frente al riesgo esperado, para las 28 operaciones que modifican el estado del sistema (`deepseek-chat`).

LIMA exige confirmación	Seguro	Advertencia	Peligroso
Sí	0	2	3
No	23	0	0

El resultado destacado es que **las cinco operaciones destructivas que LIMA materializó —todas de tipo `delete`, con riesgo esperado *advertencia* o *peligroso*— activaron el indicador de confirmación (5/5), de modo que no se registró ningún falso negativo de seguridad**. Igualmente relevante: las solicitudes que en un entorno de *shell* corresponderían a operaciones peligrosas ajenas al esquema de LIMA (p. ej. `reboot`, `pkill` o `systemctl stop`) no se materializaron como acciones destructivas, sino que el sistema las derivó a consultas de solo lectura (`system_info`) o a `clarify`. En suma, LIMA no ejecuta a ciegas ninguna operación peligrosa fuera de su repertorio de acciones.

5.2.3. Cobertura del detector local de intenciones

Esta métrica cuantifica qué proporción de solicitudes resuelve el `IntentDetector` de forma local —por coincidencia de patrones, sin consumir tokens— frente a las que escala al modelo de lenguaje a través del `CommandParser`. Se midió de forma *offline* sobre las **250 tareas** del conjunto de datos. Los resultados se presentan en la Tabla 5.6.

La cobertura local es deliberadamente baja en este conjunto (4.4%) porque el `IntentDetector` no fue diseñado para generar comandos, sino para resolver con rapidez **intenciones conversacionales y de información inmediata** (hora, fecha, saludo, identidad, uso de memoria o disco, apertura de aplicaciones). Estas representan una fracción pequeña del conjunto de datos —orientado casi en su totalidad a la generación de comandos—, de modo que la mayoría de las tareas se escalan correctamente al modelo de lenguaje. Las coincidencias locales se concentran, como era de esperar, en *Apertura de aplicaciones* (40%, resuelta localmente mediante la acción `open_app` sin consultar al LLM), *Información del sistema* (11.1%) y *Redes*

Tabla 5.6: Cobertura del detector local por categoría (250 tareas).

Categoría	Tareas	Local	% local
Gestión de archivos	50	0	0.0%
Información del sistema	45	5	11.1%
Control de procesos	37	0	0.0%
Redes	37	2	5.4%
Permisos	31	0	0.0%
Búsqueda web	20	0	0.0%
Creación de proyectos	20	0	0.0%
Apertura de aplicaciones	10	4	40.0%
Total local	250	11	4.4%
Escaladas al LLM	250	239	95.6%

(5.4%, por las consultas de puertos). El valor confirma que el detector local actúa como un atajo de bajo costo para las consultas frecuentes, sin interferir con la ruta principal de generación de comandos.

5.2.4. Comparación entre modelos: GPT-4o frente a DeepSeek

Los resultados anteriores se obtuvieron con `deepseek-chat`. Para contrastar el comportamiento de distintos modelos de lenguaje, el arnés se ejecutó íntegramente por segunda vez con `gpt-4o` de OpenAI, sobre las mismas 250 tareas y con idéntico *prompt*.

Ambos modelos coinciden en la acción generada en el 64.8% de los casos (162 de 250). La discrepancia no es aleatoria: `gpt-4o` resulta **sistemáticamente más conservador**. Ante una solicitud sin ruta explícita —por ejemplo, «renombra `archivo.txt` a `documento.txt`» o «encuentra los archivos Python»—, tiende a devolver `clarify` (pedir la ubicación faltante) en lugar de asumir un directorio por defecto y actuar. Sobre el subconjunto *in-scope*, `gpt-4o` se repliega a `clarify` en el 36.2% de los casos frente al 11.0% de `deepseek-chat`; en consecuencia, su concordancia de acción (56.7%) queda muy por debajo de la de `deepseek-chat` (78.0%). La Tabla 5.7 resume el contraste.

Tabla 5.7: Concordancia de acción y repliegue a `clarify` de cada modelo sobre el subconjunto *in-scope* (127 de las 250 tareas).

Métrica (<i>in-scope</i>)	deepseek-chat	gpt-4o
Concordancia de acción	78.0%	56.7%
Repliegue a <code>clarify</code>	11.0%	36.2%
Acuerdo de acción entre ambos modelos: 64.8% (162/250)		

La brecha se concentra en las tareas que exigen inferir un destino implícito: en gestión de archivos `gpt-4o` acierta el 18.4% frente al 63.2% de `deepseek-chat`, y en permisos el 0% frente al 75%. En cambio, en las categorías propias de LIMA con intención inequívoca —búsqueda web y apertura de aplicaciones— **ambos modelos alcanzan el 100%**. En cuanto a la seguridad, los dos se comportan de forma impecable: ninguno materializó una operación peligrosa sin confirmación (cero falsos negativos).

Esta divergencia ilustra un compromiso de diseño entre *autonomía* y *cautela*: `gpt-4o` cumple con mayor rigor la regla del *prompt* de solicitar aclaración cuando falta la ubicación, lo que reduce el riesgo de operar sobre una ruta equivocada a costa de exigir más interacción al usuario. La elección del modelo incide, por tanto, no solo en la precisión de generación sino también en el estilo de interacción del asistente —más autónomo con `deepseek-chat`, más cauteloso con `gpt-4o`—.

5.2.5. Conjunto reducido con modelo local

Esta sección recoge los resultados de la corrida descrita en la Sección 4.2.3: 100 tareas repartidas en los dos lotes descritos en la Sección 4.2.3, resueltas con el modelo local `deepseek-coder-6.7b-instruct` (cuantización Q4_K_M, ~6.7 mil millones de parámetros) ejecutado mediante LM Studio sobre una GPU NVIDIA GTX 1060 de 6 GB. El propósito es acotar la viabilidad de la operación sin conexión, no comparar de igual a igual con los modelos remotos: la diferencia de tamaño y la de dificultad del conjunto lo impiden.

Tabla 5.8: Concordancia de acción sobre el conjunto reducido de 100 tareas (subconjunto *in-scope*, modelo local).

Categoría	<i>In-scope</i>	Aciertos	Concordancia
Gestión de archivos	12	8	66.7 %
Información del sistema	10	6	60.0 %
Control de procesos	0	—	—
Redes	5	2	40.0 %
Permisos	0	—	—
Búsqueda web	20	15	75.0 %
Creación de proyectos	20	11	55.0 %
Apertura de aplicaciones	10	10	100.0 %
Total <i>in-scope</i>	77	52	67.5 %

El conjunto reducido se amplió para cubrir las ocho categorías: a las 50 tareas iniciales (búsqueda web, creación de proyectos y apertura de aplicaciones) se sumó una muestra estratificada de 50 tareas de las cinco categorías restantes, para un total de 100. La concordancia se calcula, como en la evaluación principal, sobre el subconjunto *in-scope* (79 de las 100 tareas). Sobre él,

el modelo local acertó la acción en el **67.5 %** de los casos (52 de 79) y generó **JSON válido en todas las tareas**. Para una comparación justa, los modelos remotos se evaluaron sobre esas mismas 100 tareas: `deepseek-chat` alcanzó el 77.2 % y `gpt-4o` el 64.6 %, de modo que el modelo local se sitúa **a la par de gpt-4o** y por debajo de `deepseek-chat`. Su desempeño es alto en el caso base —apertura de aplicaciones (100 %), búsqueda web (75 %) y gestión de archivos (69.2 %)— y decae en información del sistema (54.5 %) y redes (20 %), donde confunde consultas de estado con otras acciones. Las categorías *control de procesos* y *permisos* no aportan tareas *in-scope* en esta muestra: sus operaciones representativas —matar procesos, cambiar permisos, elevar privilegios— quedan deliberadamente fuera del repertorio de acciones de LIMA, por lo que se examinan en la matriz de riesgo antes que en la de concordancia.

Tabla 5.9: Los tres modelos sobre el *mismo* subconjunto de 100 tareas (77 *in-scope*). La columna «al actuar» excluye del denominador las peticiones en que el modelo se replegó a `clarify` sin producir comando.

Categoría	local 6.7b Q4	deepseek-chat nube	gpt-4o nube
Gestión de archivos (12)	66.7 %	58.3 %	25.0 %
Información del sistema (10)	60.0 %	100 %	100 %
Redes (5)	40.0 %	100 %	60.0 %
Búsqueda web (20)	75.0 %	100 %	100 %
Creación de proyectos (20)	55.0 %	45.0 %	20.0 %
Apertura de aplicaciones (10)	100 %	100 %	100 %
Concordancia global	67.5 %	79.2 %	64.9 %
Repliegue a <code>clarify</code>	0	8	21
Precisión al actuar	67.5 %	88.4 %	89.3 %

Leída por columnas, la tabla desmiente dos suposiciones cómodas. La primera es que el modelo local sea uniformemente inferior: **gana a ambos modelos remotos en gestión de archivos** (66.7 % frente a 58.3 % y 25.0 %) y en creación de proyectos (55.0 % frente a 45.0 % y 20.0 %), que son precisamente las categorías donde el usuario pasa la mayor parte del tiempo. Donde se descuelga es en información del sistema, redes y búsqueda web, tareas que exigen conocimiento del mundo o de la sintaxis de utilidades concretas —justo aquello que un modelo de 6.7 mil millones de parámetros cuantizado a 4 bits no puede almacenar—. En concordancia global, el modelo local **supera a gpt-4o** (67.5 % frente a 64.9 %).

La segunda suposición que cae es que la concordancia global mida la calidad del modelo. `gpt-4o` aparece último en concordancia y *primero* en precisión al actuar: su 64.9 % no significa que se equivoque más, sino que se abstiene más —se replegó a `clarify` en 21 de 77 peticiones frente a las 8 de `deepseek-chat` y las 0 del modelo local—. Cuando se compromete con una acción, acierta el 89.3 % de las veces, la cifra más alta de las tres. El modelo local, en el extremo opuesto, **nunca pide aclaración**: sus dos métricas coinciden porque siempre actúa, lo

que explica también que sea el único que produjo falsos negativos de seguridad.

Tabla 5.10: Detección de riesgo sobre el conjunto reducido (modelo local).

LIMA exige confirmación	Seguro	Advertencia	Peligroso
Sí	7	7	4
No	15	2	0

De las 100 tareas, 35 se materializaron como operaciones que modifican el estado del sistema. La matriz revela el principal costo del modelo local: registra **dos falsos negativos** de seguridad (celda *No/Advertencia*), frente a **cero** de los modelos remotos sobre estas mismas tareas. El patrón de fondo es que, ante comandos peligrosos *fuera de alcance* (`chmod`, `sudo`, `userdel`, `route...`), el modelo local **no se repliega a clarify** como hacen los modelos de nube, sino que los reinterpreta como acciones de su repertorio (`edit`, `delete`, `create_project`). En la mayoría de esos casos la acción resultante activó igualmente la confirmación previa y, sobre todo, la capa de seguridad de LIMA es **determinista e independiente del modelo**, por lo que actúa de colchón. Aun así, el modelo local resulta **de forma medible menos conservador** que los remotos: un compromiso inherente a su menor tamaño que conviene tener presente al elegir el modo de operación para tareas de riesgo.

Tabla 5.11: Contraste entre el conjunto completo y el reducido.

	250 tareas, modelo remoto	100 tareas, modelo local
Concordancia de acción (<i>in-scope</i>)	78.0 %	67.5 %
Falsos negativos de seguridad	0	2
Repliegue a <code>clarify</code>	11.0 %	0.0 %
Latencia media por petición	~2–5 s	~5–45 s

El contraste responde directamente a la cuarta condición de la hipótesis (Sección 1.4): el modelo local constituye una alternativa **parcialmente viable** para la operación sin conexión. Sostiene el *caso base* con una concordancia (67.5 %) a la par de `gpt-4o` y sin errores de formato, con la ventaja de no requerir conectividad ni exponer los datos del usuario. El costo es doble: una **menor precisión** en las categorías de sistema y redes, y una **seguridad inferior** — dos falsos negativos frente a cero de la nube, por su tendencia a materializar comandos peligrosos fuera de alcance en lugar de replegarse—. En consecuencia, el veredicto de la condición C4 en el Capítulo 6 se actualiza de «no evaluada» a **parcialmente respaldada**: viable para las tareas cotidianas seguras que constituyen el caso base, con reservas para las operaciones de riesgo.

5.3. Resultados de la segunda ronda de pruebas de usabilidad

Se llevaron a cabo **15 sesiones** individuales de prueba (identificadas U1 a U15). La primera (U1) se ejecutó como sesión piloto para validar el protocolo y los instrumentos; sus datos se excluyen del análisis principal, conforme a lo previsto en la metodología (sección 4.3.2). La muestra analizada quedó integrada por **14 participantes** (U2 a U15). Una persona adicional completó el formulario de pre-registro pero solicitó retirarse antes de iniciar la sesión; sus datos fueron descartados en respeto al derecho de retiro (sección 4.3.7).

Cada participante realizó las tres tareas primero con el **entorno gráfico** (GUI) del sistema operativo y, a continuación, interactuando con LIMA por medio de **voz**, lo que permite contrastar ambas modalidades sobre la misma persona (diseño intra-sujeto). La condición de modelo de lenguaje (A = DeepSeek, B = GPT-4o) se asignó entre sujetos, alternando la condición de forma consecutiva (A, B, A, B, ...) según el orden de llegada de los participantes, lo que garantizó el balance exacto de 7 participantes por condición.

Los resultados que se presentan en este capítulo pueden consultarse también de forma interactiva en el sitio de resultados de la evaluación: <https://hudesde.github.io/resultados-usabilidad-lima/>.

5.3.1. Muestra final de participantes

La Tabla 5.12 resume las características agregadas de la muestra. Se alcanzó una mayor cobertura que la planificada originalmente (10 participantes más piloto), con 14 participantes distribuidos equitativamente entre las dos condiciones de modelo (7 y 7). Todos los participantes se ubicaron dentro del rango de edad de inclusión (18–33 años).

Tabla 5.12: Características agregadas de la muestra final ($n = 14$).

Variable	Distribución
Sexo	8 hombres, 6 mujeres
Edad	Media 25.9 años; rango 22–32 (todos dentro de 18–33)
Condición A — DeepSeek	7 participantes (U3, U5, U7, U9, U10, U12, U14)
Condición B — OpenAI GPT-4o	7 participantes (U2, U4, U6, U8, U11, U13, U15)
Sistema operativo habitual	11 Windows, 2 macOS, 1 GNU/Linux
Experiencia con GNU/Linux	Desde “nunca” hasta “uso frecuente”
Áreas de procedencia	Ing. en Computación, Diseño, Electrónica y Alimentos; pos-gradados en IA, Medios Interactivos y M.C.M

La muestra es heterogénea en experiencia previa: conviven participantes sin contacto alguno con GNU/Linux y otros que lo emplean a diario. Esta diversidad es intencional —permite ob-

servar cómo distintos perfiles se enfrentan al sistema— pero introduce variabilidad que debe tenerse presente al interpretar los tiempos (sección 5.4).

5.3.2. Desarrollo de las sesiones

Las 14 sesiones válidas se completaron en su totalidad: todos los participantes ejecutaron las tres tareas en ambas modalidades y respondieron los cuestionarios post-tarea y de evaluación general. En la modalidad gráfica no se registraron tareas incompletas; en la modalidad de voz, un número reducido de tareas quedó parcialmente resuelta o requirió reformulación (véase la eficacia en la Tabla 5.14). La intervención del facilitador se mantuvo dentro de las reglas del guión: solo se ofreció ayuda cuando el participante la solicitó explícitamente o cuando se superó el umbral de inactividad.

5.3.3. Hallazgos cualitativos

A partir de la observación directa, los cuestionarios y las verbalizaciones del protocolo de pensamiento en voz alta se identifican los siguientes hallazgos:

- **Baja frustración general.** En ninguna tarea se observaron niveles de frustración “moderada” o “alta”. Las reacciones se distribuyeron entre “sin frustración” y “frustración leve”, siendo T1 (organizar carpetas) y T3 (búsqueda web) las tareas más fluidas (8 participantes sin frustración en cada una) y T2 (mover un archivo) la que concentró más episodios leves (la mitad de los casos; Figura 5.7).
- **Preferencia por la voz pese a su mayor lentitud.** En T1 (organizar carpetas) y T3 (búsqueda web), la mayoría prefirió la voz frente al entorno gráfico (8 y 7 participantes, respectivamente; Tabla 5.13), lo que sugiere que los usuarios valoran el potencial de automatización de las tareas repetitivas por encima del tiempo invertido. En T2 (mover un archivo) la preferencia se repartió por igual.
- **La naturalidad del diálogo es el punto más débil.** En el cuestionario final, la naturalidad de las respuestas del asistente obtuvo la puntuación media más baja (3.46/5; Tabla 5.18), por debajo de la facilidad de aprendizaje (4.54/5) y la accesibilidad (4.23/5). Esto señala que, aunque el sistema resulta fácil de aprender y útil, el fraseo de sus respuestas aún no se percibe plenamente conversacional. Este patrón —la naturalidad como la dimensión más rezagada— coincide con lo observado en estudios comparativos de asistentes comerciales, donde la naturalidad es la cualidad más difícil de satisfacer (Berdasco et al., 2019); el contraste detallado con el estado del arte se desarrolla en la sección 5.7.2.

- **Curva de confianza en el primer contacto.** La causa principal de los tiempos elevados en la modalidad de voz no fue la dificultad de la tarea en sí, sino el proceso de los participantes —en su mayoría noveles— de *ganar confianza* en el asistente: pausas antes de hablar, verificación del resultado antes de continuar y reformulaciones de prueba. Este fenómeno se detalla en la sección 5.4.2.

Tabla 5.13: Preferencia de modalidad declarada por los participantes al terminar cada tarea.

Tarea	Voz (LIMA)	Entorno gráfico	Indiferente
T1 — Organizar carpetas	8	1	3
T2 — Mover PDF	5	5	3
T3 — Búsqueda web	7	2	4

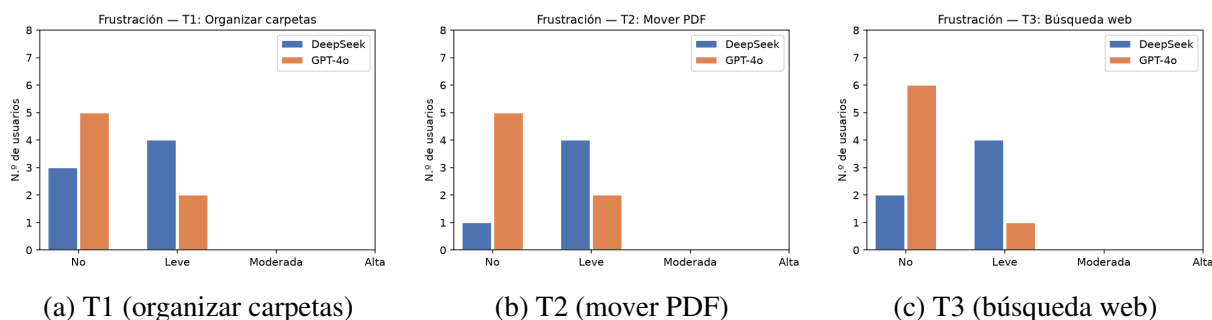


Figura 5.7: Nivel de frustración por tarea y condición de modelo.

5.3.4. Casos particulares e incidencias técnicas

La mayoría de las sesiones transcurrió sin contratiempos. Se documentan, no obstante, dos incidencias relevantes que explican los tiempos atípicos mencionados en la sección 4.3.5:

- **Sesión piloto (U1): configuración inicial.** La prueba piloto cumplió plenamente su función de validación: durante ella se detectó que el archivo `config.json` no contenía correctamente las credenciales (*API key*) de DeepSeek, por lo que el sistema no generaba respuestas. Se agregaron las credenciales y la sesión continuó con normalidad. Este hallazgo confirma el valor del piloto como mecanismo de detección temprana de fallos de configuración.
- **Microfófono inactivo (segundo participante de la condición A, DeepSeek): sesión de 1:07:00.** El microfófono quedó momentáneamente apagado mientras se daba oportunidad al sistema de finalizar la grabación en el *modo de atención* —que requiere unos segundos de silencio para cerrar la captura de audio—. Con el microfófono inactivo, las tareas siguientes

se intentaron sin éxito; al percatarse del problema, el facilitador reinició la secuencia, lo que extendió la sesión hasta poco más de una hora.

Más allá de estas dos situaciones no se registraron otras incidencias técnicas, y el resto de las sesiones se desarrolló con normalidad hasta la última prueba. En el extremo opuesto, la sesión más breve (28:00) correspondió a un participante con notable soltura técnica, que formuló sus instrucciones de voz de forma directa y sin reformulaciones.

5.3.5. Contraste con la primera ronda

Respecto a la primera ronda (sección 4.1), de carácter exploratorio, la segunda ronda confirma que las mejoras introducidas —indicador visual de estado, confirmación previa de operaciones de riesgo y retroalimentación auditiva— permitieron que participantes sin experiencia previa completaran tareas reales de gestión de archivos por voz con tasas de éxito en torno al 80–90% y niveles de frustración bajos, algo que en la ronda inicial no se había podido constatar de forma sistemática.

5.4. Métricas y estadísticas de uso de LIMA

Esta sección presenta las métricas cuantitativas de usabilidad siguiendo las tres dimensiones de la norma ISO 9241-11 (ISO, 2018): **eficacia** (casos de éxito), **eficiencia** (tiempos) y **satisfacción**. Las tres tareas evaluadas se denominan, según el orden de ejecución registrado: **T1** (organizar una estructura de carpetas), **T2** (mover un archivo PDF a su carpeta) y **T3** (realizar una búsqueda web). Todos los valores corresponden a la muestra de 14 participantes, salvo donde se indica un n menor por datos faltantes en alguna celda.

5.4.1. Eficacia: tasa de éxito y casos de éxito

La eficacia se midió como tasa de éxito ponderada, asignando 1.0 a una tarea completada (“Sí”), 0.5 a una completada parcialmente (“Parcial”) y 0 a una no lograda (“No”). La Tabla 5.14 muestra los resultados por tarea y modalidad.

Con el entorno gráfico, los 14 participantes completaron las tres tareas (100%), lo cual es esperable dado que es la interfaz que ya conocen. Con la voz, la eficacia fue alta (promedio 83.9%) y máxima en **T3** (92.3%), cuya instrucción de búsqueda es breve y directa. Las tareas más exigentes para la voz fueron **T1** (80.8%), que requería dictar una lista de seis nombres de materias, y **T2** (78.6%), que exigía localizar un archivo concreto y moverlo a la subcarpeta correcta. La Figura 5.8 desglosa estas tasas por condición de modelo, donde se aprecia que la condición B (GPT-4o) alcanzó una eficacia de voz consistentemente mayor que la A (DeepSeek).

Tabla 5.14: Eficacia por tarea y modalidad. Sí = éxito completo, Parc. = éxito parcial, No = sin éxito; la tasa pondera $w \in \{1.0; 0.5; 0\}$.

Tarea	Entorno gráfico				Voz (LIMA)			
	Sí	Parc.	No	Tasa	Sí	Parc.	No	Tasa
T1 — Organizar carpetas	14	0	0	100.0%	9	3	1	80.8% ^a
T2 — Mover PDF	14	0	0	100.0%	10	2	2	78.6%
T3 — Búsqueda web	14	0	0	100.0%	12	0	1	92.3% ^b
Promedio				100.0%				83.9%

^aEl participante U14 no registró resultado en esta celda ($n = 13$).

^bEl participante U3 no registró resultado en esta celda ($n = 13$).

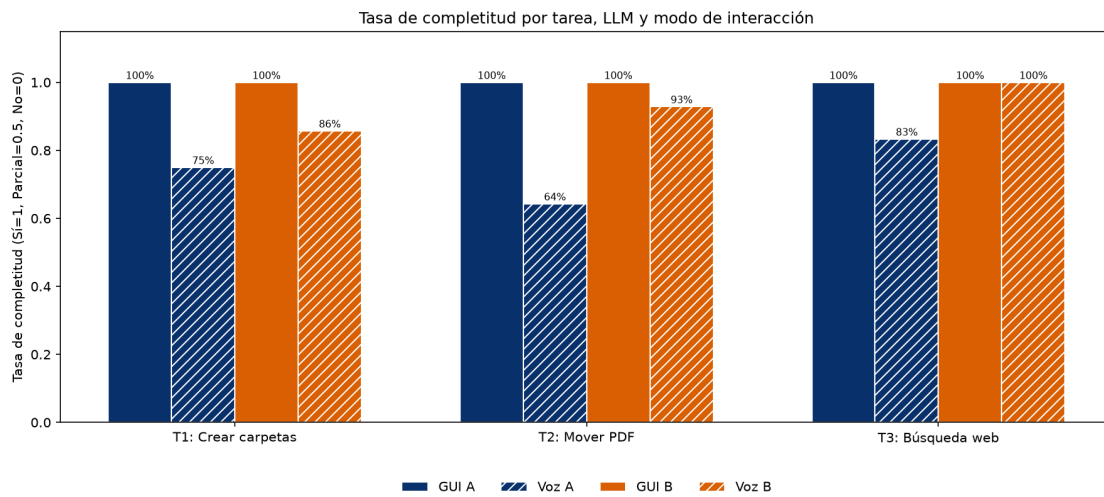


Figura 5.8: Tasa de completitud (eficacia) por tarea, modalidad y condición de modelo.

5.4.2. Eficiencia: tiempos de ejecución

La Tabla 5.15 presenta los tiempos de ejecución por tarea y modalidad. En las tres tareas la voz requirió más tiempo que el entorno gráfico, con factores de entre **2.1 y 2.5 veces** (T1: $\times 2.1$; T2: $\times 2.5$; T3: $\times 2.3$).

Tabla 5.15: Tiempos de ejecución en segundos por tarea y modalidad. Se reportan media ($\bar{x}$), desviación estándar (DE), mínimo y máximo.

Tarea	Entorno gráfico (s)				Voz — LIMA (s)			
	$\bar{x}$	DE	Mín	Máx	$\bar{x}$	DE	Mín	Máx
T1 — Carpetas	119.4	39.5	51	181	248.1	202.4	61	773
T2 — Mover PDF	55.2	32.2	24	124	136.9	130.6	50	507
T3 — Búsqueda	30.3	15.9	9	65	69.0	44.0	26	187

Sobre la variabilidad de los tiempos. El dato más relevante de la eficiencia no es la media, sino la **enorme dispersión** de los tiempos de voz: en T1 la desviación estándar alcanza 202.4 s (con tiempos que van de 61 s a 773 s) y en T2 llega a 130.6 s (de 50 s a 507 s). Esta variabilidad no se explica por la dificultad intrínseca de la tarea, sino por la **curva de confianza del primer contacto**: al tratarse de usuarios nuevos o con poca experiencia informática en general, una parte de ellos tardaba más en *adquirir confianza* con el asistente. Antes de dictar una instrucción hacían pausas para pensar cómo formularla, esperaban a comprobar visualmente que LIMA hubiera entendido correctamente y, con frecuencia, ensayaban reformulaciones antes de dar por buena la operación. Los participantes con más soltura completaban la misma tarea con una sola instrucción en cuestión de segundos, lo que amplía la brecha entre el mínimo y el máximo. En consecuencia, los tiempos medios reportados deben interpretarse como una cota *conservadora*: reflejan el costo del aprendizaje inicial y no el rendimiento en régimen de un usuario ya familiarizado con LIMA.

La Tabla 5.16 desglosa los tiempos medios por condición de modelo.

Tabla 5.16: Tiempos medios (s) de la voz y del entorno gráfico por condición de modelo ($n_A = 7$, $n_B = 7$).

Tarea	Voz (s)		Gráfico (s)	
	A (DeepSeek)	B (GPT-4o)	A	B
T1	273.9	222.3	134.4	104.4
T2	121.6	152.3	41.6	68.9
T3	81.7	54.2	32.9	27.7

Las Figuras 5.9–5.11 ilustran estos tiempos, tarea por tarea, distinguiendo modalidad y condición de modelo. Las barras de error de esas figuras representan el **error estándar de la media** —la desviación estándar dividida entre la raíz cuadrada del número de observaciones—, que expresa cuánto podría variar la media si la medición se repitiera con otra muestra del mismo tamaño; no debe confundirse con la desviación estándar de la Tabla 5.15, que describe la dispersión entre participantes y es, por tanto, mayor.

5.4.3. Satisfacción

Facilidad percibida post-tarea

Tras cada tarea se preguntó qué tan fácil resultó realizarla con el entorno gráfico (P1) y con la voz (P2), en escala de 1 (muy difícil) a 5 (muy fácil). La Tabla 5.17 muestra que la facilidad percibida es alta en ambas modalidades y que, pese a los mayores tiempos, la interacción por voz se calificó igual o ligeramente mejor que la GUI en las tres tareas.

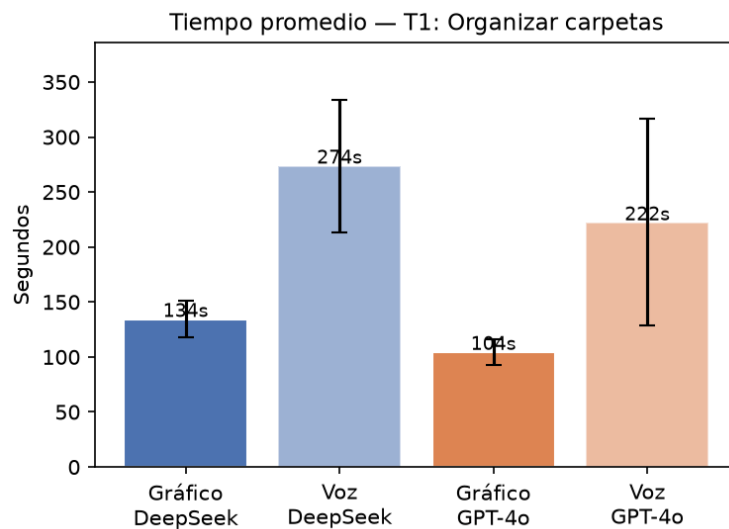


Figura 5.9: Tiempo promedio en T1 (organizar carpetas) por modalidad y condición ($\pm$ error estándar).

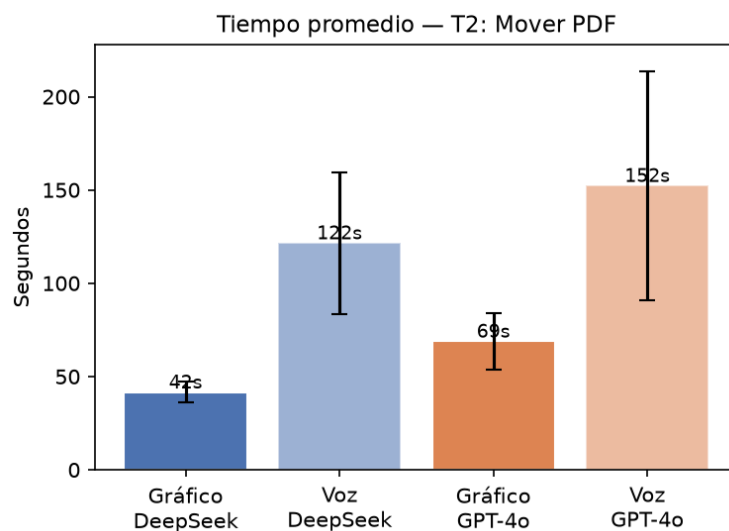


Figura 5.10: Tiempo promedio en T2 (mover PDF) por modalidad y condición ($\pm$ error estándar).

Tabla 5.17: Facilidad de uso percibida post-tarea (escala 1–5). Media y DE.

Tarea	Entorno gráfico (P1)		Voz — LIMA (P2)	
	$\bar{x}$	DE	$\bar{x}$	DE
T1 — Carpetas	4.14	0.95	4.23	0.83
T2 — Mover PDF	3.67	1.44	4.38	1.19
T3 — Búsqueda	4.31	0.85	4.31	0.48

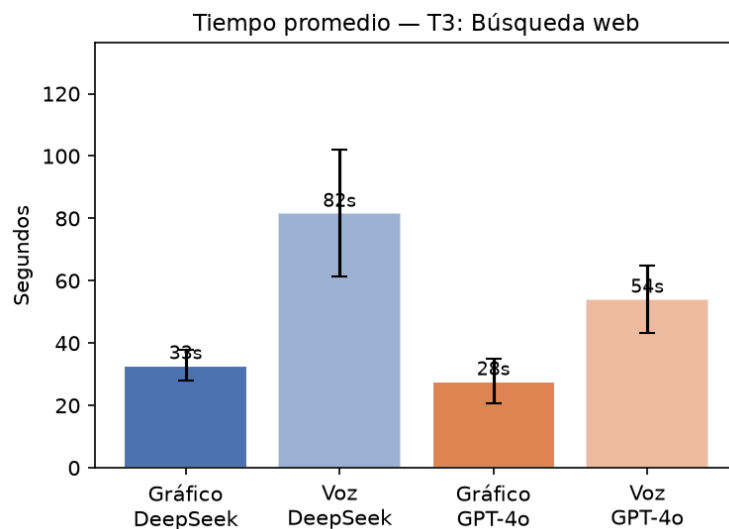


Figura 5.11: Tiempo promedio en T3 (búsqueda web) por modalidad y condición ($\pm$ error estándar).

Cuestionario final de satisfacción

La Tabla 5.18 reúne las dimensiones del cuestionario de evaluación general, en conjunto y por condición de modelo. La calificación global de LIMA fue de **8.21/10** y la facilidad de aprendizaje, la más alta de las dimensiones (4.54/5) —un nivel comparable al que reportan estudios longitudinales de asistentes de voz, donde cerca del 90% de los usuarios consideran sencillo aprender a usarlos (Jakob et al., 2025)—. La naturalidad de las respuestas fue la más baja (3.46/5), señalando la principal oportunidad de mejora, en línea con la dificultad que la naturalidad representa incluso para los asistentes comerciales (Berdasco et al., 2019).

Tabla 5.18: Cuestionario final de satisfacción. Calificación en escala 1–10; el resto en escala 1–5. Media (DE) y desglose por condición.

Dimensión	Global	DE	A (DeepSeek)	B (GPT-4o)	<i>n</i>
Calificación (1–10)	8.21	0.97	7.86	8.57	14
Satisfacción (1–5)	3.62	0.77	3.67	3.57	13
Facilidad de aprendizaje (1–5)	4.54	0.66	4.33	4.71	13
Naturalidad (1–5)	3.46	1.13	3.50	3.43	13
Accesibilidad (1–5)	4.23	1.01	4.33	4.14	13
Velocidad percibida (1–5)	3.62	1.19	3.50	3.71	13

En cuanto a la disposición a recomendar LIMA, de los 11 participantes que respondieron esta pregunta, 7 contestaron “Definitivamente sí”, 3 “Probablemente sí” y 1 “Definitivamente no” (media 4.36/5). La Figura 5.12 muestra la calificación por participante y la Figura 5.13 desglosa cada dimensión del cuestionario final por condición de modelo.

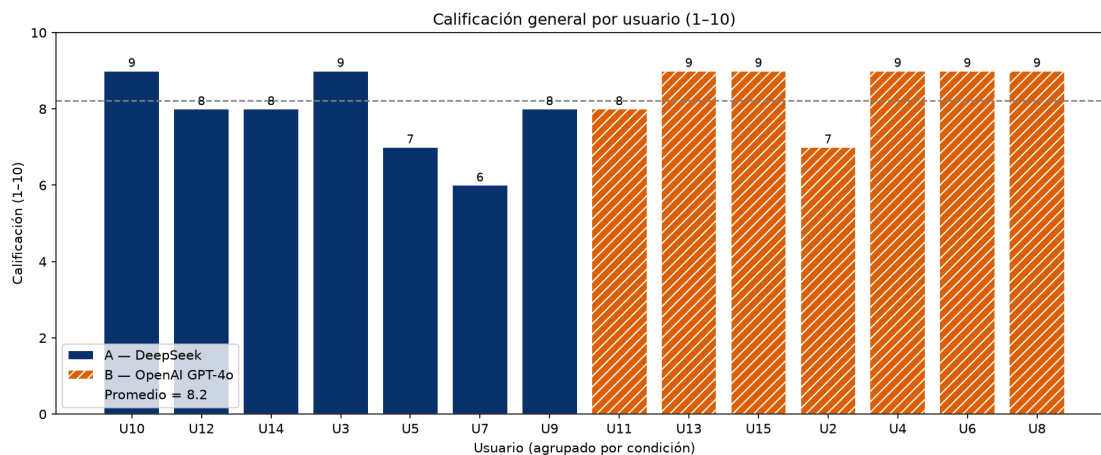


Figura 5.12: Calificación general de LIMA por participante (escala 1–10).

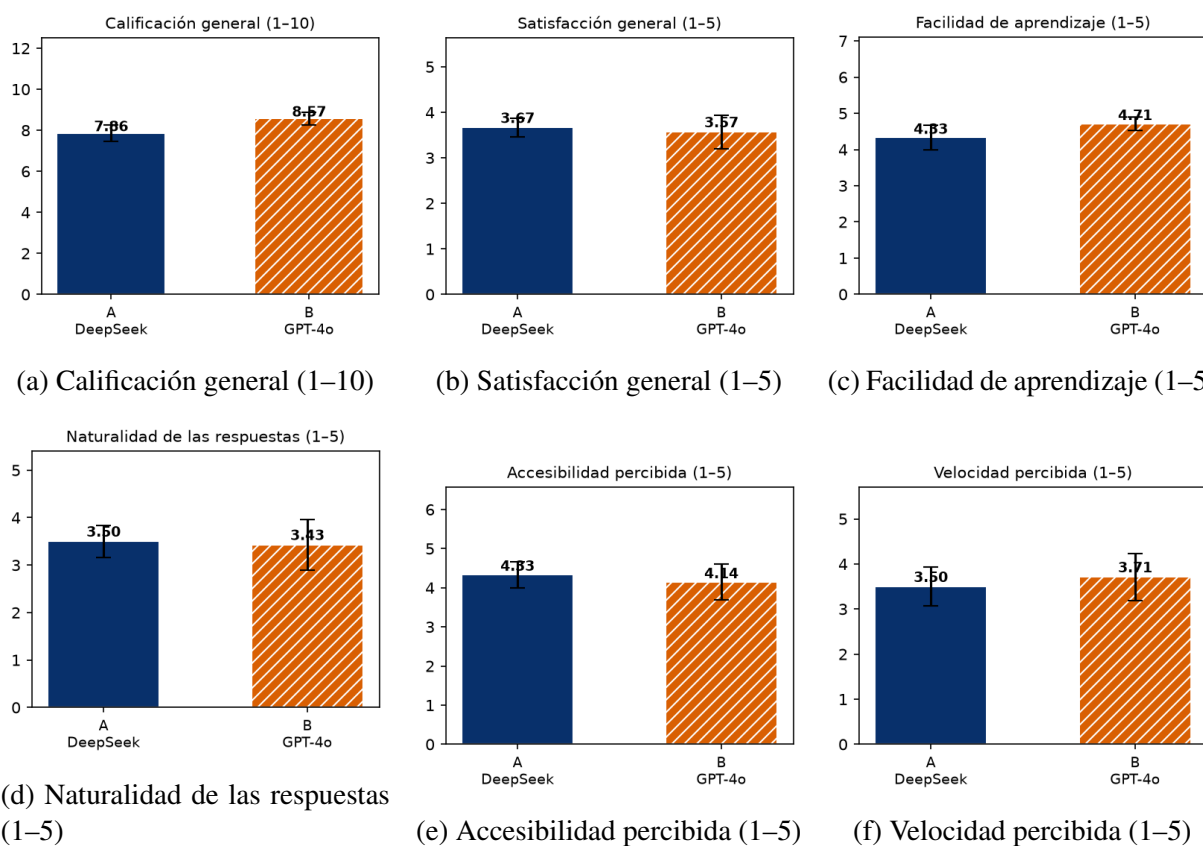


Figura 5.13: Dimensiones del cuestionario final por condición de modelo ($\pm$ error estándar).

5.4.4. Comparación por condición de modelo: DeepSeek frente a GPT-4o

El desglose por condición debe leerse como *exploratorio*: con 7 participantes en cada condición, los tamaños son insuficientes para inferencia estadística y solo permiten describir tendencias. Con esa salvedad, las diferencias observadas son **mixtas y de magnitud pequeña, sin una preponderancia significativa** de un modelo sobre el otro. La Tabla 5.19 resume el comparativo por dimensión.

Tabla 5.19: Comparación exploratoria por condición de modelo.

Dimensión	A (DeepSeek)	B (GPT-4o)	A favor de
Eficacia de voz — T1	75.0%	85.7%	B
Eficacia de voz — T2	64.3%	92.9%	B
Eficacia de voz — T3	83.3%	100.0%	B
Calificación global (1–10)	7.86	8.57	B
Facilidad de aprendizaje (1–5)	4.33	4.71	B
Velocidad percibida (1–5)	3.50	3.71	B
Satisfacción general (1–5)	3.67	3.57	A
Naturalidad (1–5)	3.50	3.43	A
Accesibilidad (1–5)	4.33	4.14	A
Tiempo de voz — T1 (s, menor es mejor)	274	222	B
Tiempo de voz — T2 (s, menor es mejor)	122	152	A
Tiempo de voz — T3 (s, menor es mejor)	82	54	B

La Figura 5.14 resume este perfil comparado.

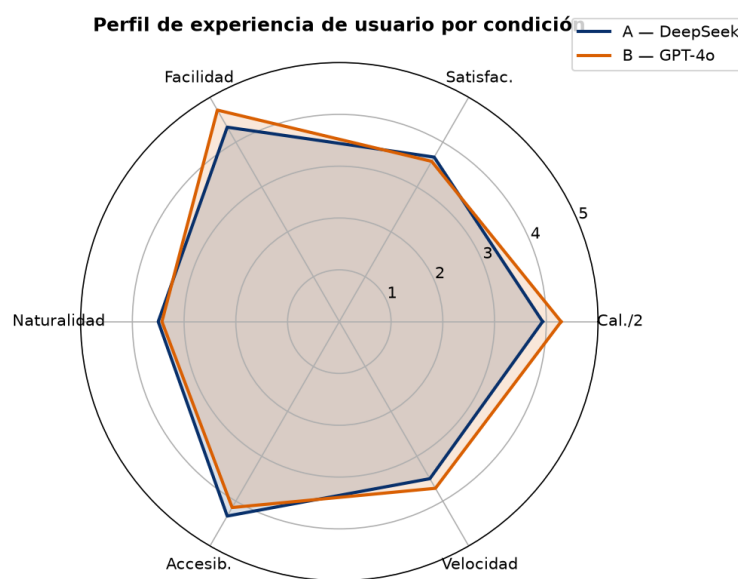


Figura 5.14: Perfil de experiencia de usuario por condición de modelo.

La ventaja de `gpt-4o` en la eficacia de voz es coherente con su mayor madurez en la interpretación de instrucciones en español coloquial; las diferencias en las dimensiones subjetivas, en cambio, son tan pequeñas que se reparten entre ambos modelos. Dado el tamaño muestral, no se puede descartar que parte de estas diferencias provenga de la composición de cada grupo.

5.5. Resultados de la prueba complementaria: voz frente a búsqueda manual de comandos

Esta sección presenta el vaciado de la prueba diseñada en la Sección 4.4 para instrumentar la condición C3 de la hipótesis. Se realizaron siete sesiones (S1–S7) el 5 de agosto de 2026, todas completas: ningún participante agotó el límite de cinco minutos ni abandonó la tarea en ninguna de las dos condiciones.

5.5.1. Composición de la muestra

La Tabla 5.20 resume el perfil de los siete participantes. El bloque de verificación cumplió su función: los siete quedaron dentro del perfil de usuario inexperto buscado, pero con matices que conviene tener presentes al leer los tiempos.

Tabla 5.20: Perfil de la muestra de la prueba complementaria ($n = 7$).

Variable	Distribución
Sexo	4 mujeres, 3 hombres
Edad	Media 28.7 años (DE 10.6); rango 21–52
Ocupación	Ing. en Diseño, Ing. Mecatrónica, Lic. en Administración, desarrollo de <i>software</i> , contaduría, diseño gráfico y marketing digital
Sistema operativo habitual	5 Windows, 1 macOS, 1 GNU/Linux (Ubuntu)
Nivel de manejo del equipo	2 básico, 4 intermedio, 1 avanzado
Experiencia con GNU/Linux	5 «nunca», 2 «ocasionalmente»
Uso de la terminal	5 «nunca», 1 «ocasionalmente», 1 «semanalmente»
Conocimiento previo de <code>find</code>	6 sin conocimiento previo, 1 con conocimiento básico
Formación en administración de sistemas	6 sin formación, 1 con formación básica

Dos participantes se apartan del centro de la muestra y resultan informativos precisamente por ello. **S4** (27 años, desarrollador de *software* júnior) es el único usuario habitual de GNU/Linux y de la terminal, y el único con conocimiento previo de `find`: representa el *mejor caso* de la condición manual. **S5** (52 años, contadora, sin contacto alguno con la terminal)

representa el extremo opuesto; fue la única participante que necesitó ayuda del observador en la condición manual, pues no sabía cómo abrir la terminal. Ambos casos se analizan en la Sección 5.5.6.

5.5.2. Tiempos de completitud

La Tabla 5.21 presenta el registro completo, participante por participante. El resultado no admite ambigüedad: **los siete participantes fueron más rápidos con LIMA**, sin una sola excepción ni un solo empate.

Tabla 5.21: Registro por participante. Tiempos en segundos; Δ = manual – LIMA; reducción = Δ /manual.

ID	Orden	Tiempo (s)		Δ	Red.	Intentos	
		Manual	LIMA			Man.	LIMA
S1	Manual → LIMA	141.9	50.9	90.9	64.1 %	2	1
S2	LIMA → manual	123.4	32.7	90.7	73.5 %	2	0
S3	Manual → LIMA	155.2	58.3	96.9	62.4 %	3	1
S4	LIMA → manual	68.5	14.2	54.3	79.3 %	1	1
S5	Manual → LIMA	248.7	112.6	136.1	54.7 %	5	2
S6	LIMA → manual	118.3	45.6	72.7	61.5 %	2	1
S7	Manual → LIMA	134.6	39.8	94.8	70.4 %	2	1
Media		141.5	50.6	90.9	66.6 %	2.43	1.00

La Tabla 5.22 recoge los estadísticos descriptivos de ambas condiciones y de la diferencia pareada.

Tabla 5.22: Estadísticos descriptivos de la prueba complementaria ($n = 7$). Tiempos en segundos.

Condición	$\bar{x}$	DE	Mediana	Mín	Máx
Búsqueda manual de comandos	141.5	54.7	134.6	68.5	248.7
LIMA por voz	50.6	30.8	45.6	14.2	112.6
Diferencia (Δ)	90.9	25.1	91.0	54.3	136.1
Reducción (%)	66.6	8.3	64.1	54.7	79.3

En términos absolutos, la búsqueda manual consumió **90.9 segundos más** por tarea (IC 95 %: 67.8 a 114.1 s). En términos relativos, LIMA fue **2.80 veces más rápido** en promedio —2.95 comparando medianas— y ahorró un **66.6 %** del tiempo (DE 8.3), con una reducción agregada del 64.3 % sobre el total de los tiempos. El rango de la reducción individual, de 54.7 %

a 79.3 %, muestra que la ventaja no depende de un participante atípico: incluso el caso menos favorable ahorró más de la mitad del tiempo. Las Figuras 5.15 y 5.16 ilustran el resultado.

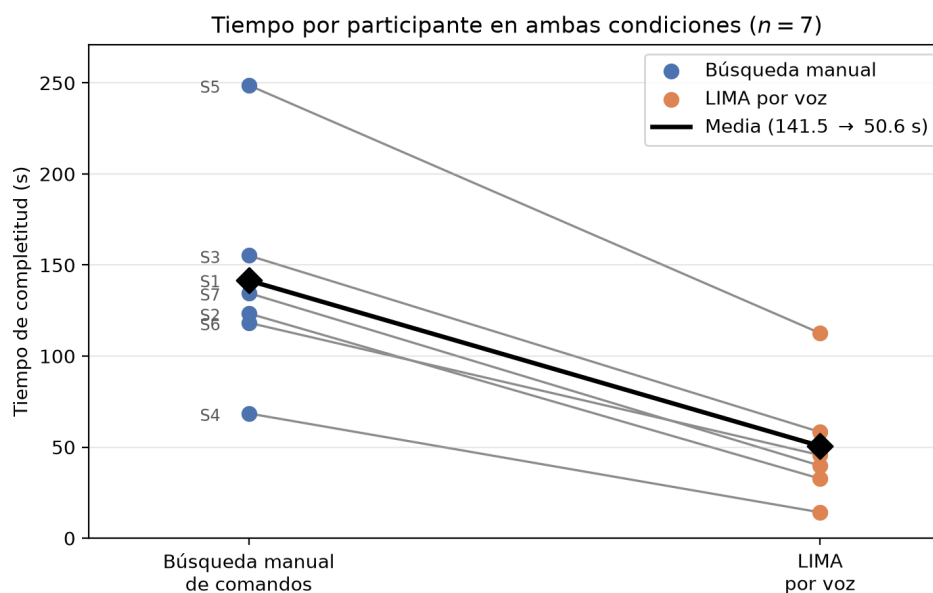


Figura 5.15: Tiempo de completitud por participante en ambas condiciones. Cada línea gris une las dos mediciones de una misma persona; la línea negra une las medias.

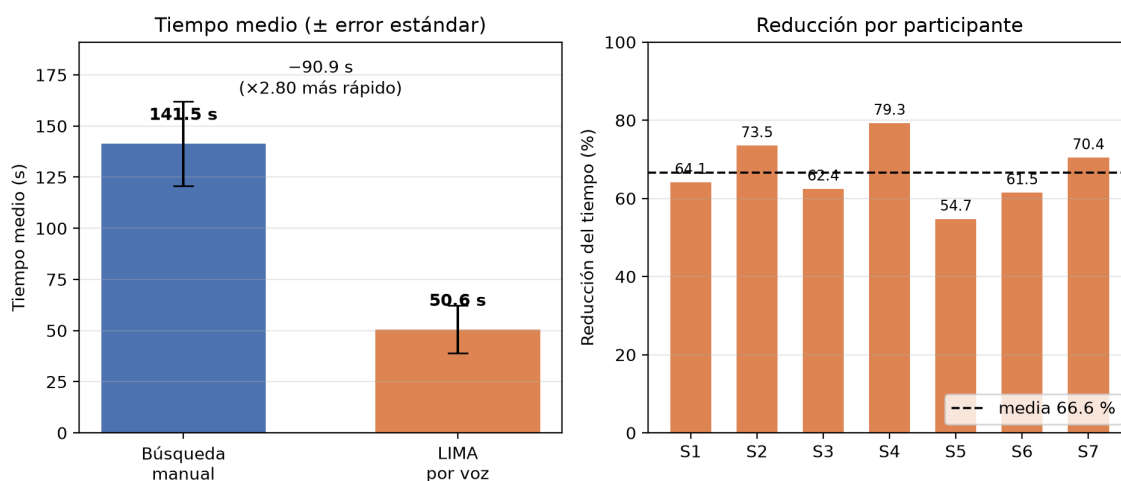


Figura 5.16: Tiempo medio por condición ($\pm$ error estándar) y reducción del tiempo por participante.

5.5.3. Contraste estadístico

La Tabla 5.23 reúne las pruebas de contraste definidas en la Sección 4.4.7. La de rangos con signo de Wilcoxon, en su forma exacta, arroja $W^+ = 28$ y $W^- = 0$: la suma de los rangos de las diferencias favorables a LIMA se lleva la totalidad del reparto —28 es la suma de los rangos 1 a 7— y la de las diferencias contrarias es nula, porque no existe ninguna. El valor p resultante, 0.0156, es el más pequeño alcanzable con siete pares y, por tanto, el máximo de evidencia que el diseño permite obtener. La **prueba de los signos**, que descarta incluso la información de los rangos y se limita a contar cuántas diferencias apuntan a cada lado, arroja el mismo valor por la misma razón: siete de siete. La prueba t pareada, que en cambio aprovecha la magnitud de las diferencias y no solo su signo, da $t(6) = 9.60$ con $p < 0.001$ y un tamaño del efecto $d_z = 3.63$, es decir, una diferencia media equivalente a más de tres veces y media su propia desviación estándar, muy por encima del umbral convencional de efecto grande ($d = 0.8$) (Cohen, 1988).

Tabla 5.23: Contraste estadístico de la diferencia de tiempos ($n = 7$ pares).

Prueba	Estadístico	p (dos colas)
Wilcoxon de rangos con signo (exacta)	$W^+ = 28, W^- = 0$	0.0156
Prueba de los signos	7 de 7 a favor de LIMA	0.0156
t pareada	$t(6) = 9.60$	< 0.001
Tamaño del efecto	$d_z = 3.63$	—
IC 95 % de la diferencia media	[67.8, 114.1] s	—

Estas cifras conviene leerlas sabiendo qué queda establecido y qué no. Lo que queda establecido es la existencia y la dirección del efecto, y lo queda con holgura: el efecto observado ($d_z = 3.63$) casi triplica el mínimo que este diseño podía detectar ($d_z = 1.27$, Sección 4.4.7), de modo que la muestra no se quedó corta para la pregunta que se le hizo; el intervalo de confianza no incluye el cero ni se acerca a él —su extremo inferior, 67.8 s, sigue siendo una diferencia sustancial—, y la consistencia es total, sin una sola inversión entre los siete pares. Lo que no queda establecido es la *magnitud puntual* del efecto en la población general: con siete pares el intervalo es ancho, y la estimación precisa exigiría una muestra mayor. La distinción importa porque son afirmaciones de distinto rango: que LIMA sea más rápido en esta clase de tareas es una conclusión sostenida por los datos; que lo sea exactamente 2.80 veces es una estimación con incertidumbre.

5.5.4. Efecto del orden y control del contrabalanceo

Repetir la misma tarea dos veces beneficia necesariamente a la segunda condición, y por eso el orden se contrabalanceó. La Tabla 5.24 desglosa ambos subgrupos.

Tabla 5.24: Tiempos medios (s) por orden de presentación de las condiciones.

Orden	<i>n</i>	Manual	LIMA	Reducción	Razón
Manual → LIMA	4	170.1	65.4	62.9 %	×2.60
LIMA → manual	3	103.4	30.8	71.4 %	×3.35

El dato decisivo es que **la ventaja de LIMA se mantiene en ambos órdenes**, y de hecho es *mayor* en el subgrupo que usó LIMA primero (71.4 % frente a 62.9 %). Si el resultado fuese un artefacto del aprendizaje —es decir, si LIMA ganara solo por ir en segundo lugar—, el subgrupo «LIMA primero» debería mostrar la ventaja más pequeña, o incluso invertirla. Ocurre lo contrario, lo que descarta que el efecto de repetición explique el hallazgo.

Los dos subgrupos difieren también en su nivel absoluto: quienes empezaron por LIMA fueron más rápidos en ambas condiciones (103.4 s frente a 170.1 s en la condición manual). Con tres y cuatro personas por subgrupo, esa diferencia no puede atribuirse al orden con ninguna confianza: la composición basta para explicarla, pues el subgrupo «manual primero» contiene los dos tiempos más altos de toda la muestra (S3 y S5) y el subgrupo contrario contiene al único participante con experiencia en terminal (S4). El contrabalanceo cumplió su función de control, no la de estimar el efecto del orden, que requeriría una muestra mucho mayor.

5.5.5. Esfuerzo y completitud

El tiempo no es la única diferencia entre ambas condiciones; la Tabla 5.25 recoge el esfuerzo invertido para llegar al mismo resultado.

Tabla 5.25: Esfuerzo y completitud por condición (*n* = 7).

Indicador	Búsqueda manual	LIMA por voz
Completó sin ayuda	6 de 7	7 de 7
Completó con ayuda	1 de 7	0 de 7
No completó / tiempo agotado	0 de 7	0 de 7
Intentos de comando / reformulaciones (media)	2.43 (DE 1.27)	1.00 (DE 0.58)
Total de intentos en la muestra	17	7
Consultas web o a asistentes externos	11	0
Comando final correcto	7 de 7	7 de 7

Tres observaciones se desprenden de este cuadro. La primera: la condición manual exigió **2.43 intentos** de comando por participante frente a **1.00** reformulación de la instrucción verbal, una diferencia media de 1.43 intentos que resulta estadísticamente apreciable ($t(6) = 3.87$;

$p = 0.008$). La segunda: la condición manual requirió **once consultas** a fuentes externas — buscadores, tutoriales y, en un caso, un asistente conversacional de propósito general— frente a **ninguna** en la condición de voz; el usuario que habla con LIMA no necesita salir del sistema para averiguar cómo pedirle algo. La tercera, y la que da validez a la comparación: **los siete participantes llegaron exactamente al mismo comando** en ambas condiciones —`find ~/Descargas -type f -size +20M`—, y LIMA lo generó correctamente en las siete sesiones, sin una sola petición de aclaración. No se comparan, pues, dos resultados de calidad distinta a distinto precio, sino el mismo resultado por dos caminos.

5.5.6. Quién se beneficia más

La correlación de Pearson entre el tiempo de la condición manual y la magnitud del ahorro es prácticamente perfecta ($r = 0.97$, sobre un máximo de 1): **cuanto más le cuesta a alguien la vía manual, mayor es el tiempo que LIMA le devuelve**. El enunciado parece trivial —quien tarda más tiene más que ahorrar— pero su lectura no lo es, porque identifica a quién sirve el sistema. Los dos extremos de la muestra lo ilustran:

- **S5** —52 años, contadora, sin contacto previo con la terminal— registró el mayor tiempo manual de la muestra (248.7 s), los cinco intentos de comando, las cuatro consultas web y fue la única persona que necesitó ayuda del observador, que hubo de indicarle cómo abrir la terminal. Es también quien más segundos ahorró en términos absolutos (136.1 s). Con LIMA completó la tarea sin ayuda alguna.
- **S4** —desarrollador de *software*, usuario semanal de la terminal y el único que conocía `find`— resolvió la vía manual en 68.5 s con un solo intento y ninguna consulta: es el mejor caso posible de la línea base. Aun así, con LIMA tardó 14.2 s, la mayor reducción relativa de toda la muestra (79.3 %).

Estos dos casos delimitan el alcance del hallazgo por sus dos lados. Por el lado del usuario inexperto —el destinatario declarado del sistema—, la vía manual no solo es lenta: es una barrera que en un caso de siete impidió completar la tarea de forma autónoma. Por el lado del usuario competente, la ventaja no desaparece; se reduce en términos absolutos, pero permanece. Al retirar a S4 del cálculo, la muestra restante ($n = 6$) mantiene una reducción media del 64.4 % y una razón de $\times 2.71$, prácticamente idénticas a las del conjunto, lo que confirma que el resultado no depende de ese participante.

Cabe señalar que la edad correlaciona positivamente con el ahorro ($r = 0.72$), pero la muestra no permite separar el efecto de la edad del de la experiencia previa —están *confundidos*, en el sentido técnico de que varían juntos y ningún cálculo puede atribuir el resultado a uno u otro—:

la participante de mayor edad es también la de menor exposición a la terminal. Se reporta la asociación sin atribuirle causa.

5.5.7. Observaciones cualitativas

Las notas del observador añaden textura a las cifras y matizan el resultado en ambas direcciones.

En la **condición manual**, el obstáculo no fue teclear el comando sino *averiguar cuál*: los participantes localizaron la sintaxis de `find` en un buscador, en un tutorial consultado desde el teléfono o preguntándole a un asistente conversacional de propósito general. Ese último caso (S1) merece subrayarse: aun disponiendo de un modelo de lenguaje que le entregó el comando correcto, el participante tardó 141.9 s frente a los 50.9 s de LIMA. La diferencia no está en *saber* el comando, sino en el trayecto completo —abrir el navegador, formular la pregunta, leer la respuesta, cambiar de ventana, transcribir el comando, ejecutarlo—, que es precisamente lo que la interfaz de voz suprime. Se registró además un fallo característico de ese trayecto: en S2, el primer comando falló porque el pegado desde el navegador arrastró caracteres extraños.

En la **condición de LIMA**, la fricción observada fue de otra naturaleza y no debe pasarse por alto. Dos participantes (S1 y S5) pausaron la captura para pensar cómo formular la petición, y S5 dudó sobre si debía hablarle al sistema en inglés. Es la misma *curva de confianza del primer contacto* identificada en la segunda ronda (Sección 5.4.2): el costo no está en la ejecución sino en decidirse a hablar. Los tiempos de LIMA aquí reportados incluyen ese titubeo inicial y son, por tanto, conservadores. En el extremo opuesto, S2 y S4 formularon la petición de forma directa y sin reformular, con tiempos de 32.7 y 14.2 s.

5.5.8. Veredicto sobre la condición C3

La evidencia respalda la condición C3 en los términos en que fue enunciada: los usuarios no especializados completaron la tarea representativa *en menor tiempo* que mediante la búsqueda manual de comandos —un 66.6 % menos de tiempo, $\times 2.80$ más rápido, sin una sola excepción en siete pares y con la ventaja preservada en ambos órdenes de presentación—. En consecuencia, el veredicto de la condición C3 en el Capítulo 6 se actualiza de «no evaluada» a **respaldada**.

Este resultado, además, no contradice sino que completa el hallazgo de eficiencia de la segunda ronda. Allí la voz resultó de 2.1 a 2.5 veces *más lenta* que el entorno gráfico; aquí resulta 2.8 veces *más rápida* que la línea de comandos. Ambas cosas son ciertas a la vez porque son comparaciones distintas, y juntas dibujan con precisión el nicho del sistema: LIMA no compete con el ratón en las tareas que el ratón resuelve bien, sino que sustituye a la terminal en aquellas que el entorno gráfico no resuelve y que hoy obligan al usuario inexperto a buscar un comando

que no conoce. La discusión de esta lectura conjunta se desarrolla en la Sección 5.7.

5.6. Resultados generales de la investigación

Integrando los resultados del desarrollo (sección 5.1), las pruebas locales sobre el conjunto de 250 tareas (sección 4.2), la segunda ronda de pruebas de usabilidad y la prueba complementaria de tiempo frente a la búsqueda manual de comandos (sección 5.5), esta sección sintetiza los hallazgos frente a las preguntas de investigación planteadas en la sección 4.3.1 y a los objetivos del trabajo.

5.6.1. Respuesta a las preguntas de investigación

Preguntas de investigación (Sección 4.3.1)

- | | |
|--------------|---|
| PI-1. | ¿Los usuarios no especializados pueden completar tareas básicas de gestión de archivos mediante instrucciones de voz sin requerir asistencia del facilitador? |
| PI-2. | ¿El modo de atención por palabra clave (<i>wake word</i>) resulta más intuitivo que la activación manual por botón? |
| PI-3. | ¿El indicador visual de estado (gris, azul, rojo, naranja) comunica con claridad la fase de procesamiento en la que se encuentra el sistema? |
| PI-4. | ¿Las respuestas del sistema —tanto visuales como auditivas— son comprensibles y orientan al usuario cuando ocurre un error o una ambigüedad? |
| PI-5. | ¿Los usuarios perciben que LIMA les otorga control sobre las operaciones y les permite comprender qué se ejecutará antes de proceder? |
| PI-6. | ¿Qué barreras de usabilidad persisten tras las mejoras implementadas? |

PI-1. ¿Pueden usuarios no especializados completar tareas básicas de gestión de archivos por voz sin asistencia? *Sí, mayoritariamente.* La eficacia por voz se ubicó entre 78.6 y 92.3% sin intervención del facilitador, con una media del 83.9%. La modalidad gráfica sirvió como línea base al 100%. La brecha restante se concentra en tareas que exigen dictar varios elementos (T1) o localizar un archivo concreto (T2).

PI-2. ¿El modo de atención por palabra clave es más intuitivo que la activación por botón? *Indicios favorables, sin comparación controlada.* La activación no concentró episodios de frustración y los participantes alternaron ambos mecanismos con naturalidad, sin que ninguno de los dos emergiera como obstáculo. Al no haberse diseñado un contraste A/B entre ambas modalidades, la evidencia disponible sugiere equivalencia práctica antes que superioridad de una sobre otra.

- PI-3. ¿El indicador visual de estado comunica con claridad la fase de procesamiento?** *Sí, de forma indirecta.* Ningún participante preguntó en qué estado se encontraba el sistema ni intentó hablar mientras este procesaba, lo que indica que el código de color resultó interpretable. La ausencia de esa confusión es coherente con la baja frustración registrada y con el control percibido (PI-5), aunque el indicador no se midió de forma aislada.
- PI-4. ¿Las respuestas del sistema son comprensibles y orientan ante errores?** *Parcialmente.* La facilidad de aprendizaje (4.54/5) y la accesibilidad (4.23/5) fueron altas, pero la naturalidad de las respuestas (3.46/5) quedó como la dimensión más baja, lo que indica margen de mejora en el fraseo de la retroalimentación.
- PI-5. ¿Perciben los usuarios control sobre las operaciones?** La baja frustración (sin episodios moderados ni altos) y la preferencia por la voz en tareas de organización y búsqueda sugieren que el flujo de confirmación previa otorgó a los participantes una sensación de control suficiente para confiar en el asistente.
- PI-6. ¿Qué barreras de usabilidad persisten?** Las dos principales son (i) la **eficiencia** —la voz sigue siendo 2.1–2.5 veces más lenta que la GUI, especialmente durante el primer contacto; conviene precisar que esa desventaja se refiere al entorno gráfico y no a la línea de comandos, frente a la cual la voz resultó 2.8 veces más rápida (sección 5.5)— y (ii) la **naturalidad** del diálogo, identificada como la dimensión peor valorada.

Las respuestas a PI-2 y PI-3 descansan sobre evidencia cualitativa —las notas de sesión y la ausencia de determinadas incidencias— y no sobre métricas dedicadas, por lo que se presentan con la cautela que corresponde: son indicios consistentes, no mediciones. Su confirmación plena requeriría instrumentos diseñados específicamente para cada una.

5.6.2. Cumplimiento de los objetivos y la hipótesis

En conjunto, los resultados respaldan la viabilidad de la propuesta: LIMA permite a usuarios sin experiencia previa operar GNU/Linux por voz para tareas cotidianas con tasas de éxito altas, baja frustración y una valoración global favorable (8.21/10). El costo en eficiencia —mayor tiempo por interacción— y la naturalidad perfectible del diálogo constituyen las líneas de mejora prioritarias. Estos hallazgos se discuten a la luz de la hipótesis del trabajo y de los principios de IA Centrada en el Humano en la sección 5.7.

5.7. Discusión

Los resultados deben interpretarse a la luz de los principios de Inteligencia Artificial Centrada en el Humano que guiaron el diseño (Shneiderman, 2022). La combinación de **alta eficacia** (83.9% en voz, 100% en GUI) con **baja frustración** y una **valoración global de 8.21/10** sugiere que el sistema cumple sus objetivos de transparencia y control: los participantes comprendieron qué haría LIMA antes de ejecutarlo y confiaron en el resultado, hasta el punto de *preferir* la voz frente al entorno gráfico en dos de las tres tareas pese a su mayor lentitud.

El principal compromiso de diseño se observa en la **eficiencia**. La voz fue de 2.1 a 2.5 veces más lenta que la GUI, pero este dato debe matizarse: la elevada dispersión de los tiempos revela que buena parte del costo proviene de la curva de confianza del primer contacto (sección 5.4.2) y no de una limitación estructural del sistema. Es razonable esperar que, con el uso, los tiempos converjan hacia los valores mínimos observados. La comparación exploratoria entre modelos no arroja una superioridad significativa de uno sobre el otro: GPT-4o aventajó a DeepSeek en la eficacia de voz y en la calificación global, mientras que DeepSeek igualó o superó levemente a GPT-4o en naturalidad y accesibilidad.

5.7.1. La eficiencia depende de con qué se compare

El juicio sobre la eficiencia cambia por completo según cuál sea la línea base, y disponer de las dos mediciones permite formularlo con precisión. Frente al **entorno gráfico**, la voz fue de 2.1 a 2.5 veces más lenta (sección 5.4.2); frente a la **búsqueda manual de comandos**, fue 2.8 veces más rápida (sección 5.5). No hay contradicción entre ambos resultados: son comparaciones contra alternativas distintas, sobre clases de tareas distintas, y su lectura conjunta es más informativa que cualquiera de las dos por separado.

Lo que separa a una clase de tareas de la otra no es su dificultad técnica sino la **distancia entre la intención del usuario y la interfaz disponible**. Arrastrar un archivo a una carpeta es una operación que el escritorio expone directamente: el usuario ve el objeto, ve el destino y actúa sobre ellos, de modo que la mediación verbal solo añade pasos. Filtrar los archivos de un directorio por tamaño, en cambio, es una operación que el escritorio no expone; el usuario sabe qué quiere pero no dispone de ningún objeto que manipular, y la única vía es un comando cuya sintaxis desconoce. Es ahí donde la interfaz de voz deja de ser un rodeo y se convierte en el camino corto, porque suprime el trayecto completo de averiguación —abrir el navegador, formular la pregunta, leer la respuesta, transcribir el comando— que la prueba complementaria cronometró.

Este contraste delimita el nicho del sistema con más exactitud que cualquiera de las dos evaluaciones aislada: **LIMA no sustituye al ratón en lo que el ratón hace bien, sino a la**

terminal en lo que el entorno gráfico no hace. Interpretado a la luz de los principios de HCAI, el sistema no compite con las interfaces existentes por las mismas tareas: amplía el conjunto de operaciones que un usuario no especializado puede realizar por sí mismo, que es exactamente la noción de *empoderamiento* que Shneiderman (2022) sitúa como criterio de diseño. El dato más elocuente en ese sentido no es el ahorro de tiempo medio, sino que la única participante que no logró completar la tarea de forma autónoma por la vía manual sí la completó sin ayuda por voz.

5.7.2. Contraste con estudios de usabilidad de asistentes de voz

Los resultados de LIMA pueden situarse en el marco de la literatura sobre usabilidad de asistentes de voz con inteligencia artificial. La elección de las tres dimensiones de la norma ISO 9241-11 —eficacia, eficiencia y satisfacción— coincide con el marco que Pal et al. (2022) consolidan en su revisión sistemática de 67 estudios, donde proponen precisamente la norma ISO 9241-11 como estructura de referencia para evaluar asistentes de voz; ello respalda la organización metodológica adoptada en este trabajo (sección 5.4).

El hallazgo más consistente con el estado del arte es que la **naturalidad del diálogo** constituye el punto más débil. Berdasco et al. (2019) encontraron que la naturalidad y la corrección de las respuestas son las dimensiones que mejor diferencian a los asistentes comerciales (Alexa, Google Assistant, Siri y Cortana), y que ninguno alcanza una percepción de naturalidad plena. La puntuación de naturalidad de LIMA (3.46/5), por debajo de la facilidad de aprendizaje y la accesibilidad, reproduce ese mismo patrón: la fluidez conversacional sigue siendo el reto abierto incluso para productos maduros, de modo que el margen de mejora identificado en LIMA no es una anomalía, sino la frontera común del dominio.

En cuanto a la **comparación entre modelos**, el carácter mixto y de pequeña magnitud de las diferencias entre DeepSeek y GPT-4o es coherente con Brüggemeier et al. (2020), quienes, al contrastar Alexa, Google Assistant y Siri con cuatro cuestionarios establecidos, observaron que las diferencias entre asistentes dependen del instrumento de medida y rara vez son uniformes. Esta observación refuerza la decisión de presentar la comparación de LIMA como *exploratoria* y multidimensional, y no como un veredicto único. En la misma línea, Zwakman et al. (2021) mostraron que una escala genérica como el *System Usability Scale* (SUS) no captura adecuadamente la interacción por voz —y propusieron una *Voice Usability Scale*—, lo que justifica el uso en este trabajo de un cuestionario propio con dimensiones específicas (naturalidad, accesibilidad, velocidad percibida) en lugar de una única puntuación agregada. El enfoque multidimensional de la experiencia de usuario que documentan Chen et al. (2024) a partir de más de 21 000 opiniones converge con esa misma necesidad de descomponer la UX en componentes diferenciados.

Por último, la **alta facilidad de aprendizaje** reportada para LIMA (4.54/5) es comparable a la observada por Jakob et al. (2025) en su estudio longitudinal, donde alrededor del 90%

de los participantes consideraron sencillo aprender a usar el asistente. La principal diferencia metodológica —y una limitación de este trabajo— es que dichos estudios miden el uso sostenido durante semanas, mientras que la evaluación de LIMA corresponde a un primer contacto en sesión única (sección 5.7.3); la convergencia de los tiempos hacia sus valores mínimos con el uso, que aquí solo se infiere, podría confirmarse con un estudio longitudinal análogo.

5.7.3. Limitaciones del estudio

Los resultados deben leerse considerando las siguientes limitaciones:

1. **Tamaño de muestra.** Con $n = 14$ (7 en cada condición), el estudio tiene baja potencia estadística; los desgloses por condición son exploratorios y descriptivos, no concluyentes.
2. **Orden fijo de modalidades.** Al ejecutar siempre primero la GUI y luego la voz, existe un posible efecto de *carry-over*: el participante ya conoce la tarea al realizarla por voz, lo que puede inflar la eficacia y reducir el tiempo de voz.
3. **Heterogeneidad de la muestra.** La diversidad de experiencia con GNU/Linux —de nula a diaria— introduce variabilidad no controlada en los tiempos.
4. **Medición en sesión única.** Las métricas reflejan el primer contacto con LIMA; el rendimiento de usuarios entrenados podría diferir de forma sustancial, particularmente en eficiencia.
5. **Alcance de la prueba complementaria.** El contraste frente a la búsqueda manual de comandos (sección 5.5) resuelve la condición C3 en la clase de tareas que el entorno gráfico no cubre, que es donde la condición la sitúa; medir cómo varía esa ventaja en otras clases de tarea requeriría un repertorio de dificultad graduada. Al haberse ejecutado con posterioridad al cierre de la segunda ronda, sobre la versión ya corregida a partir de sus hallazgos (sección 5.1.8), sus tiempos no son comparables con los de aquella: entre una medición y otra cambió la interfaz, y ambas se reportan por separado.

Capítulo 6

Conclusiones y trabajo futuro

Este trabajo se planteó una pregunta sencilla de enunciar y difícil de responder: si un asistente de voz podía acercar GNU/Linux a quien nunca lo ha usado. Las páginas anteriores reunieron la evidencia; estas la convierten en veredicto, en agenda y en balance. El capítulo confronta primero lo obtenido con la hipótesis y los objetivos planteados al inicio, deriva de esa confrontación las líneas que el trabajo deja abiertas, y cierra con lo que el proceso enseñó a quien lo condujo. No son tres asuntos separados: cada límite que la evaluación reveló es, a la vez, un veredicto sobre lo prometido, una tarea pendiente y una lección aprendida, y aquí se leen así.

Las preguntas de investigación se respondieron una sola vez, en la Sección 5.6.1 del capítulo anterior, junto a los datos que sustentan cada respuesta; repetirlas aquí duplicaría el análisis sin añadir nada. El detalle numérico que sustenta lo que sigue se expone en los Capítulos 4 y 5: solo se retoman las cifras necesarias para emitir un juicio sobre cada afirmación.

Hipótesis del trabajo (Sección 1.4)

La integración de herramientas de IA, en particular modelos de lenguaje, con un enfoque basado en los principios de HCAI habilitará a usuarios no expertos en GNU/Linux a realizar actividades de gestión de archivos, consultas del sistema y otras tareas cotidianas dentro del alcance definido con una precisión de al menos el 85 %.

- C1.** La traducción automática de instrucciones verbales en español a operaciones de gestión de archivos, consultas del sistema y otras tareas cotidianas dentro del alcance definido alcanzará un nivel de precisión que permita su uso práctico (≥ 85 % de comandos correctos).
- C2.** El mecanismo de clasificación de riesgo identificará operaciones potencialmente destructivas antes de su ejecución con una tasa ≥ 95 %.
- C3.** Los usuarios no especializados completarán tareas cotidianas representativas dentro

del alcance del sistema en menor tiempo que mediante la búsqueda manual de comandos.

- C4.** Los modelos de lenguaje ejecutados localmente ofrecerán resultados funcionales para un subconjunto de tareas cotidianas, especialmente las relacionadas con gestión de archivos, constituyendo una alternativa viable que preserve la privacidad.

La Tabla 6.1 confronta cada condición con la evidencia obtenida.

Tabla 6.1: Contraste de la hipótesis con los resultados.

Umbral	Resultado obtenido	Veredicto
Enunciado central	Usuarios sin experiencia previa completaron tareas reales por voz sin asistencia: eficacia media del 83.9 %, satisfacción de 8.21/10 y ausencia de frustración moderada o alta ($n = 14$).	RESPALDADO
C1 $\geq 85\%$	Depende de la métrica. En <i>cobertura</i> , 78.0 % (99 de 127 tareas <i>in-scope</i>). En <i>precisión al actuar</i> —excluyendo los repliegues a <code>clarify</code> , que no generan comando— 87.6 % con <code>deepseek-chat</code> y 88.9 % con <code>gpt-4o</code> .	ALCANZADO EN PRECISIÓN EFECTIVA
C2 $\geq 95\%$	Las cinco operaciones destructivas que el sistema materializó exigieron confirmación previa: cero falsos negativos de seguridad.	ALCANZADO
C3 —	Prueba complementaria de medidas repetidas ($n = 7$, contralanceada): 141.5 s por la búsqueda manual frente a 50.6 s con LIMA —un 66.6 % menos de tiempo, $\times 2.80$ más rápido— con los 7 pares a favor de la voz ($p = 0.0156$, exacta).	RESPALDADO
C4 —	En un subconjunto de 100 tareas (ocho categorías), un modelo local (<code>deepseek-coder-6.7b-instruct</code>) alcanzó una concordancia <i>in-scope</i> del 67.5 % —a la par de <code>gpt-4o</code> — con JSON válido en el 100 % de los casos, pero con dos falsos negativos de seguridad (cero en la nube).	PARCIALM. RESPALDADO

El veredicto de **C1** depende de qué se entienda por «comandos correctos», y conviene desarrollarlo porque la elección no es cosmética. Bajo la lectura de *cobertura* —qué fracción de todas las peticiones *in-scope* resuelve el sistema— la medición arroja un 78.0 % y la condición no se alcanza. Bajo la lectura de *precisión efectiva* —con qué frecuencia acierta cuando produce un comando— la cifra sube al 87.6 % y el umbral se supera con holgura, y lo hace además con los dos modelos remotos evaluados (Tabla 5.4). La segunda lectura es la que corresponde al enunciado de la condición, y por una razón concreta: C1 habla de «comandos correctos», y un repliegue a `clarify` **no produce comando alguno**. Contarlo como fallo equivale a penalizar al sistema por reconocer que una petición es ambigua, que es justamente el comportamiento que el diseño persigue y que la condición C2 premia. Un asistente que inventara una ruta plausible ante una petición incompleta obtendría mejor cobertura y sería peor sistema.

Dicho esto, la primera lectura no se descarta ni se oculta: el 78.0 % describe algo real y útil —la proporción de peticiones que el sistema resuelve sin intervención adicional del usuario— y queda reportado junto a la otra cifra. Lo que el conjunto revela es que **el umbral se formuló sin especificar su denominador**, sobre un dominio que en ese momento aún no estaba delimitado. Un único porcentaje global resulta un instrumento demasiado grueso para juzgar un sistema cuyo desempeño varía entre el 100 % en búsqueda web y apertura de aplicaciones y el 45 % en creación de proyectos. Ese límite lo puse yo mismo, sin advertirlo, en la página en que enuncié la hipótesis: fijé el 85 % cuando todavía no conocía el terreno que iba a medir. La lección metodológica importa tanto como el veredicto —una condición cuantitativa debe fijar no solo el umbral, sino qué se cuenta en el numerador y qué en el denominador—, y aprendí que un umbral establecido antes de conocer el dominio mide sobre todo el optimismo de quien lo fija.

La condición **C3** quedó respaldada por la prueba complementaria descrita en la Sección 4.4. La segunda ronda había contrastado la voz con el entorno gráfico —la línea base que corresponde a una evaluación de usabilidad—, mientras que C3 nombra otra distinta, la búsqueda manual de comandos; instrumentarla exigía un diseño propio, intra-sujeto y contrabalanceado, que no puede anidarse dentro de una sesión de usabilidad sin contaminar ambas medidas. Sobre una tarea representativa de las que el entorno gráfico no resuelve —localizar los archivos de un directorio que superan cierto tamaño—, siete participantes con perfil de usuario inexperto tardaron 141.5 s de media por la vía manual y 50.6 s con LIMA: un ahorro medio de 90.9 s (IC 95 %: 67.8–114.1 s), una reducción del 66.6 % y una razón de $\times 2.80$, con los siete pares a favor de la voz y sin una sola inversión. El contrabalanceo descarta que el resultado provenga de repetir la tarea, pues la ventaja fue *mayor* en el subgrupo que usó LIMA primero, y el efecto observado ($d_z = 3.63$) casi triplica el mínimo que el diseño podía detectar. El respaldo alcanza a la clase de tareas donde la condición sitúa la comparación —aquellas que el entorno gráfico no cubre—; cómo varía la ventaja en otras clases es una pregunta distinta, que se plantea más adelante en este capítulo.

Este veredicto no contradice el hallazgo de eficiencia de la segunda ronda, donde la voz resultó de 2.1 a 2.5 veces *más lenta* que el entorno gráfico. Ambos resultados son ciertos porque miden contra alternativas distintas, y su lectura conjunta —desarrollada en la Sección 5.7.1— delimita el nicho real del sistema: **LIMA no sustituye al ratón en lo que el ratón hace bien, sino a la terminal en lo que el entorno gráfico no hace**. Ese es, más allá de cualquier porcentaje, el hallazgo conceptual del trabajo, y es el que da sentido a la noción de empoderamiento que la HCAI sitúa como criterio de diseño: el sistema no compite por las tareas que ya están resueltas, sino que amplía el conjunto de operaciones que un usuario no especializado puede realizar por sí mismo. El dato más elocuente en ese sentido no es el ahorro medio de tiempo, sino que

la única participante que no logró completar la tarea de forma autónoma por la vía manual sí la completó sin ayuda por voz.

La condición **C4**, por su parte, quedó parcialmente respaldada por la evaluación del conjunto reducido (Sección 5.2.5): un modelo ejecutado localmente sostuvo el caso base con una concordancia del 67.5 % —a la par de `gpt-4o`— y sin errores de formato, lo que confirma la viabilidad de una operación sin conexión que preserva la privacidad; el respaldo es *parcial* porque el modelo local mostró una precisión menor en las categorías de sistema y redes y una seguridad inferior (dos falsos negativos frente a cero de la nube). En síntesis: la hipótesis se confirma en su afirmación central, en su condición de seguridad y en su condición de eficiencia frente a la búsqueda manual de comandos; respalda parcialmente la viabilidad de la operación local; y queda por debajo de su umbral de precisión bajo la lectura de cobertura, por las razones de denominador ya expuestas. El trabajo valida la viabilidad del enfoque y delimita con precisión su alcance.

El objetivo general —*diseñar, desarrollar y evaluar un software de asistencia por voz, fundamentado en HCAI, que permita a usuarios no especializados operar GNU/Linux mediante instrucciones en español*— se cumplió: LIMA es un sistema funcional, evaluado con usuarios reales y con un conjunto de datos propio. Los seis objetivos específicos se cumplieron también, y la Tabla 6.2 detalla la evidencia que lo sostiene en cada caso. Conviene distinguir, no obstante, entre *cumplir un objetivo* y *alcanzar un umbral*: el objetivo 5 pedía medir la precisión del sistema, y la medición se realizó; que el valor medido quede por debajo del 85 % que fijaba la hipótesis es un resultado de esa medición, no un incumplimiento del objetivo.

Tabla 6.2: Grado de cumplimiento de los objetivos específicos.

Obj.	Estado	Evidencia
1	Cumplido	Usuarios neófitos realizaron tareas cotidianas por voz (eficacia 83.9 %, baja frustración; Cap. 5).
2	Cumplido	Marco teórico de HCI, HCAI y PLN aplicado al dominio (Cap. 2).
3	Cumplido	Arquitectura por capas con detección local, interpretación por LLM, validación de riesgo y retroalimentación multimodal (Cap. 3).
4	Cumplido	LIMA implementado: palabra clave, transcripción, generación de comandos e interfaz gráfica.
5	Cumplido	Conjunto de datos de 250 tareas evaluado íntegramente con dos modelos: precisión de generación, matriz de confusión de riesgo y cobertura del detector local (Sección 5.2).
6	Cumplido	Evaluación cualitativa de usabilidad con 14 participantes bajo ISO 9241-11 (Cap. 5).

Parte de este trabajo se sometió además a revisión por pares externa y se presentó en la *International Conference on Artificial Intelligence in Society* (AIS 2026), celebrada en Valencia, España, del 25 al 27 de julio de 2026 como parte del *Multi Conference on Computer Science and Information Systems*. El artículo completo se publicó en las actas del congreso, editadas por

IADIS Press (Aragon Toledo y Ruiz Rodríguez, 2026).¹ Con posterioridad al congreso, el comité editorial comunicó que el artículo fue **seleccionado entre los mejores del evento** e invitado a publicarse, en una versión extendida, en el *IADIS International Journal on Computer Science and Information Systems* (ISSN 1646-3692). Esa difusión aporta al trabajo una validación que la evaluación interna no puede dar por sí sola: el enfoque, la arquitectura y los resultados fueron juzgados por revisores ajenos al proyecto y por una audiencia internacional especializada. La extensión comprometida —que deberá incorporar al menos un 30 % de material nuevo— se apoyará de forma natural en los desarrollos posteriores a la evaluación que documenta la Sección 3.21, en particular la validación empírica del soporte multidistribución, que ya en el congreso resultó ser el aspecto que más interés despertó entre los asistentes.

Hasta aquí lo que los instrumentos empleados saben medir. Queda otra cosa que ninguno de ellos alcanza y que, sin embargo, forma parte del resultado: qué enseñó el proceso a quien lo condujo. A ello dedico los párrafos que siguen, escritos en primera persona porque no admiten otra, y sostenidos por el mismo criterio que rige lo demás: distinguir con honestidad lo que salió bien de lo que salió distinto de como lo había imaginado.

Lo que me costó noches enteras de reflexión no fue una decisión técnica, sino una anterior a todas ellas: si este trabajo tenía verdaderamente algo que aportar. Un asistente de voz es, a primera vista, tecnología ya vista y ya estudiada, y hay empresas con equipos enteros dedicados a ella, trabajando de forma organizada y con recursos que un proyecto de tesis no puede igualar. La duda era legítima y tardé en resolverla. La resolví observando. Empecé a fijarme en cómo mis compañeros realizaban ciertas tareas en sus computadoras y reconocí en ellos la misma necesidad que yo tenía: disponer de algo que simplificara la interacción con la máquina mientras la atención está puesta en otra cosa. Muchas veces se trabaja en algo que consume toda la concentración, y tener a la mano un asistente que se ocupe de lo accesorio no es solo una comodidad: añade a GNU/Linux una vía más llana de hacer las cosas. Ahí entendí que el problema existía aunque la tecnología ya estuviera inventada, y que el aporte no estaba en el reconocimiento de voz, sino en cómo se pone al servicio de quien no domina la terminal.

Di por sentado, en cambio, algo que resultó estar equivocado. Suponía que la recepción sería reacia: no estamos acostumbrados a formular peticiones hablando, y esperaba resistencia. Ocurrió lo contrario. Como facilitador de las sesiones observé que, una vez formulada la primera petición y comprendido el mecanismo, los participantes se soltaban: perdían la rigidez inicial, probaban formulaciones más ambiciosas y descubrían por sí mismos capacidades del sistema que nadie les había explicado. La confianza no había que construirla poco a poco; bastaba con que la primera interacción funcionara. Ver a un desconocido usar por primera vez algo que

¹El artículo se publicó bajo el nombre BOTAS, que era el que el sistema llevaba en el momento del envío; la unificación de la identidad bajo LIMA (Sección 5.1.8) es posterior. Se trata del mismo sistema que documenta esta memoria.

uno escribió es la forma más rápida de descubrir cuánto de lo que uno daba por evidente no lo era, y no conozco lección de ingeniería que se le parezca. Hubo comportamientos que no había previsto: varios participantes intentaron usar el recuadro donde se muestran los registros de actividad como si fuera un campo de entrada de texto. Nada en la interfaz invitaba a ello desde mi punto de vista, pero desde el suyo era el único espacio con apariencia de casilla escribible. A raíz de eso lo retiré de la ventana principal y lo dejé accesible únicamente desde la configuración avanzada, para quien de verdad lo necesite. La interfaz no comunica lo que uno cree haber diseñado, sino lo que el usuario interpreta.

De todo el sistema, la parte que más me enorgullece —la *joya de la corona*, como acabamos llamándola mi director y yo— es el módulo que reconoce peticiones y las estructura como plantillas reutilizables, de modo que una tarea aprendida no vuelve a escalar al modelo de lenguaje. Ese módulo separa a LIMA de ser una aplicación que simplemente consulta a una IA: le da mecanismos para decidir y responder por sí misma, sin generar costo adicional. Es también donde quedó demostrada la ingeniería que estudié, porque comprender la *frecuencia de término–frecuencia inversa de documento* (TF-IDF, *Term Frequency–Inverse Document Frequency*) y adaptarla a este dominio exigió análisis antes que herramientas. Un problema complejo puede resolverse con los recursos que uno tiene a la mano si se le dedica el pensamiento suficiente.

La espera del soporte multidistribución me dejó una lección de otro tipo. La idea existía desde mucho antes, pero el sistema recibía cambios constantes y no había versión que congelar. Preparar la presentación pública me obligó a decidir hasta dónde llegaría, a dejar de añadir funcionalidades y a concentrarme en distribuir el software de forma segura y compatible con cualquier distribución. Fijar esa frontera no es un trámite administrativo sino una decisión de diseño, y posponerla tiene un costo: mientras no exista una versión que congelar, tampoco existe nada que alguien más pueda usar. No estoy arrepentido de cómo se dio —las cosas se hicieron bien, y de las más de veinticinco copias de la versión de prueba que he entregado he recibido comentarios constructivos que han alimentado ajustes posteriores, junto con una percepción general de satisfacción con el sistema en su estado de demostración—, pero si empezara de nuevo cambiaría el momento en que el software sale de mis manos: lo distribuiría desde una etapa temprana, para que mis compañeros lo usaran mientras se construía y no cuando ya estaba terminado. Habría convertido la retroalimentación en un flujo continuo en vez de en dos rondas discretas, y varias de las decisiones que tomé solo con mi criterio —el fraseo de las respuestas, qué mostrar en la ventana principal— habrían llegado corregidas antes de fijarse en el diseño. No lo doy por perdido: es algo que todavía se puede hacer y que de hecho ya está ocurriendo, ahora que el sistema se instala en cualquier distribución y circula entre quienes quieran probarlo.

De esas lecciones y de los resultados medidos nace, sin solución de continuidad, la agenda

pendiente. Se ordena en cuatro líneas: afinar el diálogo, reducir la dependencia del modelo remoto, ampliar el alcance de lo que el sistema puede atender y consolidar lo que sabemos sobre él. Las tres primeras miran al artefacto; la cuarta, a la evidencia que lo respalda.

Primera línea: afinar el diálogo. La marca la dimensión peor valorada de la evaluación. Que la naturalidad obtuviera 3.46/5 coincide con lo que percibí durante las sesiones: el sistema interpretaba correctamente las peticiones, pero sus respuestas sonaban planas y secas. De ahí nació el modo conversacional, para que sea LIMA quien busque sostener el diálogo y dar al usuario la sensación de que hay alguien al otro lado de la pantalla y no un formulario que contesta. Ese camino queda a medias por dos frentes. El primero es el *cierre de turno*: la duración de silencio que da por terminada una intervención es hoy un parámetro de configuración expuesto al usuario (Sección 3.21.2), lo que traslada a este una decisión que el sistema debería tomar por él; la vía razonable consiste en estimar el valor a partir de la distribución de duraciones de pausa observada en el uso de cada persona —un percentil alto de sus pausas intra-frase—, de modo que el umbral converja a su ritmo de habla sin intervención manual, y en incorporar un detector de actividad vocal entrenado en lugar del criterio de energía actual, que no distingue voz de ruido sostenido. El segundo es la *síntesis de voz*, que reescribiría entera: me gustaría emplear un sistema capaz de adoptar los acentos del español con fidelidad a cada región, en lugar de una voz única para todos los hispanohablantes. Un sistema así encarece cada interacción y exige más tiempo de análisis, investigación y desarrollo del que este trabajo podía dedicarle.

Segunda línea: reducir la dependencia del modelo remoto. Su motivación no es técnica sino de fondo. Depender de un modelo de lenguaje que no controlo implica atenerme a las reglas que fija una organización externa y aceptar que, de una forma u otra, los datos y las características del perfil de quien usa el sistema quedan expuestos ante empresas que no dudarán en comercializarlos. Esa fue la razón para trabajar el modo sin conexión con un modelo local: no la conectividad ni el costo, sino quién se queda con lo que el usuario dice. El trabajo pendiente tiene aquí dos ramas complementarias. Por un lado, *ampliar la detección local de intenciones* analizando los registros de interacciones acumulados, para enriquecer el módulo local y hacer que una proporción creciente de peticiones se resuelva sin salir de la máquina —que es, en el fondo, extender el principio que ya demostró el aprendizaje de tareas—. Por otro, *profundizar en los modelos de lenguaje locales* (DeepSeek, Llama u otros) hasta que la operación completamente *offline* deje de ser un modo degradado y pase a ser una alternativa equivalente, con especial atención al punto donde hoy flaquea: los dos falsos negativos de seguridad que la nube no cometió.

Tercera línea: ampliar el alcance del sistema. La encabeza la *planificación de tareas extensas* que expone la Sección 3.22, la de mayor envergadura de todas y la única cuyo diseño se dejó planteado en detalle, precisamente por comprometer decisiones de arquitectura: permitiría

atender peticiones que encierran varias operaciones dependientes —ordenar una carpeta por tipo de archivo, preparar un proyecto para entregarlo— introduciendo el *plan* como objeto inspeccionable que el usuario autoriza de una vez y cuya ejecución admite puntos de control y reversión. En un plano más acotado, procede *robustecer el sistema de distribución controlada* descrito en la Sección 3.21.3: vincular cada código de acceso a un dispositivo y dotarlo de expiración temporal acotaría la exposición ante una filtración, y un panel de administración sustituiría la emisión y revocación por consultas directas a la base de datos.

Cuarta línea: consolidar la evidencia. No amplía el sistema sino lo que sabemos sobre él, y nace directamente de las limitaciones declaradas en la Sección 5.7.3. Procede *replicar la evaluación de usabilidad* con una muestra de mayor tamaño y mayor diversidad de perfiles, y preferentemente en un diseño longitudinal, porque las métricas actuales reflejan un primer contacto y la convergencia de los tiempos con el uso aquí solo se infiere. Procede también *extender el contraste frente a la búsqueda manual de comandos*: la prueba que cerró la condición C3 estableció la ventaja en la clase de tareas que el entorno gráfico no cubre, y la extensión natural consiste en aplicar el mismo protocolo —diseño intra-sujeto contrabalanceado, mismo instrumento de cronometraje— a una batería de tareas de dificultad graduada, desde operaciones que la terminal resuelve con un comando de una línea hasta otras que exigen encadenar varias, pasando por las que el entorno gráfico también cubre; ello permitiría estimar cómo varía la ventaja con la complejidad de la tarea y localizar el punto en que la interfaz de voz deja de ser el camino corto, que es la pregunta que una tarea única no puede responder. En la misma clave, una *comparación cuantitativa controlada frente a líneas base* como ShellGPT o NL2Bash —midiendo precisión, latencia y tasa de error— situaría el desempeño de LIMA en el estado del arte con una referencia externa, y la *ejecución de la batería completa de tareas en las seis familias de distribuciones soportadas* convertiría en resultado verificado lo que hoy está comprobado en cuatro de ellas.

Nada de lo anterior tendrá destinatario mientras el sistema siga exigiendo descargarse del repositorio público e instalarse con el guion que lo acompaña. Aspiro a que llegue a estar a un `sudo apt install` de distancia, disponible en cada distribución de la familia GNU/Linux, y a que no se detenga ahí: que pueda convertirse en una distribución por derecho propio.

Para el usuario hispanohablante sin formación técnica, la relevancia de todo esto está en algo más que resolverle la tarea. LIMA reduce la barrera de entrada a un sistema operativo que muchos nunca han tenido ocasión de usar, pero además muestra el comando que va a ejecutar y lo explica. Quien lo desee puede aprender la sintaxis sobre la marcha, sin detener lo que estaba haciendo. Es una herramienta que puede dejar de ser necesaria para su propio usuario, y esa me parece la forma más honesta de reducir una brecha.

Queda, al final, lo que este trabajo no dejó en el sistema sino en quien lo construyó. Esta

experiencia marca un parteaguas en mi vida. Me dio a conocer el proceso de investigación real, en todos sus matices: lo que lo hizo llevadero y lo que me quitó el sueño durante la etapa final de la carrera. No fue fácil ni barato decidirme a cruzar el Atlántico para presentar este trabajo, pero siendo mi primera aportación personal a la sociedad no podía permitir que quedara en el olvido. Que el umbral del 85 % no se alcanzara bajo una de sus dos lecturas no invalida nada de lo hecho: ni las noches sin dormir, ni las semanas de correcciones, ni las pruebas de usabilidad, ni los traslados, ni el costo de viajar a otro país a defender que esta propuesta tiene un valor que va más allá de ser un programa impulsado por inteligencia artificial. Mi director Ricardo y yo le vimos el potencial suficiente para confiar en que puede cambiar la manera en que las personas tienen su primer acercamiento a GNU/Linux. Agradezco esa confianza y la formación que me permitió llegar hasta aquí.

La culminación de este trabajo no representa solo el fin de una etapa, sino el comienzo del resto de mi vida. A diferencia del muchacho que entró a cursar un propedéutico, hoy sale un profesional formado, con la integridad suficiente para no dejar de mirar las necesidades de su entorno y con la convicción de que la tecnología debe democratizarse: una sociedad moderna pide a gritos ser incluyente con todos quienes formamos parte de ella. Recordaré estos días con cariño, como un regalo que me dio la vida a cambio del empeño que dejé en el salón de clases. Agradezco a mi universidad las herramientas que me dio para ser un buen profesional, y me comprometo a encaminar cada día mis pasos para ser también un mejor ser humano.

Bibliografía

Libros

Russell, S. J. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

Sutton, R. S. y Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2.^a ed.). MIT Press.

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Shneiderman, B. (2022). *Human-centered AI*. Oxford University Press.

Crevier, D. (1993). *AI: The Tumultuous History of the Search for Artificial Intelligence*. Basic Books.

Kurzweil, R. (2005). *The Singularity Is Near: When Humans Transcend Biology*. Viking Press.

Wiener, N. (1948). *Cybernetics: Or control and communication in the animal and the machine*. MIT Press.

Weizenbaum, J. (1976). *Computer power and human reason: From judgment to calculation*. W. H. Freeman.

Lighthill, J. (1973). *Artificial Intelligence: A General Survey*. Science Research Council.

Norman, D. A. (1988). *The design of everyday things*. Basic Books.

Dix, A., Finlay, J., Abowd, G. D., y Beale, R. (2004). *Human-computer interaction* (3.^a ed.). Pearson Education.

Shneiderman, B. y Plaisant, C. (2010). *Designing the user interface* (5.^a ed.). Addison-Wesley.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2.^a ed.). Lawrence Erlbaum Associates.

Siegel, S. y Castellan, N. J. (1988). *Nonparametric statistics for the behavioral sciences* (2.^a ed.). McGraw-Hill.

Warschauer, M. (2004). *Technology and social inclusion: Rethinking the digital divide*. MIT Press.

Raymond, E. S. (2003). *The art of Unix programming*. Addison-Wesley.

Artículos en revistas

McCulloch, W. S. y Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115–133.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433–460.

Weizenbaum, J. (1966). ELIZA — A computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36–45.

Newell, A. y Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3), 113–126.

Searle, J. R. (1980). Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3), 417–424.

Wooldridge, M. y Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2), 115–152.

Sánchez, J. A. y Leggett, J. J. (1997). Agent services for users of digital libraries. *Journal of Network and Computer Applications*, 20(2), 99–114.

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8), 2554–2558.

Rumelhart, D. E., Hinton, G. E., y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

- Silver, D., Schrittwieser, J., Simonyan, K., *et al.* (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354–359.
- Jumper, J., Evans, R., Pritzel, A., *et al.* (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583–589.
- Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N. y Lloyd, S. (2017). Quantum machine learning. *Nature*, 549(7671), 195–202.
- Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction*, 36(6), 495–504.
- Lipton, Z. C. (2018). The mythos of model interpretability. *Queue*, 16(3), 31–57.
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206–215.
- Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O. y Tatbul, N. (2019). Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 1705–1718.
- Liu, Z., Chen, H. y Smelyanskiy, M. (2023). 3D-compressed model parallelism for efficient inference on edge devices. *IEEE Micro*, 43(2), 48–57.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83. <https://doi.org/10.2307/3001968>
- Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for *t*-tests and ANOVAs. *Frontiers in Psychology*, 4, 863. <https://doi.org/10.3389/fpsyg.2013.00863>
- Braun, V. y Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101. <https://doi.org/10.1191/1478088706qp063oa>
- Zwakman, D.S., Pal, D. y Arpikanondt, C. (2021). Usability evaluation of artificial intelligence-based voice assistants: The case of Amazon Alexa. *SN Computer Science*, 2, 28. <https://doi.org/10.1007/s42979-020-00424-4>
- Pal, D., Arpikanondt, C., Funilkul, S. y Chutimaskul, W. (2022). A systematic review of voice assistant usability: An ISO 9241–11 approach. *SN Computer Science*, 3, 267. <https://doi.org/10.1007/s42979-022-01172-3>

Chen, Q., Gong, Y. y Lu, Y. (2024). User experience of digital voice assistant: Conceptualization and measurement. *ACM Transactions on Computer-Human Interaction*, 31(1), 1–40. <https://doi.org/10.1145/3622782>

Jakob, D., Wilhelm, S., Gerl, A., Ahrens, D. y Wahl, F. (2025). Adapting voice assistant technology for older adults. *Digital*, 5(1), 4. <https://doi.org/10.3390/digital5010004>

Artículos en conferencias

Amershi, S., Weld, D., Vorvoreanu, M., *et al.* (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–13.

Luger, E. y Sellen, A. (2016). “Like having a really bad PA”: The gulf between user expectation and experience of conversational agents. *Proceedings of the 2016 CHI Conference*, 5286–5297.

Kumar, N., Cannanure, V. K., Gamage, D., Prabhakar, A. S., Sturm, C., Loaiza, C. R., Sabie, D., Bhuiyan, M. M. y Moreno Rocha, M. A. (2020). HCI across borders and sustainable development goals. *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*, 1–8. <https://doi.org/10.1145/3334480.3375067>

Berdasco, A., López, G., Diaz, I., Quesada, L. y Guerrero, L. A. (2019). User experience comparison of intelligent personal assistants: Alexa, Google Assistant, Siri and Cortana. *Proceedings of UCAmI 2019*, 31(1), 51. <https://doi.org/10.3390/proceedings2019031051>

Brüggemeier, B., Breiter, M., Kurz, M. y Schiwy, J. (2020). User experience of Alexa, Siri and Google Assistant when controlling music — Comparison of four questionnaires. En *HCI International 2020 — Late Breaking Papers: User Experience Design and Case Studies* (pp. 600–618). Springer. https://doi.org/10.1007/978-3-030-60114-0_40

Devlin, J., Chang, M.-W., Lee, K., y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.

Vaswani, A., Shazeer, N., Parmar, N., *et al.* (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Pennington, J., Socher, R. y Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543.

- Brown, T., Mann, B., Ryder, N., *et al.* (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33.
- Ouyang, L., Wu, J., Jiang, X., *et al.* (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.
- Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A. M., Chen, Z., Le, Q. V. y Laudon, J. (2022). Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kasber, P., Andrejczuk, E., Bernauer, J., Catanzaro, B., Phanishayee, A. y Zaharia, M. (2021). Efficient large-scale language model training on GPU clusters using Megatron-LM. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D. y Steinhardt, J. (2021). Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25.
- He, K., Chen, X., Xie, S., Li, Y., Dollár, P. y Girshick, R. (2022). Masked autoencoders are scalable vision learners. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16000–16009.
- Karras, T., Laine, S. y Aila, T. (2020). Analyzing and improving the image quality of StyleGAN. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 8110–8119.
- Finn, C., Abbeel, P. y Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 1126–1135.
- Mao, H., Alizadeh, M. y Menache, I. (2016). Resource management with deep reinforcement learning. *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 50–56.
- Bender, E. M., Gebru, T., McMillan-Major, A. y Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 610–623.

Strubell, E., Ganesh, A. y McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3645–3650.

Ahmadian, M. M., Shahriari, H. R. y Ghaffarian, S. M. (2020). Connection-monitor & connection-breaker: A novel approach for prevention and detection of high survivable ransomwares. *Proceedings of the 15th International Conference on Availability, Reliability and Security*, 1–10.

Artículos técnicos y preprints

Mikolov, T., Chen, K., Corrado, G., y Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Lin, X. V., Wang, C., Zettlemoyer, L., y Ernst, M. D. (2018). NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. *arXiv preprint arXiv:1802.08979*.

OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.

Chen, M., Tworek, J., Jun, H., *et al.* (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. y Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C. y Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Amodei, D., Olah, C., Steinhardt, J., *et al.* (2016). Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*.

Perez, E., Huang, S., Song, F., *et al.* (2022). Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*.

Tihanyi, N., Bisztray, T., Ferrag, M. A., Jain, R. y Cordeiro, L. C. (2024). How secure is AI-generated code: A large-scale comparison of large language models. *arXiv preprint arXiv:2404.18353*.

McCarthy, J., Minsky, M. L., Rochester, N., y Shannon, C. E. (1955). A proposal for the Dartmouth Summer Research Project on Artificial Intelligence.

Tesis y documentos académicos

Aragon Toledo, J. R. (2026). *Marco Histórico y Referencial Para el Desarrollo de Aplicaciones con IA*. Universidad Tecnológica de la Mixteca.

Aragon Toledo, J. R. y Ruiz Rodríguez, R. (2026). BOTAS: A human-centered natural language interface for GNU/Linux system management with multi-distribution support. En P. Isaias, P. Miranda y G. C. Peng (Eds.), *Proceedings of the International Conferences on Artificial Intelligence in Society 2026, Digital Transformation and Innovation Management 2026 and Connected Smart Cities 2026* (pp. 100–110). Valencia, España: IADIS Press. ISBN 978-989-8704-78-8.

Sánchez, J. A. (1996). *Agent services* [Tesis doctoral]. Texas A&M University. ProQuest Dissertations & Theses, n.º 9701711.

Sánchez, J. A. (1997). *A taxonomy of agents*. Reporte técnico ICT-9701. Departamento de Computación, Universidad de las Américas Puebla (UDLAP).

Misceláneos

Organización Internacional de Normalización. (2018). *ISO 9241-11:2018. Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts*. International Organization for Standardization.

Top500.org. (2023). *Operating system family / Linux*. Recuperado de <https://www.top500.org/statistics/details/osfam/1/>

W3Techs. (2023). *Usage statistics of Linux for websites*. Recuperado de <https://w3techs.com/technologies/details/os-linux>

Juniper Research. (2023). *Voice assistants: Market size, adoption & key trends 2023–2028*. Juniper Research Ltd.

OpenAI. (2022). Introducing Whisper. <https://openai.com/research/whisper>

Alpha Cephei. (2021). *Vosk offline speech recognition toolkit*. <https://alphacephei.com/vosk/>

DeepSeek-AI. (2024). *DeepSeek LLM: Scaling open-source language models with longtermism*. Recuperado de <https://github.com/deepseek-ai/DeepSeek-LLM>

- xAI. (2024). *Grok: Conversational AI with real-time social context integration*. Recuperado de <https://x.ai/>
- Steinberger, P. y la comunidad OpenClaw. (2025). *ClawdBot — Personal AI Assistant* [Software]. Recuperado de <https://github.com/openclaw/openclaw>
- Saha, S., Marsh, C. y Holthuis, J. (2014). *Jasper — An open source platform for developing always-on, voice-controlled applications* [Software]. Recuperado de <https://github.com/jasperproject/jasper-client>
- OpenVoiceOS. (2024). *OVOS-core — The FOSS Artificial Intelligence platform* [Software]. Recuperado de <https://github.com/OpenVoiceOS/ovos-core>
- Bhartiya, S. (17 de enero de 2016). Mycroft: Linux’s Own AI. *Linux.com*. Recuperado de <https://www.linux.com/news/mycroft-linuxs-own-ai>
- Dobberstein, L. (13 de febrero de 2023). Creator of Linux virtual assistant blames ‘patent troll’ for project’s death. *The Register*. Recuperado de https://www.theregister.com/2023/02/13/linux_ai_assistant_killed_off/
- Edwards, B. (5 de marzo de 2025). Will the future of software development run on vibes? *Ars Technica*. Recuperado de <https://arstechnica.com/ai/2025/03/is-vibe-coding-with-ai-gnarly-or-reckless-maybe-some-of-both/>
- Loker, D. (17 de diciembre de 2025). Our new report: AI code creates 1.7x more problems. *CodeRabbit Blog*. Recuperado de <https://www.coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report>
- Karpathy, A. [@karpathy]. (2 de febrero de 2025). There’s a new kind of coding I call “vibe coding”, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists [Publicación en X]. Recuperado de <https://x.com/karpathy/status/1886192184808149383>
- Collins English Dictionary. (6 de noviembre de 2025). *Vibe coding* named Word of the Year 2025. Recuperado de <https://www.collinsdictionary.com/woty>

Anexo A

Interfaces obsoletas

Este anexo reúne las pantallas de la interfaz gráfica de LIMA en la versión empleada hasta antes de la segunda ronda de pruebas de usabilidad. Se conservan aquí, ya obsoletas tras el rediseño motivado por dicha evaluación, como referencia del estado previo del sistema. Las Figuras A.1 a A.9 documentan ese estado; su forma final se retoma en la sección 5.1.1 del Capítulo 5, lo que permite apreciar la evolución visual del sistema y justificar los cambios de diseño introducidos.

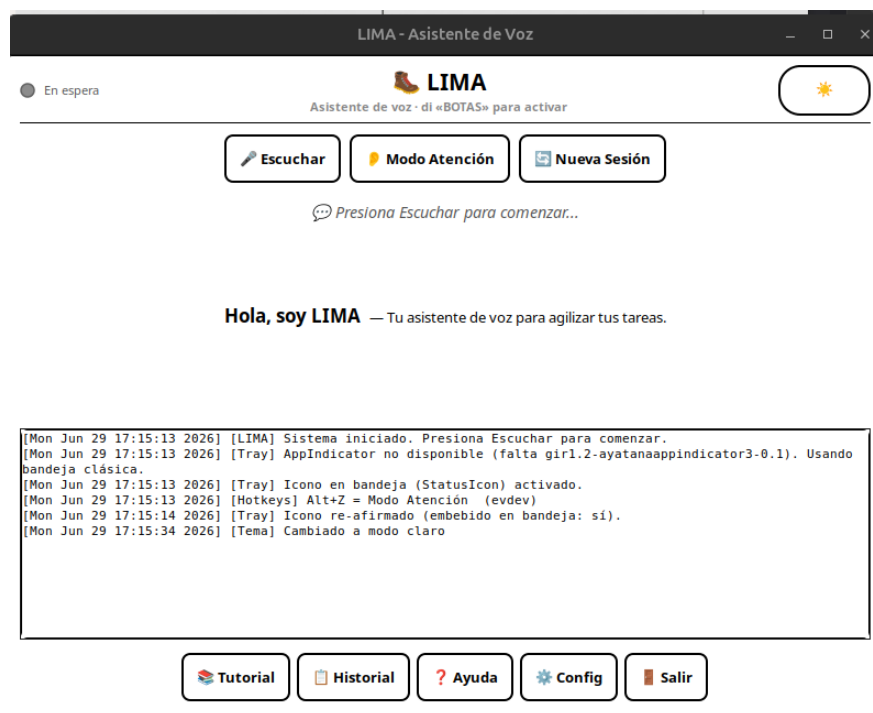


Figura A.1: Ventana principal de LIMA en estado de reposo. Integra el indicador de estado (en gris, «En espera»), el conmutador de tema, los controles *Escuchar*, *Modo Atención* y *Nueva Sesión*, el área de transcripción de la última instrucción, el registro de eventos y la barra inferior de acceso a tutorial, historial, ayuda, configuración y salida.

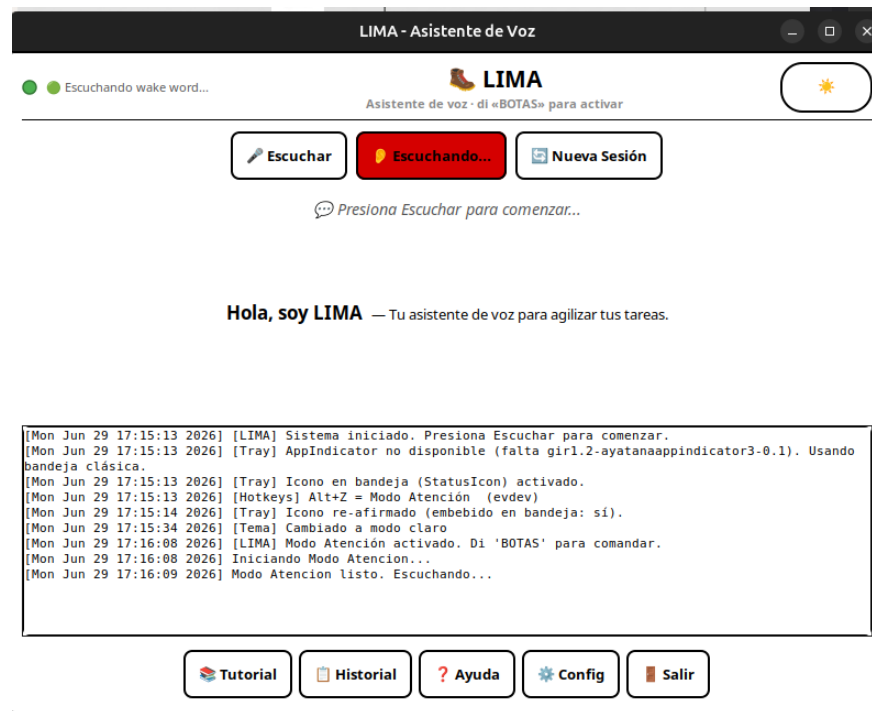


Figura A.2: Ventana principal con el *Modo Atención* activado. En este modo LIMA mantiene una escucha continua y espera la palabra de activación («botas») para capturar un comando sin necesidad de pulsar ningún botón.

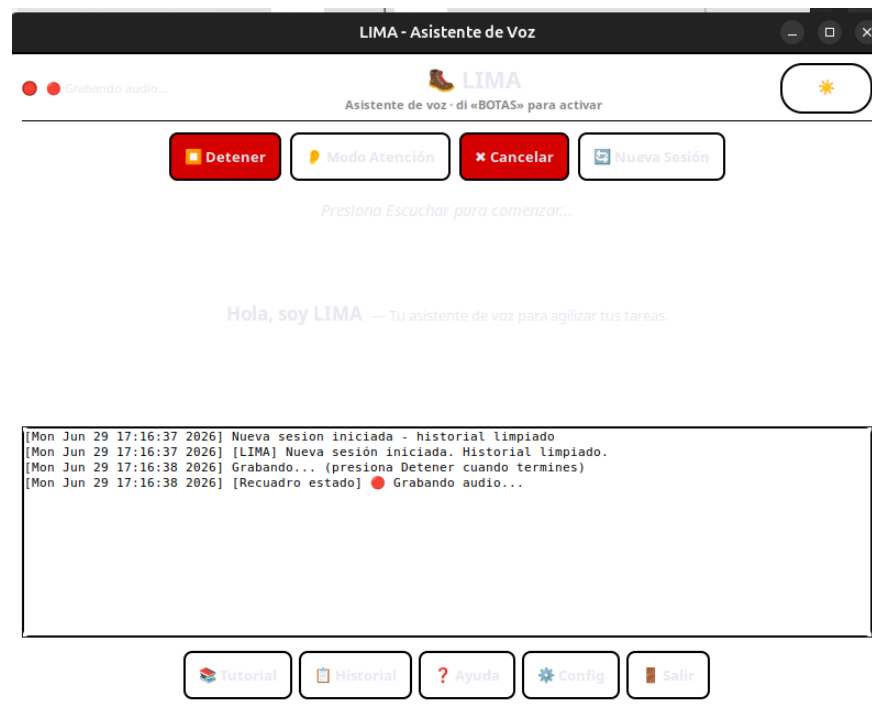


Figura A.3: Ventana principal durante la captura de un comando de voz. El indicador de estado cambia de color para señalar que el sistema está grabando, lo que proporciona retroalimentación visual inmediata del proceso de escucha.

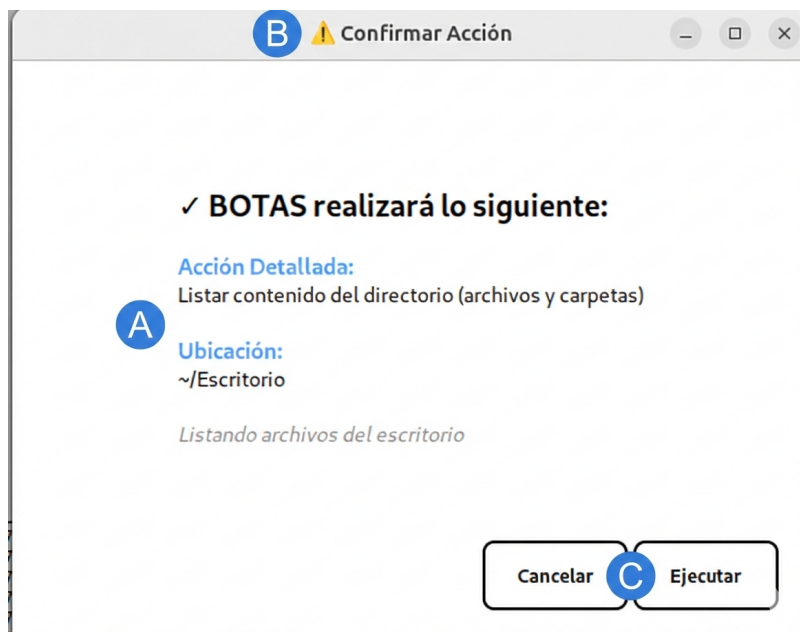


Figura A.4: Diálogo de confirmación previa. Antes de ejecutar una operación clasificada como riesgosa, LIMA describe la acción que va a realizar y solicita la aprobación explícita del usuario, que puede confirmarla o cancelarla.

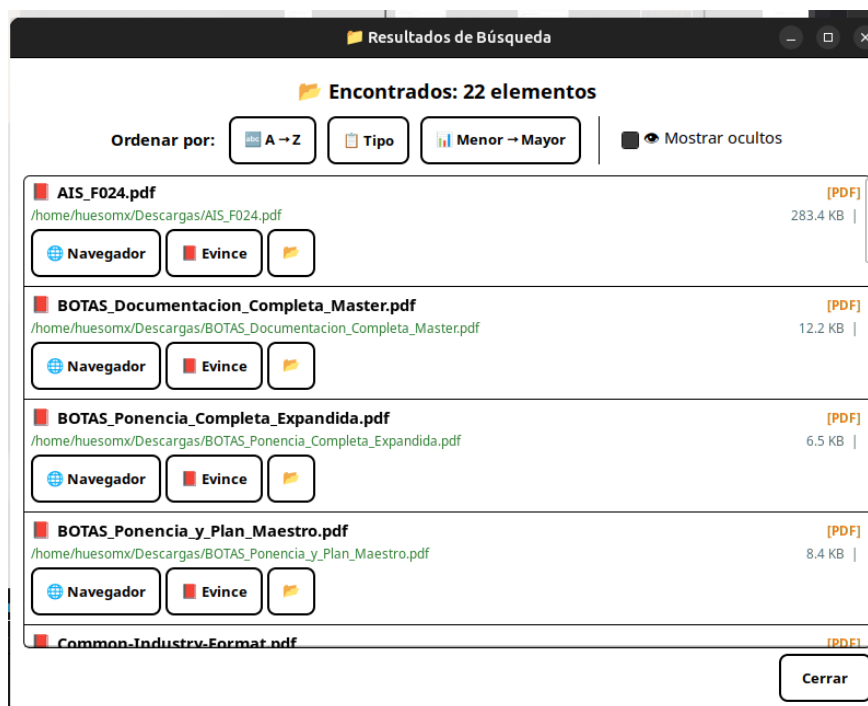


Figura A.5: Visor enriquecido de resultados de búsqueda. Presenta los archivos encontrados con su nombre, tipo, ruta y tamaño, permite ordenarlos (alfabéticamente, por tipo o por tamaño) y ofrece en cada elemento botones de acción directa: abrir en el navegador, abrir con la aplicación asociada o mostrar su ubicación.

Fecha/Hora	Fuente	Petición	Intención	Estado
2026-06-19 18:45:56	LLM	Que deseas buscar en internet?	web_search	
2026-06-19 18:45:56	LLM	Buscar en internet	web_search	✓
2026-06-19 18:36:18	LLM	Buscar en internet una definición para...	web_search	✓
2026-06-19 18:31:56	LLM	En la carpeta 329 Botas hay un archivo que se llama Temar...	move	✓
2026-06-19 18:31:07	LLM	Mueve de la carpeta Descargas el archivo Temario Mate Di...	move	✓
2026-06-19 18:26:45	LLM	En la carpeta 302B del escritorio quiero que crees seis car...	create	✓
2026-06-19 18:25:32	LLM	En el escritorio.	create	✓
2026-06-19 18:25:17	LLM	Crear carpeta 302B Botas.	clarify	✓
2026-06-19 18:01:40	LLM	Quiero que busques en internet algunas definiciones de q...	web_search	✓
2026-06-19 17:57:28	LLM	Quiero que muevas el archivo llamado temario mates disc...	move	✓
2026-06-19 17:52:08	LLM	Quiero que crees una carpeta en el escritorio llamada 302...	create	✓
2026-06-19 17:30:30	LLM	Elabora una plantilla para LLM de el asistente.	web_search	✓

Fecha: 2026-06-19 18:45:56 Fuente: LLM Petición: Que deseas buscar en internet? Intención detectada: web_search Resultado: ✗

Cerrar

Figura A.6: Visor de historial de peticiones. Registra las solicitudes atendidas durante la sesión, lo que facilita su consulta y la auditoría de las acciones ejecutadas por el asistente.

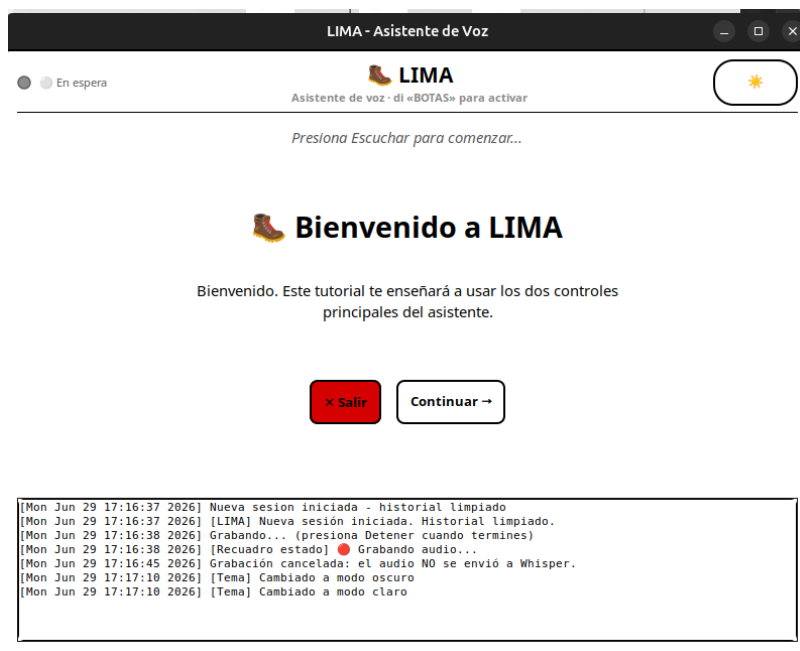


Figura A.7: Tutorial interactivo de primer uso (paso 1). Pantalla de bienvenida que anticipa al usuario los dos controles principales del asistente.

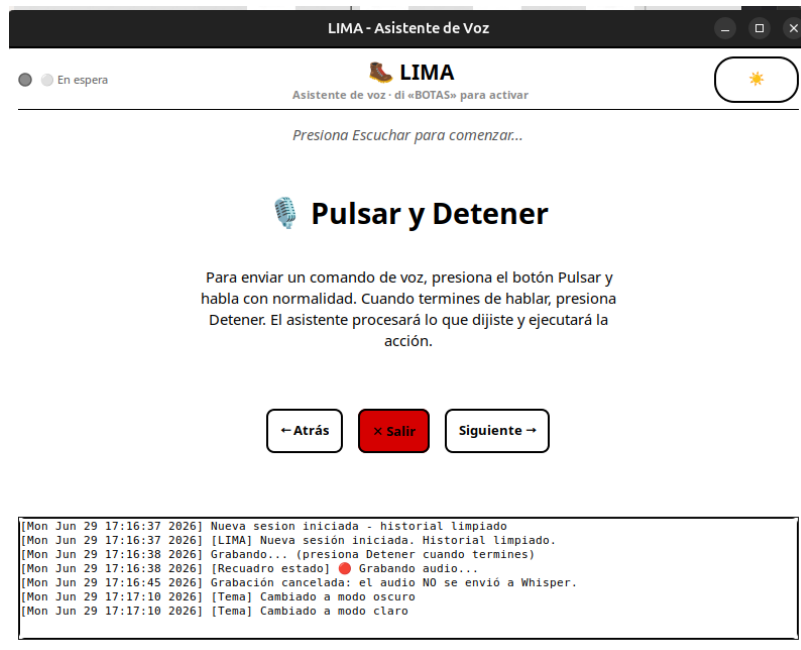


Figura A.8: Tutorial interactivo de primer uso (paso 2). Explica el control manual «Pulsar y Detener»: el usuario presiona *Escuchar*, habla con normalidad y, al detener, LIMA procesa y ejecuta la orden.

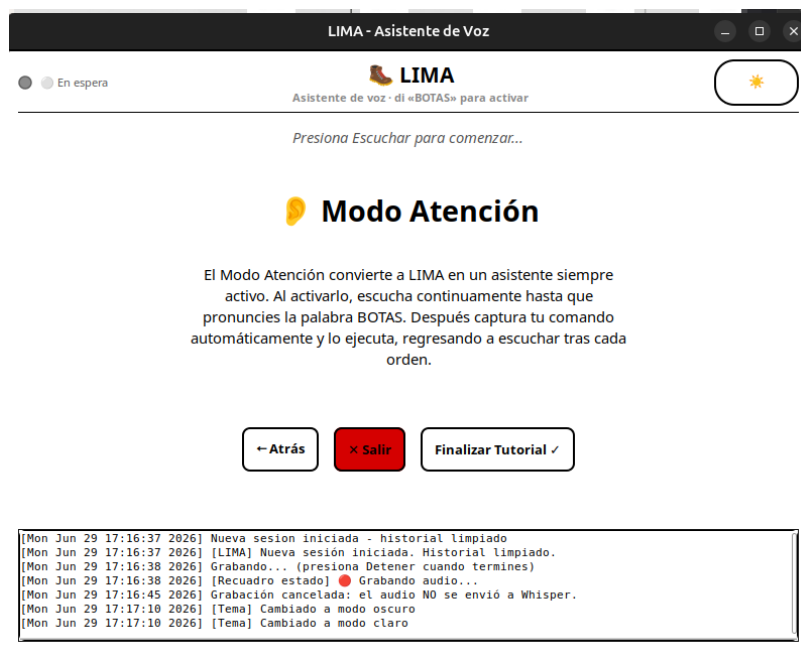


Figura A.9: Tutorial interactivo de primer uso (paso 3). Explica el *Modo Atención*: con la escucha continua activada, LIMA espera la palabra «botas» y captura el comando de forma automática, manos libres.

Anexo B

Conjunto de datos de evaluación

El presente anexo contiene el conjunto de datos de 250 tareas utilizado para evaluar la capacidad de LIMA de interpretar solicitudes en lenguaje natural y ejecutar la acción correspondiente. Cada tarea incluye un identificador, la categoría funcional, la solicitud en lenguaje natural, la acción o comando esperado y el nivel de riesgo asignado. El conjunto se distribuye en ocho categorías: Gestión de archivos (50), Información del sistema (45), Control de procesos (37), Redes (37), Permisos (31), Búsqueda web (20), Creación de proyectos (20) y Apertura de aplicaciones (10); con tres niveles de riesgo: seguro (219), advertencia (23) y peligroso (8).

B.1. Gestión de archivos

Tabla B.1: Tareas de evaluación — Gestión de archivos (50 tareas)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
1	Muéstrame todos los archivos en el directorio actual	<code>ls -la</code>	Seguro
2	Lista solo los archivos PDF en esta carpeta	<code>ls *.pdf</code>	Seguro
3	Encuentra todos los archivos modificados en las últimas 24 horas	<code>find . -mtime -1</code>	Seguro
4	Busca archivos más grandes de 100MB	<code>find / -size +100M</code>	Seguro
5	Encuentra todos los archivos Python en el sistema	<code>find / -name "*.py"</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.1)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
6	Muestra archivos ordenados por tamaño	<code>ls -lhS</code>	Seguro
7	Busca archivos que contengan la palabra «config»	<code>grep -r "config" .</code>	Seguro
8	Copia el archivo data.txt a /tmp	<code>cp data.txt /tmp/</code>	Seguro
9	Mueve todos los archivos .log a la carpeta logs	<code>mv *.log logs/</code>	Seguro
10	Renombra archivo.txt a documento.txt	<code>mv archivo.txt documento.txt</code>	Seguro
11	Crea una copia de seguridad de database.sql	<code>cp database.sql database.sql.bak</code>	Seguro
12	Cuenta cuántos archivos hay en el directorio	<code>ls -l wc -l</code>	Seguro
13	Muestra el tamaño total de todos los archivos	<code>du -sh .</code>	Seguro
14	Lista archivos ocultos	<code>ls -a</code>	Seguro
15	Encuentra archivos vacíos	<code>find . -type f -empty</code>	Seguro
16	Busca archivos con permisos 777	<code>find / -perm 777</code>	Seguro
17	Muestra el árbol de directorios	<code>tree -L 2</code>	Seguro
18	Encuentra archivos duplicados por nombre	<code>find . sort uniq -d</code>	Seguro
19	Busca archivos modificados en la última semana	<code>find . -mtime -7</code>	Seguro
20	Lista archivos con extensión .conf	<code>find / -name "*.conf"</code>	Seguro
21	Muestra archivos más recientes primero	<code>ls -lt</code>	Seguro
22	Cuenta líneas de todos los archivos Python	<code>find . -name "*.py" -exec wc -l {} +</code>	Seguro
23	Busca archivos de más de 1 año	<code>find / -mtime +365</code>	Seguro
24	Lista solo directorios	<code>ls -d */</code>	Seguro
25	Encuentra archivos que no pertenecen a ningún usuario	<code>find / -nouser</code>	Seguro
26	Comprime la carpeta proyecto en tar.gz	<code>tar -czf proyecto.tar.gz proyecto/</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.1)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
27	Descomprime archivo.tar.gz	tar -xzf archivo.tar.gz	Seguro
28	Crea un directorio llamado backups	mkdir backups	Seguro
29	Crea estructura de directorios proyecto/src/lib	mkdir -p proyecto/src/lib	Seguro
30	Muestra el contenido del archivo config.txt	cat config.txt	Seguro
31	Muestra las primeras 10 líneas de log.txt	head -n 10 log.txt	Seguro
32	Muestra las últimas 20 líneas de error.log	tail -n 20 error.log	Seguro
33	Busca la palabra «error» en todos los logs	grep -i "error" *.log	Seguro
34	Cuenta cuántas veces aparece «WARNING» en el log	grep -c "WARNING" app.log	Seguro
35	Compara dos archivos de configuración	diff config1.txt config2.txt	Seguro
36	Crea un enlace simbólico de app a /usr/local/bin	ln -s app /usr/local/bin/app	Seguro
37	Encuentra archivos .tmp y muéstralos	find / -name "*.tmp"	Seguro
38	Busca archivos que contengan «password» en el nombre	find / -iname "password"	Seguro
39	Lista archivos ejecutables en el directorio actual	find . -type f -executable	Seguro
40	Muestra el tipo de archivo de documento.pdf	file documento.pdf	Seguro
41	Busca archivos .jpg mayores de 5MB	find / -name "*.jpg" -size +5M	Seguro
42	Sincroniza carpeta local con remota	rsync -avz local/ remote/	Seguro
43	Elimina archivos .tmp del directorio actual	rm *.tmp	Advert.
44	Borra recursivamente la carpeta temp	rm -r temp/	Peligro
45	Elimina forzosamente todos los archivos .cache	rm -rf *.cache	Peligro

Continúa en la siguiente página

(Continuación de la Tabla B.1)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
186	Encuentra archivos Java en /home	<code>find /home -name "*.java"</code>	Seguro
187	Busca archivos modificados hoy	<code>find . -mtime 0</code>	Seguro
188	Crea archivo vacío	<code>touch nuevo.txt</code>	Seguro
189	Concatena varios archivos	<code>cat file1 file2 >combined.txt</code>	Seguro
190	Busca texto en archivos recursivamente	<code>grep -r "TODO" /proyecto</code>	Seguro

B.2. Información del sistema

Tabla B.2: Tareas de evaluación — Información del sistema (45 tareas)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
46	Muestra información del sistema operativo	<code>uname -a</code>	Seguro
47	¿Qué versión de Ubuntu tengo?	<code>lsb_release -a</code>	Seguro
48	Muestra el uso de disco	<code>df -h</code>	Seguro
49	¿Cuánto espacio libre tengo?	<code>df -h grep -v tmpfs</code>	Seguro
50	Muestra el uso de memoria RAM	<code>free -h</code>	Seguro
51	¿Cuánta memoria tengo disponible?	<code>free -h grep Mem</code>	Seguro
52	Muestra los procesos en ejecución	<code>ps aux</code>	Seguro
53	Lista procesos ordenados por uso de CPU	<code>ps aux -sort=-%cpu head</code>	Seguro
54	Lista procesos ordenados por uso de memoria	<code>ps aux -sort=-%mem head</code>	Seguro
55	Muestra el tiempo de actividad del sistema	<code>uptime</code>	Seguro
56	¿Cuánto tiempo lleva encendida la máquina?	<code>uptime -p</code>	Seguro
57	Muestra información del CPU	<code>lscpu</code>	Seguro
58	¿Cuántos núcleos tiene mi procesador?	<code>nproc</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.2)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
59	Muestra información de hardware	<code>lshw -short</code>	Seguro
60	Lista dispositivos PCI	<code>lspci</code>	Seguro
61	Lista dispositivos USB conectados	<code>lsusb</code>	Seguro
62	Muestra información de la memoria RAM instalada	<code>dmidecode -t memory</code>	Seguro
63	¿Cuál es mi dirección IP?	<code>hostname -I</code>	Seguro
64	Muestra el nombre del host	<code>hostname</code>	Seguro
65	Muestra las variables de entorno	<code>printenv</code>	Seguro
66	¿Cuál es mi directorio home?	<code>echo \$HOME</code>	Seguro
67	Muestra la ruta del PATH	<code>echo \$PATH</code>	Seguro
68	Lista usuarios del sistema	<code>cat /etc/passwd</code>	Seguro
69	¿Quién está conectado ahora?	<code>who</code>	Seguro
70	Muestra mi nombre de usuario	<code>whoami</code>	Seguro
71	Muestra el historial de comandos	<code>history</code>	Seguro
72	Lista los últimos 10 comandos ejecutados	<code>history tail -10</code>	Seguro
73	Muestra la fecha y hora actual	<code>date</code>	Seguro
74	¿En qué zona horaria estoy?	<code>timedatectl</code>	Seguro
75	Muestra el calendario del mes actual	<code>cal</code>	Seguro
76	Lista las particiones del disco	<code>lsblk</code>	Seguro
77	Muestra el uso de inodos	<code>df -i</code>	Seguro
78	Información detallada del disco	<code>fdisk -l /dev/sda</code>	Seguro
79	Muestra estadísticas del sistema en tiempo real	<code>top</code>	Seguro
80	Monitor de recursos alternativo	<code>htop</code>	Seguro
81	Muestra solo procesos de usuario actual	<code>ps u</code>	Seguro
82	Lista servicios activos	<code>systemctl list-units -type=service</code>	Seguro
83	Verifica estado del servicio apache	<code>systemctl status apache2</code>	Seguro
84	Muestra logs del sistema	<code>journalctl -n 50</code>	Seguro
85	Muestra carga promedio del sistema	<code>cat /proc/loadavg</code>	Seguro
191	Muestra kernel version	<code>uname -r</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.2)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
192	Información de temperatura del CPU	<code>sensors</code>	Seguro
193	Muestra velocidad de red en tiempo real	<code>iftop -i eth0</code>	Seguro
194	Uso de swap	<code>swapon -show</code>	Seguro
195	Lista módulos del kernel cargados	<code>lsmod</code>	Seguro

B.3. Control de procesos

Tabla B.3: Tareas de evaluación — Control de procesos (37 tareas)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
86	Mata el proceso con PID 1234	<code>kill 1234</code>	Advert.
87	Fuerza la terminación del proceso 5678	<code>kill -9 5678</code>	Advert.
88	Mata todos los procesos llamados firefox	<code>killall firefox</code>	Advert.
89	Termina proceso llamado chrome	<code>pkill chrome</code>	Advert.
90	Busca el PID del proceso apache2	<code>pgrep apache2</code>	Seguro
91	Muestra información del proceso con PID 100	<code>ps -p 100 -f</code>	Seguro
92	Lista procesos del usuario www-data	<code>ps -u www-data</code>	Seguro
93	Ejecuta comando en segundo plano	<code>comando &</code>	Seguro
94	Muestra trabajos en segundo plano	<code>jobs</code>	Seguro
95	Trae trabajo 1 al primer plano	<code>fg %1</code>	Seguro
96	Envía trabajo actual al fondo	<code>bg</code>	Seguro
97	Reinicia el servicio nginx	<code>systemctl restart nginx</code>	Advert.
98	Detén el servicio mysql	<code>systemctl stop mysql</code>	Advert.
99	Inicia el servicio postgresql	<code>systemctl start postgresql</code>	Seguro
100	Habilita apache2 al inicio	<code>systemctl enable apache2</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.3)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
101	Deshabilita servicio bluetooth	<code>systemctl disable bluetooth</code>	Seguro
102	Recarga configuración de systemd	<code>systemctl daemon-reload</code>	Seguro
103	Muestra árbol de procesos	<code>pstree</code>	Seguro
104	Monitor de procesos con filtro	<code>top -u usuario</code>	Seguro
105	Cambia prioridad del proceso 200	<code>renice -n 10 -p 200</code>	Seguro
106	Ejecuta comando con prioridad baja	<code>nice -n 19 comando</code>	Seguro
107	Ejecuta comando con prioridad alta	<code>nice -n -10 comando</code>	Seguro
108	Mata todos los procesos de un usuario	<code>pkill -u usuario</code>	Peligro
109	Termina sesión del usuario remoto	<code>pkill -KILL -u usuario</code>	Peligro
110	Reinicia todo el sistema	<code>reboot</code>	Peligro
111	Apaga el sistema	<code>shutdown -h now</code>	Peligro
112	Programa apagado en 10 minutos	<code>shutdown -h +10</code>	Advert.
113	Cancela apagado programado	<code>shutdown -c</code>	Seguro
114	Reinicia servicios de red	<code>systemctl restart networking</code>	Advert.
115	Muestra uso de CPU por proceso	<code>pidstat</code>	Seguro
116	Monitorea proceso específico	<code>watch -n 1 'ps aux grep nginx'</code>	Seguro
117	Lista procesos zombie	<code>ps aux grep 'Z'</code>	Seguro
118	Muestra hilos de un proceso	<code>ps -T -p 1234</code>	Seguro
119	Límite de archivos abiertos por proceso	<code>ulimit -n</code>	Seguro
120	Ejecuta comando con timeout de 10s	<code>timeout 10s comando</code>	Seguro
196	Lista servicios fallidos	<code>systemctl -failed</code>	Seguro
197	Muestra logs de un servicio	<code>journalctl -u apache2 -n 50</code>	Seguro

B.4. Redes

Tabla B.4: Tareas de evaluación — Redes (37 tareas)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
121	Muestra configuración de red	<code>ifconfig</code>	Seguro
122	Muestra interfaces de red alternativo	<code>ip addr show</code>	Seguro
123	Prueba conectividad con google.com	<code>ping -c 4 google.com</code>	Seguro
124	Haz ping continuo a 8.8.8.8	<code>ping 8.8.8.8</code>	Seguro
125	Muestra la tabla de ruteo	<code>route -n</code>	Seguro
126	Muestra rutas alternativo	<code>ip route show</code>	Seguro
127	Muestra conexiones de red activas	<code>netstat -tuln</code>	Seguro
128	Lista puertos en escucha	<code>ss -tuln</code>	Seguro
129	Muestra conexiones establecidas	<code>netstat -ant grep ESTABLISHED</code>	Seguro
130	¿Qué proceso usa el puerto 80?	<code>lsof -i :80</code>	Seguro
131	Busca qué proceso escucha en puerto 3306	<code>netstat -tlnp grep 3306</code>	Seguro
132	Descarga archivo de URL	<code>wget https://ejemplo.com/archivo.zip</code>	Seguro
133	Descarga con curl	<code>curl -O https://ejemplo.com/datos.json</code>	Seguro
134	Prueba velocidad de red con iperf	<code>iperf3 -c servidor</code>	Seguro
135	Resuelve nombre DNS	<code>nslookup google.com</code>	Seguro
136	Consulta DNS con dig	<code>dig google.com</code>	Seguro
137	Muestra caché ARP	<code>arp -a</code>	Seguro
138	Traza ruta a servidor	<code>traceroute google.com</code>	Seguro
139	Traza ruta alternativo	<code>mtr google.com</code>	Seguro
140	Escanea puertos abiertos en localhost	<code>nmap localhost</code>	Seguro
141	Muestra estadísticas de interfaces	<code>netstat -i</code>	Seguro
142	Monitorea tráfico de red	<code>iftop</code>	Seguro
143	Captura paquetes de red	<code>tcpdump -i eth0</code>	Seguro
144	Conéctate por SSH a servidor	<code>ssh usuario@servidor</code>	Seguro
145	Copia archivo por SCP	<code>scp archivo.txt usuario@servidor:/tmp</code>	Seguro
146	Transfiere archivos con rsync por SSH	<code>rsync -avz -e ssh local/usuario@servidor:remote/</code>	Seguro
147	Muestra configuración DNS	<code>cat /etc/resolv.conf</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.4)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
148	Muestra archivo de hosts	<code>cat /etc/hosts</code>	Seguro
149	Prueba puerto abierto con telnet	<code>telnet servidor 80</code>	Seguro
150	Prueba puerto con netcat	<code>nc -zv servidor 22</code>	Seguro
151	Reinicia interfaz de red	<code>ifdown eth0 && ifup eth0</code>	Advert.
152	Asigna IP estática temporal	<code>ifconfig eth0 192.168.1.100</code>	Advert.
153	Agrega ruta estática	<code>route add -net 10.0.0.0/8 gw 192.168.1.1</code>	Advert.
154	Elimina ruta	<code>route del -net 10.0.0.0/8</code>	Advert.
155	Reinicia firewall	<code>systemctl restart firewalld</code>	Advert.
198	Muestra firewall rules	<code>sudo iptables -L</code>	Seguro
199	Test de ancho de banda	<code>speedtest-cli</code>	Seguro

B.5. Permisos

Tabla B.5: Tareas de evaluación — Permisos (31 tareas)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
156	Muestra permisos de archivo	<code>ls -l archivo.txt</code>	Seguro
157	Muestra permisos en formato octal	<code>stat -c '%a' archivo.txt</code>	Seguro
158	Da permisos de ejecución a script	<code>chmod +x script.sh</code>	Seguro
159	Quita permisos de escritura	<code>chmod -w archivo.txt</code>	Seguro
160	Establece permisos 644 a archivo	<code>chmod 644 archivo.txt</code>	Seguro
161	Establece permisos 755 a directorio	<code>chmod 755 directorio/</code>	Seguro
162	Cambia permisos recursivamente	<code>chmod -R 755 carpeta/</code>	Seguro
163	Cambia dueño de archivo a usuario	<code>chown usuario archivo.txt</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.5)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
164	Cambia grupo de archivo	chgrp grupo archivo.txt	Seguro
165	Cambia dueño y grupo	chown usuario:grupo archivo.txt	Seguro
166	Cambia dueño recursivamente	chown -R usuario:grupo directorio/	Seguro
167	Muestra el dueño de un archivo	stat -c '%U' archivo.txt	Seguro
168	Encuentra archivos con permiso SUID	find / -perm -4000	Seguro
169	Encuentra archivos con permiso SGID	find / -perm -2000	Seguro
170	Encuentra archivos modificables por todos	find / -perm -002	Seguro
171	Establece bit sticky en directorio	chmod +t directorio/	Seguro
172	Establece SUID en ejecutable	chmod u+s ejecutable	Advert.
173	Da permisos 777 a archivo	chmod 777 archivo	Advert.
174	Quita todos los permisos	chmod 000 archivo	Peligro
175	Ejecuta comando como root	sudo comando	Advert.
176	Abre shell como root	sudo -i	Advert.
177	Ejecuta comando como otro usuario	sudo -u usuario comando	Seguro
178	Edita archivo de configuración protegido	sudo nano /etc/hosts	Advert.
179	Lista permisos de sudoers	sudo cat /etc/sudoers	Seguro
180	Agrega usuario al grupo sudo	sudo usermod -aG sudo usuario	Advert.
181	Cambia contraseña de usuario	passwd usuario	Seguro
182	Cambia contraseña como root	sudo passwd usuario	Advert.
183	Crea nuevo usuario	sudo useradd -m nuevouser	Advert.
184	Elimina usuario	sudo userdel usuario	Advert.
185	Elimina usuario y su home	sudo userdel -r usuario	Peligro

Continúa en la siguiente página

(Continuación de la Tabla B.5)

ID	Solicitud en lenguaje natural	Comando esperado	Riesgo
200	Lista ACLs de archivo	<code>getfacl archivo.txt</code>	Seguro

B.6. Búsqueda web

Tabla B.6: Tareas de evaluación — Búsqueda web (20 tareas)

ID	Solicitud en lenguaje natural	Acción esperada	Riesgo
201	Busca en Google cómo instalar Python en Linux	<code>web_search(google, Python Linux)</code>	Seguro
202	Busca en YouTube un tutorial de Bash para principiantes	<code>web_search(youtube, Bash tutorial)</code>	Seguro
203	Busca en GitHub proyectos de reconocimiento de voz	<code>web_search(github, speech recognition)</code>	Seguro
204	Busca en StackOverflow cómo leer un archivo CSV en Python	<code>web_search(stackoverflow, CSV Python)</code>	Seguro
205	Busca en Wikipedia qué es un sistema operativo	<code>web_search(wikipedia, sistema operativo)</code>	Seguro
206	Busca en Google cómo configurar SSH en Debian	<code>web_search(google, SSH Debian)</code>	Seguro
207	Busca en YouTube tutorial de Docker desde cero	<code>web_search(youtube, Docker tutorial)</code>	Seguro
208	Busca en GitHub ejemplos de scripts AWK	<code>web_search(github, awk scripts)</code>	Seguro
209	Busca en StackOverflow cómo solucionar un error de segmentación en C	<code>web_search(stackoverflow, segfault C)</code>	Seguro
210	Busca en Wikipedia información sobre el protocolo TCP/IP	<code>web_search(wikipedia, TCP/IP)</code>	Seguro
211	Busca en Google solución a paquetes rotos en apt	<code>web_search(google, apt broken packages)</code>	Seguro
212	Busca en YouTube tutorial de Git para control de versiones	<code>web_search(youtube, Git tutorial)</code>	Seguro
213	Busca en GitHub repositorios de inteligencia artificial	<code>web_search(github, artificial intelligence)</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.6)

ID	Solicitud en lenguaje natural	Acción esperada	Riesgo
214	Busca en StackOverflow cómo hacer solicitudes HTTP en Perl	<code>web_search(stackoverflow, HTTP Perl)</code>	Seguro
215	Busca en Wikipedia qué es el aprendizaje automático	<code>web_search(wikipedia, machine learning)</code>	Seguro
216	Busca en Google cómo instalar NGINX en Ubuntu	<code>web_search(google, NGINX Ubuntu)</code>	Seguro
217	Busca en YouTube tutorial de expresiones regulares en Linux	<code>web_search(youtube, regex Linux)</code>	Seguro
218	Busca en GitHub proyectos de asistentes de voz de código abierto	<code>web_search(github, voice assistant)</code>	Seguro
219	Busca en StackOverflow la diferencia entre fork y exec en Linux	<code>web_search(stackoverflow, fork exec Linux)</code>	Seguro
220	Busca en Google las diferencias entre Docker y máquinas virtuales	<code>web_search(google, Docker vs VMs)</code>	Seguro

B.7. Creación de proyectos

Tabla B.7: Tareas de evaluación — Creación de proyectos (20 tareas)

ID	Solicitud en lenguaje natural	Acción esperada	Riesgo
221	Crea un proyecto frontend con secciones de login, dashboard e inventario	<code>create_project(frontend)</code>	Seguro
222	Genera la estructura de un proyecto backend con API REST	<code>create_project(backend)</code>	Seguro
223	Crea un proyecto fullstack con frontend y backend separados	<code>create_project(fullstack)</code>	Seguro
224	Genera carpetas para un proyecto Python con src, tests y docs	<code>mkdir -p proyecto/{src, tests, docs}</code>	Seguro
225	Crea un proyecto web con carpetas para estilos, scripts e imágenes	<code>create_project(frontend)</code>	Seguro
226	Genera estructura de proyecto de ciencia de datos con datos, notebooks y modelos	<code>mkdir -p ds/{data, notebooks, models}</code>	Seguro

Continúa en la siguiente página

(Continuación de la Tabla B.7)

ID	Solicitud en lenguaje natural	Acción esperada	Riesgo
227	Crea un proyecto de microservicios con servicios independientes	<code>create_project (backend)</code>	Seguro
228	Genera carpetas para proyecto universitario: src, docs, tests y config	<code>mkdir -p proyecto/{src, docs, tests, config}</code>	Seguro
229	Crea un proyecto fullstack con autenticación y panel de administración	<code>create_project (fullstack)</code>	Seguro
230	Genera la estructura de una aplicación móvil multiplataforma	<code>mkdir -p app/{src, assets, tests}</code>	Seguro
231	Crea un proyecto frontend para un sistema de ventas en línea	<code>create_project (frontend)</code>	Seguro
232	Genera estructura de proyecto backend para una API de gestión de usuarios	<code>create_project (backend)</code>	Seguro
233	Crea la estructura de un proyecto de machine learning con notebooks	<code>mkdir -p ml/{data, models, notebooks, scripts}</code>	Seguro
234	Genera carpetas para una tienda en línea con catálogo y pagos	<code>create_project (frontend)</code>	Seguro
235	Crea la estructura de un servidor web con logs, config y htdocs	<code>mkdir -p server/{logs, config, htdocs}</code>	Seguro
236	Genera proyecto modular con módulos para base de datos, API y UI	<code>mkdir -p app/{db, api, ui}</code>	Seguro
237	Crea un proyecto backend con soporte para consultas GraphQL	<code>create_project (backend)</code>	Seguro
238	Genera estructura de automatización con scripts, tests y registros	<code>mkdir -p auto/{scripts, tests, logs}</code>	Seguro
239	Crea la estructura de un proyecto fullstack con React y Express	<code>create_project (fullstack)</code>	Seguro
240	Genera proyecto de documentación técnica con secciones de API, guías y ejemplos	<code>mkdir -p docs/{api, guides, examples}</code>	Seguro

B.8. Apertura de aplicaciones

Esta categoría agrupa solicitudes para **abrir una aplicación por voz sin especificar un archivo** (acción `open_app`). Son tareas que LIMA resuelve de forma local mediante el

IntentDetector —sin escalar al modelo de lenguaje— y que, por tanto, quedan *dentro del alcance* del sistema (sección 4.2.2).

Tabla B.8: Tareas de evaluación — Apertura de aplicaciones (10 tareas)

ID	Solicitud en lenguaje natural	Acción esperada	Riesgo
241	Abre Visual Studio Code	<code>open_app(code)</code>	Seguro
242	Inicia el navegador Firefox	<code>open_app(firefox)</code>	Seguro
243	Abre la terminal	<code>open_app(terminal)</code>	Seguro
244	Lanza LibreOffice Writer	<code>open_app(libreoffice_writer)</code>	Seguro
245	Abre la hoja de cálculo (LibreOffice Calc)	<code>open_app(libreoffice_calc)</code>	Seguro
246	Abre el explorador de archivos	<code>open_app(nautilus)</code>	Seguro
247	Inicia el editor de texto gedit	<code>open_app(gedit)</code>	Seguro
248	Abre el reproductor VLC	<code>open_app(vlc)</code>	Seguro
249	Lanza GIMP	<code>open_app(gimp)</code>	Seguro
250	Abre el navegador Chromium	<code>open_app(chromium)</code>	Seguro

Anexo C

Instrumentos de evaluación cualitativa

Este anexo reúne los instrumentos empleados en las pruebas de usabilidad. Las secciones C.1 y C.2 reproducen, respectivamente, el formato de consentimiento informado y el guión del facilitador de la **segunda ronda** (la evaluación formal, junio de 2026). Las secciones C.3 y C.4 conservan los instrumentos de la primera ronda exploratoria (enero de 2026), de los que derivaron los anteriores.

C.1. Formato de consentimiento informado — Segunda ronda

A continuación se reproduce, tal como se entregó y firmó al inicio de cada sesión, el consentimiento informado de la segunda ronda de pruebas de usabilidad.

Universidad Tecnológica de la Mixteca (UTM)

Ingeniería en Computación

LIMA

Linux Intelligent Middleware Assistant

Formulario de Consentimiento Informado

Segunda Tanda de Pruebas de Usabilidad – 2026

Investigador responsable: José Ramón Aragón Toledo

Tesis de Licenciatura, 2026

1. Descripción del Estudio

Usted ha sido invitado(a) a participar en un estudio de evaluación de usabilidad de **LIMA** (*Linux Intelligent Middleware Assistant*), un asistente de voz para la gestión de archivos y consulta de información del sistema en entornos GNU/Linux, desarrollado como parte de una tesis de licenciatura.

La sesión tendrá una duración aproximada de **35 a 45 minutos** e incluirá **3 tareas prácticas**:

1. **Gestión de archivos:** mover archivos mediante instrucciones de voz.
2. **Consulta del sistema:** solicitar datos sobre el sistema operativo y los recursos del equipo.
3. **Búsqueda web:** realizar una búsqueda de información en internet a través del asistente.

Con su autorización, la sesión podrá ser grabada en audio y/o video con fines exclusivamente académicos.

2. Confidencialidad y Derechos del Participante

- Sus datos serán **codificados** bajo un número de participante y nunca se asociarán públicamente a su identidad. Solo el investigador responsable tendrá acceso a la información recopilada.
- Los datos se conservarán un **máximo de 2 años** y no serán compartidos con terceros ni utilizados para otros fines.
- Su participación es **completamente voluntaria**. Puede **retirarse en cualquier momento** y **solicitar la eliminación de sus datos** sin consecuencia alguna.
- Tiene **18 años de edad o más**.

3. Datos del Participante

(Complete con letra de molde o encierre en un círculo la opción correspondiente.)

Nombre completo: _____

LIMA – Linux Intelligent Middleware Assistant

Segunda Tanda de Pruebas de Usabilidad

Edad: _____ años **Lugar y fecha:** _____**Grado de estudios** (*encierre en un círculo*):

Licenciatura / Maestría / Doctorado / Otro: _____

Experiencia con tecnología (*encierre en un círculo*):

Básica / Intermedia / Avanzada

Experiencia previa con asistentes de voz (*encierre en un círculo*):

Ninguna / Poca / Regular / Mucha

¿Ha usado la terminal / línea de comandos de Linux? (*encierre en un círculo*):

Sí / No

Condición experimental asignada (*encierre en un círculo*):**A** – DeepSeek / **B** – OpenAI GPT-4o

4. Firma de Consentimiento

Yo, _____, declaro haber leído y comprendido el presente documento y acepto participar voluntariamente en este estudio.

Firma del participante**Firma del investigador**

José Ramón Aragón Toledo

Fecha: _____*Fecha:* _____**Número de participante:** _____

Contacto: Para cualquier duda relacionada con este estudio, puede comunicarse con el investigador responsable: José Ramón Aragón Toledo.

C.2. Guión de prueba de usabilidad — Segunda ronda

A continuación se reproduce el guión del facilitador utilizado en la segunda ronda. El documento estandariza los escenarios de las tres tareas, los criterios de éxito, los archivos esperados y las preguntas de cada cuestionario.

Guión de Prueba de Usabilidad — Segunda Tanda

LIMA — Linux Intelligent Middleware Assistant

Universidad Tecnológica de la Mixteca — Instituto de Computación
Tesis de Licenciatura, 2026

Tipo de estudio	Prueba de usabilidad comparativa intra-sujeto (ejecución con entorno gráfico vs. ejecución por voz con LIMA). El orden entorno gráfico → voz es fijo para todos.
Condiciones (modelo)	A — DeepSeek o B — OpenAI GPT-4o . Se asigna por participante (ver Hoja de Observador).
Duración estimada	40–55 minutos
Tareas	(1) Organizar carpetas del semestre (2) Mover el temario (3) Investigar la POO

Este documento es solo el guión de lectura y conducción.

Los datos de la sesión y *todas* las respuestas (tiempos, éxito, preguntas post-tarea, cuestionario final y observaciones) se registran en la **Hoja de Observador**, que es un documento aparte.

Perfil del usuario objetivo (criterio de reclutamiento — no leer al usuario)

Tipo de usuario al que va dirigida la prueba:

Usuarios sin conocimiento previo en GNU/Linux que tienen la necesidad de aprender a usarlo.

Características que debe cumplir cada usuario:

- **Edad entre 18 y 25 años** (las pruebas están enfocadas a estudiantes).
- **Poca o nula experiencia técnica en GNU/Linux**: no haberlo usado nunca, o casi nunca (usuario novato en este entorno).
- Manejar a nivel usuario un sistema operativo de escritorio (Windows o macOS): crear carpetas, mover archivos, navegar en internet.
- No tener formación avanzada en administración de sistemas ni uso habitual de la línea de comandos.
- Tener una motivación real o potencial para aprender Linux (p. ej. estudiante que lo requiere para una materia o trabajo).

Criterio de la muestra (cuota de género): al menos la mitad de los usuarios deben ser mujeres.

Verificar el cumplimiento del perfil antes de iniciar y **marcarlo en la Hoja de Observador**.

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)**Pasos antes de iniciar la sesión:**

1. Verificar que LIMA esté ejecutándose con la condición correcta (**DeepSeek** si es condición A; **OpenAI GPT-4o** si es condición B).
2. Verificar que el **Escritorio esté limpio** (sin una carpeta **302-A** previa).
3. Colocar en `~/Descargas` varios archivos `.pdf` (al menos 3), uno de ellos nombrado de forma reconocible para la materia objetivo, p. ej. `Temario_Programacion.pdf`.
4. Verificar que la conexión a internet esté activa.
5. Tener preparado el cronómetro y la **Hoja de Observador** impresa.
6. Silenciar todas las notificaciones del sistema.

TEXTO PARA LEER AL USUARIO

Leer al usuario en voz alta y con calma:

“Buenos días/tardes y muchas gracias por su tiempo. Le pedimos su apoyo para evaluar un asistente de voz para computadoras con el sistema GNU/Linux, llamado **LIMA**.

Queremos comentarle que a nosotros únicamente se nos encargó realizar estas pruebas para encontrar errores y posibles mejoras en la aplicación; más allá de eso no estamos involucrados en su desarrollo, así que puede darnos su opinión con total libertad y franqueza.

Es muy importante que sepa que **estamos evaluando al sistema, no a usted**. No hay respuestas correctas ni incorrectas; cualquier dificultad que encuentre nos ayuda a mejorar el software.

Le pido que, mientras trabaja, **piense en voz alta**: cuénteme lo que está pensando, lo que intenta hacer o lo que le genera dudas.

Si en algún momento desea tomar un descanso, puede pedirlo. También es libre de **retirarse en cualquier momento** sin ningún problema. ¿Tiene alguna pregunta antes de comenzar?”

Escenario general — contexto del usuario (leer al usuario primero, antes de cualquier tarea)

“Para esta prueba le pedimos que se ponga en el papel de un **estudiante universitario del grupo 302-A**. Está por iniciar el semestre y, entre sus tareas, tiene una que es **obligatorio realizar en el sistema operativo GNU/Linux**: no puede usar Windows ni macOS para hacerla.

A lo largo de la sesión iremos resolviendo, paso a paso, lo que ese estudiante necesita hacer en su computadora con Linux.”

Cómo trabajaremos cada tarea (leer al usuario, ya que entró en el papel)

“En cada tarea le pediré que la haga de dos formas: **primero usando el entorno gráfico** de la computadora (con el ratón y el teclado, como lo haría normalmente) y, **después, pidiéndosela por voz** al asistente. Así podremos comparar cómo se siente de una y de otra forma.”

Marco de métricas (metodología — no leer al usuario)

En cada tarea se registran **tiempo** y **éxito** tanto de la versión con *entorno gráfico* como de la versión *por voz* (en la Hoja de Observador). Con esos datos se calculan, al final, los tres indicadores de usabilidad (ISO 9241-11):

- **Eficacia (tasa de éxito)**. Grado en que la tarea se completa correctamente.

Eficacia = $\frac{\text{tareas completadas}}{\text{tareas intentadas}} \times 100$. Se compara la eficacia con *entorno gráfico* contra la eficacia *por voz*.

- **Eficiencia (tiempo).** Recursos (tiempo) empleados para lograr la tarea.

$\Delta t = \frac{t_{\text{gráf}} - t_{\text{voz}}}{t_{\text{gráf}}} \times 100$. Un valor *positivo* significa que LIMA fue más rápido; *negativo*, más lento.

- **Satisfacción.** Componente subjetivo: opinión del usuario recogida con las escalas y las preguntas post-tarea y finales.

Nota: la “tasa de éxito” se calcula sobre el total de intentos. El orden entorno gráfico → voz es fijo para todos los usuarios.

Criterios de éxito y de no éxito (metodología — no leer al usuario)

Aplicar de forma uniforme en todas las tareas, tanto en la versión con entorno gráfico como en la versión por voz.

Se considera ÉXITO cuando:

1. El usuario logra realizar la tarea **como se esperaba**.
2. El usuario logra realizar la tarea **de otra manera no prevista**, alcanzando igualmente el resultado.

Se considera NO ÉXITO cuando:

1. El usuario **no puede completar** la tarea.
2. El usuario **excede los 12 minutos** en la misma tarea.
3. El **facilitador interviene** y le ayuda a completar la tarea.

Regla de oro del facilitador (cómo conducir — no leer al usuario)

Nunca indique CÓMO realizar una tarea; solo diga QUÉ se desea lograr. No describa pasos, ni señale botones, menús o atajos.

- Si el usuario se bloquea, no le dé la solución: **sugíerale un par de opciones o espere a que él proponga una** y, en ese caso, invítelo a intentarla (“puede intentarlo de esa manera”).
- Sobre el tiempo: no le diga un límite exacto, pero tampoco dé a entender que dispone de tiempo ilimitado. Pídale trabajar a un ritmo cómodo. El límite interno de 12 minutos por tarea es solo para el facilitador.
- Recuerde: si el facilitador resuelve la tarea por el usuario, **se considera NO éxito**.

TAREA 1: Organizar las carpetas del semestre

Escenario (leer al usuario)

“Como estudiante del grupo 302-A que está por entrar al semestre, lo primero que necesita es **organizar sus materias**. Quiere crear, en el **Escritorio**, una carpeta llamada **302-A**, y dentro de ella una carpeta para cada una de sus **6 materias**:”

- Matemáticas discretas
- Cálculo integral para ingeniería
- Cultura y sociedad
- Teoría de la computación
- Bases de datos
- Programación orientada a objetos

“Puede usar abreviaturas o el nombre completo, como prefiera.”

Objetivo: Crear en el Escritorio la carpeta 302-A con 6 subcarpetas (una por materia). El nombre exacto puede abreviarse; lo que se evalúa es la estructura.

PARTE A — Versión con entorno gráfico (apuntar y dar clic)

Leer al usuario:

“Empecemos. Le pido que, **usando únicamente el entorno gráfico** (el ratón y el teclado, como lo haría en cualquier computadora), arme en el Escritorio esa organización de carpetas que acabamos de describir: la carpeta del grupo y, dentro, una carpeta por cada materia. Hágalo a un ritmo cómodo, con naturalidad, y recuerde pensar en voz alta.”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Inicia el cronómetro al primer movimiento del usuario y registra el tiempo en la **Hoja de Observador** (columna *Entorno gráfico*). No ayudes salvo bloqueo total. Marca allí los criterios de éxito de la versión con entorno gráfico.

Transición (leer al usuario)

“¡Muy bien! Ya tiene su semestre organizado usando el entorno gráfico. Ahora que ya hizo esta tarea, **¿le gustaría probar a hacerla otra vez, pero esta vez pidiéndosela por voz al asistente** que trae instalado esta máquina? Vamos a comparar cómo se siente de una forma y de la otra.”

PARTE B — Versión por voz (asistente LIMA)

Indicar al usuario:

“Perfecto. Para que no se mezcle con lo anterior, imagine que es un grupo distinto: pídale al asistente, **por voz**, que cree una carpeta y sus materias dentro. Háblele con naturalidad, como le hablaría a alguien más.”

Solo si lo necesita, ejemplo de cómo podría pedirlo (no obligatorio):

“Crea en el escritorio una carpeta llamada 302-A y dentro crea carpetas para mis materias: matemáticas discretas, cálculo, cultura y sociedad, teoría de la computación, bases de datos y programación orientada a objetos.”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Para evitar mezclar resultados con la parte de entorno gráfico, antes de la versión por voz **elimina la carpeta ya creada** (o pide la estructura con otro nombre y elimínala después). Inicia el cronómetro y registra en la **Hoja de Observador** (columna *Voz*). Marca allí los criterios de éxito de la versión por voz.

→ **Hoja de Observador:** Completa la **tabla de métricas** y las **Preguntas Post-Tarea 1** en la Hoja de Observador.

TAREA 2: Mover el temario a su materia

Escenario (leer al usuario)

“Ya tiene su semestre organizado. Ahora, un profesor le compartió el **temario de la materia de Programación Orientada a Objetos** y usted lo descargó: está, junto con otros PDF, en su carpeta de **Descargas**. Necesita **encontrar ese PDF** y **moverlo** a la carpeta de la materia de Programación Orientada a Objetos que acaba de crear.”

Objetivo: Localizar entre los PDF de ~/Descargas el archivo del temario de Programación Orientada a Objetos (**Temario_Programacion.pdf**) y moverlo a la subcarpeta de *Programación orientada a objetos* dentro de 302-A.

PARTE A — Versión con entorno gráfico (apuntar y dar clic)

Leer al usuario:

“Igual que antes, primero **usando el entorno gráfico**: le pido que mueva el temario de Programación Orientada a Objetos, que está entre los PDF de su carpeta de Descargas, a la carpeta de esa materia.”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Inicia el cronómetro y registra en la **Hoja de Observador** (columna *Entorno gráfico*). Marca allí los criterios de éxito de la versión con entorno gráfico.

Transición (leer al usuario)

“Perfecto, ya movió su temario a donde corresponde. Como en la tarea anterior, **ahora probemos lo mismo pero por voz**: le pediremos al asistente que busque y mueva el archivo por usted.”

PARTE B — Versión por voz (asistente LIMA)

Indicar al usuario:

“Pídale al asistente, **por voz**, que busque el PDF del temario en Descargas y lo mueva a la carpeta de su materia. Háblele con naturalidad.”

Solo si lo necesita, ejemplo de cómo podría pedirlo (no obligatorio):

“Busca en Descargas el PDF del temario de programación orientada a objetos y muévelo a la carpeta de esa materia en 302-A.”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Antes de la versión por voz, **regresa el PDF a Descargas** para que el usuario pueda repetir la operación. Inicia el cronómetro y registra en la **Hoja de Observador**

(columna *Voz*). Marca allí los criterios de éxito de la versión por voz.

→ **Hoja de Observador:** Completa la **tabla de métricas** y las **Preguntas Post-Tarea 2** en la Hoja de Observador.

TAREA 3: Investigar qué es la Programación Orientada a Objetos

Escenario (leer al usuario)

“Ya tiene el temario de Programación Orientada a Objetos en su carpeta, pero antes de la primera clase quiere llegar preparado. Necesita **buscar en internet** una **definición o algunos ejemplos de la Programación Orientada a Objetos** para entender de qué trata la materia.”

Objetivo: Realizar una búsqueda en internet sobre la definición o ejemplos de la Programación Orientada a Objetos y abrir los resultados en el navegador.

PARTE A — Versión con entorno gráfico (apuntar y dar clic)

Leer al usuario:

“Como siempre, primero **usando el entorno gráfico**: le pido que busque en internet una definición o algunos ejemplos de la Programación Orientada a Objetos.”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Inicia el cronómetro y registra en la **Hoja de Observador** (columna *Entorno gráfico*). Marca allí los criterios de éxito de la versión con entorno gráfico.

Transición (leer al usuario)

“Muy bien, ya encontró información por su cuenta. Para terminar, **hagamos esta última búsqueda por voz**: que sea el asistente quien busque por usted.”

PARTE B — Versión por voz (asistente LIMA)

Indicar al usuario:

“Pídale al asistente, **por voz**, que busque en internet la definición o ejemplos de la Programación Orientada a Objetos.”

Solo si lo necesita, ejemplo de cómo podría pedirlo (no obligatorio):

“*Busca en internet qué es la programación orientada a objetos y algunos ejemplos.*”

INSTRUCCIONES PARA EL FACILITADOR (protocolo interno — no leer al usuario)

Inicia el cronómetro y registra en la **Hoja de Observador** (columna *Voz*). Marca allí los criterios de éxito de la versión por voz.

→ **Hoja de Observador:** Completa la **tabla de métricas** y las **Preguntas Post-Tarea 3** en la Hoja de Observador.

Cierre de la sesión

Cuestionario final (leer al usuario)

“Hemos terminado las tareas. Para cerrar, me gustaría hacerle unas preguntas sobre su experiencia general con el asistente. No hay respuestas correctas ni incorrectas; me interesa su opinión sincera.”

Pasos de cierre (no leer al usuario)

1. Aplica el **Cuestionario Final (Evaluación General)** leyendo cada pregunta en voz alta; registra las respuestas en la **Hoja de Observador**.
2. Completa las **Notas y comentarios** y las **observaciones del facilitador/observador**.
3. Agradece al usuario su tiempo y participación.
4. Realiza el **reseteo del entorno** (eliminar 302-A, devolver el PDF a Descargas, cerrar ventanas) según la lista de la Hoja de Observador, antes del siguiente usuario.

→ **Hoja de Observador:** Todo el cierre (cuestionario final, tabla resumen, notas, evaluación del observador y reseteo del entorno) se completa en la Hoja de Observador.

Fin del guión de la sesión de prueba.

C.3. Formato de consentimiento informado — Primera ronda

Formulario de Consentimiento Informado

Plurix — Asistentes de Voz para Gestión de Archivos

Fecha de elaboración: 29 de enero de 2026

Consentimiento de Participación en Estudio de Usabilidad

Investigador: Jose Ramon Aragon Toledo

Fecha: _____

Descripción del Estudio

Este es un estudio de usabilidad diseñado para evaluar la facilidad de uso de la aplicación **Plurix**, un sistema de asistentes de voz para gestión de archivos. Su participación es completamente voluntaria.

Durante esta sesión usted realizó 4 tareas prácticas con la aplicación. Se midió el tiempo de cada tarea y se recopiló su opinión mediante preguntas breves. La sesión duró aproximadamente 25–35 minutos.

Confidencialidad

- Sus datos serán codificados bajo un número de participante.
- Solo el investigador tendrá acceso a la información.
- Los datos se almacenarán de forma segura y cifrada.
- Los datos se conservarán durante el período de análisis (máximo 2 años).

Derechos del Participante

Al firmar este documento, usted reconoce que:

- Ha leído y entendido esta información.
- Puede retirarse del estudio en cualquier momento sin dar explicaciones.
- Puede solicitar la destrucción de sus datos en cualquier momento.
- Tiene 18 años de edad o más.
- Acepta participar voluntariamente en el estudio.

Riesgos y Beneficios

Riesgos: Este estudio presenta riesgos mínimos. Puede experimentar fatiga leve o frustración temporal si encuentra dificultades con la aplicación. Puede solicitar un descanso en cualquier momento.

Beneficios: Su participación contribuirá directamente a mejorar la aplicación Plurix para futuros usuarios.

Firma de Consentimiento

Yo, _____, declaro que he leído este documento, entiendo el propósito del estudio y acepto participar voluntariamente.

Firma del participante:

Firma del investigador:

Fecha: _____

Fecha: _____

Hora: _____

Hora: _____

Número de participante: _____

Contacto: Si tiene preguntas sobre este estudio, puede contactar al investigador Jose Ramon Aragon Toledo.

C.4. Guión de prueba de usabilidad — Primera ronda

A continuación se reproduce el guión utilizado durante la primera ronda de pruebas de usabilidad individuales, realizada el 30 de enero de 2026. El documento sirvió como guía para el facilitador durante cada sesión y establece los escenarios, los criterios de éxito y las métricas de cada tarea.

Datos generales del estudio

- **Investigador:** Jose Ramon Aragon Toledo
- **Fecha de elaboración:** 29 de enero de 2026
- **Tipo de evaluación:** Prueba de usabilidad individual

- **Sistema evaluado:** Plurix — Asistentes de Voz para Gestión de Archivos
- **Asistentes disponibles:** Nova (archivos), Alloy (búsqueda), Onyx (sistema), Fable (web)

Instrucciones para el participante

Se proporcionó al participante la siguiente información antes de comenzar:

1. El objetivo de la prueba es evaluar la aplicación, **no** al participante.
2. Se solicitó pensar en voz alta durante toda la sesión.
3. El participante podía solicitar ayuda o detenerse en cualquier momento.
4. Se informó que la sesión sería grabada (pantalla y audio) con su consentimiento.
5. Se registraron: nombre del participante, edad, experiencia con Linux y experiencia con asistentes de voz.

Tarea 1: Búsqueda de archivos

- **Asistente:** Alloy (búsqueda)
- **Escenario:** «Estás trabajando en tu computadora y necesitas encontrar un archivo llamado `tesis_final.txt` que guardaste en alguna parte de tu carpeta personal.»
- **Objetivo:** Localizar el archivo `tesis_final.txt` dentro del directorio de trabajo.
- **Comando sugerido:** `find $HOME -name "tesis_final.txt"`
- **Criterios de éxito:**
 1. El asistente reconoce correctamente la instrucción de voz.
 2. El comando de búsqueda se genera correctamente.
 3. El archivo se localiza exitosamente.
 4. El participante comprende la respuesta del sistema.

Métricas registradas: Tiempo de inicio, tiempo de fin, duración total, intentos de voz, estado (completada sin ayuda / completada con ayuda / no completada), errores del sistema (tipo), notas de observación.

Preguntas de control posteriores (escala 1–5):

1. ¿Qué tan fácil fue completar esta tarea?
2. ¿Fue claro cómo formular la instrucción de voz?
3. ¿La respuesta del sistema fue comprensible y útil?
4. ¿Sentiste que tenías control sobre lo que el sistema iba a hacer?
5. ¿La velocidad de respuesta fue adecuada?

Tarea 2: Mover archivos

- **Asistente:** Nova (archivos)
- **Escenario:** «Necesitas organizar tus archivos. Quieres mover los archivos de tipo `.pdf` de la carpeta `Descargas` a la carpeta `Documentos`.»
- **Objetivo:** Mover archivos PDF de un directorio a otro.
- **Comando sugerido:** `mv $HOME/Descargas/*.pdf $HOME/Documentos/`
- **Criterios de éxito:**
 1. El asistente reconoce correctamente la instrucción.
 2. El comando se genera correctamente (ruta origen y destino).
 3. Los archivos se mueven exitosamente.
 4. El participante comprende el resultado.

Métricas registradas y preguntas de control: Idénticas a la Tarea 1.

Tarea 3: Eliminar archivos temporales

- **Asistente:** Nova (archivos)
- **Escenario:** «Tu carpeta de trabajo se ha llenado de archivos temporales con extensión `.tmp` que ya no necesitas. Pide al asistente que los elimine.»
- **Objetivo:** Eliminar archivos con extensión `.tmp` del directorio de trabajo.
- **Comando sugerido:** `rm $HOME/trabajo/*.tmp`
- **Criterios de éxito:**
 1. El asistente reconoce la instrucción correctamente.
 2. El sistema muestra una advertencia de seguridad antes de ejecutar (nivel de riesgo: advertencia).
 3. El comando se genera correctamente.
 4. El participante entiende la advertencia y confirma la acción.

Métricas registradas y preguntas de control: Idénticas a la Tarea 1.

Tarea 4: Búsqueda en Internet

- **Asistente:** Fable (web)
- **Escenario:** «Necesitas investigar sobre un tema para tu clase. Pide al asistente que busque información sobre “inteligencia artificial en la educación” en Internet.»
- **Objetivo:** Realizar una búsqueda web a través del asistente de voz.
- **Comando sugerido:** `xdg-open "https://www.google.com/search?q=inteligencia+`

■ Criterios de éxito:

1. El asistente reconoce la solicitud.
2. Se genera la URL de búsqueda correctamente.
3. Se abre el navegador con los resultados.
4. El participante considera útil el resultado.

Métricas registradas y preguntas de control: Idénticas a la Tarea 1.

Resumen de la prueba

Al finalizar las cuatro tareas, el facilitador registró:

- Tiempo total de la sesión.
- Número total de errores del sistema.
- Tasa de éxito general (tareas completadas sin ayuda / total de tareas).
- Observaciones generales del comportamiento del participante.

Evaluación general del sistema

Se formularon las siguientes preguntas abiertas al finalizar:

1. ¿Cuál fue tu impresión general de la aplicación?
2. ¿Qué fue lo que más te gustó?
3. ¿Qué fue lo que menos te gustó o te resultó más difícil?
4. ¿Usarías esta aplicación en tu vida diaria? ¿Por qué?
5. ¿Tienes alguna sugerencia para mejorar la aplicación?

Protocolo para el evaluador

1. Preparar el entorno de prueba antes de la llegada del participante.
2. Verificar que los archivos de prueba estén en su ubicación esperada.
3. Leer las instrucciones al participante de forma neutral.
4. No intervenir durante la ejecución de tareas, salvo si el participante solicita ayuda.
5. Registrar métricas objetivas inmediatamente después de cada tarea.
6. Tomar notas de verbalizaciones y comportamientos espontáneos.
7. Al terminar, agradecer al participante y confirmar la eliminación posterior de grabaciones.

Archivos de prueba esperados

La estructura de directorios preparada para cada sesión incluía los siguientes archivos:

- `$HOME/tesis_final.txt` — Archivo objetivo de la Tarea 1.
- `$HOME/Descargas/documento1.pdf`, `documento2.pdf`, `documento3.pdf` — Archivos para la Tarea 2.
- `$HOME/trabajo/temp1.tmp`, `temp2.tmp`, `temp3.tmp`, `datos.txt` — Archivos para la Tarea 3 (solo los `.tmp` deben eliminarse).

Anexo D

Código fuente de módulos principales

Este apéndice reproduce los *prompts* del sistema empleados por el módulo `CommandParser.pm` (Sección 3.10), omitidos en el cuerpo del capítulo por su extensión. El primero es la versión compacta para modelos locales; el segundo, la versión completa para GPT-4o.

Listing D.1: *Prompt* compacto para modelos locales (`_get_local_system_prompt()`)

```
Interpreta comandos de voz para gestionar archivos en Linux.
Responde UNICAMENTE con JSON valido, sin texto adicional.

Formato:
{"action":"ACCION","target_type":"file|directory|system|web",
 "target_path":"RUTA","source_path":"","parameters":{}},
"requires_confirmation":false,"tts_response":"BREVE"}

Acciones: create, list, search, edit, move, rename, delete,
          info, answer, create_project, system_info,
          web_search, analyze_text, clarify

Reglas:
- Rutas con ~/ (Documentos, Descargas, Escritorio, etc.)
- requires_confirmation=true SOLO para delete
- tts_response maximo 12 palabras
- Si falta ubicacion: action="clarify",
  tts_response="En que carpeta?"
- Si puedes responder del historial: action="answer"
- Búsqueda siempre recursiva

Parametros de search: pattern, extension, date_exact,
                     date_before, date_after, content_contains, size_min, size_max
Parametros de create: content (texto del archivo),
                     subdirs (subcarpetas)
Parametros de system_info: info_type
                          (processes|memory|disk|network|ports)
Parametros de web_search: query, site
                          (google|youtube|github|stackoverflow|wikipedia)
Parametros de analyze_text: word (palabra especifica,
```

```
opcional), top_n (limite resultados, default 20)
```

Ejemplo: "lista archivos del escritorio"

```
{ "action": "list", "target_type": "directory",
  "target_path": "~/Escritorio", "parameters": {},
  "requires_confirmation": false,
  "tts_response": "Listando escritorio" }
```

Listing D.2: *Prompt* del sistema para GPT-4o (`_get_system_prompt()`)

Eres un asistente que interpreta comandos de voz para gestionar archivos en Linux.

Tu tarea es convertir el texto del usuario en un objeto JSON estructurado. DEBES responder UNICAMENTE con JSON valido, sin texto adicional.

IMPORTANTE - MEMORIA DE CONVERSACION:

- Tienes acceso al historial de la conversacion
- Los mensajes previos incluyen [RESULTADO DE EJECUCION], [DETALLE], [LISTA] y [RUTA_ACTUAL]
- [RUTA_ACTUAL] indica donde se realizo la ultima operacion (ej: ~/Escritorio)
- Cuando el usuario dice "de estos", "de esos", "los anteriores", se refiere a los resultados previos
- Cuando el usuario dice "esa carpeta", "ese archivo", "ahi" se refiere a la [RUTA_ACTUAL] previa
- USA [RUTA_ACTUAL] para operaciones sobre elementos de busquedas anteriores
- Si el usuario pregunta "cuantos son archivos" despues de listar, CUENTA los archivos del [DETALLE] previo
- Si puedes responder con la informacion del historial, usa action="answer" con tts_response conteniendo la respuesta

IMPORTANTE - RUTAS RELATIVAS Y CARPETAS PARCIALES:

- Si el usuario menciona una carpeta SIN ruta completa, construye la ruta relativa a
~/Documentos/GitHub/Plurix/Plurix/
- Carpetas especiales del proyecto de pruebas:
 - * "pruebas usabilidad" ->
~/Documentos/GitHub/Plurix/Plurix/pruebas_usabilidad
 - * "semestre actual" ->
.../pruebas_usabilidad/semestre_actual
 - * "descargas pendientes" ->
.../pruebas_usabilidad/descargas_pendientes
 - * "entregas" ->
.../pruebas_usabilidad/entregas
 - * "archivos temporales" ->
.../pruebas_usabilidad/archivos_temporales
- Para mover archivos entre carpetas de pruebas, usa las rutas completas
- La busqueda siempre es RECURSIVA (incluye subcarpetas automaticamente)

Estructura del JSON:

```
{
  "action": "string",
  "target_type": "string",
  "target_path": "string",
  "source_path": "string",
  "parameters": {},
  "requires_confirmation": bool,
  "tts_response": "string"
}
```

ACCIONES SOPORTADAS:

1. ARCHIVOS Y CARPETAS:

- create: Crear archivos o directorios
- list: Listar contenido de directorios
- search: Buscar archivos
- edit: Abrir/editar archivo o carpeta (usa xdg-open)
- move: Mover archivos/carpetas
- rename: Renombrar
- delete: Eliminar (SIEMPRE requires_confirmation=true)
- info: Informacion de archivo/carpeta
- analyze_text: Analizar frecuencia de palabras en un archivo
- answer: Responder una pregunta usando informacion del historial (solo tts_response)

2. PROYECTOS:

- create_project: Crear estructura de proyecto

3. SISTEMA:

- system_info: Informacion del sistema (procesos, memoria, red, disco)

4. INTERNET:

- web_search: Buscar informacion en internet (Google, YouTube, GitHub, StackOverflow, Wikipedia)

IMPORTANTE – BUSQUEDA EN INTERNET:

- Cuando el usuario pide "buscar en internet", "buscar en Google/YouTube/etc", usa action="web_search"
- Parametros: query (lo que se busca), site (google, youtube, github, stackoverflow, wikipedia)
- Si no se especifica sitio, usar "google" por defecto

IMPORTANTE – EDICION DE ARCHIVOS:

- Si el usuario pide "abrir", "editar", "mostrar" un archivo, usa action="edit"
- Si se refiere a archivos de búsquedas previas ([LISTA]), usa el primer archivo de la lista

FILTROS DE BUSQUEDA AVANZADA:

Para search, puedes usar parametros de fecha:

- "date_exact": "hoy", "ayer", "hace 3 dias", "15/01/2026", "15 de enero"
- "date_before": "15/01/2026"
- "date_after": "10/01/2026"
- "extension": "pdf", "jpg", "txt"
- "pattern": "*. {jpg,png}"
- "content_contains": "error"

```
- "size_min": "100MB"
- "size_max": "1GB"
- "size_exact": "500KB" (tamaño aproximado +/- 10%)

CREAR ARCHIVOS CON CONTENIDO:
{
  "action": "create",
  "target_type": "file",
  "target_path": "~/archivo.py",
  "parameters": {
    "content": "# Codigo aqui\nprint('Hola mundo')\n"
  },
  "requires_confirmation": false,
  "tts_response": "Creando archivo con codigo"
}

CREAR CARPETAS CON SUBCARPETAS:
{
  "action": "create",
  "target_type": "directory",
  "target_path": "~/Documentos/Proyecto",
  "parameters": {
    "subdirs": ["src", "docs", "tests"]
  }
}

INFORMACION DEL SISTEMA:
{
  "action": "system_info",
  "target_type": "system",
  "target_path": "",
  "parameters": {
    "info_type": "processes|memory|disk|network|ports"
  },
  "requires_confirmation": false,
  "tts_response": "Mostrando informacion del sistema"
}

EJEMPLOS:

Usuario: "Crea un archivo hola.c con un hola mundo"
{
  "action": "create",
  "target_type": "file",
  "target_path": "~/hola.c",
  "parameters": {
    "content": "#include <stdio.h>\n\nint main() {\n    printf(\"\\nHola mundo\\n\\n\");\n    return 0;\n}\n"
  },
  "requires_confirmation": false,
  "tts_response": "Creando archivo hola punto c"
}

Usuario: "Busca fotos en Descargas de ayer"
{
```

```
"action": "search",
"target_type": "file",
"target_path": "~/Descargas",
"parameters": {
  "pattern": ".*{jpg,jpeg,png,gif,bmp}",
  "date_exact": "ayer"
},
"requires_confirmation": false,
"tts_response": "Buscando fotos de ayer"
}
```

Usuario: "Busca archivos que contengan la palabra error"

```
{
  "action": "search",
  "target_type": "file",
  "target_path": "~/Documentos",
  "parameters": {
    "content_contains": "error"
  },
  "requires_confirmation": false,
  "tts_response": "Buscando archivos con la palabra error"
}
```

Usuario: "Encuentra archivos de mas de 100 MB en Descargas"

```
{
  "action": "search",
  "target_type": "file",
  "target_path": "~/Descargas",
  "parameters": {
    "size_min": "100MB"
  },
  "requires_confirmation": false,
  "tts_response": "Buscando archivos grandes"
}
```

Usuario: "Busca en YouTube tutorial de Perl"

```
{
  "action": "web_search",
  "target_type": "web",
  "target_path": "",
  "parameters": {
    "query": "tutorial de Perl",
    "site": "youtube"
  },
  "requires_confirmation": false,
  "tts_response": "Buscando en YouTube"
}
```

REGLAS:

1. Responde SOLO con JSON valido
2. Para archivos de codigo, SIEMPRE genera el contenido completo y funcional
3. Usa rutas con ~/Documentos, ~/Descargas, ~/Escritorio
4. requires_confirmation=true SOLO para delete
5. Mensajes TTS breves (maximo 12 palabras)

6. NO ejecutes comandos peligrosos (sudo, rm -rf /, etc.)

IMPORTANTE - DIRECTORIO NO ESPECIFICADO:

Si el usuario NO indica donde crear/buscar/operar:

```
{
  "action": "clarify",
  "target_type": "",
  "target_path": "",
  "parameters": {
    "question": "En que carpeta deseas hacerlo?
    Por ejemplo: Documentos, Escritorio, Descargas"
  },
  "requires_confirmation": false,
  "tts_response": "En que carpeta deseas hacerlo?"
}
```

EJEMPLOS ESPECIFICOS PARA PRUEBAS DE USABILIDAD:

Usuario: "Mueve el archivo manual SQL basico de
descargas pendientes a entregas"

```
{
  "action": "move",
  "target_type": "file",
  "source_path": "~/Documentos/GitHub/Plurix/Plurix/
  pruebas_usabilidad/descargas_pendientes/
  manual_sql_basico.pdf.md",
  "target_path": "~/Documentos/GitHub/Plurix/Plurix/
  pruebas_usabilidad/entregas",
  "parameters": {},
  "requires_confirmation": false,
  "tts_response": "Moviendo archivo a entregas"
}
```

Usuario: "En entregas crea una carpeta proyecto final IA
con subcarpetas src docs data y tests"

```
{
  "action": "create",
  "target_type": "directory",
  "target_path": "~/Documentos/GitHub/Plurix/Plurix/
  pruebas_usabilidad/entregas/proyecto_final_IA",
  "parameters": {
    "subdirs": ["src", "docs", "data", "tests"]
  },
  "requires_confirmation": false,
  "tts_response": "Creando carpeta con subcarpetas"
}
```